

The G17 technical reference

An independently recovered model of the Apple M5 GPU and its tensor unit,
and the compiler and runtime built on it

Spencer Bryngelson

Manuscript

AGXForge revision 7e39ba977e95, 2026-10-04

The AGXForge project: a native compiler for the Apple M5 GPU
<https://github.com/sbryngelson/AGXForge>

For the expert reader

The model of the G17 GPU and its in-core tensor unit given here was recovered independently of Apple. Each rule is stated with the evidence that established it, what it predicts about a program, and how the project’s compiler and runtime use it. The [showcase](#) reports application results. The [machine model](#) (MM) is the evidence record; a citation such as MM [25.91](#) links to the section that establishes a claim, and an opcode, as in tensor MMA, 16-bit × 16-bit, no accumulator (op5107), links to its entry in [appendix C](#). [Section 0.7](#) gives the kinds of statement used.

To judge the model quickly, start with these parts of the document.

1. **One checked program, two execution paths** ([sections 0.3](#) and [0.4](#), p. 8). A program the project compiled, traced from IR to instruction bytes, runs through Metal and below it, where the project’s runtime submits through Apple’s private IOPGPU interfaces, and gives the same exact output on both.
2. **Tensor layouts and arithmetic** ([sections 10.4](#) to [10.6](#), p. 100). The lane-to-element map was measured with one-hot probes, and a kernel has to keep three coordinate systems apart. The reduction tree was fixed with stimuli on which rival models predict different bits. A hand calculation shows that the accumulator enters first; Apple’s library adds C last and so differs by one rounding.
3. **Register chaining** ([section 10.9](#), p. 111). A D tile feeds the next MMA with no shuffle in this compiler’s kernels and with a row rotation in canonically packed ones; both cases were measured. The compiler allows a feed only where a measurement covers it.
4. **The compiler on one program** ([section 15.2](#), p. 162). Selection, tensor lowering, register allocation under lifetime and reservation rules, and scoreboard waits, shown on the same program as item 1.
5. **Native execution and its validation** ([sections 16.1](#) and [17.2](#), p. 170). These chapters separate what the native runtime builds from what stays Apple’s, report each level of validation on its own, and state where the evidence stops.

Four other experiments changed the reading of earlier results. A sentinel test showed that the register namespace ends at R125 ([section 2.1](#)); a release destroys a value as it is read ([section 2.5](#)); in a control with one wait left out, the consumer read zeros ([section 9.1](#)); and removing one metadata byte makes every MMA return zero ([section 10.7](#)). [Section 0.6](#) lists the limits; [chapter 14](#) indexes what is open.

Contents

0	Summary	7
0.1	Measured hardware and software	7
0.2	Recovered behaviour	7
0.3	Project components and execution paths	8
0.4	The running example	8
0.5	Applications	11
0.6	Limits of the evidence	12
0.7	Conventions	12
I	The machine	14
1	Device and execution model	15
1.1	Platform versions	15
1.2	Threads, simdgroups, threadgroups and grids	17
1.3	Core resource model	17
1.4	Residency and occupancy rules	18
1.5	Forward progress	19
1.6	Clock states and timing	20
2	The register file	22
2.1	Register namespace	23
2.2	Register classes	24
2.3	Narrow register fields	26
2.4	Operand modifiers	26
2.5	Release semantics	28
2.6	Loop-carried values	30
2.7	Encoding holes	31
2.8	Register budgets and spills	32
3	Instruction encoding	33
3.1	Instruction lengths and framing	33
3.2	Opcode identity and coverage counts	35
3.3	Field maps	36
3.4	Scoreboard field encoding	39
3.5	Program boundaries	40
3.6	Apple's decoder	41
3.7	Opcode renumbering in macOS 27	42
3.8	Decoder admission and execution	42
4	Scalar arithmetic	45
4.1	Integer arithmetic	45
4.2	Compares, flags and selects	47
4.3	fp32 arithmetic	49

4.4	fp16 arithmetic	50
4.5	Conversions	51
4.6	Special-function instructions	52
4.7	Scalar dependences and issue	53
5	Control flow and the execution mask	55
5.1	Execution-mask instructions	55
5.2	Predication and if-conversion	57
5.3	Branches and loops	58
6	Special registers	61
6.1	Register numbers and builtins	61
6.2	Slot-29 metadata numbering	64
6.3	Read latency and waits	64
7	Memory	66
7.1	Address computation	66
7.2	Load and store forms by width	68
7.3	Immediate-address and vector forms	70
7.4	Masked and predicated access	71
7.5	Out-of-range accesses	71
7.6	Threadgroup memory	72
7.7	Imageblock (per-core tile memory)	73
7.8	Constants and uniforms	75
7.9	Textures	76
8	Cross-lane operations	78
8.1	Shuffle-xor mask range	78
8.2	Late operands	79
8.3	Broadcast and inactive lanes	79
8.4	Native reductions	79
8.5	Other cross-lane forms	80
9	Synchronization	81
9.1	The scoreboard	82
9.2	Barriers and fences	85
9.3	Atomics	87
9.4	Ordering by scope	90
10	The tensor unit	93
10.1	Access through Metal	94
10.2	MMA instruction forms	95
10.3	MMA operand modifiers	98
10.4	Fragment layouts	100
10.5	Coordinate systems and packing	103
10.6	Arithmetic by input type	104
10.7	Ordering and the tensor enable	108
10.8	Tensor loads and stores	110

10.9	Chaining and the memory bridge	111
10.10	Data movement across simdgroups	113
10.11	Quantized formats	114
10.12	Reductions	117
10.13	Legacy simdgroup-matrix engine	118
10.14	Compiler rules for tensor code	120
11	Kernel metadata and the image	122
11.1	__GPU_METADATA slots	122
11.2	Constant program and record	125
11.3	ABI classes	127
11.4	Object, library and archive bytes	128
12	Performance	131
12.1	Tensor issue rate and latency	132
12.2	Bottlenecks in the studied kernels	133
12.3	Scalar issue	134
12.4	Cost of registers and spills	135
12.5	Residency and occupancy measurements	136
12.6	Memory bandwidth, latency and instruction fetch	139
12.7	Dispatch costs and counters	141
12.8	GPU clock and power	143
12.9	Cost models	144
12.10	Limiting resources by workload	146
13	Submission below Metal	150
13.1	Apple-owned layers	150
13.2	Device, context and queue setup	151
13.3	Allocations and address windows	151
13.4	Code placement and the program counter	152
13.5	Launch state and command pages	152
13.6	Submit record and IOGPU call	153
13.7	Completion and callback ownership	154
13.8	Multi-record Submits and dependent dispatches	154
13.9	Faults, hangs and recovery	155
13.10	Scope of the measured graphs	155
14	Open questions	156
14.1	The device, registers, encoding and metadata (chapters 1, 2, 3, 11)	156
14.2	Scalar arithmetic, control flow, special registers and cross-lane operations (chapters 4, 5, 6, 8)	157
14.3	Memory and synchronization (chapters 7, 9)	157
14.4	The tensor unit and performance (chapters 10, 12)	158

II	Compilation and delivery	160
15	The compiler	161
15.1	Input routes	161
15.2	The running example in the compiler	162
15.3	General selection and allocation rules	164
15.4	Tensor lowering	165
16	Delivery and validation	166
16.1	Image and validation ladder	166
16.2	Model graphs	167
III	Execution	168
17	Execution paths	169
17.1	Execution through Metal	169
17.2	Native execution below Metal	170
17.3	Comparison of the two paths	172
A	Corrections to earlier interpretations	173
A.1	The device, registers, encoding and metadata (chapters 1, 2, 3, 11)	173
A.2	Scalar arithmetic, control flow, special registers and cross-lane operations (chapters 4, 5, 6, 8)	174
A.3	Memory and synchronization (chapters 7, 9)	175
A.4	The tensor unit and performance (chapters 10, 12)	176
A.5	Superseded counts and wordings	179
B	Scalar instruction semantics	185
B.1	Integer add, subtract and saturate	185
B.2	Integer multiply and multiply-add	186
B.3	Logic, shifts and bit operations	187
B.4	Moves	188
B.5	Compares, flags and selects	189
B.6	fp32 arithmetic	191
B.7	fp16 and mixed-width float arithmetic	192
B.8	Conversions	192
B.9	Special-function instructions	193
B.10	Cross-lane and special-register reads	193
B.11	Control flow	194
B.12	Notes on the tables	194
C	Opcode index	196
D	Source code locations	204

0 Summary

This document describes the Apple M5 GPU (the G17) at the level beneath Metal: its instruction bytes, what they do, and how work reaches the hardware. From that description the project built a compiler that emits native G17 code, and two ways to run it: through Metal, and below Metal, where the project's own runtime submits work through Apple's private IOGPU interfaces and Metal is never loaded ([figure 0.1](#)).

0.1 Measured hardware and software

Every measurement here was made on one machine: an Apple M5 Pro (the H17s GPU, `applegpu_g17s`) with 20 GPU cores and 48 GB of memory. The stated platform is macOS 26.6.2 (build 25G83) with the Xcode 27.0 toolchain (Metal front end 32023.921, against an OS compiler at 32023.886; MM [22](#), MM [25.91](#)). The below-Metal records carry that OS build in their own evidence. Most Metal-path evidence files record no OS build, so for those results the build is the stated platform, not a recorded one.

A result on this part does not establish behaviour on another G17 product, another OS build or another generation ([section 1.1](#)). Where something has been checked across parts or builds, the text says so.

0.2 Recovered behaviour

Apple documents how to program the GPU and its neural accelerators through Metal, and nothing beneath that interface. The project has recovered much of that lower layer and tested it on the hardware.

- In the register file, the usable namespace ends at R125, and an instruction naming any register from R126 to R143 is dropped whole, without an error. Register operands carry a lifetime modifier, which a few short forms fix. A *release* frees the register as it is read, and a later reader must not rely on its contents: controlled probes read zero, but what a released register returns is not established in general. These rules constrain register allocation, loops and composition ([chapter 2](#)).
- Instructions are 2 to 22 bytes in even steps, and the opcode does not fix the length: each instance carries its own. Apple's code uses every even length from 2 to 16, plus 18- and 22-byte forms such as texture write (`op10909`) and texture sample (`op14661`); no 20-byte instruction has been seen. Operands are linear functions of scattered bits, recovered as field maps. The tensor-instruction encoder reproduces 800 of 800 compiler-emitted instructions byte for byte. Of the 280 opcodes the project names, 193 are confirmed by execution ([chapter 3](#)).
- For the tensor unit, Apple's decoder admits ten tensor MMA forms. The mapping from each lane's registers to matrix elements was measured with one-hot probes over 6,651 dispatches. For half inputs, the exact reduction order of one $16 \times 16 \times 16$ multiply-accumulate, including where the accumulator enters, reproduces 5,120 of 5,120 tile elements bit for bit, where a sequential chain, a balanced tree and an exactly rounded sum each match fewer than 2,800; other input types are recorded at single points ([chapter 10](#)).
- Ordering uses no wait instruction. The producer of a late value names one of eight scoreboard slots, and the consumer carries an eight-bit wait mask. If the slot's bit is left out, the consumer reads the

register before the value lands (zeros, in the measured tensor case) and nothing reports an error ([chapter 9](#)).

- Conversions, fused operations and special-function instructions have been measured, some of them input by input. The same chapters give the address law, the load and store forms, threadgroup memory, the imageblock, constants and atomics ([chapters 4 to 9](#)).
- In the kernel image, every byte across 421 delivered archives has a known source. Apple’s loader requires 148 constants, and the meaning of 12 of their bytes is still open. Metal checks the container’s format and does not inspect the code in it ([chapter 11](#)).
- In the submission machinery, the launch words, the Submit record, the kernel entry and the completion path have been recovered well enough to run complete computations without Metal, within the measured resource graphs ([chapter 13](#)).

0.3 Project components and execution paths

The project authors the IR, the compiler (instruction selection, register allocation, encoding and tensor lowering), the native instruction bytes, the object, metadata and container around them, an executor for each path, and the verification tools. [Figure 0.1](#) shows both paths and names the Apple components each one meets.

- Through Metal ([section 17.1](#)), the project hands Metal an image. Metal builds the pipeline from its archived code and submits the work.
- Below Metal ([section 17.2](#)), the native runtime takes the program bytes directly. It uploads them, writes each launch’s resource and launch state, builds the Submit records, and calls IOGPU itself. No Metal pipeline, command buffer or device exists in that process.
- Below both paths, the same Apple components remain: IOGPU, the AGX kernel driver and the firmware.

Build and validation use one more Apple component. The compiler’s encoders, its whole-program checks and the CPU emulator decode programs with Apple’s G17 decoder, loaded from `GPUCompiler.framework` ([chapter 3](#)). That decoder is not used when the GPU runs a program, but it is the reason compiling and validating currently require macOS. Metal source also passes through Apple’s `xcrun metal` front end before it reaches the project’s IR ([section 15.1](#)).

0.4 The running example

The running example computes a $17 \times 19 \times 16$ matrix product, half-precision inputs and fp32 output, and then adds 1.0 to one output element. Later chapters come back to it at each layer and check it there. The bytes are the project compiler’s output. Recompile on the CPU, the program is byte-identical to the one that ran on the GPU below Metal: 1,232 bytes, SHA-256 8bb67a3c65919cb7fa9a6f2e70d152cb2d8086fba0cade7437f18d1d8114c667, register count 76.

The source is seven lines of the project’s IR: three buffers, `tensor_matmul(A, B, C, M=17, N=19, K=16)`, a load, an fp32 add and a store ([section 15.2](#)).

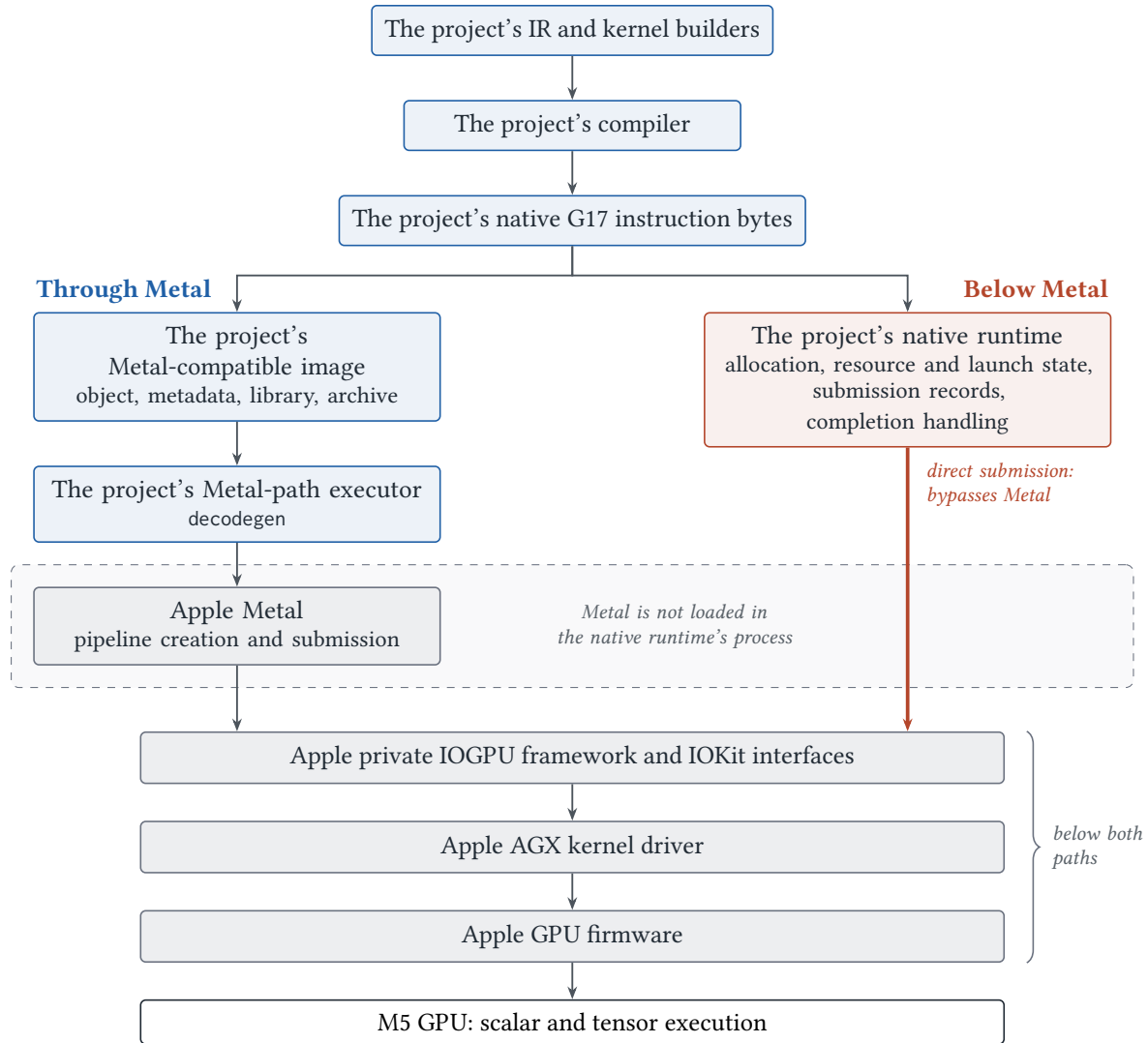


Figure 0.1. The two execution paths. Both execute the project's own native GPU code. The native runtime bypasses Metal's pipeline creation and submission, builds the measured resource and launch state itself, and submits through Apple's private IOGPU interfaces. Apple's kernel driver and firmware remain in use. Blue: the project's components; red: the project's native runtime; grey: Apple's.

The 17×19 output does not divide into 16×16 tiles, so the product is two tile rows by two tile columns, and the second row and column are partial. The compiler emits 101 instructions, grouped by phase in [table 0.1](#).

Table 0.1. Instructions of the $17 \times 19 \times 16$ example program by phase, as read with Apple’s decoder, with what each group does.

Phase	Instructions	What they do
addresses	1 special-register read, 16-bit (op14060), of the lane index, then shifts, ANDs and adds	compute each lane’s row, column and byte offset (section 10.4)
masks	20 range mask, immediate width (op612) and 16-bit AND (op437)	enable only the 17 rows and 19 columns that exist (section 7.4)
loads	4 tensor load, half/bf16 (op12674) and 4 masked tensor load, half/bf16 (op12675)	gather two A and two B fragments, eight bytes per lane each
multiply	4 tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107)	one per output tile, since $K = 16$ is one slice
stores	2 tensor store (op17257) and 6 masked tensor store (op17258)	write the four tiles, masked where partial
epilogue	special-register read (op14059), plain 32-bit load (op12682), move immediate, three-operand form (op11842), fp32 add (op998), plain 32-bit store (op17229), end of program (op684)	add 1.0 to one element

The first tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107), is ten bytes, a70405112204a4620202. Apple’s decoder reads it as follows:

```
op5107 R24..R31 <- 0x01000000, 1, R20_R21_R22_R23, 0, 2, R68_R69_R70_R71, 0, 2
      D      flags word      A fragment      lifetime, type      B fragment      lifetime, type
```

- **D, R24–R31**, is an eight-register fp32 accumulator group: the output tile at application rows 0–15, columns 0–15. In the instruction’s canonical coordinates lane 5 holds elements (4, 4)–(4, 7) in its first four registers and (5, 4)–(5, 7) in the next four. The compiler packs every tile with the B map, so its store writes the same registers to application rows 2 and 10, columns 4–7. The two labelings differ by a one-bit rotation of the row index ($\text{rot11}(2) = 4$, $\text{rot11}(10) = 5$); A is packed the same way, so the rotation only relabels the free M index and the product is correct in application coordinates ([section 10.5](#)).
- **A, R20–R23, and B, R68–R71**, are four-register fragments of 16-bit values. Type code 2 means half.
- **The flags word 0x01000000** has bit 24 set: wait for scoreboard slot 0. All eight tensor loads write slot 0. Each load’s own control word, 0x01100000, both publishes slot 0 and waits on it ([sections 3.4](#) and [9.1](#)). On an MMA whose loads fill slot 7, clearing the matching wait bit made the multiply read its fragments before the loads landed, and it produced zeros (measured, MM 6); that control was run on slot 7, not on this program.
- Each fragment is read by two of the four MMAs. Its first reader leaves it live (modifier 0) and its second, the last use, releases it (16); nothing reads the register after that ([section 2.5](#)).
- **The 1 after the flags word** is a constant: no encoding bit varies it ([section 3.3](#)).
- Each output element sums 16 products in a fixed tree: adjacent pairs, then pairs eight apart, rounding to nearest-even fp32 at every step (the rule fitted on half inputs, which this program uses). This from-zero form starts from the first of the four partial sums, where the accumulating form adds C first ([section 10.6](#)).

Read from the bytes, the epilogue is ordered by these waits:

1. The special-register read (op14059) of the threadgroup position publishes slot 0.
2. The plain 32-bit load (op12682) waits on slot 0 for its index register and publishes slot 7.
3. The fp32 add (op998) waits on slot 7 for the loaded value.
4. The store needs no wait: its value comes from an ALU instruction, and ALU results are interlocked ([section 4.7](#)).

Alongside the code, the compiler reports what an executor needs: the three bindings with their element types, which binding is written, the 24 distinct (opcode, length) forms used, an empty constant pool, the register count and the system registers read. The image author turns that into the object, metadata, library and archive Metal loads ([chapter 11](#)).

The program was then run on the GPU below Metal (measured; docs/g17-first-dispatch-trials.md, “A second authored tensor program below Metal”). The native runtime wrote the program at allocation 0, offset 0x6c0. Relative to the first tensor program it had run, the measured resource graph needed only these field changes:

- two moved view offsets;
- one C pointer;
- the kernel word at offset 0x3f8, from 0xa to 0xc;
- two binding pointers;
- A, B and C windows of 544, 608 and 1,292 bytes, which are exactly 17×16 halves, 16×19 halves and 17×19 floats.

Three fresh processes each submitted the program through IOGPU with no Metal loaded. In each, 323 of 323 fp32 outputs were bit-exact, inputs and guards were intact, both completion callbacks arrived and no recovery event occurred. The outputs hash to ec43027260f5a0b8dd59526c17102ffe00066852f10af42a91103c36e778faf6, the same as an independently computed reference and as the earlier run through Metal ([chapter 13](#)). That is one program at one shape; how far each layer generalises is stated in the chapters cited above.

0.5 Applications

- InternLM2.5-1.8B decodes on the compiler’s kernels, run through Metal. Single stream at a 1,024-token context, it runs at 1.14× the decode rate of mlx-lm’s generate_step. Several other models and workloads are measured, each with its own baseline and limits ([the showcase](#)).
- The compiler was also run on code it was not written for. Of 125 compiled sources from a frozen 197-source census, 101 have run and been checked on the hardware, and none crashes the driver (MM 25.149.1).
- A CPU emulator, working from the program bytes, executes one position of a shipped batched-decode graph and of single-stream decode, dispatch by dispatch: 316 of 316 and 172 of 172 dispatches byte-identical to the GPU, and a 16 MB prefill slice identical (MM 25.197). It is an internal verification tool, and it uses Apple’s decoder.
- MiniLM-L6 sentence retrieval and Qwen2.5-0.5B-Instruct generation ran through the native submission path, each with independently checked outputs (MM 25.210; [section 17.2](#)).

0.6 Limits of the evidence

- Nothing here is claimed for other parts or builds, and the OS build is verified per record only where a record carries it ([sections 0.1](#) and [1.1](#)).
- The InternLM2 decode lead over mlx-lm ([section 0.5](#)) is due to the kernels' design. In a matched study, Apple's compiler, given the same kernels, ran them 10–15 percent faster with identical tokens, so the lead does not show that native instruction control is needed. The tensor-unit kernels were not part of that study (MM [25.211](#)).
- Below Metal, complete computations run with checked results, but not faster than the compared Apple paths (MM [25.210.5](#)).
- [Section 17.2](#) keeps four kinds of evidence apart: stage checks against actual inputs, comparisons with the original checkpoint, exact differentials, and one auxiliary control, which fails.
- Validation uses Apple's decoder. An independent decoder for the compiler's own programs was started and then paused, so nothing here establishes portability (MM [25.209](#)).
- Each chapter states what is not known next to the rule it qualifies. [Chapter 14](#) indexes them.

0.7 Conventions

Each claim is marked with one of five kinds of statement:

- *measured*: run on this GPU and compared with an independent reference, with a control that could have failed;
- *Apple's compiler emits*: read from Apple's compiled code; it shows a pattern works, not that the hardware requires it;
- *decoder*: what Apple's decoder prints or admits, which is not evidence of execution;
- *this compiler*: a choice the project's compiler makes, not a hardware rule;
- *open*: not established either way.

Opcodes are named first and numbered second, as in “tensor load, half/bf16 (op12674)”. The numbers are Apple's opcode ids. The names are this project's, chosen by measured function, and are not Apple's spellings ([appendix C](#)). Registers are written by hardware number (R0–R125), with their role stated where it matters.

[Table 0.2](#) lists where to start for each topic.

Table 0.2. Where to start reading, by topic.

To understand...	Read
how an instruction's bytes become operands register lifetimes, and what a released register returns	chapter 3 , then section 9.1 for the wait fields sections 2.4 to 2.6
the tensor unit, from the math to the bytes	chapter 10 , starting at section 10.4
why a kernel is slow	chapter 12
what orders what in memory	chapter 9 , with chapter 7 for the forms
the Metal bypass	section 0.3 , then chapter 13 , then section 17.2
how the compiler uses all this	Part II
a specific opcode	appendix C , or appendix B for scalar semantics

Part I

The machine

1 Device and execution model

All measurements were made on one Apple M5 Pro GPU (G17, the 20-core H17s) under the software listed in [section 1.1](#). Later chapters assume the execution model set out here. Mechanisms are in [chapters 2 to 11](#) and measured costs in [chapter 12](#).

These terms are used throughout the document:

- *Lane*: one thread’s slot in a simdgroup. *Simdgroup*: 32 lanes that issue one instruction stream together under an execution mask ([chapter 5](#)). *Threadgroup*: one or more simdgroups that share threadgroup memory and can meet at a threadgroup barrier ([section 9.2](#)). *Grid*: all threadgroups of one dispatch.
- *Core*: one of the 20 GPU cores. Per-core figures divide a GPU-wide count by 20.
- *Resident*: a threadgroup (or simdgroup) is resident while it holds a core’s resources, from launch until it ends. *Residency* or *occupancy*: how many are resident at once.
- *Registers used*: the highest 32-bit register a program names, plus one. *Declared register count*: the value in per-kernel metadata slot 0 ([section 11.1](#)).
- *Warm* and *cold* refer to the GPU’s performance state ([section 1.6](#)).
- “MM x” cites the project’s machine model, docs/g17-tensorops-machine-model.md.

1.1 Platform versions

The measurements were taken on one machine and one toolchain, on the platform in [table 1.1](#) (MM 22, MM 25.66, MM 25.91). The below-Metal evidence ([chapter 13](#)) records the OS build, 25G83; the Metal-path evidence files do not, so for those measurements the build is the stated platform rather than a per-file record. On 2026-10-02 the machine was updated to macOS 27.0.1 (build 26A434). Nothing in this reference was re-measured on that build; tools/g17platform.py --check reports the change, and the table gives the platform the figures were taken on. The update renumbered Apple’s decoder; the tools translate back to the numbering used here ([section 3.7](#)).

Table 1.1. The measured platform: machine, OS, driver and toolchain versions, with how each was read.

Component	Value	How read
Machine	Mac17, 8, Apple M5 Pro	<code>sysctl hw.model</code>
GPU	“Apple M5 Pro”, 20 cores, Metal 4; Apple’s internal names H17s, <code>applegpu_g17s</code>	<code>system_profiler</code> ; the core count is the campaign’s own record (below)
macOS	26.6.2, build 25G83; Darwin 25.6.0	<code>sw_vers</code> , <code>uname</code>
GPU user driver	AGXMetalG17X.bundle 353.14	MM 25.141.20
Metal compiler	metal 32023.921 (metalfe-32023.921.6), target <code>air64-apple-darwin25.6.0</code> , against an OS-side GPU compiler at 32023.886	<code>xcrun metal --version</code> ; MM 22
Xcode / SDK	Xcode 27.0 (27A266a), macOS SDK 27.0	<code>xcodebuild -version</code>
Host compiler	Apple clang 21.0.0 (clang-2100.3.34.2)	<code>clang --version</code>
Apple’s AGX3 decoder	<code>inside GPUCompiler.framework/Versions/32023/Libraries/libLLVM.dylib</code>	section 3.6

`sysctl -n hw.ncpu` returns 18, the CPU core count, which the project has mistaken for the GPU core count before. The GPU core count is not readable through `sysctl` and is carried as the campaign’s own record; a platform pin that confused the two would put a wrong divisor under every per-core figure (MM [25.66](#)). `tools/g17platform.py --check` compares the running machine with this record and names each field that differs, with what it bears on: the compiler version on lowering, scheduling and every “Apple’s compiler emits” statement; the OS build on driver behaviour and loader variants; the part on everything (MM [25.66](#)).

Results on this part do not establish results on any other part, OS build or compiler: another G17 product (a different core count, a different populated register file, other loader variants) is a different population, and no row of the project’s ledger claims otherwise (MM [0.1](#), MM [25.66](#) row T9, MM [25.119](#)). MM [25.119](#) groups results by how likely they are to change on another part, OS build or compiler:

- *Properties of the toolchain*, which follow Apple’s compiler rather than the silicon and are the likeliest to move: which source forms reach the tensor unit, metadata layout classes, the constant-program laws, which opcodes Apple’s compiler selects, spill counts (MM [25.119](#)).
- *Properties of the core count and clock*: residency counts, speeds, power (MM [25.119](#)).
- *Encodings and arithmetic*, the least likely to move within this generation: field maps, the register namespace, the tensor unit’s reduction arithmetic, which “has held across every G17 measurement in this repository” (MM [25.119](#)). Several conventions are conserved from Apple’s earlier G13 generation and its public reconstruction in Mesa: the length-bit cluster in the encoding ([section 3.1](#)) and the command-stream layout below Metal, which decodes field for field against Mesa’s G13 description ([chapter 13](#)). Mesa’s model was used only to suggest hypotheses and is not evidence for G17.
- Apple’s decoder admits the same ten tensor MMA opcodes when told the CPU is g17s, g17g or g18, and decodes the same bytes as unrelated instructions for g17, g16 and g15 targets (MM [2](#)). This is a property of Apple’s decoder tables and is no evidence about other silicon.

The portable parts of the project’s own work (byte-identity pins, static checks, the metadata determinant over the committed corpus) rerun anywhere on the CPU; the platform facts each carry a named re-measurement instrument (MM [25.119](#)).

1.2 Threads, simdgroups, threadgroups and grids

- SIMD width is 32 (measured; `threadExecutionWidth` is the driver constant 32 for every pipeline, MM 25.141.20, MM 25.147). A tensor fragment is a per-lane register tuple across those 32 lanes (chapter 10).
- A threadgroup holds at most 1,024 threads. `maxTotalThreadsPerThreadgroup` is a field of the compiler's reply that the driver copies; 0 or absent becomes 1,024 (driver, read statically, MM 25.141.20). It read 1,024 for every pipeline tested, from 4 to 512 live scalars per thread and 64 to 32,768 bytes of threadgroup memory (measured, MM 25.47), so it does not vary with register use.
- Registers are per lane and never cross simdgroups. Every hand-off between simdgroups goes through memory and a barrier (section 9.2); cross-lane movement inside a simdgroup is chapter 8.
- Threadgroup memory is limited to 32,768 bytes per threadgroup, a hard limit enforced by Apple's GPU compiler on this toolchain (48, 60 and 64 KiB refused; 96 KiB drops the compiler connection; MM 7, MM 0.1). Metal's pipeline creation does not check the declared size: a container declaring 1 MiB builds and reports 1,048,576 back (MM 25.147; section 11.4).
- Position registers (lane index, simdgroup index, threadgroup and grid positions) are special registers read by instruction; chapter 6 is the table.
- Grids are dispatched in threadgroup order, and the driver prepares fast-division constants to split a linear threadgroup index into three dimensions (MM 25.141.20). The ordering consequence is section 1.5.

1.3 Core resource model

The model in table 1.2 is a *logical* account of measured concurrency: which streams run side by side without slowing each other, and which contend. It is not a floorplan and does not locate or count physical units (MM 0.2).

Table 1.2. Logical resources of one core and their measured concurrency behaviour.

Resource	Measured behaviour	Source
Four SIMD issue slots per core	One, two or four simdgroups each running a dependent tensor MMA chain take 32.8, 32.4 and 32.8 μ s; eight take 57.5, sixteen 112.6, thirty-two 224.3: flat to four, linear beyond	measured, MM 8
Tensor unit, one slot per SIMD	$16 \times 16 \times 16$ per issue, 1,024 flop per cycle per core at saturation; about 5.5 simdgroups per core saturate it	measured, MM 8; chapter 10
fp32 / integer ALU	about 4.1×10^{12} fp32 adds per second machine-wide, consistent with 128 fp32 lanes per core at about 1.6 GHz; no dual issue between fp32 and int32 (ffma + iadd mixed 3.61 ns per op against 3.64 predicted for one shared port and 2.09 for dual issue)	measured, MM 0.15
Legacy simdgroup-matrix engine	overlaps the tensor unit completely at 16 and 32 simdgroups per core; its fp32-accumulate forms contend with fp32 FMA (overlap 0.37 or less), its half-accumulate form mostly does not (0.66–0.95)	measured, MM 9; section 10.13
Per-simdgroup scoreboard	eight slots ordering late values (loads, special-register reads, atomics) against their consumers	measured, MM 6; section 9.1
Load/store path	per-lane addresses; no hidden tile-layout unit	measured, MM 23.2; chapter 7

“Contentends” and “overlaps” in this table are exchange rates measured at stated occupancies; what physically is shared was not identified (“about 8 fma of budget per legacy issue” is a rate, not a mechanism; MM 9). Partial overlap at one or two simdgroups per core is the in-order issue of a single simdgroup, not a shared unit (MM 9). Performance figures are in [chapter 12](#).

1.4 Residency and occupancy rules

These are the rules a compiler or runtime may rely on; the measured residency counts, the per-core staircase and what registers and spills cost are in [sections 12.4](#) and [12.5](#).

1. Residency follows the registers a program uses, not the count it declares (measured, MM 25.115). The same bytes declaring 19 and declaring 125 reach the same peak every time (1,814–1,896 resident at 2,048 launched simdgroups); only using more registers lowers the peak. The declared count (metadata slot 0) is inert for residency between 19 and 125, and inert for execution: code using R0–R123 runs correctly declaring 124, 127, 128, 160 or 224 (MM 10). Below Metal the count travels to the hardware in a launch-state packet ([section 11.1](#), [chapter 13](#)); whether that packet’s value affects residency was not isolated.
2. The driver caps threadgroups per core at $\min(3072 / \text{threads_per_threadgroup}, 96)$ (static read of the driver, MM 25.141.20): 3,072 threads per core, 96 simdgroups of 32. Nothing in that path reads the register count; any register-dependent limit sits below the driver or in the compiler’s reply. The measured peak at 19 registers, 95.0 simdgroups per core (MM 25.121), sits at this cap.
3. Register pressure lowers residency, by a kernel-dependent amount. Two probes disagree by about a factor of two at high register counts (about 64 against about 32 simdgroups per core near 125

registers; the counts and their kernels are [section 12.5](#)), and which kernel property the difference follows is open (MM [25.121](#)). A scheduler must use the count measured for its own kernel family.

4. The per-core threadgroup-memory budget is 57,344 bytes (56 KiB) (measured, MM [25.49](#)): two threadgroups of 28,672 bytes fit on a core, one of 28,688 does, and the 16-byte step costs 25 percent. With the 32 KiB per-threadgroup limit, at most one threadgroup above 28 KiB is resident per core.
5. For 1,024-thread threadgroups, no residency count may be assumed. The forward-progress probe ran 200 threadgroups of 1,024 threads, one producer last, with no consumer starving (17–21 μ s); had fewer than 200 been co-resident, the earliest consumers would have spun to the 200,000-poll cap before the last-dispatched producer ran (MM [25.141.1](#)). That implies at least 10 per core. The driver formula gives 3 per core (60 GPU-wide) (MM [25.141.20](#)). The two numbers have not been reconciled, and the 800-threadgroup row (480 of 799 at the cap) does not give a simultaneous count, because waves can reach the cap one after another. Do not size a spin-waiting grid of 1,024-thread groups from either number ([section 1.5](#)).

Not known. The physical register capacity per core and its allocation granularity. Residency is not one fixed pool divided by per-simdgroup demand (a fixed-pool model under-predicts the measured count about fourfold at 126 registers; [section 12.5](#), MM [24.2](#) row H4, MM [25.49](#)), and whether that means a larger file than any published part or registers that are not all physically resident is not separable with these instruments (MM [25.20](#)). Metal’s public API has no occupancy counter on this part (MM [25.48](#); [section 12.7](#)).

1.5 Forward progress

The probe is one dispatch (tools/g17gpuprobes/fp.m): consumers spin, capped at 200,000 polls, until an atomic count reaches the number of producers placed at the end of the grid (MM [25.141.1](#)). [Table 1.3](#) gives its results.

Table 1.3. Forward-progress probe results by grid shape, with consumers capped at 200,000 polls (MM [25.141.1](#)).

Grid (threadgroups $\times$ threads)	Producers	Consumers at the cap
64 to 1,200 $\times$ 32	1, the last	0 (none waited more than 94 polls)
2,000 $\times$ 32	1	1,688 of 1,999
20,000 $\times$ 32	1	18,350 of 19,999
20 to 200 $\times$ 1,024	1	0
800 $\times$ 1,024	1	480 of 799

These are observations of one probe family in the tested configurations, not a scheduling guarantee. Below, each is used only to restrict what code may assume.

- Among resident threadgroups, no starvation was observed. Up to 1,200 single-simdgroup threadgroups and up to 200 of 1,024 threads, every consumer saw its producer within the cap (at most 94 polls in the 32-thread rows).
- Past residency, waiters starved. With 2,000 threadgroups the first resident wave (about 1,690, about 84 per core at that register count) spun to its cap before the producer, dispatched last, ran, and the threadgroups that succeeded were the high-numbered ones. That fits in-order dispatch of threadgroups

with no preemption of a spinning one in these runs (MM 25.141.1); no run tested whether preemption can occur under other conditions, so code must not rely on preemption to make progress.

- In the persistent-kernel runs (16 phases, totals exact), grid barriers among resident threadgroups all completed without reaching the cap, and a barrier cost more than a dispatch boundary at every measured shape (MM 25.141.1; costs in [chapter 12](#)). A grid that waits on itself has to be sized to resident capacity at its own register and threadgroup-memory use, measured for that kernel ([section 1.4](#)).

The following may not be assumed:

- that a threadgroup waiting on another threadgroup will see it run, unless both are resident; a spin-wait on a producer that is not resident may wait indefinitely. In the probes, polls were capped, and waiters beyond residency spun to the cap without seeing their producer run (MM 25.141.1; MM 17 rule 40, MM 0.15);
- that “in order” means anything finer than dispatch order of threadgroups: completion order, and ordering of memory effects between threadgroups, are separate questions answered in [section 9.4](#) (visibility needs a device fence on both sides; ordering comes from an atomic);
- that a timeout protects the machine. Killing the host process does not cancel a kernel: an unbounded loop kept the GPU at 100 percent with nothing dispatching until a reboot (MM 25.116), and an unbounded data-driven trip count once rebooted the machine (MM 25.90). Loops in this project’s compiler are capped so that termination is provable ([section 2.6](#), [chapter 5](#)).

Within those configurations, the measurements support a persistent kernel that sizes its grid to resident capacity at its own register and threadgroup-memory use and pulls work from an atomic queue; any other hand-off between threadgroups goes through a dispatch boundary (MM 17 rule 40).

1.6 Clock states and timing

Timing on this GPU follows the rules below; the measurements are in [section 12.8](#).

- The top clock is about 1.62 GHz: 1,620 MHz read from `powermetrics` at 100 percent residency in the top bin; 1.63 GHz is implied by the saturated tensor issue rate of 32 cycles per MMA (measured, MM 8, MM 20).
- A run shorter than about 1.5 ms runs at whatever frequency the GPU is idling at, and a longer one speeds up partway (MM 25.121); a cold GPU ran the same kernel about 4 to 4.8 times slower than after sustained load (MM 22, MM 25.115). Timings need a saturating warm-up in the same process (the project uses 4,096 threadgroups of 256) and a reference dispatch to detect a clock change.
- The governor reacts to command-buffer shape. Metal System Trace records the GPU performance state (Minimum, Medium, Maximum) per interval; a workload issuing seven short command buffers per step spent 27–78 percent of its time in Medium, while one long command buffer per step stayed at Maximum (two recordings, direction reproduced, size not; MM 0.15).
- In-run variation was 0.4–1.3 percent, but the same configuration in two processes differed by 9–13 percent; compare arms interleaved in one process (MM 16, MM 25.126; further figures at the start of [chapter 12](#)).
- Several results published earlier in the project were artefacts of the idle clock (a residency “shortfall”, a “bimodal” issue rate); each is corrected where it is used (MM 25.115, MM 25.122).

Evidence. MM 0.1, MM 0.2, MM 0.15, MM 2, MM 7, MM 8, MM 9, MM 10, MM 16, MM 17 (rule 40), 20, 22, 24.2 (row H4), 25.20, 25.47, 25.48, 25.49, 25.66, 25.90, 25.91, 25.115, 25.116, 25.119, 25.121, 25.122, 25.126, 25.141.1, 25.141.20, 25.147.

2 The register file

Each lane of a simdgroup owns a private file of 32-bit general registers, addressed by number in the instruction and also viewed as 16-bit halves and as 2-, 4- and 8-register tuples. Beside it sits a small uniform file, written once per dispatch. There is no separate accumulator file for the tensor unit, no register renaming visible to software, and no interlock that protects a value a program has said it no longer needs. Register sources carry a *modifier* (a few short forms fix it) that can negate it, take its absolute value, or *release* it, and a release is destructive.

Register allocation on this GPU, and the composition of kernels, must respect three facts:

1. Fields can name R0–R143, but the hardware executes only R0–R125. An instruction naming a higher register does not fault; it is discarded whole, with no error ([section 2.1](#)).
2. Fields can be narrower than the namespace: certain forms can name only R0–R15, and others cannot name particular registers at all ([sections 2.3](#) and [2.7](#)). Register choice has to satisfy each form’s constraints, which a graph colouring over one homogeneous register file does not capture.
3. Lifetime is encoded in the instruction and applies per lane, under the execution mask. The last reader of a value marks it released, the release destroys the lane’s copy (controlled probes then read zero, [section 2.5](#)), and no hardware mechanism notices a later read ([sections 2.4](#) to [2.6](#)). Liveness is a correctness property of the emitted bytes and must be computed per lane over the control-flow graph, including back edges.

When kernels are composed, a value that stays live across a region boundary (a loop header, a fused stage, a second GEMM) must be kept live in the bytes, inside the 126-register pool, in registers every consuming form can name.

Register notation:

- *Rn*: general register n, 32 bits per lane. *RnL*, *RnH*: its low and high 16-bit halves. *Ra...Rb*: a tuple (for example R20_R21_R22_R23, a 16-bit tensor fragment).
- *MRegister id*: the number Apple’s decoder uses for a register; the general block starts at 105, so Rn has id 105 + n (verified over 300 registers in 60 shaders; docs/agx3-oracle.md section 6). The project’s code writes “r119” for id 119, which is R14; this chapter always writes hardware numbers.
- *Slot domain*: an operand field value counting halves, $2 \times n + h$ ($h = 0$ low, 1 high) (decoder, MM 25.139.1).
- *Modifier*: the immediate operand that follows a register source ([section 2.4](#)). *Release* (weight 16), *keep* (weight 32).
- *Late value*: a result that arrives after its producer issues and needs a scoreboard wait ([section 9.1](#)).
- *Registers used*: the highest 32-bit register named in the main program, plus one; it equals the declared count this compiler writes into metadata slot 0 ([section 11.1](#)).
- The running example ([section 0.4](#)) is used in [section 2.5](#).

[Table 2.1](#) lists the chapter’s main statements and their standing.

Table 2.1. Main statements of the chapter and their standing.

Statement	Standing
R0–R125 execute; naming R126–R143 squashes the whole instruction	measured (MM 10, MM 25.18)
Release destroys the lane’s value, per lane, gated by the execution mask	measured (MM 6, MM 25.95)
A released register’s later read returns 0	observed in every controlled run; not established as a rule (section 2.5)
Loop-carried values must not be released before the back edge	measured, with a machine-hanging failure (MM 25.116)
$8N + K \leq 115$ and $8N + 4M \leq 108$	measured frontiers of the kernels tested (MM 11)
Field widths and holes	decoder (Apple’s decoder reading back the encoder’s bytes)

2.1 Register namespace

Allocate only R0–R125: 126 registers per lane, fifteen eight-register groups at bases R0, R8, ..., R112 (measured, MM 10, MM 0.7). Any instruction that names R126, R127 or R128–R143 in *any* register operand is squashed as a whole: no register write, no memory write, no atomic, no tensor-unit time, and no error from the compiler, the driver or the API (measured, MM 17 rule 1). Nothing faults, and a stray high operand shows up only as wrong output, so the allocator has to enforce the bound when it assigns registers.

The boundary first showed up as a store of a tuple reaching R126 that failed to round-trip, independent of the declared count. A follow-up wrote through R126 and R127 and read zeros from the output buffer, and was recorded as R126 and R127 *reading as zero and dropping writes*, like a hardwired zero register. The rival reading is that the hardware discards any instruction that names them. The two predict different results for an instruction that reads R126 and stores it: the first stores 0, the second stores nothing. That experiment could not separate them, because the output buffer had been zero-filled before the dispatch and “stored 0” and “stored nothing” leave the same bytes. With the buffer filled with a sentinel instead, the sentinel survived, so the store had not executed (MM 19, MM 25.20). The same test was then repeated in every operand role: ALU sources and destinations, store sources of three widths, a store’s index register, a load’s destination group, an atomic’s destination, and the tensor MMA’s accumulator, A and B operands (MM 10). In each the whole instruction is discarded, not only the register access. For R128–R143 the result is unchanged by declared register counts from 12 to 255, by one against 32 simdgroups, and by a 32-against a 1,024-thread pipeline limit (MM 25.18). The wide RMSNorm’s first check (section 2.5), which passed against an output that carried no sentinel, had the same flaw.

agxforge/g17/registerdomain.py encodes the rule: `ALLOCATABLE_MAX = 125`, `REGISTER_COUNT = 126`, `MAX_FP32_ACCUMULATOR_GROUPS = 15`, group starts on a multiple of 8 (this compiler). The boundary is measured on one part; whether another G17 implementation backs the higher names with storage is not tested (MM 25.18).

Beyond R125, the encoding and the decoder distinguish four cases.

- *R126 and R127* are the top pair of a 128-register architectural file that this part populates only to R125. Three routes agree on 128: the half-register move’s seven-bit source field plus a one-bit file selector (128 halves $\times$ 2 files = 128 registers, measured in this repository); the five-bit accumulator selector’s eighteen groups, of which exactly fifteen lie inside R0–R125, one straddles (R120–R127) and two lie wholly beyond; and, as a hypothesis source only, Mesa’s G13 model of 256 sixteen-bit slots. Sentinel stores rule out a hardwired zero and unique encodings rule out aliases, and no metadata or launch control enables the two registers (MM 24.2 row H1, MM 25.20). Whether anything *reserves* the top pair, rather than the part simply populating 126 of 128, is open; only documentation could settle it.
- *R128–R143* are decoder-visible names outside that file, reachable only through fields wide enough to express them (accumulator-selector groups 16 and 17, nine-bit slot fields). Load and store register fields reach R127 only, so for those forms R126/R127 are expressible and R128–R143 are not (MM 24.2 row H1, MM 25.38). Prior art’s larger uniform file is selected by operand type, not by a high register number, so it does not explain them (MM 25.18).
- *The encoding* admits R0–R143 and rejects 144–255 (MM 24.2 row H2).
- *Accumulator selector values 18–31* decode to no operand at all: null targets with no name, absent from every compiled witness, never dispatched because malformed tensor encodings have hung the GPU (decoder, MM 25.19). An assembler must reject them.

Table 2.2 gives the highest legal base register for each value class (measured, MM 0.7, MM 23.2).

Table 2.2. Highest legal base register per value class.

Value	Registers	Alignment	Highest base
scalar 32-bit	1	1	R125
int8 / uint8 tensor fragment	2	2	R124
half / bfloat tensor fragment	4	4	R120
fp32 tensor fragment, fp32 / int32 accumulator	8	8	R112

2.2 Register classes

Apple’s decoder tables survive with register and register-class names (3,567 registers, 500 classes); instruction names do not (decoder, docs/agx3-oracle.md section 6). Table 2.3 lists the classes used in this document, with member counts read from those tables (tools/agx3meta classes, members).

Table 2.3. Register classes in Apple’s decoder tables, with width and member count.

Class	Width	Members	What it is
GPR32	32	144	R0–R143
GPR16	16	288	R0L, R0H, ..., R143H
GPR32tup2 / tup4	64 / 128	143 / 141	2- and 4-register tuples
GPR32tup8_alignedrc	256	18	the eighteen 8-aligned groups R0–R7 ... R136–R143
IR32	32	16	IR0–IR15, MCRegister ids 89–104
IRGPR32	32	160	R0–R143 followed by IR0–IR15
FLAGR / FLAGwritable	16	17 / 15	FLAG0–FLAG14 plus the constant sources FLAGTRUE, FLAGFALSE (chapter 5)

A 16-bit operand names a half of a general register; the slot domain numbers halves $2n + h$, confirmed through Apple’s decoder (MM [25.139.1](#)). Writes and releases act per half ([section 2.5](#)): a 16-bit write leaves the other half’s contents and history untouched. Measured for special-register read, 16-bit (op14060): with R holding $7t + 0xABCD0000$, writing its low half with the lane index left $0xABCD0000 | \text{lane}$, three runs identical; it does not zero-extend (MM [25.145.4](#)).

Register fields are encoded in units of the tuple width, so a tuple base must be naturally aligned (2, 4 or 8); an unaligned request cannot be expressed and must be refused, not rounded (MM [0.4](#), MM [2](#); the encoder bug that rounded silently is [section 3.8](#)). The tensor unit’s operands use three classes found nowhere else in the table: class 335 (8-register tuple, the destination of every 32-bit-accumulator form), class 159 (4-register tuple) and class 41 (2-register tuple); there is no fourth, hence no sub-byte fragment class (decoder, MM [25.6](#)). Eight-bit slot fields come in two addressing conventions (half-indexed: the value 128 names R64, on 3,970 operands; register-indexed: 128 names R128, on 638), and one scalar form carries one of each, so field width alone does not say which registers a field reaches (decoder, MM [25.38](#)).

The IR file proper is the 16-member class IR32. IRGPR32 is a class that *admits* the IR registers alongside the general ones (R0–R143 followed by IR0–IR15, 160 members); its operands decode as ordinary registers, value 105 = R0, and all 229 corpus writes by its users (32-bit saturating add (op10239), multiply by immediate, 16-bit addend (op10824), multiply by immediate minus register (op11060), bit interleave, Morton encode (op14147)) land in Rn that ordinary instructions read (MM [25.59](#)). The corpus uses only its general members. What the 16 IR registers hold, and whether any executed program names one, is not recorded (open).

The uniform file is a per-dispatch file of 32-bit uniforms $u_0, u_1, \dots$, written by the *constant program* (a second program Apple’s compiler emits, run once per dispatch before main; [section 11.2](#)) and read by main-program sources. Apple’s decoder writes a uniform operand as $\text{bin}(\text{op}_0, \text{const}(K), S)$, where op_0 is the uniform register file, decoder tag (op_0), and K counts 16-bit halves, so the uniform is $K // 2$; a 64-bit value occupies $u(2k), u(2k+1)$ (decoder, MM [25.120](#)). In tensor (MMA) programs u_0 – u_5 are reserved and constant-program writes start at u_6 ; outside MMA programs some constant programs write below u_6 (Apple’s compiler emits, MM [25.120](#)). A uniform destination must be written with one value across the active lanes (measured by emulation against Apple’s GPU, MM [25.179](#)). Uniforms also serve as operands of ordinary forms: the mask operand of 16-bit AND, 32-bit result (op428) in its pool form is a uniform halfword preloaded from the slot-13 pool (MM [25.141.17](#), MM [25.145](#); [section 7.8](#)).

Uniform / staging write (op592) stages operands for texture messages in the message staging file, decoder tag (op4); the writer names a scoreboard slot and the message instruction waits on it (decoder over 189,898 staging writes, MM 25.120.1; [section 7.9](#), [section 3.4](#)).

FLAG0–FLAG14 are an allocatable predicate resource with pressure, not one condition code; compact forms reach FLAG0–FLAG6 only (decoder, isa/g17-special-registers.toml). They are covered in [chapter 5](#).

2.3 Narrow register fields

Each operand field has its own width, and the allocator must compute, per value, the highest register every form touching it can name. The authoring table records field widths per (opcode, length) form, and this compiler derives each value’s ceiling from them (cc._caps), with a hand list only for forms outside the table (this compiler, agxforge/g17/cc.py _needs_narrow). The binding narrow field today is the four-byte special-register read’s four-bit destination, R0–R15 (decoder and this compiler, MM 25.110.1, agxforge/g17/cc.py). Other forms carry field-specific ceilings: the fp16 conversions author destinations to R61 with holes ([section 2.7](#)), and in measured kernels the fma’s first source and a load’s index register were confined to narrow registers by their forms’ fields at the time (MM 25.144.3).

This compiler keeps R4–R15 (twelve registers) as its narrow pool and places a value that is only ever an ALU first source in the wide pool (R16 and up) first, to keep the narrow registers free (this compiler, MM 25.146). For one witness the policy makes this compiler declare 17 registers where Apple’s compiler declares 3; which policy gives better occupancy is unmeasured (MM 25.146). Refusals name the pool (“12 narrow + 110 wide available”); when they occur, the narrow pool, not the file, ran out (MM 25.144.3).

The widths now used were measured after the early scalar field map proved wrong. That map read the ALU destination as five bits and the load, store, special-register-read and move-immediate destinations as four bits in byte 0, which implied a 32-register file (the ALU destination saturated at 31 in kernels needing 32 to 64 live values). Apple’s own shaders contradicted that (39 percent exceed R31; driver shaders reach R112). The ceiling was then measured on the hardware, with one value pinned into one high register and read back, one dispatch per process, with the expected value 107 preregistered every time. R20, R40, R64, R100, R120, R124 and R125 read back correctly, and R126 and above were squashed (measured, agxforge/g17/cc.py comment, spike/accel/re/hireg.py). The fields were re-measured at seven and eight bits (extension bits in byte 2 and byte 5). The hand list also hid one under-measured field: a register-register bitwise form that it held to the narrow registers had a destination field recorded as five bits. The field was six bits, which went unnoticed until something was allocated above R15 (this compiler, agxforge/g17/cc.py).

2.4 Operand modifiers

Every register source is followed by a modifier immediate. Over 12,000 corpus instances of five opcodes it takes exactly the values {0, 2, 16, 18, 20, 32, 34, 36}: bit 1 (2) negate, bit 2 (4) absolute value, bit 4 (16) release, bit 5 (32) keep, composing freely (18 = negate + release, 34 = negate + keep) (Apple’s compiler emits; agxforge/g17/auth.py, isa/g17-source-modifier-operand.toml). A destination’s modifier takes only 0 or 32: a definition is always a keep. Value 0 carries no lifetime information and preserves the register (the running example’s first reads, [section 2.5](#), execute exactly with 0). Floating subtraction is

fp32 add (op998) with a negate modifier on the second source; $a - b$ and $a + (-b)$ compile to the same bytes (Apple’s compiler emits, MM 25.146).

In the ALU and tensor forms the destination’s raw operand (operand 1, which the decoder prints as one wide integer and older notes call the “flags word”) packs several fields: bits 4–5 the destination’s lifetime, bits 20–23 a fill slot, bits 24–31 the scoreboard wait mask (measured, MM 25.145, MM 6; the fields are section 3.4). In the tensor MMA the keep bit is operand 1 weight 32, physically byte 4 bit 3 (auth. carriers(5106, 1)[5] = [(4, 3, 0)]), and it is hardware-inert: a conservative assertion that the next MMA writes the same destination tuple (section 10.3).

MM 6 and agxforge/g17/auth.py give bit 5 as keep; MM 25.59 reports that 32-bit saturating add (op10239)’s 101 instances carry “source modifier 16 (32 for a high-half source)” in unsigned kernels and 24 (40) in signed ones. These are consistent: in the saturating add the modifier carries an extra bit 3 (weight 8) that selects signed saturation (signed arm executed and clamped to INT_MIN..INT_MAX, MM 25.59), so the field’s low bits are not uniform across forms. The 16 / 32 split itself is consistent with release and keep: $16 + 8 = 24$ and $32 + 8 = 40$ preserve it. MM 25.59 records a correlation over 101 instances, not a measurement isolating the bit, and nothing shows a high-half *selector* at weight 32. A compiler may treat 32 as keep in the saturating add as in the general census; it may not assume that every form’s modifier is limited to the four general bits.

In the forms of table 2.4, the modifier cannot be chosen freely.

Table 2.4. Forms whose modifier cannot be chosen freely, with the consequence for a compiler. Each row is measured or decoder-checked.

Form	Fact	Consequence	Source
four-byte 32-bit AND (op424), 32-bit OR (op13575), 32-bit XOR (op17771)	no lifetime bit: always release their sources	use the longer form for a source read again	measured, MM 24.8 row T4, ledger g17-the-four-byte-bitwise-releases-its-sources
uniform-address atomic, returning old (op10094), ten bytes	the value’s lifetime (operand 9) is pinned at 16	two atomics sharing a value, or the value stored afterwards, read 0 (fuzz seed 5012: 0 where 24)	measured, MM 25.144.6
128-bit vector store (op17256)	the stored tuple’s lifetime (operand 1) is 16 in every witness; no keep value measured	a value read after the store read 0 on 32 of 32 lanes; store a copy	measured, MM 25.144.6
SIMD shuffle, constant lane, broadcast (op14157)	the source lifetime is operand 3, not operand 1; a template’s 16 released x	$y = \text{broadcast}(x); z = \text{fma}(x, x, y)$ stored 0	measured, MM 25.144.6
imageblock store	operand 6 is the coordinate’s lifetime (0 keep, 16 release)	releasing it collapsed all 32 lanes to element 0 (one-bit control)	measured, MM 6
tensor MMA, 16-bit $\times$ 16-bit, accumulate (op5106)	the accumulator’s modifier (operand 10) has no single-bit carrier in the 80-bit field map	cannot be authored; a multi-bit encoding is not excluded	decoder, MM 6

The position of the lifetime operand varies by form (32-bit register move (op586): operand 3; plain 32-bit store (op17229)/8: operands 1 and 6 for value and index; the fp16 fma, register form: operands 3, 5, 7), and each must be written from liveness (MM 25.117, MM 25.196, agxforge/g17/cc.py). All four modifier bits must be written, not only the lifetime: witnesses taken from a mutation walk rather than from Apple’s code can carry 0 or 52 (keep, release and absolute value at once), and inheriting them made four opcodes in a naming sweep return the same value for every input (agxforge/g17/auth.py).

2.5 Release semantics

A release (modifier 16) destroys the lane’s value, per lane, under the execution mask, and a later write restores the register. Every controlled probe of a released register read zero, but that is not established as a rule (see the end of this section).

Each of the following was measured in executed programs.

- The contents do not survive, in the controlled probes. An instruction carrying 16 as the first reader of a register made every later reader in that program see 0, and the register read 0 at a store. Modifier 32, or another first-reader form, preserved it (MM 6). The imageblock store’s coordinate behaved the same way, so the effect is not specific to the tensor feed (MM 6).
- The release acts per lane, gated by the execution mask. Every earlier release had run on all 32 lanes, so two mechanisms fit equally well: the release frees a register the simdgroup shares, or it clears each lane’s copy. A partial mask separates them. $y = t + 3054$ was read by 32-bit subtract immediate (op11666)/12 inside a region gated on $t < \text{bound}$, with that read’s lifetime written $32 \rightarrow 16$ and decoded back; after the region every lane stored $y + 0$ (MM 25.95, tools/g17/releasemask.py). Table 2.5 gives the four arms.

Table 2.5. Lanes reading 0 after a release under each execution mask (MM 25.95).

Arm	Region	Patched	Lanes reading 0
control	$t < 16$	no	none
partial	$t < 16$	yes	0–15 only; 16–31 read y exactly
all on	$t < 32$	yes	all 32
all off	$t < 0$	yes	none

In the partial arm only the lanes that executed the release lost y , and in the all-off arm none did: the release acts on each lane’s copy, an instruction no lane executes releases nothing, and a simdgroup-wide free is ruled out. A compiler must therefore treat a value released inside a branch as dead only on the lanes that took the branch (section 2.6, rule 3).

- The register is cleared, not handed to another simdgroup, within the measured bound. Group 0 released y (one lifetime operand, the only bytes that differ from the keep arm), stayed resident for about 13,100 instructions while 8,191 other groups (about 300 instructions each, 16 distinctive live values each) launched and finished, then re-read y : 0 in 3 of 3 rounds, never another simdgroup’s value, while group 1 (not released) read y in the same dispatch (MM 25.95, isa/g17-execution-release-reuse-results.json). This bounds when reuse could occur but does not exclude reuse.

- A write restores the register: 230 release → rewrite → read chains in 15 bit-exact tensor bodies, rewritten by ALU results, immediate moves, loads and a tensor MMA; for example fp32 add (op998) releases R19, fp32 multiply (op3290) writes R19, and the add reads it (MM 25.95).
- Order inside one instruction depends on the form. fp32 multiply squaring $x * x$ with a release on the first read is bit-exact, and fp32 compare-select, fmax/fmin (op9700) naming one register in two released slots is harmless: in these ALU forms the release takes effect after the sources are read (MM 25.145, MM 25.117). But plain 32-bit store (op17229) naming one register as both value and index, with the *index* slot releasing it, stores 0 on every lane: the store reads its value after the index's release has cleared it (5f04030a01061040 decodes to [R5, 16, 2066, expr, 0, R5, 16]; clearing operand 6 alone restores the value). Apple's compiler releases neither slot in all 142 such stores among 32,162 (MM 25.117). Rule: never release, in one slot, a register another slot of the same instruction reads, except in forms measured safe.
- A write cancels the destination's pending release. An in-place accumulate that reads C with a release and writes $D = C + \dots$ with $D = C$ is bit-exact (MM 25.145).
- Release acts per 16-bit half. A 16-bit write revives only its half; the other keeps its history (MM 25.145).

Whether a released register reads 0, or can still return its old value, is unresolved (the sources' differing wordings are listed in [appendix A](#)). The evidence on each side is as follows.

- every controlled measurement read 0: adjacent reads, one ALU op later, about 7 instructions later, past a barrier, under partial masks, after 13,100 instructions of residency (MM 25.95); and the M 2048 register-O attention whose 32-bit read of R33 needed its released high half to be zero, which passed 20 of 20 in fresh processes once the 16-bit special-register read was shown not to zero-extend ([section 2.2](#); MM 25.147, MM 25.145.4);
- the only “old value” observation is a loop kernel (the wide RMSNorm's scale loop, [section 2.6](#)) that passed one check and then failed with the same bytes; the check that passed had no sentinel in the output, so leftover correct values could have hidden unwritten ones (MM 25.142.7, agxforge/g17/releasecheck.py header).

The old-value branch is weakly evidenced but not excluded. A compiler may assume that a released register never shows another simdgroup's value within the measured bound, and that rewriting it restores it. It may not assume it reads 0, and may not assume it keeps its value. It may read one only where both outcomes give the same answer, which is the rule this project's emulator applies (it admits such a read only when the old value is zero, MM 25.145) and the rule the attention loop's latch relies on (the counter a finished lane released is the zero that ended its loop). *Correction:* MM 0.15's “A released register: reads zero” and MM 17 rule 37's “every later read returns zero” state an observation as a rule.

Of the never-written registers, untouched R2, R20 and R60 read zero as operands in a whole-program measurement; R0 does not (measured, MM 25.145; the emulator admits such reads only in its whole-program tier). Do not rely on R0.

Apple's compiler reads released registers in one place. Its legacy simdgroup MMA programs release R0 and R1 (address arithmetic) and then issue legacy 8×8 simdgroup MMA, fp32 (op2842) with accumulator R0_R1 (Apple's compiler emits, agxforge/g17/releasecheck.py). The checker treats this as a zero initialisation and exempts it; it is a statement about what Apple emits, not a measurement of the read. The checker treats Apple's 6,594 corpus programs as ground truth and is required to report zero findings on them; this exemption, one for cross-lane value operands and a few path-sensitivity rules were each added for a named false positive there (agxforge/g17/releasecheck.py).

Worked example: release at the last read in the running example’s four MMAs (compiled by this project’s compiler; decoded by Apple’s decoder; executed below Metal, 323 of 323 exact). The product is $17 \times 19 \times 16$, so the M and N axes each need two 16-wide tiles (the second masked) and $K = 16$ needs one step: four tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107) instructions, each reading one A tile and one B tile. Each tile is read twice, so its first read carries modifier 0 and its second, last read carries 16 (table 2.6).

Table 2.6. Tile modifiers in the running example’s four tensor MMAs, with the release on each tile’s last read.

Offset	Bytes	D (accumulator tile)	A tile, modifier	B tile, modifier
0x29a	a70405112204a4620202	R24–R31 (rows 0–15, cols 0–15)	R20_R23, 0	R68_R71, 0
0x2a4	2704a5112204a4420204	R32–R39 (rows 0–15, cols 16–31)	R20_R23, 16 (last read)	R72_R75, 0
0x2ae	a70485832206a4420202	R40–R47 (rows 16–31, cols 0–15)	R56_R59, 0	R68_R71, 16 (last read)
0x2b8	2704a5832206a4620204	R48–R55 (rows 16–31, cols 16–31)	R56_R59, 16 (last read)	R72_R75, 16 (last read)

R20_R23 holds A rows 0–15 (two tensor loads at byte offsets 0 and 256, eight rows of 32 bytes each); R56_R59 holds A rows 16–31, of which only row 16 exists (masked loads at 512 and 768); R68_R71 and R72_R75 hold B columns 0–15 and 16–31 (row stride 38 bytes, so offsets 0 / 304 and 32 / 336). The A modifier is operand 4, carried by byte1 bit 7 (weight 32) and byte2 bit 5 (weight 16); the B modifier is operand 7, carried by byte5 bit 1 (16) and byte8 bit 0 (32) (isa/g17-tensor-mma-fieldmap.json). The first two instructions differ in four bytes: byte 2 bit 5 sets A’s release ($0 \rightarrow 16$); D moves from R24 to R32 through byte 0 bit 7, byte 2 bit 7 and byte 7 bit 5 (weights 8, 32 and 16 on the tuple’s base register); byte 9 moves B from R68 to R72. Nothing reads R20–R23, R56–R59 or R68–R75 afterwards; each D tile is read once, as two four-register halves, by the tensor stores that follow.

Placing the release on the *first* read of any tile would have zeroed one of the four products on every lane. Three properties of the schedule make the placement trivial here and are what a general allocator must establish: the four MMAs run on every lane (no mask divides them), there is no loop (no back edge reads a tile again), and no instruction reads a tile in two slots.

2.6 Loop-carried values

A loop compiled by this compiler (measured, MM 25.116), bounded by a register ($i < \text{trips}$) and capped by a constant ($i < 255$) so that it was “bounded on paper”, never returned. The host was killed at its 600 s timeout and the kernel kept running: with nothing dispatching, the GPU read 100 percent device utilisation until a reboot. Decoded offline, the latch was `icmp(i1 < trips)` followed by the cap `icmp(i1 < 255)`, and the cap compare, the instruction added to make termination provable, carried a release on the counter. The next trip read the cleared counter, `i` stuck at 1, and neither bound could ever be reached. The compiler’s liveness took a value’s last textual read as its last live point; a loop header’s `phi` carries no uses, so the latch value, read only inside the body, looked dead after its last mention. Every earlier runtime-loop test stored its counter after the loop, which kept it live and hid the defect.

A loop phi that inherits its entry value’s register must keep that value live into the loop (measured, MM 25.142.7, MM 25.144.6). In the wide RMSNorm, the scale loop’s index phi started at the thread index t and took t’s register; t’s last mention before the loop released it, so every thread stored as thread 0 (only out[0] and out[1024] written). This holds even when the instruction falling into the header itself reads the entry value: the value’s last point is the loop header label, because that instruction would otherwise be its last reader by index ($c = B[t] + 7$; $x = c + 1$; loop $i = \text{phi}(c, i + 1)$ returned 2 instead of 1009 on 32 of 32 lanes; 29 of 150 fuzzed loop programs had the pattern, all with phis starting at constant 0, which a released register also read, so they were right by luck).

This compiler applies three liveness rules.

1. A back edge *reads* its header phis’ latch values; liveness must treat it so (this compiler: Alloc._cfg_uses).
2. Refuse a release of any loop-carried register (one read in the body before the body writes it) after the body’s last write of it (MM 25.116).
3. Compute releases per lane over every path a lane can take, with masks and loops, to a fixed point: a value released inside a branch is dead only on the lanes that took it, so it may not be treated as dead after the join (MM 25.95). Liveness taken from the last mention in instruction order is wrong for loops, where a back edge reads a value after its last mention.

agxforge/g17/releasecheck.py implements rule 3 on emitted bytes, walking each lane’s paths with the execution-mask counter and joining released-half sets by union to a fixed point; it runs after every compile, held to zero findings on Apple’s corpus (this compiler, MM 25.144.6; section 2.5). The check has to be static, because the looping kernel at the start of this section could not be stopped from the host (MM 25.116). Because a released read can pass a check once, loop kernels must also be verified against sentinel-filled outputs (MM 17 rule 37).

2.7 Encoding holes

Some forms cannot name particular registers: no byte pattern the encoder knows decodes back to them. These are found by authoring each register and reading it back through Apple’s decoder (decoder only); table 2.7 lists them.

Table 2.7. Registers each holed form cannot author, found by reading each authored register back through Apple’s decoder.

Form	Operand	Registers it cannot author	Source
half to fp32 widening (op1004)/12	destination	R14, R15, R30, R31, R46, R47, and R62 and above (the encoder reads back d + 2, or refuses a forced bit)	MM 25.133, agxforge/g17/cc.py
fp16 fma, register form (op798)/12	low-half destination	R18, R19, R26, R27, R50, R51, R58, R59 (low halves)	MM 25.196
fp16 fma, register form (op798)/12	operand 4 / operand 6 sources	R17L / R18L	MM 25.196

Correction: the fp16 fma’s holes are the listed set, not “registers 2 and 3 mod 8”; MM 25.136.6’s widening holes (d = 14 and 15) are a subset of the set above (appendix A).

To avoid the holes, this compiler’s allocator sweeps the admissible indices once from the encoder rather than typing them in (`_encodable_dests`, `_encodable_fma16_sources`), and offers them only to values those forms define or read. A third, hole-aware allocation attempt, used only after two refusals so earlier programs keep their bytes, gives unconstrained values the holed forms’ unauthorable registers first (MM 25.136.6). The holes surfaced as compile crashes in a fuzzer (seed 6) and as refusals of loop arms, never as wrong bytes, because every emitted operand is read back (section 3.8).

Before per-length slot read-back existed, two forms misencoded silently, without raising: ray-tracing block load, 32-bit (`op11994`)/16’s operand 5 named R105 for every register from R16 up, and fp32 times half multiply (`op3291`) and two sibling forms at four bytes wrapped R64 and above by writing past byte 4 (decoder, MM 25.88). The encoder now reads every slot back through the decoder-measured layout for its length and refuses a value it cannot hold.

Not known. Whether these registers are unreachable in the encoding or only in the project’s field maps (a carrier bit Apple’s instances never vary would be absent from the maps).

2.8 Register budgets and spills

The capacity rules are measured (MM 11, MM 17 rules 2 and 3).

- One flat pool of 126 registers: accumulators, fragments and scalars trade capacity; there is no separate accumulator resource and no aliasing. Widths: fp32 accumulator 8, fp32 operand fragment 8, half or bfloat fragment 4, int8 operand pair 2, scalar 1.
- At most 15 live eight-register fp32 accumulator groups in straight-line code (122 registers touched; 16 spill 24 stores), 14 in a loop with one A and one B fragment live.
- With one A and one B fragment live, the measured trade lines are $8N + K \leq 115$ for N accumulators against K live scalars and $8N + 4M \leq 108$ against M 16-bit fragments, exact at every point tested. The constants include the measured kernels’ own overhead registers; they are frontiers of those kernels, and the hardware bound is the 126-register pool.
- Spill points in Apple’s compiler are scheduling outcomes, where its backend schedule overflows the pool, not register-file boundaries (peaks at the cliffs 118–125). Count both spill forms when diagnosing: fragment spill store (`op595`) and scalar spill store, 32-bit XOR into the staging file (`op17789`); a scalar-heavy kernel had 0 of the first and 74 of the second (Apple’s compiler emits, MM 0.7, MM 11, MM 13).
- This compiler does not spill: it refuses a program that does not fit, naming the pool that ran out (this compiler, MM 25.113, MM 25.144.3).

What spills, registers and occupancy cost in time is measured in sections 12.4 and 12.5; residency against registers used is a rule in section 1.4.

Evidence. MM 0.4, MM 0.7, MM 0.15, MM 2, MM 6, MM 10, MM 11, MM 13, MM 17 (rules 1–3, 37), 19, 23.2, 24.2 (rows H1, H2), 24.8, 25.6, 25.18, 25.19, 25.20, 25.38, 25.59, 25.88, 25.95, 25.110.1, 25.113, 25.116, 25.117, 25.120, 25.133, 25.136.6, 25.139.1, 25.141.17, 25.142.7, 25.144.3, 25.144.6, 25.145, 25.145.4, 25.146, 25.147, 25.179, 25.196; `agxforge/g17/releasecheck.py`, `agxforge/g17/cc.py`, `agxforge/g17/auth.py`, `agxforge/g17/registerdomain.py`, `isa/g17-tensor-mma-fieldmap.json`.

3 Instruction encoding

A G17 program is a byte stream of variable-length instructions aligned only to two bytes. Apple’s toolchain has no mnemonics for them. An operand’s bits are spread across the instruction, often non-contiguous, sometimes inverted, and placed differently at each length an opcode takes. Apple ships a decoder for this stream inside its GPU compiler but withholds the printer. The project calls the decoder directly, and every structural statement in this reference comes from it; statements about execution come from dispatches.

Three kinds of statement about an encoding are easily confused: *what Apple’s decoder recognises* (an opcode, a length, typed operands), *what can be authored* (bytes this project’s encoder can produce for a requested operand set, read back through the decoder) and *what the hardware does with those bytes*.

Errors in this project have come from each boundary between them. They include a decoder-clean back edge that was wrong in its whole second half, a “free” bit that changed a compare’s result, a refused tensor bit that changed the product, an encoder that rounded an odd register silently, and a name inferred from a structural twin that was wrong 44 times in 44. Each rule below says which of the three it concerns, and [section 3.8](#) gives the guards that check authored bytes against each boundary.

Terms:

- *Form*: an (opcode, length) pair, written for example tensor load, half/bf16 (op12674)/16. Field maps are per form, never per opcode ([section 3.1](#)).
- *byteN[k]*: bit k (0 = least significant) of byte N of the instruction, byte 0 first in the stream. *Bit n* (linear): byte $n // 8$, bit $n \% 8$.
- *Operand*: a position in the decoder’s operand list (operand 0 first). Registers print as Rn ([chapter 2](#)); immediates as integers; a base address as `expr`: (a pointer into the decoder’s process, meaningless outside it).
- *Operand 1*: in ALU, memory and tensor forms, the destination’s raw operand, printed as one wide integer (older notes: the “flags word” or “control word”). It packs the destination modifier (bits 4–5), a fill slot (bits 20–23) and a wait mask (bits 24–31) ([section 3.4](#)).
- *Witness*: an instance of a form in Apple’s compiled code. *Repair-walk witness*: bytes reached by mutating a neighbour until the decoder accepts them, with no Apple instance.
- *D1, D2, D3*: the three coverage counts ([section 3.2](#)).
- *Corpora*: `isa/g17-corpus-programs.jsonl` (6,594 programs, 365,590 instructions) and the vendor corpus (9,556 functions, 4,644,438 instructions), union 1,893 forms (MM [25.51](#), MM [25.68](#), MM [25.78](#)).

3.1 Instruction lengths and framing

Lengths are even, from 2 to 22 bytes, and belong to the instance, not the opcode (decoder). Apple’s decoder frames three populations as follows.

- *Apple’s two corpora* (5,010,028 instructions, 1,893 forms; `isa/g17-vendor-corpus-forms.json`): every even length from 2 to 16, plus 18 bytes (4 forms, 31,458 instructions, among them texture write (op10909)) and 22 bytes (30 forms, 53,629 instructions, among them texture sample (op14661)); no 20-byte instance. The 18- and 22-byte instances are 1.7 percent of the total.

- *The 216-opcode census* (isa/g17-opcodes.toml, docs/agx3-oracle.md section 7) spans 2 to 16 bytes; none of the 18- or 22-byte opcodes above is in it, so a range read from this census stops at 16.
- *The decoder-admitted universe* (6,719 opcodes, [section 3.2](#)): the 6,000 opcodes with no Apple witness take their width from decoding a repair-walk encoding, which gives 4 to 16, 18 and 22 bytes (7 opcodes at 18, 38 at 22) and again no 20 (tools/g17isamap.py repair_walk_widths). Such a width means the decoder accepts the form, not that anything emits it. Whether a 20-byte length exists is not known; no source contains one and no source excludes it.

In the 216-opcode census, 64 opcodes occur at more than one length: tensor load, half/bf16 (op12674) at 12 or 16 bytes, move immediate, two-operand form (op554) at 4 or 8, move immediate, three-operand form (op11842) at 2, 8 or 10 (docs/agx3-oracle.md section 7). Field positions move with the length: fp32 fused multiply-add (op2190)’s operand-1 high code sits at 2^{24} - 2^{28} in its 8-byte form, 2^{30} - 2^{31} in the 16-byte form and 2^{32} / 2^{36} in the 12-byte form (MM [25.53](#)); one byte offset can mean different fields in loads and stores of different lengths (MM [25.24](#)), so field maps must be per form. One map per opcode was the common defect behind five separate fixes: for 1,301 of 1,401 per-form specifications the opcode-wide authoring map had been probed on an instance of another length (32-bit OR (op13575)’s 16-byte map reached byte 9 of its 4-byte form) (MM [25.88](#)).

[Table 3.1](#) collects the known length rules.

Table 3.1. Known length rules by population, each a statement about Apple’s decoder’s framing.

Population	Rule	Standing
general	lengths are a base plus optional extensions; the length-bit cluster {32, 63, 64, 79, 80} is conserved from G13	decoder, 895-opcode length_bits work (MM 25)
class 7 (byte0 low nibble 7), byte1 = 0, byte6 in {0x60, 0x68, 0x70, 0x78}	length = 12 + 4 × byte10[0]	decoder; 9,913 instances (MM 25)
class 7, byte1 = 0, byte6 in {0x83, 0xa3, 0xa4, 0xa9, 0xaa, 0xab, 0xae, 0xaf}	constant per byte6 (12 or 10)	decoder; 13,571 instances (MM 25 , agxforge/g17/dis.py)
class 7, byte1 = 0, byte6 = 0x10 / 0xa0 / 0xa2	bit 63 / bit 18 / bit 18 selects the length	causal against the decoder, 60 of 60, 37 of 37, 15 of 15 decodable flips (MM 25)
class 7, byte1 = 0, byte6 = 0xa1 (lengths 10, 12, 14, 16)	open; bit 45 explained 81.5 percent correlationally and moved the length in 0 of 60 flips	bounded negative (MM 25)
move immediate, three-operand form (op11842)	2 bytes exactly when bit 15 = 0 (1,682 of 1,682); the 10-byte driver is not a function of the always-present bits	decoder (MM 25.62)
store, sub-form 01, texel/word-slot (op17235), two-component store (op17244)	8 bytes exactly when bit 63 = 0; 14 needs bit 63 = 1, not sufficient	decoder (MM 25.62)
128-bit vector load (op12709)	14-byte form when it carries bit 37 or a displacement above 255	decoder and Apple’s compiler (MM 6)
uniform / staging write (op592)	4-byte form’s K field stops at 126; Apple uses the 8-byte form for K 128–134	Apple’s compiler emits (MM 25.120)
every opcode	bits 0, 1, 2 never vary within an opcode	decoder (MM 25.62)

For class 7, docs/agx-isa-status.md states, from a tiling solution, that 16 bytes need *both* byte10 bit 7 and bit 0 (0x21 → 12, 0x80 → 12, 0x81 and 0xc1 → 16); MM 25 quotes _WIDE7_EXT as $12 + 4 \times \text{byte10}[0]$, which reads 0x21 as 16. They were derived from different populations. Checked against the lengths Apple’s decoder assigned in the corpus’s span index: in the _WIDE7_EXT population (byte6 in {0x60, 0x68, 0x70, 0x78}) byte10 takes only 0x80, 0x81, 0xc1 and once 0xe1, bit 7 is always set, and $12 + 4 \times \text{byte10}[0]$ holds on all 12,001 such class-7 spans (9,913 of them with byte1 = 0, MM 25’s count); byte10 = 0x21 occurs only in other byte6 groups (0xa3, 0xa2, 0x83, 0xa8, 0xa0), where its instances are 12 bytes except under 0xa8 (16) and 0xa0 (14). Each formula holds on its own population and the two agree where they overlap, but neither is a general class-7 rule: “both bits” fails for byte6 = 0xa8 (0x21, 16 bytes). Tiling alone cannot settle the question, because two disjoint length assignments satisfy all 364 tiling constraints (docs/agx-isa-status.md). The running example follows the _WIDE7_EXT rule: its tensor loads are 12 bytes with byte10 = 0x80 and 16 bytes with byte10 = 0x81, and in that listing the 16-byte ones are exactly those with a displacement of 256 bytes or more.

Ground truth for framing is Apple’s decoder (section 3.6); the project’s own framer cannot replace it. The framer “landed” exactly on 22,184 of 22,187 constant programs but framed them identically to Apple in only 1,725 (7.8 percent), because a wrong framing falls back into phase on the short fixed tail of end-of-program and two-byte fillers; it misframes 200 of 200 current programs within their first instructions (MM 25.88, MM 25.209).

3.2 Opcode identity and coverage counts

Opcode numbers are indices into Apple’s instruction table as built for macOS 26.6.2 (25G83). That MCInstrDesc table has 17,796 entries (stride 48; the first halfword equals the index for all of them), of which 17,779 declare at least one operand (isa/g17-agx3meta-instrs.txt; sources quoting one or the other are reconciled in appendix A). The macOS 27 build’s table has 17,905 entries in a different order; this reference keeps the 25G83 numbering (section 3.7). Apple built its LLVM with instruction names disabled, so the name table holds “0”, “1”, “2” and there is no assembly-string table; every opcode name in this reference is a project name from measured function, recorded in isa/g17-opcode-glossary.json, not an Apple spelling (decoder, docs/agx3-oracle.md section 6). Mechanically available per opcode: operand count, number of definitions, register class and operand type per operand (type 4 = PC-relative), scheduling class, flag words; lengths are not a table property (agxforge/g17/model.py).

The decoder admits 6,719 opcode records (6,718 found by mutation and breadth-first walks from real encodings, plus camera/video kernel op, function unknown (op13754), added from the vendor corpus) (MM 24.8 row T1, MM 25.58). The declared set overstates the ISA: in the atomic family 208 opcodes are declared and 29 admitted (tools/g17admit.py).

Table 3.2 gives the three form counts (generated, docs/g17-isa-reference.md); they are never summed.

Table 3.2. The form counts D1, D2 and D3, with the dispatched forms outside the verified class.

Count	Forms	What it requires
D1 structural	7,546	the decoder can address the form (opcode, length); 7,549 with the 25G83 decoder (section 3.7)
D2 encoded	1,118	an encoding Apple wrote, with no unforced and no undetermined bits
D3 own instruments (any of nine)	340	an own hardware instrument checked its semantics (D3 verified class, an isolated one-opcode probe returning the predicted value: 89)
dispatched, not in the verified class	472	it ran; no isolated probe confirmed the name (236 of these carry other semantic evidence)
dispatched, no semantic evidence of any kind	236	it ran and nothing has read its output

- D1 is known to undercount. Five tensor MMA forms ($\text{fp32} \times \text{fp32}$ and $16\text{-bit } A \times \text{fp32 } B$, each with and without accumulator, and int8 widening without accumulator) are admitted by the decoder but missing from the admission walk: a selector bit that also changes an operand’s register class cannot be reached one flip at a time (`docs/g17-isa-reference.md`). D1 counts what the contract records, not what the decoder accepts (MM [24.8](#) row T1).
- D2 counts forms fully specified by Apple-witnessed, bit-settled evidence, which is a different property from being emitted by the compiler; 91.7 percent of D2 forms reconstruct their own witness and 57 do not (MM [25.87](#)). D2 is held back by missing per-form evidence: 6,579 of 6,858 non-D2 forms failed on absence, not on unknown bits. Its last gains came from hardware checks of drifting bits (MM [25.87](#), MM [25.58](#); the successive counts are in [appendix A](#)).
- The nine D3 columns are instruments with different failure modes (a name an isolated probe confirmed; the one library candidate surviving a form’s cases; a family law), so only “any of the nine” is quoted (`docs/g17-isa-reference.md`).
- *Correction:* `docs/g17-isa-cartography.md`’s D1 975, D2 690 and two-MMA admission are a superseded snapshot ([appendix A](#)).

The counts do not give the number of independent semantic questions. The earlier “527” is retracted (21 of 25 multi-member name-stem classes contain opcodes with different functions), and naming by structural twin was right 0 times in 44 on the verified set (MM [25.81](#), `docs/g17-isa-cartography.md`). Forms Apple’s compiler never emits are absent from both corpora, so corpus reading cannot find them (MM [25.78](#)).

3.3 Field maps

A field map is linear per form: $\text{decoded operand} = \text{base} + \sum(\text{bit_is_set}[i] \times \text{weight}[i])$, negative weights for inverted bits (MM [23.3](#)). Operand domains (`agxforge/g17/auth.py`): *raw* (the value the decoder prints: immediates, modifier words), *slot* (register $\times 2 + \text{half}$, ordinary registers), *index* (class-relative member index, for aligned tuple classes; [section 2.2](#)). Each instruction bit is classified by flipping it and asking the decoder (`agxforge/g17/encode.py`): *opcode* (the flip decodes as another opcode), *forced* (the flip is undecodable; the recorded value is the only legal one), *operand* (the flip moves a printed operand), *invisible* (nothing printed changes). The classification has four complications.

- A value bit can have several carriers that are not interchangeable. fp32 multiply (op3290)’s operand 3 value bit 4 is carried by byte6[3] inverted and by byte8[2]; byte6[3] also turns operand 2 into an expression. 1,031 of 9,282 sampled operands have such a bit (agxforge/g17/auth.py).
- A bit’s role can depend on other bits: 547 of 1,401 per-form specs carry an opcode- or forced-class bit that Apple’s own instances vary (1,132 bits; MM 25.87, MM 25.88).
- Some immediates are table codes rather than literal values (camera/video kernel op, function unknown (op13754)’s operand 2 prints 8, 224,776 and 183,494); the codec refuses to write them as numbers (MM 25.58).
- An invisible bit can still change what executes (section 3.8).

A field map counts as established when it rebuilds Apple’s instances. In the best case, a 38-bit map for 32-bit register compare to 16-bit result (op11462)/10, fitted on 1,637 instances, rebuilds 33,473 of 33,475 vendor instances with zero differences (decoder, MM 25.52). Across the corpus the rates are lower: corpus-fitted maps re-author 92.0 percent of in-sample instructions byte-exactly but 55.4 percent of held-out vendor instructions (2026-09-06 snapshot, docs/agx-isa-status.md); the encoder’s held-out re-authoring now stands at 80.8 percent (MM 25.88).

Worked example: the operand fields of a tensor MMA (compiled, decoded; executed in the running example). The first MMA of the running example (offset 0x29a), tensor MMA, 16-bit × 16-bit, no accumulator (op5107), is a7 04 05 11 22 04 a4 62 02 02. Apple’s decoder prints R24...R31 <- 16777216, 1, R20_R21_R22_R23, 0, 2, R68_R69_R70_R71, 0, 2. The field map (isa/g17-tensor-mma-fieldmap.json, entry 5107, reference 2f0005102200a4420200) reproduces every operand from the bytes; it records each bit’s effect as a flip from its reference, and this instance differs from the reference in seven bits (3, 7, 10, 24, 42, 61, 73). Table 3.3 lists each operand and its carriers.

Table 3.3. Operands of the running example’s first tensor MMA and the bits that carry them in this instance.

Operand	Value	Carried by (in this instance)	Meaning
0, D	R24	byte0[7] = 1 (+8), byte7[5] = 1 (+16); byte2[3], byte2[4], byte2[7] = 0	8-register destination; field in units of 8 registers
1	16,777,216 = 2^{24}	byte1[2] = 1 (2^{24}); byte0[3] = 0 (no 2^{31}); byte4[3] = 0 (no keep, weight 32)	wait on scoreboard slot 0 (the tile loads publish slot 0); not a keep, since the next MMA writes another tuple
2	1	no variable bit in the map	a constant of this form
3, A	R20	byte3[0] = 1 (+16 from the reference’s R4)	4-register tuple
4	0	byte1[7] (32), byte2[5] (16), both 0	A’s modifier: neither keep nor release (section 2.5)
5	2	byte6[2] = 1 (clear gives 3); byte8[2] = 0 (transpose adds 32)	A is half (3 would be bfloat)
6, B	R68	byte5[2] = 1 (+64), byte9[1] = 1 (+4)	4-register tuple
7	0	byte5[1] (16), byte8[0] (32)	B’s modifier
8	2	byte7[6] = 1; byte8[3] = 0 (transpose)	B is half

The remaining bits are structural or inert: byte8[1] = 1 selects this no-accumulator form (clearing it decodes as tensor MMA, 16-bit × 16-bit, accumulate (op5106)); byte5[7], byte8[4] and byte6[0] would select fp32-B, fp32-A and the int8 family (MM 0.4, MM 2); bits 0–2 are constant. Across the accumulate form’s reference all 80 bits are accounted for: 39 claimed by mapped fields, 25 structural, 16 decoder-inert,

none unaccounted (MM 6). Its hardware single-bit census ran 80 of 80 flips with none refused by the pipeline and none faulting; every decoder-inert bit was hardware-inert (measured, MM 2). The project’s MMA encoder reproduces 800 of 800 compiled MMAs byte-exactly from decoded operands (MM 2).

Worked example: a tensor load’s operand 1 (compiled, decoded). The first tile load, 47 00 47 0a 08 00 60 2a 00 80 80 08, is tensor load, half/bf16 (op12674)/12: R20_R21 <- 17825792, 241, expr, 0, R5, 0, 0, 2, 15: destination pair (operand 0), operand 1, the base address pair (operand 3, printed as an expression), index R5 with modifier 0 (operands 5, 6), displacement 0 (operand 7), element code 2 (operand 8) and mask 15 (operand 9); operands 2 and 4 are immediates not discussed here. Operand 1 is 17,825,792 = 0x01100000, exactly what agxforge/g17/memenc.py authors as ((slot + 1) << 20) | (wait_mask << 24) with slot 0 and wait mask 0b00000001:

- bits 20–23 = 1: this load *publishes* scoreboard slot 0 (the decoder prints slot + 1; 0 means none);
- bit 24: this load *waits* on slot 0, where the special-register read that feeds its index R5 published. Single-bit flips of this instance through Apple’s decoder locate the carriers: clearing byte7[5] removes 2^{24} (the wait); setting byte6[3] or byte6[4] changes the slot field (to 2 and 3); setting byte2[5] adds 32, the destination keep weight (decoder only, this instance). The slot carriers differ between the tensor load’s four field variants (12 and 16 bytes, with and without a residue bit byte1[6]), so they must not be copied from one variant to another (MM 25.26, MM 25.32). The element code (operand 8) is the access width in bytes, 1, 2, 4, 8 or 16 across the load family (MM 25.37). The index’s modifier is 0 because the next load reads R5 again; a load releases its index with 16 (memenc index_last).

Worked example: the scalar tail (compiled, decoded, executed). The running example’s last five instructions compute $C[\text{tg.x}] = A'[\text{tg.x}] + 1.0$ on a scalar path (table 3.4).

Table 3.4. The running example’s scalar tail, five instructions from special-register read to store.

Offset	Bytes	Decoded	Fields that matter
0x49e	9c9c1006	special-register read (op14059): R9 ← 1048576, SR_TG_X	operand 1 = 2^{20} : publishes slot 0. Destination R9 in byte0[7:4] (a four-bit field, section 2.3); special register number in byte1
0x4a2	0f084312182210c0 410080000000	plain 32-bit load (op12682)/14: R16 ← 137464119296, 2066, expr, 0, R9, 0, 0, 4	operand 1 = $2^{37} + 8 \times 2^{20} + 2^{24}$: publishes slot 7, waits on slot 0 (for R9); index R9 kept (read again by the store); element code 4
0x4b0	1c80423e6020000c	move immediate, three-operand form (op11842)/8: R17 ← 68736253952, 1065353216	immediate 0x3F800000 = 1.0f, scattered (below)
0x4b8	0900044a0220a02a 24081a00	fp32 add (op998)/12: R18 ← 148176371712, R16, 16, R17, 16	operand 1 = 0x2280000000: bit 31 waits on slot 7 (the load); both sources released (last reads)
0x4c4	2f08431201061040	plain 32-bit store (op17229)/8: R18, 16, 2066, expr, 0, R9, 16, 0, 4	value R18 and index R9 both released; distinct registers, so legal (section 2.5)

The move immediate’s 32-bit value is scattered across six byte ranges, a layout predicted and confirmed byte-exactly on 0xDEADBEEF and 0x12345678 in the scalar campaign (docs/agx-isa-status.md): value bits 6:0 in byte1[6:0], 10:7 in byte4[4:1], 12:11 in byte5[3:2], 20:13 in byte6, 24:21 in byte7[3:0], 31:25 in byte3[7:1]. Here byte3 = 0x3e gives bits 25–29, byte7 = 0x0c gives bits 23–24, everything else is zero: 0x3F800000. Its destination R17 is seven bits: byte0[7:4] = 1 plus byte2[6] (+16); single flips of this instance

through the decoder give `byte2[7] = +32` and `byte2[3] = +64` (decoder only). Operand 1’s bits 33 and 37 in the load and the add are not explained here.

3.4 Scoreboard field encoding

Semantics (what a slot counts, when a wait is needed, which producers are late) are in [section 9.1](#). A producer of a late value names the slot it fills, printed as `slot + 1` in operand 1 bits 20–23; a consumer names the slots it waits on in operand 1 bits 24–31, bit `24 + s` for slot `s`; there is no separate wait instruction (measured, MM 6, MM 0.8). [Figure 3.1](#) draws the layout, and [table 3.5](#) gives the carriers in each form.

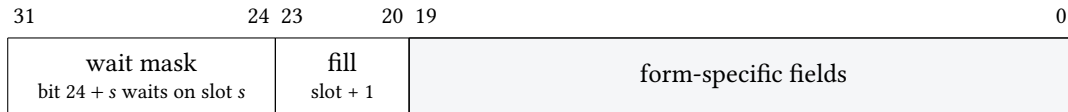


Figure 3.1. Scoreboard fields in operand 1, for the forms that carry them there. A producer of a late value writes the slot it fills as `slot + 1` in bits 20–23 (0 means none); a consumer sets bit `24 + s` for each slot `s` it waits on. The plain 32-bit load (op12682) of the running example carries `0x01800000`, so it waits on slot 0 and fills slot 7. Other forms place the same fields elsewhere ([table 3.5](#)).

Table 3.5. Where each form encodes its scoreboard fill slot and wait mask. Sources follow the table.

Form	Fill slot	Wait mask
general loads, atomics, special-register read (op14059), imageblock load	operand 1 bits 20–23 (slot + 1)	operand 1 bits 24–31
tensor load, half/bf16 (op12674)	operand 1 bits 20–23; carriers per field variant (section 3.3)	operand 1 bits 24–31 (byte7[5] carries bit 24 in the 12-byte variant above)
128-bit vector load (op12709)	byte4[3] + 2 × byte4[4] + 4 × byte6[4], printed as slot + 1	operand 1 bits 24–31
tensor MMA (same carriers in all ten field maps; causally measured on the 16-bit forms)	none	slots 0–4 at byte1[2..6], slot 5 at byte7[7], slot 6 at byte2[6], slot 7 at byte0[3]
12-byte ALU forms	none	byte0[3] = slot 7; byte1[2] = slot 0 on 32-bit add immediate (op10279) and 32-bit add (op10282)
14-byte stores	none	byte9[5] = slot 7 (the carrier this compiler’s encoder writes)
plain 32-bit store (op17229), 8-byte form	none	operand 1 bits 24–31 on 15 corpus instances, each matching its pending slots; this compiler’s encoder has no carrier for them
128-bit vector store (op17256)	none	operand 1 bits 24–31, every bit used (13,140 instances in a 314-object census, each one-hot value present)
tensor store (op17257), masked tensor store (op17258)	none	zero in all 876 and 118 compiled instances

Table 3.5, continued

Form	Fill slot	Wait mask
uniform / staging write (op592)	0 into the uniform file; 1–8 (slot + 1) into the staging file; 8-byte form: field in byte5[0..2], value + 1	none

Sources: MM 6 (MMA mask causal over 43 dispatches, then 20 of 20 preregistered; vector-load formula, correcting an inverted one; the vector store’s full mask, refuting an earlier “three-bit store limit” that was an encoder-table artifact; the tensor stores’ zero byte), MM 25.121 (ALU slot-0 bit), MM 25.120.1 (uniform / staging write (op592) over 178,582 vendor instances: uniform writes always 0, staging writes 1 in the 4-byte form and 1–8 in the 8-byte form, none crossing), MM 25.146 and isa/g17-wait-mask-census.json (the 8-byte store), agxforgel/g17/memenc.py, agxforgel/g17/cc.py. The decoder census behind the table (every instance’s wait bits equal the slots its sources are still pending on, 126 forms, MM 25.96) and what each mask means are section 9.1. A form whose wait bits are not located (for example fp16 fma, register form (op798)) gets a loaded source through a waited copy (MM 25.196).

byte4[3] has three unrelated meanings. It is the first term of the vector load’s slot formula, an opcode selector in tensor load, half/bf16 (op12674) (flipping it gives plain 64-bit load, pair address (op12673), with 9 operands instead of 10), and the keep bit in the tensor MMA (MM 24.2 row R6, MM 25.1), so a measurement of it in one of these forms does not carry over to the other two.

Two earlier misreadings took a regularity in Apple’s code for a hardware mechanism.

- *The store “consumer token”*. Early analysis found bits in stores that looked like a token with a wait counter, a countdown rule and a loop rule. They are bits 5:4 of the store’s source register number, on 632 of 632 stores; the “rules” were Apple’s compiler allocating registers in descending order (MM 19).
- *The MMA “result ring”*. The MMA’s high operand-1 bits appeared one-hot, rotating through eight positions, and were read as tags of the accelerator’s in-flight results. They are the eight-bit wait mask over load slots; they looked one-hot because each MMA waited on exactly its own load group (MM 19, MM 6). The causal test that settled this is in section 9.1.

3.5 Program boundaries

- The delivered __text is a 64-byte prologue, the main program, then filler, in 421 of 421 delivered archives; the prologue in the released class is end of program (op684) followed by thirty two-byte filler (op13483), 0e000000 then thirty 0600, and the entry offset is 64 (this compiler and the ABI-v5 class, MM 23.2, MM 25.148). The scalar campaign stripped the same fixed 0x40-byte prologue from Apple’s minimal kernels (docs/agx-isa-status.md).
- end of program (op684) is 4 bytes, 0e 00 00 00, identical in all 1,169 objects examined, and the last instruction of every program; composed multi-GEMM programs remove intermediate ENDS (decoder; Apple’s compiler emits; MM 15).
- two-byte filler (op13483), 06 00, is the compiler’s nop and pads bodies; nearly every kernel contains it (142,452 instances in one corpus) (MM 25.68).

- Per-kernel metadata slot 32 is absent for main programs of 30 instructions or fewer and present (valued by length and back edge) above, a law of Apple’s compiler on this toolchain, not a device guarantee ([section 11.1](#)). While this compiler could author only the slot-present layout it padded bodies with two-byte fillers to at least 31 instructions (MM 15); both layouts are now generated (`agxforge/g17/mdgen.py slot32_for`).
- Code is position-independent. An identical body is bit-exact at stream positions 1–12, after 0–4,096 bytes of padding and at offsets 2–40,002 bytes: no setup or alignment sequence is needed (measured, MM 15). Metal places the code at offset +0x2740 of the dispatch’s first mapped 64 KiB region (measured, MM 25.147).
- A branch target is offset + displacement (decoder; [section 5.3](#) for semantics), landing on a real instruction boundary for 1,867 of 1,867 corpus branches (offset plus size: 12.9 percent). skip branch, no active lanes (op462) and conditional back-edge branch (op458) are both 10 bytes; the skip’s bytes 5–9 are constant `00 00 00 00 00`, the back edge’s `00 ff c7 ff 7f`. A back edge authored with a zeroed tail decoded cleanly but was wrong in its whole second half, and the error showed up only on execution (`docs/agx3-oracle.md` sections 12–13). A G17 branch names no condition; the condition comes from execution-mask state ([chapter 5](#)).

3.6 Apple’s decoder

The per-generation compiler plugins (`applegpu_g13p` to `applegpu_g18p` on macOS 26) implement 30 of 32 plugin entry points; the two stubs are assembly emission and object emission, so `-S` is unreachable for every AGX target. `air-objdump` reports “no instruction printer” for `agx2/agx3` (and “no disassembler” for `agx1`), which implies that the `MCDisassembler` exists. In `GPUCompiler.framework/Versions/32023/Libraries/libLLVM.dylib` the `agx3 llvm::Target` has its disassembler constructor (+0x80) set and its printer constructor (+0x88) null; Apple withheld only the printer (`docs/agx3-oracle.md` sections 1–3).

The decoder is reached by writing a printer factory into +0x88; `LLVMCreateDisasm` then succeeds. `printInst` is vtable slot 4, found by pointing all 32 slots at recording stubs. The `MCInst` lives in `LLVMDisasmInstruction`’s frame, so operands must be copied inside `printInst`: reading them after return gave 69.13 percent invalid operand kinds; copying inside gave 0 invalid over 89,645 operands in 290 objects. `tools/agx3meta.c` dumps the instruction, operand, register-class and register tables (strides 48, 6, 32 and 24). No privileges, entitlements or patched binaries are involved (`docs/agx3-oracle.md` sections 4–8). The in-process form is `tools/libagx3dis.dylib`, behind `agxforge.g17.model.decode`; both this compiler’s read-back guards and the project’s emulator decode through it, so neither runs on another machine (MM 25.209).

Several things break on an OS update (`docs/agx3-oracle.md` section 9). The library path contains `Versions/32023`; the `llvm::Target` field indices and every table stride are build properties (`agx3meta` re-validates strides and refuses; `agx3dis` checks the disassembler constructor is non-null); the vtable slot of `printInst` is not checked at run time, and a reorder would silently capture nothing (“opcode 0 on every line”). The register, class and instruction tables are committed snapshots held equal to the binary’s output by test, so after an update the tests fail rather than the snapshots silently going stale. The update to macOS 27 moved the numbering and those tests failed ([section 3.7](#)).

MM 25.63 reports that “our decode” consumes 9,556 of 9,556 vendor bodies exactly and agrees with Apple’s boundaries on 400 of 400 sampled bodies (67,787 instructions of the classes once disputed). That decode is `agxforge.g17.model.decode`, which calls Apple’s decoder (MM 25.209), so the result shows the wrapper

reproduces Apple’s framing; it is not an independent confirmation. Neither of the two independent attempts matches Apple’s decoder. One is the framer (section 3.1). The other, a decoder learned from Apple-decoded programs, framed and identified 99.98 percent of 746,000 held-out instructions but matched only 83.2 percent of operand tokens, left 16.7 percent unlearned, and got 636 (0.085 percent) wrong without any refusal, on bits that never varied in training; it is paused (MM 25.209). An independent decoder would have to be derived from per-length field maps with an encode-then-compare self-check that turns every uncertain decode into a refusal (MM 25.209).

3.7 Opcode renumbering in macOS 27

macOS 27.0.1 (26A434) ships a rebuilt AGX3 LLVM at the same path. The decoder still runs, but TableGen re-sorted its tables: the instruction table grew from 17,796 to 17,905 entries and the register table from 3,567 to 3,685 entries, so end of program (op684) decodes as 706. The encoding did not move. All 184,349 instructions of the 6,594 corpus programs decode at their recorded offsets and lengths, and each old opcode the corpus uses lands on exactly one new one. After the update 344 of the 722 test modules failed, among them the snapshot tests.

tools/g17renumber.py generates a table through which agx3dis, libagx3dis and agx3meta print the 25G83 ids, so every opcode and register number in this reference refers to that build. Registers and register classes map by name, since their names survived Apple’s name stripping. Opcodes map from 3,244 encodings decoded under both builds: the corpus, the form record’s witnesses and their opcode-bit flips, and the full decodes retained beside Apple-compiled programs. Between those anchors the two descriptor tables are aligned, keeping only the placements that every maximum alignment agrees on; 11,583 opcodes map that way and 934 are left unmapped as ambiguous. Built from the corpus anchors alone, the alignment placed 2,684 of 2,687 held-out witnessed opcodes correctly, none wrongly, and left three unmapped. The new build also writes a two-bit tag into bits 29–30 of some immediates, mostly in memory and tensor forms; 114 fitted entries undo it per opcode and operand. At start-up each decoder decodes eight canary instructions and refuses a build whose canaries match neither numbering. With the translation the corpus decodes as recorded, every operand value in the form record reads back (41,246 immediates and 29,635 registers), and 1,603 retained full decodes match token for token (isa/g17-agx3-renumber.json).

The new decoder also differs in ways that are not numbering effects. It rejects the repair-walk witnesses of op9994 and op9995 (no glossary entry) and of 64-bit no-return atomic, max/min (op9996), so D1 is 7,546 on 26A434 against 7,549 on 25G83; Apple’s own encoding of op9996, compiled from a 64-bit atomic_max, still decodes. Of the single-bit flips in the form record, it rejects 38 that the 25G83 decoder read as another opcode and reads 7 that it rejected as opcodes new in the 26A434 build; operand classes are unchanged. And three operand slots carry the bit-29 tag in every corpus instance with no record of their 25G83 values: uniform-address no-return compare-exchange (op10023) operand 1, uniform-address compare-exchange, returning (op10095) operand 2, and op9996 operand 1. They are decoded without translation, and no statement in this reference rests on them.

3.8 Decoder admission and execution

Neither a decoder rejection nor an acceptance settles what executes (docs/agx3-oracle.md section 12):

- An authored subtraction the decoder rejected computed 0 - x on hardware (the rejection was correct).

- byte9[2] of the 14-byte load is rejected by the decoder yet executes as a narrow load: 0x12345678 with it clear, 0x00000078 with it set, both predicted (measured).
- Six decoder-refused one-bit variants of tensor MMA, 16-bit × 16-bit, accumulate (op5106) are no-ops on the accumulator and eight change the result; no encoding property separates the two groups, and the decode side is exhausted (measured, MM 2, MM 25.25, MM 25.36).
- Apple’s compiler emits a decoder-refused encoding: int8 MMA streams contain a700a518220ea182c00a; clearing structural bit 71 alone gives a 10-byte tensor MMA, int8 widening to int32 (op10384). With bit 71 set, the other 79 single flips never yield an MMA, and bit 71’s meaning is open (MM 25, MM 25.71). The project’s decoder wrapper names this variant and resumes ten bytes on.
- Names R128–R143 decode and do not execute (section 2.1). A zeroed conditional back-edge branch (op458) tail decodes and is wrong (section 3.5).
- MMA admission is gated on the CPU name passed to the decoder: g17s, g17g and g18 admit the same ten tensor MMA opcodes; the 122 other table entries in scheduling classes 170–175 are unreachable within three flips of the seeds, a bounded negative (decoder, MM 2, MM 25.4 “F4, narrowed”, MM 25.13).
- Metal’s pipeline creation checks no instruction bytes: an undecodable 0xff fill and a removed end-of-program both build (section 11.4).

A bit the decoder ignores and Apple’s corpus writes both ways can still change the result, so free bits need an execution check. 16-bit compare-select against immediate (op11482)/14 bit 11.6 returns 1, 1, 1, 0 set and 1, 1, 1, 1 clear, three runs each (measured, MM 25.88). Drifting free-bit verdicts were adopted only after a hardware check (13 forms, then two single-instance forms whose 28 and 11 bits were executed), 252 drift bits that change no emitted byte were deliberately left unadopted, and the compare-select’s bit stays live (MM 25.58). Nor does a clean decode show that a form works: forty repair-walk opcodes in one class ignore both inputs and return a constant of the encoding (docs/g17-isa-cartography.md).

In the encode-then-decode-back guard, the encoder starts from a measured reference with its variable bits cleared, solves each requested operand, emits, decodes the result through Apple’s decoder, and refuses unless opcode, length and every requested value round-trip (MM 23.3). The guard was added after the encoder had relied on field arithmetic alone: the campaign’s encoder accepted a byte sequence whose field arithmetic reached a zero residual for an *odd* register base, and silently rounded it to the aligned tuple, because the tuple field cannot express the low bit (MM 2, MM 23.3). Later read-back gaps of the same kind were found and closed: float compare-select, csel form (op9749)/8 asked for register id 441 and wrote 425 until operands were read back; fp32 plus half, half result (op1015)/4 accepted 2147483680 and emitted 32 because operands with an opcode-level code table were skipped (MM 25.88). Every memory entry point in agxforge/g17/memenc.py performs the same check (MM 23.3). The guard checks only that encoder and decoder agree; it does not check arithmetic, layout, wait masks or hardware admission (MM 23.3).

The project’s own dispatch gate refuses by Apple’s instruction flags (may load, may store, branch, call, return, terminator; memory, atomic, texture) and refuses opcodes with no Apple witness unless explicitly overridden; 83 of the 86 opcodes this compiler can emit carry an Apple witness (this compiler, MM 25.147).

Not known. Per-length field maps for most of D1 (D2 is 1,118 of 7,546); the 0xa1 class-7 lengths; the meaning of int8-MMA bit 71; why 36 narrow-accumulator table rows are refused although structurally like admitted forms; the remaining meaning of tensor load, half/bf16 (op12674)’s residue bit byte1[6] (MM 25, MM 25.6, MM 25.32, MM 25.58, MM 25.71).

Evidence. MM 0.4, MM 2, MM 6, MM 15, MM 19, MM 23.2, MM 23.3, MM 24.2 (row R6), 24.8 (row T1), 25 (introduction), 25.1, 25.4, 25.6, 25.13, 25.24, 25.25, 25.26, 25.32, 25.36, 25.37, 25.51, 25.52, 25.53, 25.58, 25.62, 25.63, 25.68, 25.71, 25.78, 25.81, 25.87, 25.88, 25.120, 25.120.1, 25.121, 25.147, 25.148,

25.196, 25.209; docs/agx3-oracle.md, docs/agx-isa-status.md, docs/g17-isa-reference.md, isa/g17-tensor-mma-fieldmap.json, isa/g17-agx3meta-instrs.txt, agxforgel/g17/memenc.py, agxforgel/g17/mdgen.py.

4 Scalar arithmetic

Beside the tensor unit, every G17 lane runs an ordinary scalar ALU: integer arithmetic on 32-bit registers and 16-bit halves, fp32 and fp16 arithmetic, conversions, compares that write flags or select values, and five single-instruction special functions. A compiler targeting this machine must reproduce the ALU bit for bit, because a tensor kernel’s address arithmetic, quantization and epilogues all run here.

Apple does not document this ALU. Many of its forms were first named from Apple’s decoder tables and compiler output, and a later audit of those names against execution corrected 7 of 100 uniquely fitted forms (MM 25.90). Each semantic below was measured on inputs chosen so that the rival readings give different answers. Several results bear directly on code generation. The integer condition codes are a structured field, and one code that every source called “equal” is a bit test (section 4.2). The machine has three subnormal policies, one per execution path, so a value can change when it passes from one path to the next; and both fused multiply-adds, fp32 and fp16, round once, proven on inputs where once and twice differ (sections 4.3 and 4.4). On positive normal inputs the hardware reciprocal square root is an exact, tabulable function with a scaling law, while hardware exp2 is deterministic but has no compact description; the difference determines how kernels that use them are checked (section 4.6).

The full per-opcode table, with lengths and standing, is [appendix B](#).

Conventions:

- Opcodes are cited as glossary name then number, “32-bit add (op10282)”; “/N” is the encoded length in bytes, and a semantic belongs to the (opcode, length) forms that ran. Forms Apple’s compiler emits that have no glossary entry are given a label and marked “(no glossary entry)”.
- Rn is general register n (R0–R125 usable, [chapter 2](#)); RnL and RnH are its 16-bit halves. a32 and a16 are 32- and 16-bit sources; zext16 and sext16 are zero and sign extension of a half; K8 is an 8-bit float-coded immediate (isa/g17-float-immediate.toml).
- Each register source carries a **modifier operand** (section 2.4): negate (2) and absolute value (4) act on values and are used here; release (16) and keep (32) govern lifetime (MM 6).
- FTZ: subnormal inputs and results flushed to zero. RNE: round to nearest, ties to even. A canonical NaN is the single pattern a form returns for every NaN input.

Many forms below can release a source. Treat a later read of a released register as “zero or the old value”, and read one only where both outcomes agree (MM 25.95, MM 25.145, MM 25.145.4, agxforge/g17/releasecheck.py; the rule and its evidence are [section 2.5](#)).

4.1 Integer arithmetic

Integer results are the low 32 bits of the exact result unless a form says otherwise. The families are conventional. Their edge cases below were each measured on an input where the candidate readings disagree ([appendix B](#) lists every form and its source).

Add, subtract and saturating forms

32-bit add (op10282), 32-bit add immediate (op10279) with an 8-bit immediate, 32-bit subtract (op11667) and 32-bit subtract immediate (op11666) wrap. The forms that mix widths zero-extend, and each was separated from sign extension by an input with the half's top bit set:

- add, 32-bit plus zero-extended 16-bit (op10283): $d = a_{32} + \text{zext16}(b)$ (MM 25.145.1);
- add 16-bit to 32-bit (op10285): $d_{32} = \text{zext16}(a) + b_{32}$. Its census operands were special-register values below 2^{15} , where zero and sign extension agree, so a lane kernel with a full-width operand was needed (MM 25.195);
- 16-bit plus 16-bit, 32-bit result (op10286) widens: $0xFFFF + 1 = 65536$, refuting a 16-bit wrap;
- 16-bit add immediate (op10289) does wrap at 16 bits, $d_{16} = a_{16} + \text{imm}$ (4,096 lanes; “immediate + 1” refuted, MM 25.195).

32-bit saturating add (op10239) and 32-bit saturating subtract (op11624) clamp; the receipt that fixed them chose unsigned saturation over signed saturation and wrapping on 32 lanes (MM 25.145). Their signedness is not an opcode: in the corpus, the saturating add carries source modifier 16 (32 for a high-half source) in every unsigned kernel and 24 (40) in every signed one (MM 25.59), and 32-bit arithmetic shift right immediate (op16805) executed only with modifier 24 (MM 25.145.1). Reading the value 8 as a signedness bit fits both; that it is a general integer-modifier rule is *open*.

Multiply

32-bit multiply (op10825) and 32-bit multiply-add (op10826) keep the low 32 bits. Around them sits a family of immediate and half-width variants (appendix B) whose decoder names were repeatedly wrong: the three subtracting forms (multiply by immediate minus immediate (op11059), multiply by immediate minus register (op11060), multiply-subtract (op11063)) carry the map name madd, and three 64-bit product forms carry mulhi though they compute full products (glossary). signed-by-unsigned 16-bit multiply-add (op10838) computes $\text{sext16}(a) * \text{zext16}(b) + c_{32}$, 60 of 60 over seven records (MM 25.59). There is no integer divide: $A[t] / (B[t] \mid 1)$ lowers to a reciprocal sequence through hardware reciprocal (op3658) that touches fourteen forms (Apple's compiler emits, MM 25.80).

Logic, shifts and bit operations

- 32-bit shift left by register (op14392) uses the amount's low 7 bits and gives 0 when the amount is 32 or more, not a shift modulo 32 (measured, glossary).
- find most significant bit (op9986) returns 0xFFFFFFFF for an input of 0 (the rival readings, 0 and a leading-zero count, were refuted); clz is derived from it (receipt fit, MM 25.145).
- The arithmetic shift right immediate sign-fills; the other right shifts, by register and by immediate, are logical.
- The 4-byte register forms release their sources. 32-bit AND (op424)/4, 32-bit OR (op13575)/4 and 32-bit XOR (op17771)/4 have no lifetime bit and always release both sources; all three matched Apple's compiler element for element on disjoint operands (measured, MM 24.8 row T4, MM 25.77). A source needed again requires the /10 form or a copy.
- 16-bit AND, 32-bit result (op428) has a uniform mask. Authored with a register as its mask, it returned zero on every lane, because its second operand is a uniform halfword of the kernel's constant pool

and not a register; the pool sits after the binding pointers (uniform K is pool halfword $K - 4 \times$ buffers). Zeroing or swapping that pool fails all 2,048 outputs of a q4 quantized matrix-vector chain (whole-program, MM 25.139.1, MM 25.141.17, MM 25.145). The pool format is chapter 11's. Extracting a half from a register takes 16-bit AND with immediate, 32-bit result (op426) instead, which reads one half with its source kept and is exact for L and H (MM 25.139.1).

Moves and immediates

32-bit register move (op586) and 16-bit register move (op590) copy a register or one half. move immediate, two-operand form (op554) has run only with immediate 0 (1,228 of 1,231 corpus uses are 0), and 16-bit move immediate into a half (op555) is claimed only for 0 (MM 25.145.1, MM 25.184). move immediate, three-operand form (op11842), 10 bytes, carries a full 32-bit immediate. 32-bit OR with immediate, seven-bit destination (op13574) reaches R64 and above and can carry a load wait, so OR with 0 is the register copy into the upper file (measured, glossary). Immediate size selects the form (Apple's compiler emits, every probe executed, isa/g17-comparison-semantics.json): a select keeps both-immediate arms 0..255 in an immediate-arm select (op11373, no glossary entry) and moves 256 and above, and every negative value, to 32-bit compare-select, four registers (op11375); a compare against an immediate holds 0..256 in 32-bit compare-select, three registers (op11365) and moves 257 and above to the four-register form.

4.2 Compares, flags and selects

A comparison and its select are one instruction (Apple's compiler emits). $a < b ? x : y$ on int emits a single 32-bit register compare-select, immediate arms (op11372)/14 or a four-register select, and no separate compare (isa/g17-comparison-semantics.json). Compares that must survive as predicates write a flag register: 32-bit compare with immediate into a flag (op10369), "the flag compare" below, is the loop-exit compare, /10 for a uint counter and /6 for an int one (MM 25.73). The flag registers FLAG0-FLAG14 are an allocatable class (chapter 2).

Integer condition field

The decoder prints an integer condition as a number from 8 to 15. Authoring 32-bit compare, OR-chained into a flag (op11313) at each printed code and reading the flag on four operand pairs fixed the decode (measured, isa/g17-condition-codes.toml):

```
printed cc = 8 | (signed << 2) | (gt << 1) | lt      equality: lt = gt = 0
```

Table 4.1. Integer condition codes by printed number, with the result of 32-bit compare, OR-chained into a flag on four operand pairs.

printed	relation	(10,20)	(20,10)	(10,10)	(-5,3)
8	unsigned equal	false	false	true	false
9	unsigned less	true	false	false	false
10	unsigned greater	false	true	false	true
12	signed equal	false	false	true	false
13	signed less	true	false	false	true
14	signed greater	false	true	false	false
15	predicted “not equal”; measured otherwise	false	false	true	true

In [table 4.1](#), signed and unsigned order disagree on the pair (-5, 3), so that column identifies bit 2 as signedness rather than a correlated bit. The decode predicted that codes 11 and 15 mean not-equal (lt or gt), but code 15 is true on (10,10) and (-5,3) only, which is the bit test described below.

An encoder writes a field code, not the printed number. For the immediate-arm compare-select the field codes are 0 equal, 1 unsigned less-than and 5 signed less-than (printed = 8 | field), and field 4 behaves as 0, as the decode predicts. Every code returns exactly 0 or 1, so the remaining seven relations derive by swapping operands and by complementing with one XOR: ten relations at four orderings, 40 of 40 (measured, ledger/notes.toml [g17-all-ten-relations-from-three-codes]).

Select field code 1

32-bit compare-select, four registers (op11375) computes $d = (a \text{ cc } b) ? x : y$ with two field codes. Code 0 is a $> b$. Code 1 was recorded as $a == b$, from witness pairs (1,2), (2,1), (1,1) and (5,5), and shipped in this compiler as CSEL_CCC['eq']. Those four pairs cannot tell equality from a bit test: the unequal pairs share no bit and the equal pairs are nonzero, so both readings give the same answers. The error was found when a switch, lowered to a chain of selects that tested a 0/1 compare value against 0, chose the wrong arm on every lane. The discriminating run (measured, MM [25.90](#); ledger/claims.toml [g17-csel-code-one-is-a-bit-test]): $x = 0..31$ on 32 lanes,

- against 16: selected exactly $x \geq 16$, in both operand orders (symmetric, so not an order relation);
- against 0: never selected (equality would select lane 0);
- against 5: selected exactly the 24 lanes with $x \& 5 \neq 0$ (equality gives one lane, $\geq$ gives 27).

Field code 1 therefore computes $(a \& b) \neq 0$. *Correction:* the $a == b$ reading (MM [25.90](#)’s “Was” column) is wrong; cc now names code 1 test and refuses a select that asks for eq.

The four-register select’s field codes 0 and 1 print as 14 and 15; tools/g17emu.py uses the printed numbers and the ledger the field numbers. The printed-15 truth row of [table 4.1](#) (false, false, true, true) is exactly the bit test: $10 \& 20 = 0$, $20 \& 10 = 0$, $10 \& 10 = 10$, and $-5 \& 3 = 3$. g17emu reads printed 14 as $a > b$ and refuses inputs at or above 2^{31} , matching the ledger’s open signedness for code 0. Field 1 (printed 15) can be relied on as the bit test, never equality. *Open:* the bit test on operands with high bits set (the test used $x < 32$), and code 0’s signedness above 2^{31} . 16-bit compare-select, four registers (op11507) is plain equality; on separating cases it fits $a == b$ alone (MM [25.90](#)).

Flags, chains and immediates

- `&&` and `||` chain through flag-reading compares: 32-bit compare, AND-chained into a flag (op11310) $\text{flag} = (x \text{ cc } K) ? c16 : 0$, and the OR-chained compare flag $= (x \text{ cc } y) ? T : \text{flag_in}$ (measured, glossary; more chain forms in [appendix B](#)).
- `if (x == 0)` and `if (x != 0)` compile to the same flag compare with printed code 12 and differ only in the exec form's invert bit (decoder, isa/g17-exec-mask.txt, [section 5.1](#)).
- Apple's compiler never emits $\geq$ or $\leq$ against an immediate: $x \geq K$ becomes $x > K-1$ and $x \leq K$ becomes $x < K+1$ (isa/g17-scalar-isa.toml).
- A flag's numeric value is form-specific. Read back through flag to 16-bit register (op14099) after one compare form (op10379, no glossary entry), a flag read 1 for true and 2 for false (ledger/notes.toml); the authored OR-chained compare read 2 for true and 1 for false (isa/g17-condition-codes.toml). Both are measured on their own forms. Use a flag only as a predicate.
- The flag compare's /10 form has a second operand layout selected by operand 1's 2^{31} bit (1,421 vendor instances against 19,260); no source shape reaches it and its meaning is *open* (MM [25.73](#), MM [25.76](#)).

Floating-point compares

Every ordered relation is false when either operand is NaN, `!=` is true on NaN, and `-0` equals `+0`, for float, half and bfloat (measured, isa/g17-comparison-semantics.json). fp32 compare-select, fmax/fmin (op9700) is the four-register float select; its codes are 0 `==`, 1 `<`, 2 `>`, 4 `!=`, 5 `≥`, 6 `≤`, and Apple's compiler writes 3 for `min(float, float)` and 7 for `max` (agxforge/g17/cc.py). For 3 and 7 the hardware tests a first: a NaN a selects y, then a NaN b selects x, and with both NaN it returns y. A model that tested b first disagreed with the receipts on lanes 7 and 23 (measured, MM [25.145](#)). `llvm.maxnum(x, 0)` returns 0 for NaN x, so `relu(NaN) = 0` (MM [6](#)).

4.3 fp32 arithmetic

fp32 add (op998), fp32 multiply (op3290) and fp32 fused multiply-add (op2190) are RNE. There is no subtract instruction: `a - b` is the add with the negate modifier on b, exact since negation is exact (Apple's compile, MM [25.180](#)). Literal operands select different forms (fp32 FMA, literal addend (op2192); fp32 FMA, literal multiplicand (op2198); fp32 FMA with two float immediates (op2200); fp32 add with float immediate (op1006)), and `a * 3.5f` selects a form distinct from `a * b` (MM [25.73](#)).

Two fp32 FMA forms were tested for single rounding, on inputs where fused and unfused evaluation round differently.

- The fp32 fused multiply-add: 0 of 16 rounding-witness lanes double-rounded (MM [25.90](#)). Its four-byte form needs the accumulator and destination in one register.
- The literal-addend FMA, from Apple's `fma(a[i], b[i], 1.0f)` over 4,096 lanes whose draws put the +1 inside the product's discarded bits: on the 3,186 lanes where once and twice differ, the fused model matches all and the unfused model misses all (measured, MM [25.184](#), isa/g17-rounding-results.json).

Subnormals are handled three ways. The fp32 ALU flushes subnormal inputs and results to zero (fadd, fmul, the immediate fma forms, the reciprocal); the tensor unit's fp16, bf16 and fp32-accumulator paths

flush nothing; and the measured half ALU forms keep half subnormals (section 4.4). An MMA can produce a subnormal that the next fp32 ALU instruction to read it flushes to zero (measured, MM 5, MM 17 rule 22). The flush keeps the sign. A GELU epilogue that passed within its bound at M16 to M64 failed in 57 of 65,536 elements at M512, all with x near -52 , where $1/(1 + 2^t)$ is about $3.6e-39$; the GPU returned -0 , and with FTZ in the reference every arm passed (measured, MM 25.109).

fp32 NaN results are the canonical `0x7fc00000` (g17emu EVIDENCE; MM 5). fp32 saturate (op904) is encoded as $\text{sat}(a + K)$ with $K = \pm 0$ and maps -0 to $+0$, values above 1 to 1 and negatives to 0 (32 lanes); NaN into it is unmeasured. *Correction:* the map's "fsat" is fine x -derivative, saturated (op1062), a cross-lane form (section 8.5); saturate is the form above (MM 25.90). fp32 truncate toward zero (op3818) keeps the sign of a zero result ($-0.3 \rightarrow -0$), and round to nearest even (op3770) is Metal rint.

`air.dot.vNf32` is an fp32 multiply of lane 0 then one fma per lane, left to right; a pairwise order differs from Apple's on 66 of 256 random lanes, while an fp64 model of Apple's chain agrees on all (MM 25.180). Under default fast math $x \neq x$ folds to false, so a NaN guard must be `isnan(x)` or `(as_type<uint>(x) & 0xffffffff) > 0x7f800000` (measured, MM 25.50).

4.4 fp16 arithmetic

The measured half forms run on 16-bit halves of the general registers, keep subnormals, and return the half canonical NaN `0x7e00`.

fp16 fma, register form (op798) (`ffma.f16.112`, 12 bytes) computes binary16 $a * b + c$ rounded once (measured, MM 25.196).

- Operands: destination 0; sources 2, 4 and 6; their lifetimes 3, 5 and 7. A source is a low half (register id $425 + n$) or a high half ($281 + n$) of word register n ; Apple's own `fma(half, half, half)` reads a high half (r283). Authoring Apple's operands reproduces Apple's bytes, `3800060a2320a40245018000`.
- The test: 196,608 triples in three cases (the whole finite binary16 range by bit pattern, and two cancelling cases with 24,000 subnormal results each). 0 differ from binary16($a * b + c$) computed exactly; the multiply-then-add control differs on 6,246 to 54,951 per case; with both addends taken from the two halves of one packed word, 0 of 196,608 differ.
- Encoding holes (decoder): the low-half destination cannot author r18, r19, r26, r27, r50, r51, r58 or r59, operand 4 cannot author r17L and operand 6 cannot author r18L. The load-wait field is not located.

Worked example: exact 4-bit dequantization in two fp16 fmas (*this compiler*; executed bit-exact on hardware, MM 25.196). Integer code packs a 4-bit weight q into a half's mantissa under the bits of `1024.0h`, so the half's value is $h = 1024 + 2^k q$ ($k = 0, 4$ or 6 , masks `0x000F00F0` and `0x03C000F0`). One fp16 fma recovers q exactly as $h * 2^{-k} - 2^{10-k}$: every intermediate is a small integer, so the single rounding is exact. A second fp16 fma, `fma16(q, s, b)`, applies the scale and bias with one rounding and writes the weight straight into the half A fragment of a tensor tile. On a sample of 29,360,128 real weights the result equals the old fp32-then-fp16 dequant on every weight (the fp32 sum of a bf16 product and bias is already exact), so the model's logits do not move; the projection gets 10 to 12 percent faster because the dequant leaves the fp32 pipe (128 fp16 fmas per trip, none of the integer-to-float, fp32 fma or widening forms).

The mixed-width forms below are all measured.

- half add with float immediate (op776): $\text{half}(a + K8)$. A NaN input returns 0x7e00, not its payload (104 and 90 NaN lanes); subnormals are kept (a flushing model misses 965 lanes of an $h + 1/64$ kernel built to reach subnormal results); rounding once and through fp32 are the same function on all 65,536 halves and every positive immediate, checked exhaustively on the CPU, so this form needs no rounding test (MM 25.195).
- fp32 add, half result (op1014): $\text{half}(a32 + b32)$, one rounding, overflow to inf, NaN $\rightarrow$ 0x7e00, a tiny negative $\rightarrow$ -0. fp32 plus half, half result (op1015) keeps half subnormals. fp32 multiply, half result (op3306): a width receipt's 0x3e00 rules out a bf16 result (MM 25.145.1).
- Apple's compiler selects the fp16 fma (/12) for fma on half and a different form (op862/10, no glossary entry) for $a * b$ (MM 25.73).

4.5 Conversions

Table 4.2. Rounding and edge behaviour of the measured conversions.

Conversion	Form	Rounding and edges	Standing
fp32 $\rightarrow$ half	fp32 to half narrowing (op1016)	IEEE RNE; zero mismatches over 1,510,400 fp32 patterns, subnormal results included; NaN $\rightarrow$ 0x7e00	measured (MM 6)
fp32 $\rightarrow$ bfloat	fp32 to bfloat16 narrowing, unconfirmed (op1048) is the candidate	RNE except that an fp32 subnormal input returns a zero with its sign (5,879 of 5,912 subnormal patterns); NaN $\rightarrow$ 0x7fc0; denormal enable and fast-math off change nothing	function measured (MM 6, MM 17 rule 21); instruction identity unresolved
half $\rightarrow$ fp32	half to fp32 widening (op1004)	exact	measured
int32/uint32 $\rightarrow$ fp32	32-bit integer to fp32 (op11179)/10	RNE; operand 2: 4 unsigned, 5 signed	measured against 8 rivals (MM 25.90)
int16/uint16 $\rightarrow$ fp32	16-bit integer to fp32 (op11180)	exact; code 2 unsigned, 3 signed	measured (MM 25.195)
uint32 $\rightarrow$ half	u32 to half (op11182)	RNE; past half's range +inf, not 65504 (2,992 lanes)	measured (MM 25.184)
fp32 $\rightarrow$ int32/uint32	fp32 to 32-bit integer (op9320)/10	mode operand 1: truncate toward zero, saturate, NaN $\rightarrow$ 0	measured (MM 25.186)
fp32/half/bf16 $\rightarrow$ fp8	fp8 pack, vector, from fp32/half/bf16 (op13618)	RNE, no saturation, every NaN $\rightarrow$ canonical NaN, over all 2^{32} fp32 patterns	measured

Remaining conversions (16-bit integer to half, half to integer, uniform-destination forms, fp8 unpack) are in [appendix B](#).

Apple's float-to-int casts $(\text{uint})x$ and $(\text{int})x$ are fp32 to 32-bit integer (op9320) with code 4 or 5 and mode operand 1; the instruction bytes are 2f00001a2200ae02b003 and 2f00001a2200ae027001. Over 4,096 lanes each, drawn to cover fractions of both signs, ties, negatives, values past 2^{31} and 2^{32} , NaN, $\pm\text{inf}$ and ± 0 : for $(\text{uint})x$, truncate-and-saturate matches every lane, RNE-and-saturate misses 326 and truncate-and-wrap misses 1,974; for $(\text{int})x$, truncate-and-saturate matches every lane and RNE-and-saturate misses 653

(measured, MM 25.186). On (uint)(-5.5), which Metal leaves undefined, the instruction gives 0; this compiler now emits it in place of its own earlier decomposition (appendix A).

Because the conversion saturates, it maps NaN to 0, +Inf to 2,147,483,647 and -Inf to -2,147,483,648 on its own, and the usual “safe” maxnum/minnum clamp placed before it changes the NaN result: it sends NaN to the clamp’s lower bound (-127 for an int8 quantizer) (measured, MM 25.50, MM 17 rule 17).

Signedness codes are per form: 4/5 on the 32-bit integer-to-fp32 and fp32-to-integer conversions, 2/3 on the 16-bit integer-to-fp32 conversion. On the integer-to-float forms operand 2 is signedness: an unsigned code converts a negative int32 to a value near 2^{32} (MM 25.90). Canonical NaNs by destination: fp32 0x7fc00000, half 0x7e00, bfloat 0x7fc0.

4.6 Special-function instructions

rsqrt seed (op3850), hardware exp2 (op1272) and hardware log2 (op2570) are what Apple’s compiler selects for rsqrt, exp2 and log2, each at 10 bytes (MM 25.73); hardware reciprocal (op3658) and sqrt-safe rsqrt (op3978) complete the set. sqrt-safe rsqrt returns 1.0 at 0 where the seed returns +inf, so fast_sqrt’s $x * \text{rsqrt}(x)$ gives 0 at 0 where the seed would give NaN (MM 6). A trig-family op, function unmeasured (op3946) is emitted by cos, sin and cospi.

Each special function was dispatched with a sentinel over every output, compared with the correctly rounded value, and run twice (table 4.3). The columns come from three different input populations, so percentages should not be compared across columns.

Table 4.3. Accuracy and structure of the special-function instructions. Each column uses its own input population.

Function	1,024-input probe (MM 25.136.3)	Dense or full measurement	Structure
rsqrt seed (op3850)	974 correctly rounded, 39 one ulp high, 11 low	all 3×2^{24} inputs of three exponent bands: correctly rounded on 16,051,395 of 16,777,216 (parity, mantissa) keys, +1 ulp on 628,246, -1 ulp on 97,575; at most 0.82 ulp (MM 25.141.16)	exact function: $\text{seed}(x * 4^k) == \text{seed}(x) * 2^{-k}$ bit for bit, 16,777,216 of 16,777,216 at $k = -8$ and $k = +5$; $\text{op3850}(0) = +\text{inf}$
hardware reciprocal (op3658)	931 exact, 91 high, 2 low		within one ulp; a subnormal result flushes to signed zero (MM 25.109)
hardware exp2 (op1272)	632 exact, 387 high, 5 low	496,131 inputs on t in $[-127, 1]$: 64.2 percent exact, 35.6 percent +1 ulp, 0.17 percent -1 ulp; within $1.03\text{e-}7$ relative over t in $[-30, 0]$ (MM 25.144.2)	deterministic (0 of 496,131 differ across dispatches); no compact law
hardware log2 (op2570)			Apple’s compiler emits it; accuracy not measured

The rsqrt result came from sweeping whole bands of 2^{24} inputs at three exponents ($E = 127, 111$ and 137) and testing the law $\text{seed}(x * 4^k) == \text{seed}(x) * 2^{-k}$ across them. It held on every input at both shifts, so one table over (exponent parity, mantissa) reproduces the instruction everywhere on positive normals; the

table is `isa/g17-rsqrts-seed.npz` and reproduces all 3×2^{24} hardware values. The same method does not work for hardware `exp2`. The analogous law $\text{exp2hw}(t + n) == \text{exp2hw}(t) * 2^n$ holds on only 88 percent of inputs at $n = 1$ and 4 percent at $n = 40$, because the fp32 add that forms $t + n$ re-rounds the fraction and the instruction depends on its full input. The best constant-bias model, $\text{f32}(\text{exp2}(t) + 0.36 \text{ ulp})$, matches 97.1 percent, and a dense table would need about 10^9 entries (MM 25.144.2).

Kernels that use `rsqrt` and `reciprocal` are checked differently from kernels that use `exp2`.

- `rsqrt` and `reciprocal` are made correctly rounded on the GPU by an exact integer midpoint test. For a candidate a , the midpoint to its successor is $(2Ma + 1)2^{Ea-151}$; $1/\text{sqrt}(x)$ lies below it exactly when $Mx(2Ma + 1)^2 > 2^{452-Ex-2Ea}$, and $1/g$ exactly when $Mg(2Ma + 1) > 2^{301-Eg-Ea}$. The products are formed in 15-bit limbs with 32-bit multiply (`op10825`); a tie is impossible because $2Ma + 1$ is odd and at least 2^{24} ; and $y0 - 1 + [f \text{ above mid}(y0 - 1, y0)] + [f \text{ above mid}(y0, y0 + 1)]$ is correctly rounded whenever the seed is within one ulp. On the probe the corrected values equal the correctly rounded ones on 1,024 of 1,024 (measured, MM 25.136). Scope: positive normal inputs with a normal result; a seed two ulps off is not repaired. Alternatively, the seed alone is reproducible from its table.
- `exp2` has neither route, since no cheap exact test decides 2^t against a midpoint, and no compact model reproduces hardware `exp2`. A kernel that uses hardware `exp2` (attention softmax, GELU) is checked against an *enclosure*: each output must lie inside the interval a reference derives when `exp2` and `reciprocal` may each be $-1, 0$ or $+1$ ulp off (MM 25.109, MM 25.114, MM 25.144.2), and is never claimed bit-exact. Where bit-exactness is required this compiler uses `exp2_soft`, a fixed RNE sequence (clamp to $[-125, 125]$, split off `rint(t)`, a degree-7 Taylor polynomial in Horner form, the integer part added to the exponent field), within one ulp of 2^t and exact on 90.6 percent of 800,000 inputs (MM 25.136). For replay the emulator reads hardware `exp2` and hardware `reciprocal` from sparse tables of the inputs real programs feed them and refuses any other input (MM 25.145.4).

Library functions have these measured edges (MM 6): `air.fast_tanh(t)` returns NaN for $t \geq 44.364$; `air.fast_divide` and `air.erf` do not lower on this toolchain.

4.7 Scalar dependences and issue

Which producers need a wait, and what an unwaited consumer reads, are [section 9.1](#)'s: ordinary ALU to ALU is interlocked (52 of 52 adjacent pairs, MM 25.90), late values (loads, atomic returns and the other memory producers) are not, and whether an ALU consumer of an MMA result is interlocked is *open* (MM 6).

- Without the wait, scalar consumers read the old contents: a compare read the register's previous value (`isa/g17-scalar-isa.toml`), the six-byte fp32 fma got 0 of 32 lanes right (MM 25.90), and an indirect load differing in one wait bit got 1 of 32 lanes right without it and 32 of 32 with it (MM 25.145).
- Some forms cannot carry the wait. On 32-bit register compare-select, immediate arms (`op11372`) the load-wait position of the other forms is an opcode bit; on fp16 fma, register form (`op798`) the field is not located. *This compiler* routes such an operand through one waited copy, 32-bit add immediate (`op10279`) with 0 (`agxforge/g17/cc.py`, MM 25.196).
- A load whose result is never read is still in flight. Reusing its destination while the load was outstanding let it land late and overwrite the new value (rows 2 and 9 of 16 failed); a register an outstanding load will fill must not be written (measured, MM 25.114).
- Special-register reads publish a slot but need no wait in the measured form ([section 6.3](#)).

- Issue intervals, latency classes and the absence of fp32/int32 dual issue are [chapter 12](#)'s.

Safe assumptions. The numerics above hold for the forms and lengths named, within each measurement's stated domain. For a released register nothing holds beyond "zero or the old value".

Not known. NaN into saturate or truncate; negate and abs on the float select; the bit test with high-bit operands; the mode-SET layout of 32-bit compare with immediate into a flag (op10369); hardware log2's accuracy; the trig op's function; the integer-modifier signedness bit beyond two opcodes; half results where rounding once and through fp32 differ on forms other than half add with float immediate (op776) and u32 to half (op11182).

Evidence. MM 5, MM 6, MM 17 (rules 17, 21, 22), MM 24.8 (row T4), MM 25.50, MM 25.59, MM 25.73, MM 25.76, MM 25.77, MM 25.80, MM 25.90, MM 25.95, MM 25.109, MM 25.114, MM 25.136, MM 25.139.1, MM 25.141.16, MM 25.141.17, MM 25.144.2, MM 25.145, MM 25.145.1, MM 25.145.4, MM 25.180, MM 25.184, MM 25.186, MM 25.192, MM 25.195, MM 25.196; isa/g17-condition-codes.toml, isa/g17-comparison-semantics.json, isa/g17-scalar-isa.toml, ledger/notes.toml, ledger/claims.toml, tools/g17emu.py, agxforged/g17/cc.py.

5 Control flow and the execution mask

A G17 simdgroup executes 32 lanes in lockstep. Divergence is handled by an execution state per lane: instructions that change it (push a condition, flip it for an else, pop it), and two kinds of branch that move the program counter only when the state of the whole simdgroup allows. Most conditionals do not branch; Apple’s compiler predicates them.

The execution state behaves as a per-lane nesting counter, active when zero, not as a stack of masks (measured for the push and one-level pop; for the other forms, a model that balances every corpus program; [section 5.1](#)). Structured control flow is built on that counter, and three of its consequences first appeared as faulty programs. Code after a join runs on no lanes unless the join restores the state ([section 5.1](#)). A loop nested inside a guarded region needs its own push before the loop, or the latch’s pop removes the region’s level ([section 5.3](#)). A branch decides only whether the simdgroup skips work that no lane needs; which lanes run is set by the counter ([sections 5.2](#) and [5.3](#)).

Names used below:

- The exec family (one 4-byte encoding word, [section 5.1](#)): exec push, if, from flag (op582); exec pop, restore mask (op577); while-exec, set mask in place (op579); the inverting if (op583), the else forms (op575, op576) and the plain while (op578), which have no glossary entries; end of program (op684).
- Branches (10 bytes): conditional back-edge branch (op458), skip branch, no active lanes (op462), call (op450); return (op580) ends a subroutine.
- A **flag** is a per-lane predicate register (FLAG0-FLAG14) written by a compare such as 32-bit compare with immediate into a flag (op10369) ([section 4.2](#)). FLAGTRUE is a constant source that always holds.
- “Counter n” is the lane’s nesting counter.

5.1 Execution-mask instructions

The family shares one encoding word (decoder; agxforge/g17/cf.py, isa/g17-exec-mask.txt). Bit 4 of byte 0 separates end of program (op684, 0e 00 00 00) from the family; bit 4 of byte 2 separates exec forms from branches; kind = b0.5 + 2 × b0.6; and b2.1 inverts the predicate ([table 5.1](#)).

Table 5.1. Exec-family and branch opcodes by kind, inversion bit b2.1 and branch bit b2.4.

kind	b2.1 = 0	b2.1 = 1	with b2.4 = 1 (branch)
0 if	exec push, if, from flag (op582)	op583 (no glossary entry)	stays exec (op573 / op574, no glossary entries)
1 pop	exec pop, restore mask (op577)	op577 (b2.1 dead)	skip branch, no active lanes (op462)
2 else	op575 (no glossary entry)	op576 (no glossary entry)	conditional back-edge branch (op458)
3 while	op578 (no glossary entry)	while-exec, set mask in place (op579)	call (op450)

- The count is b0.7 (high) and b2.3 (low), and it is a code table, not an integer: 00 → 1, 01 → 0, 10 → 2, 11 → 3. An encoder that writes the count as a plain 2-bit number emits pop 0 when it means pop 1. Apple never emits count 3, nor a single pop deeper than 2.
- The predicate is b1[4:6], an 8-entry lookup: [74, 75, 76, 77, 78, 79, 3, 2], register ids for FLAG0-FLAG5 and then two entries in another register file, which Apple uses to push a level unconditionally (FLAGTRUE). Pop reads no predicate.
- Auxiliary operand 0 (bits b1.0, b1.1, b2.5-b2.7, b3.5, b3.7) takes only a few witnessed values per kind; its meaning is *open*, and hard-coding one value writes an instruction Apple never emitted 4.6 percent of the time.
- An encoder built from this field map re-encodes all 14,195 control-flow instructions in Apple’s corpus byte for byte. It reached that only after the auxiliary field was carried as a field; as a constant it scored 94.34 percent.

Worked example: an exec push and its join (decoded; the second executed). 1e 00 00 0e: b0 = 0x1e has bit 4 set, b0.5 = b0.6 = 0 (kind if), b2 = 0x00 (exec, not inverted), count code 00 = 1: exec push, if, from flag (op582), one level on FLAG0. 3e 03 40 0e: b0 = 0x3e gives kind 1, b2 = 0x40 has bit 4 clear, count code 00 = 1: exec pop, restore mask (op577), pop 1. The second word is the join; it was identified after a prediction failed (measured, isa/g17-scalar-isa.toml). A forward branch authored one instruction past the compiler’s own target, predicted to store 114, stored 0. A kernel was then built whose post-join code writes a second value that does not depend on the branch (table 5.2).

Table 5.2. Stored values with the join present and replaced by filler, on the taken and not-taken paths.

join	taken (n = 2)	not taken (n = 7)
present	(114, 31)	(1868193279, 96)
replaced by filler	(0, 0)	(1868193279, 96)

Without the join the branch-independent value was also lost, on the taken path only. With the output pre-filled with 0xDEADBEEF the sentinel remained, so the store did not execute. Code after a join therefore retires with no active lanes unless the join restores the state, and a branch must target the join.

The counter semantics, per lane and with n the instruction’s count, are these (measured for the push and one-level pop forms; whole-program for the rest; MM 25.145.2, agxforge/g17/releasecheck.py):

- **if n**: an active lane (counter 0) stays 0 if its condition holds, else becomes n; an inactive lane adds n.
- **else n**: 0 becomes n; n becomes 0 if the condition holds; any other value is unchanged.
- **pop n**: counter = max(0, counter - n).
- **while n**: an active lane whose condition fails becomes n (and waits for the matching pop n); otherwise unchanged.
- FLAGTRUE always holds; the inverting if, the b2.1 = 1 else (no glossary entries) and while-exec, set mask in place negate the condition. A condition is the lane’s flag value, which stays the same until a compare rewrites it.

Two observations rule out a push/pop stack in favour of the counter. Apple’s corpus contains 338 top-level loops of the shape while 2 / back edge / pop 2 with no push; a stack would pop levels never pushed, while the counter model balances all 6,594 corpus programs. And Apple over-pops on early-exit paths (in one probe a branch leaves a region two levels deep and lands on pop 2 then pop 1), which only a

saturating counter tolerates; that one is an inference from the correctness of Apple’s code (decoder, isa/g17-exec-mask.txt). This is the model Asahi/Mesa call r0l. For every executed receipt that uses a push and one-level pops, the counter and a stack agree. The in-place while and the two-level pop are needed by the imageblock kernels’ K loop and are admitted only on whole-program evidence. Nesting depth 1 to 6 with a store per level emits one if 1 per level and pops that sum to the pushes (depth 6: pop 2, pop 2, pop 2); else and while are net-neutral.

Correction: the 4-byte backward branch in isa/g17-scalar-isa.toml (de 6f f3 1e) is the first four bytes of the 10-byte branch below (agxforge/g17/cf.py; [appendix A](#)). Author branches from the 10-byte map.

In the branch encoding (decoder, verified on 65 of 65 branches) b2.0 is set, and a signed two’s-complement displacement has its sign at b9.6 and weights 2 through 2^{47} scattered over bytes 0–9 (no weight-1 bit, so displacements are even). The target is the branch’s own offset + displacement, not the end of the instruction.

5.2 Predication and if-conversion

Apple’s compiler predicates first (Apple’s compiler emits; isa/g17-exec-mask.txt, isa/g17-scalar-isa.toml). A single-sided if emits no control-flow instruction: the store becomes a predicated store form, or the region is if-converted to a compare-select ([section 4.2](#)). Two or three nested levels with one assignment each still if-convert. The exec machinery appears once a region has two sides, side effects at more than one level, or a loop. Forcing a real branch took a conditional block of eight statements or more; below eight the block is predicated and no branch is emitted.

Apple places skip branch, no active lanes (op462) directly after a region’s exec push, if, from flag (op582); it is taken when no lane of the simdgroup is active and lands on the region’s pop, so it never changes which lanes run. On a 512-threadgroup grid with a selector matching no arm, both skips were taken and the program wrote nothing (measured, MM [25.141.2](#)).

This project’s compiler lowers a conditional as pure predication by default (measured, MM [25.139.9](#), MM [25.141.2](#)), so every dispatch walks the masked-off arms: +2.4 μ s per dispatch for two arms and +38 μ s per four-op chain in an uber kernel. With the skip branch emitted (opt-in), an uber arm costs what the plain kernel costs (131.2 against 131.0 μ s; 68.8 against 68.7).

For lanes that are masked off:

- A release inside a region acts only on the lanes that executed the releasing instruction: with the region gated on $t < 16$, a released value read 0 on lanes 0–15 only, and with the region gated on $t < 0$ nothing was released (measured, MM [25.95](#)), so a value released in one arm is dead only on the lanes that took that arm. [Chapter 2](#) holds the release rules.
- Load waits are per consumer, so a use after a join still waits for a load whose first consumer was skipped (this compiler, MM [25.141.2](#)).
- Cross-lane reads inside a partially active simdgroup are a separate hazard ([chapter 8](#)).

5.3 Branches and loops

The back edge, conditional back-edge branch (op458), is taken iff any lane of the simdgroup is active. Every backward branch in Apple's corpus is this branch (581 across 401 programs), every one of those programs also contains a while form, and Apple never emits call (op450) backward (decoder, isa/g17-exec-mask.txt). exec push, if, from flag (op582) gates the back edge; while-exec, set mask in place (op579) sets the state in place and does not gate it (glossary). The loop shape Apple uses and this compiler emits is a latch pop, compare, push, back edge, with one more pop at the exit (measured, MM 25.139.9).

A loop inside a region needs its own push, because the first trip's latch pop removes one level. At top level that is harmless, but inside a guarded region it removes the region's level, and the guarded-off lanes then run everything after the loop. The defect appeared when an uber kernel's masked-off arm stored threadgroup 0's rows; with a selector that matched no arm, both arms stored them. The design measured for loops starts with a preheader push, which the lowering had never emitted. This compiler now emits $\theta < 1$ and an exec push before a loop nested in a region, keeping every top-level loop's bytes (measured, MM 25.139.9).

A loop whose trip count is a runtime value lowers to an unsigned compare against a capped count. The cap is required, since without it termination is not provable (measured, MM 25.90). Apple's compiler emits 32-bit compare with immediate into a flag (op10369)/10 and fp32 to u32 into the uniform/staging file (op9328)/10 for such a loop (MM 25.73).

A loop-carried value must not be released. A release on the counter in the cap compare left a kernel unable to reach either bound, and it ran until a reboot (measured, MM 25.116; the case is described in section 2.6). The same family of defect, a loop phi's entry value released before the loop, made every thread store as thread 0 (MM 25.142.7, MM 25.144.6). The rules (a back edge reads its header phi's latch values; refuse a release of any loop-carried register after the body's last write of it) are section 2.6's, and the dispatch hazard is section 1.5's.

Table 5.3 lists loops executed below Metal (measured; programs authored by this compiler and submitted without Metal, each checked against the exact expected output with guards, callbacks and recovery state; docs/g17-first-dispatch-trials.md).

Table 5.3. Loop programs authored by this compiler and run below Metal, each checked against the exact expected output.

Trial	Program	Result	What it establishes
Bounded per-lane GPU loop	88 bytes: each lane starts at its lane index and adds 7 per trip; one- and three-trip variants differ only at code byte 55 (compare immediate 0x80 against 0x82); 32-bit compare with immediate into a flag (op10369), exec push, if, from flag (op582), a 10-byte conditional back-edge branch (op458) at byte 62 with displacement -38 landing on the body at byte 24, exec pop, restore mask (op577), store, end	64 of 64 words exact: 7..70 for one trip, 21..84 for three (the back edge ran twice)	bounded cyclic control flow for this program and layout
Per-lane divergent loop bounds	146 bytes: bound loaded from A[lane], runtime compare with a cap of 3, conditional back-edge branch (op458) at byte 120 with displacement -82 to byte 38; byte-identical code under two input profiles	bounds 1,3 alternating gave 7, 22, 9, 24, ... 69, 84; reversed gave 21, 8, 23, 10, ... 83, 70: each lane computed lane + 7 × A[lane] beside neighbours with different trip counts	per-lane divergent trip counts in one simdgroup
Runtime loop cap and zero bound	the same 146 bytes	bounds 5,3: every lane i + 21 (over-cap lanes stopped at three trips); bounds 0,3: even lanes i + 7, odd lanes i + 21	the cap clamps; this emitted loop tests at the bottom, so a zero bound still runs the body once
Bounded loop inside a divergent region	140 bytes: every lane stores its index, then only lanes with lane < 32 enter a two-trip loop; pushes at offsets 50, 68 and 106, pops at 96, 120 and 132, conditional back-edge branch (op458) at 110 with displacement -38 to the body at 72	lanes 0-31 i + 14, lanes 32-63 i	the preheader push keeps the outer guard; not arbitrary nesting

The zero-bound result comes from this loop shape and is not a machine property. A compiler that needs zero-trip semantics must guard the loop or test before the body.

Apple’s own program that uses call and return (op580) runs 32 of 32 lanes correctly in this project’s image path, and this compiler inlines calls (whole-program, MM 25.90). Indirect calls are *open*.

Safe assumptions.

- The exec forms and branches Apple emits can be emitted with the field map of [section 5.1](#); they are byte-identical to code that executed. No novel exec encoding (count 3, an unwitnessed auxiliary value, a predicate index Apple never selects) has been executed (isa/g17-exec-mask.txt).
- The counter semantics hold for the exec push and the one-level exec pop (measured); for the in-place while and the two-level pop they hold only as whole programs use them. The else rule and the saturation are model statements consistent with all corpus programs, not separate measurements.

- Branch-taken conditions (the skip branch when no lane is active, the back edge when any lane is) are measured for the skip-taken case and the executed loops; the skip branch's not-taken behaviour under partial activity follows from the same programs.

Not known. The auxiliary operand's meaning; where the counter is held and its width; indirect calls.

Evidence. MM [25.73](#), MM [25.90](#), MM [25.95](#), MM [25.116](#), MM [25.139.9](#), MM [25.141.2](#), MM [25.142.7](#), MM [25.144.6](#), MM [25.145.2](#); [isa/g17-exec-mask.txt](#), [isa/g17-scalar-isa.toml](#), [agxforging17/cf.py](#), [agxforging17/releasecheck.py](#), [docs/g17-first-dispatch-trials.md](#).

6 Special registers

Lane index, simdgroup index, threadgroup position and grid position come from special registers, read with special-register read (op14059) into a 32-bit register or with special-register read, 16-bit (op14060) into one half. Every kernel reads at least one; the tensor path reads the lane index to generate fragment addresses.

Three separate numberings identify a special register:

1. the **hardware number** in the read instruction’s source field, a lookup that Apple’s compiler tables do not contain and whose field and values depend on the encoded length ([section 6.1](#));
2. the **register id** Apple’s decoder prints (an LLVM register id with a proposed SR_* name), numbered differently from the hardware field ([section 6.1](#));
3. the **slot-29 entry** in the kernel’s metadata, which declares the registers a kernel reads in a third, non-monotone numbering ([section 6.2](#)).

Metal builtins do not map one-to-one onto registers: several are computed from a register the kernel reads, so “the kernel for builtin B reads register R” does not mean R is B ([section 6.1](#)). Whether a read must be waited on is [section 6.3](#).

“SR130” means the special register whose 4-byte read form carries byte 0x82 (130 decimal) in its source field; MM writes registers this way. “Number” is that byte with bit 7 removed (0x82 & 0x7f = 2). A “builtin” is a Metal attribute such as `[[thread_index_in_simdgroup]]`.

6.1 Register numbers and builtins

The read instruction’s source field holds a hardware number. The operand is declared as a special-register class, but its value fits no base-plus-index rule in any of the project’s encoding domains. It was recovered as a lookup by requiring each field bit to be constant across all instances that print the same register and to differ across registers, then checking the result is a bijection (decoder, isa/g17-special-register-numbers.txt). The numbering is the hardware’s own: threads-per-threadgroup x, y, z are consecutive numbers 24, 25, 26, while their LLVM ids are 55, 57, 59. Field position and numbering depend on the form:

- 4-byte form (1,116 corpus instances): byte 1 bits 0–6; byte 1 bit 7 is set in every witness and is not part of the number;
- 8-byte form (154 instances): byte 1 bits 0–1 and byte 3 bits 3–6, with a different numbering (lane index is 30, not 2). The 8-byte form is special-register read, 16-bit (op14060) (Apple’s compiler emits it at /8, MM [25.73](#)); Apple’s decoder reports its byte 1 as the register, so an 8-byte read of the simdgroup index prints 131, not 133 (agxforge/g17/mdgen.py). Key an 8-byte read by the decoder’s register name, not by byte 1.

Only registers on the observed list may be emitted; nothing predicts the number of a register the corpus never read (*this compiler*, agxforge/g17/cc.py).

[Table 6.1](#) is the reference list of special registers. Builtin is the Metal attribute whose one-builtin compile reads the register (compile-witnessed, isa/g17-special-registers.json); the SR_* names are proposed

from Apple’s library vocabulary and are not themselves witnessed. “Corpus reads” counts decoded reads over 679 objects and the read form used, 32-bit or 16-bit (decoder, isa/g17-special-registers.toml). Apple’s corpus defines 200 special registers (66 named SR_*) and reads 26 of them, counting flags.

Table 6.1. Special registers by 4-byte read byte, with their numbers, decoder id, builtin, corpus reads and standing.

4-byte byte (SR)	Number (4-byte / 8-byte)	Register id, proposed name	Builtin and meaning	Corpus reads	Standing	Source
0x82 (130)	2 / 30	45, SR_SIMD_ELEM	thread_index_in_simdgroup: lane index 0–31 (also what thread_index_in_quadgroup compiles to read)	183, 16-bit	measured: drives tensor address generation in every executed tensor kernel	MM 0.8
0x85 (133)	5 / 31	46, SR_SIMD_GRP	simdgroup_index_in_threadgroup	4, 16-bit	measured: tile ownership in executed multi-simdgroup kernels; this compiler masks it & 3 for up to four simdgroups	MM 0.8, MM 25.93, agxforge/g17/cc.py
0x94 (148)	20 / 23	37, SR_PVQUAD	not witnessed as a builtin	0	decoder only	numbers file
0x95 (149)	21 / 24	38, SR_PVSIMD	lanes before this one; equals the lane index with every lane active	15, 16-bit	measured with every lane active only	MM 25.145.1
0x96 (150)	22 / -	65, SR_TVQUAD	not witnessed	0	decoder only	numbers file
0x97 (151)	23 / 41	66, SR_TVSIMD	active lanes in all; 32 with every lane active	15, 16-bit	measured with every lane active only	MM 25.145.1
0x98–0x9a (152–154)	24–26 / 35–37	55, 57, 59, SR_TG_X/Y/Z_SIZE	threads_per_threadgroup.x.y.z	x 3, y 4, z 3, 16-bit	x: whole-program (sources reading 152 match Apple’s GPU output); y, z compile-witnessed	MM 25.149.1, agxforge/g17/mdgen.py
0x9c–0x9e (156–158)	–	54, 56, 58, SR_TG_X/Y/Z	threadgroup_position_in_grid.x.y.z	x 543, y 3, z 4, 32-bit	x measured (every tensor body reads it); y, z values compile-witnessed	MM 0.8, MM 25.145.1
0xa0–0xa2 (160–162)	–	61, 62, 63, SR_TP_IN_GRID_X/Y/Z	thread_position_in_grid.x.y.z	x 73, y 32, z 34, 32-bit	x measured; y, z values compile-witnessed	MM 0.8, MM 25.145.1
0xa4–0xa6 (164–166)	36, 37, - / 16–18	26, 27, 28, SR_LOCAL_X/Y/Z	thread_position_in_threadgroup.x.y.z	x 1, z 1, 16-bit	x measured; y, z values compile-witnessed	MM 0.8, agxforge/g17/cc.py
0xa7 (167)	39 / 15	25, SR_LIN_ID	thread_index_in_threadgroup	13, 16-bit	compile-witnessed	json, numbers file
0xa8, 0xaa (168, 170)	40, 42 / 32–34	51, 52, 53, SR_TG_DISP_X/Y/Z_SIZE	51 is threadgroups_per_grid.x; y and z have no witnessed builtin	3 each, 16-bit	compile-witnessed (x only)	json, tools/g17emu.py

Table 6.1, continued

4-byte byte (SR)	Number (4-byte / 8-byte)	Register id, proposed name	Builtin and meaning	Corpus reads	Standing	Source
0xac, 0xad (172, 173)	44, 45 / -	23, 24, SR_LIBXDIM/YDIM	unknown	–	decoder only	numbers file
0xea (234)	106 / -	71, SR_VRID_SET	unknown	–	decoder only	numbers file

“Values compile-witnessed” means the register number and builtin are witnessed by compilation, but no dispatch with more than one threadgroup or thread along that axis has run on hardware; on an axis of extent 1 every reading agrees (MM 25.145.1).

No register holds `quadgroup_index_in_threadgroup` or `simdgroups_per_threadgroup`; both are computed. The first reads SR_SIMD_ELEM and SR_SIMD_GRP (slot 29 carries [52, 53, 80] with its store’s grid position, `agxforge/g17/mdgen.py`), and the second expands to the whole threads-per-threadgroup vector, because it is computed from it (`isa/g17-class-bytes.txt`, `tools/g17synth.py`). So a one-builtin compile lists the registers its builtin is computed from, and that list alone does not identify any of them. *Correction*: the early `isa/g17-scalar-isa.toml` names bytes 0x82 and 0x98 after these two builtins; they are the lane index, measured by every executed tensor kernel that generates fragment addresses from it, and threads-per-threadgroup x (MM 0.8, `isa/g17-special-register-numbers.txt`, `agxforge/g17/cc.py`; [appendix A](#)).

Not known. What SR_PVSIMD holds when lanes are inactive. Measured with every lane active, SR_PVSIMD reads “lanes before this one” (the lane index) and SR_TVSIMD reads 32; a vote kernel built on them matches Apple’s `simd_prefix_exclusive_sum(1)` (MM 25.145.1). Separately, Apple’s divergent `simd_broadcast_first` reads register 38 (SR_PVSIMD) with the 16-bit read as the first of four instructions, and MM 25.144.6 describes it there as “the active-lane mask”; that role is inferred from the decoded sequence, not measured. With any lane inactive, a count of active lanes before this one and a mask are different values; neither reading may be assumed under partial activity, only the full-mask values. This compiler’s metadata author refuses 149 and 151 in any case, because they have no measured slot-29 entry (MM 25.115.1).

The 16-bit read writes only the half it names. With $y = 7t + 0xABCD0000$ live, the read wrote y ’s low half with the lane index and y then read `0xABCD0000 | lane`, three runs identical: the read does not zero-extend (measured, MM 25.145.4, `isa/g17-execution-halfwrite-results.json`). The probe was run for a delivered 2,048-row attention kernel that wrote a register’s low half with the 16-bit read, then shifted the whole register; it could be bit-exact on hardware only if the stale high half read as zero, either through zero-extension or because a released half reads zero. With zero-extension ruled out, that kernel depended on its released high half reading 0, which it did in every run but which is not a rule ([section 2.5](#)). The compiler now shifts only the low half (16-bit shift right, 32-bit result (`op17016`)) (MM 25.145, MM 25.145.1).

Two further constraints apply when emitting reads:

- The 4-byte read’s destination field is four bits (r0-r15), so read destinations compete for the narrow register pool (decoder and this compiler, MM 25.110; [chapter 2](#)).
- Reading threadgroup position, thread position in threadgroup, SR_PVSIMD, SR_TVSIMD, SR_SIMD_ELEM or SR_SIMD_GRP clears the per-kernel “every thread independent” metadata flag (Apple’s compiler emits, 47 one-variable probes, `agxforge/g17/cc.py`; [chapter 11](#)).

6.2 Slot-29 metadata numbering

The kernel’s metadata (per-kernel slot 29) lists the special registers the kernel reads, but in its own numbering. It is a separate namespace from the hardware numbers above (MM 0.8); the metadata format is [chapter 11](#)’s. [Table 6.2](#) gives the known entries.

Table 6.2. Slot-29 metadata entries for the special registers.

Register (4-byte byte)	Slot-29 entry	Standing	Source
SR_SIMD_ELEM (130)	52	measured for the active tensor form: 1,039 exact (130, 156) objects all carry [0, 52]	MM 0.8, agxforge/g17/mdgen.py
SR_SIMD_GRP (133)	53	compiled alone it carries [53]; with grid position [53, 80]; the multi-simdgroup tensor class (52, 53)	MM 25.93, agxforge/g17/mdgen.py
threads_per_threadgroup.x (152)	4	one object reading 152 alone carries [4]; five sources reading it with others agree and match Apple on hardware	MM 25.149.1, agxforge/g17/mdgen.py
threadgroup_position_in_grid .x .y .z (156–158)	0, 1, 2	measured	MM 0.8
thread_position_in_grid .x .y .z (160–162)	80, 81, 82	measured	MM 0.8
thread_position_in_threadgroup .x .y .z (164–166)	48, 49, 50	measured	MM 0.8

- The bases are a lookup. $156 \rightarrow 0$, $160 \rightarrow 80$ and $164 \rightarrow 48$ are not monotone in the register number, so entry = register - 80, which fits the middle row, is refuted by both others. Only the axis increment within a base is arithmetic.
- The SR130 entry depends on the read form. Decoder forms whose first printed register byte is also 130 but whose full identity (byte 1, width, byte 3) differs map differently, so no universal $130 \rightarrow 52$ rule exists (MM 0.8).
- For 43 sources the slot-29 law predicted the vector in Apple’s own compile exactly in 37, and the other 6 named 152 or a five-register set, which the later entries cover (MM 25.149.1).
- *Correction:* MM 0.8’s table lists SR_SIMD_GRP as “not generalized”, with no slot-29 entry; the entry is 53 (MM 25.93, agxforge/g17/mdgen.py; [appendix A](#)).

6.3 Read latency and waits

A special-register read publishes a scoreboard slot (slot semantics: [section 9.1](#); field positions: [section 3.4](#)), and Apple’s compiler waits on it, but a read that publishes on slot 0 is readable by the next instruction with no wait (measured, MM 25.117, MM 25.121). Apple’s wait is a scheduling choice, and its measured cost is within run-to-run spread.

An early ledger entry recorded that a store reading a special-register value directly stored 0, and attributed it to the store taking a value that had not yet arrived. This compiler then waited before such stores, and MM 6, MM 0.8 and MM 25.90 list the read among late values for that reason. A latency probe later

tested that explanation against others (tools/g17srlatency.py, every arm at one threadgroup and at 8,192 groups, three rounds, predictions committed; isa/g17-execution-sr-latency-results.json, 90 of 90 rows reproduced); [table 6.3](#) gives the arms.

Table 6.3. Arms of the special-register latency probe and what each store wrote.

Arm	Store	Stored
d0 to d4	$C[t] = t$ straight off the read, 0 to 4 independent instructions between	0
d0_wait	d0 with the store waiting on the read's slot	0
sr5	read writes R5; store value R5, index R4 (distance 0)	t
same_release	$O[u] = u$ with u from an ALU, value and index both R16 (no special register at all)	0
same_op6_cleared	that store with only the index's lifetime $16 \rightarrow 0$	u

Neither distance nor a wait fixed the failing arms, and distance 0 worked once the value and the index were different registers. Every failing arm used one register as both the store's value and its index, with the index slot releasing it: operand 6 = 16 on plain 32-bit store (op17229) clears the register before the value is read. The defect was this aliased release ([chapter 2](#)), which breaks any value stored at its own index, whether or not it came from a special register.

Apple's compiler waits uniformly (measured, MM [25.121](#)), after all 27 special registers it reads and at every distance, including 218 first consumers four or more instructions later; 3,496 of 3,497 first consumers at distance 1 wait, and 8,808 of 8,825 first consumers over 6,594 programs (MM [25.144.8](#)). A chain of 4,000 reads with every consumer waiting ran 1.092 ms against 1.096 unwaited at the full grid and 0.1528 against 0.1521 ms at one simdgroup, inside run-to-run spread, every lane right. The waited encoding is operand 1 bit 24 (byte 1 bit 2) of 32-bit add immediate (op10279) and 32-bit add (op10282). Delivered programs from this compiler read special registers unwaited 6,604 times and are bit-exact on hardware (MM [25.145](#)).

Safe assumptions.

- A slot-0 special-register read can be consumed at distance 0 without a wait. The evidence covers special-register read (op14059) values consumed by stores and adds, and both read forms in delivered programs.
- Another fill slot cannot be assumed to behave the same. The same 4,000-read program with each read publishing on slot 7 (operand 1 bits 20–23 = 8) stored nothing, waited or not. The read uses the fill slot it is given, and for this read slot 7 does not behave like slot 0 (measured, unexplained, MM [25.121](#)).
- A wait on a slot-0 read is harmless if a scheduler prefers uniformity.

Correction: “a read_sr fills a scoreboard slot like a load, so its first consumer must wait on it” (MM [6](#), MM [0.8](#), MM [25.90](#)) is superseded for slot-0 reads by MM [25.117](#) and MM [25.121](#).

Evidence. MM [0.8](#), MM [6](#), MM [25.73](#), MM [25.90](#), MM [25.93](#), MM [25.110](#), MM [25.115.1](#), MM [25.117](#), MM [25.121](#), MM [25.144.8](#), MM [25.144.6](#), MM [25.145](#), MM [25.145.1](#), MM [25.145.4](#), MM [25.149.1](#); isa/g17-special-register-numbers.txt, isa/g17-special-registers.toml, isa/g17-special-registers.json, isa/g17-scalar-isa.toml, isa/g17-class-bytes.txt, agxforge/g17/cc.py, agxforge/g17/mdgen.py, tools/g17emu.py, tools/g17synth.py.

7 Memory

A G17 program reaches memory through ordinary per-lane loads and stores. Each instruction names a base (a device binding, threadgroup memory, the imageblock, or a register-held pointer), an index (a register or an immediate), a byte displacement and an element width; the width of the access is fixed by the *opcode*, with no width field. The tensor unit’s “tensor loads” are per-lane gathers of the same kind, at addresses that ordinary ALU instructions compute, through the same load and store machinery and the same eight-slot scoreboard (MM 7; [section 10.8](#)).

Every memory access has an *address contract* (which bytes are touched) and an *arrival contract* (when the value exists in a register or in memory), and the hardware checks neither. A wrong binding rank stores nothing and returns the fill value; a load far past its buffer returned 0 at every tested offset; a consumer that does not wait on a load’s scoreboard slot reads the register’s previous contents; a released imageblock coordinate collapses every lane to element 0; an undeclared imageblock reads 0; a mis-encoded texture coordinate reads texel (0, 0). All of these completed with status 0 and no fault. This chapter calls a rule safe only where a control could have failed it, and such controls showed that several fields first read as address fields are arrival (scoreboard) fields. The arrival contract is [chapter 9](#)’s subject ([section 9.1](#)); this chapter covers the address side and the storage kinds.

Terms used below:

- *Rn*: 32-bit general register n, named here by role (“R32, the per-lane index register”). Register-file rules, including the destructive *release* lifetime (modifier value 16), are [chapter 2](#).
- *op0* and *op4*: the decoder’s tags for the uniform register file and for the message staging file that texture messages read. A *binding expression* is how the decoder prints a base: `bin(op0, const(K), 8)` for a device base, `bin(op0, const(K), 2)` for a threadgroup base; K is the binding constant and the last number the base’s scale.
- *Binding rank*: a buffer’s position in the binding list the program image declares ([section 7.1](#)).
- *Operand 1*: the control word after a memory instruction’s first operand. Bits 20–23 name the slot a producer fills (as slot + 1); bits 24–31 are the instruction’s own wait mask ([section 9.1](#)).
- *Element code*: the access-width field of tensor and vector loads: 1, 2, 4, 8 or 16 bytes (MM 7, MM 25.37).
- *Immediate-address form*: a load or store whose index and displacement are both immediates ([section 7.3](#)).
- “Below Metal” means a pure IOGPU submission with no AGXMetal driver loaded (docs/g17-first-dispatch-trials.md; the interface is [chapter 13](#)).
- Encoding and field maps are [chapter 3](#); metadata and the constant-program format [chapter 11](#); bandwidth, latency and instruction-fetch numbers [chapter 12](#).

7.1 Address computation

A register-indexed access forms its address as follows (measured on the 128-bit vector load and through the emulator’s byte-for-byte agreement with shipped dispatches; the other source shapes below are what Apple’s compiler emits):

```
address = base + index_register x element_width + displacement      (width and displacement in bytes)
```

Element width scales only the index; the displacement is unscaled. A constant address has no index register, width 1 and a byte displacement. The immediate-address family (section 7.3) uses a different formula, and tensor addresses are this formula applied to an index register that ordinary arithmetic computes (below).

Two project files give conflicting answers on whether the displacement exists and what it counts. `isa/g17-addressing-rules.json` compiled `C[tid] = A[tid + k]` with `uint tid` for six element types and found that no operand of the load moved with `k`: Apple’s compiler instead added `k` to the index with 32-bit add immediate (`op10279`), and `k = 1, 2, 4, 8` encoded as immediate 1, 2, 4, 8 for `char` through `long`. Its conclusion was “the load carries NO displacement field” and `address = base + (index + displacement) × sizeof`. `isa/g17-address-formula.txt` compiled `f[i + 7]`, `w[i + 7]` and `L[i + 7]` and found a displacement operand carrying 28, 14 and 56: bytes, not elements. Both observations are correct for the source shapes they compiled. On hardware, setting the 128-bit vector load’s displacement to 4, 8, 12 and 20 moved the access by exactly that many bytes in 10 of 10 runs (MM 25.141.18), and the repository’s emulator, which models every register-indexed access as `index × element + displacement` in bytes, agrees byte for byte with the GPU on 316 of 316 and 172 of 172 shipped dispatches (MM 25.197).

Compile-side and decoder facts about the same operands:

- A constant address: `u[1]` is 4, `u[7]` 28, `u[64]` 256 for `uint` (Apple’s compiler emits, `isa/g17-address-formula.txt`).
- For `A[tid + k]` with an unsigned 32-bit index, Apple’s compiler puts `k` into the index with the add immediate (1 to 255) or, at 256 and above, 32-bit add (`op10282`) or add, 32-bit plus zero-extended 16-bit (`op10283`), and leaves the displacement 0 (Apple’s compiler emits, `isa/g17-addressing-rules.json`). The file does not record why; a reader may not assume the displacement can replace that add when an unsigned index sum may wrap.
- The add immediate’s 8-bit immediate is scattered across the word (value bits 0–1 at `byte1[0:2]`, 2–4 at `byte3[5:8]`, 5 at `byte5[7]`, 6–7 at `byte8[0:2]`); the map predicted 15 never-compiled values byte for byte (decoder, same file).

Correction: “the load carries no displacement field” (`isa/g17-addressing-rules.json`) holds only for its unsigned `tid + k` sweep. The field exists and counts bytes (MM 7, MM 25.141.18, `isa/g17-address-formula.txt`).

Rules a compiler has to follow (Apple’s compiler emits, `isa/g17-address-formula.txt`, `isa/g17-addressing-rules.json`):

- *No index-scale field.* `u[i * 4]` emits the same load as `u[i]`, with the multiply in a separate ALU instruction.
- *Width is the opcode.* `char`, `short` and `half` use 16-bit load (`op12646`) and 16-bit store (`op17193`); `int` and `float` use plain 32-bit load (`op12682`) and plain 32-bit store (`op17229`); `long` uses plain 64-bit load (`op12691`) and the store `op17238` (no glossary entry); `int4` uses 128-bit vector load (`op12709`) and 128-bit vector store (`op17256`).
- *Index width is the opcode too.* A `ulong` index selects immediate-address one-word load (`op12688`), preceded by a 64-bit shift pair that scales the index.
- *A runtime base plus index* (`A[j + tid]`, `j` loaded) is a load of `j`, a 32-bit add, then the indexed load.
- *Misalignment* does not select a form: an `int` at byte offset 1 through a `char`-pointer cast compiles to the same plain 32-bit load and store; whether hardware permits it for scalars is open. A vector load displaced by 4 bytes (not 16-byte aligned) “lands whole” (measured, MM 25.141.18).

- No negative displacement was ever emitted (the compiler folds negative offsets elsewhere); the field's signedness is open.

The address is formed in 64 bits (measured, `isa/g17-predication.json`). A load of `A[1073741824 + tid]`, 2^{32} bytes past a 256-word buffer, was chosen so that a 32-bit byte wrap would land back inside the buffer; it returned 0, not the wrapped word ([section 7.5](#)).

A device base's binding constant is $K = 4 \times$ the binding rank, and the rank is the buffer's position in the list the image declares, not the Metal slot number: a kernel binding only buffer C makes C rank 0 (measured, `agxforge/g17/cc.py`; MM 0.8). The list order is internal bindings ascending, then user bindings ascending; reconstructed from the declaration it reproduces Apple's list in 10,964 of 12,044 corpus kernels (91.0%), never differing by order alone (Apple's compiler emits, `agxforge/g17/resource.py`). A wrong rank makes a store return its fill value at status 0 (measured, same file). Below Metal, declaring one unused buffer between A and C changed exactly one code byte, the store's descriptor operand, from 0x04 to 0x08; swapping two 64-bit entries of the argument block swapped which buffer a load read; and an entry pointed at an unviewed offset of an allocation, or at a different registered allocation, was followed by loads and stores alike (measured, `docs/g17-first-dispatch-trials.md`, "Authored device atomic below Metal" and "Input descriptor address selection" onward). Arbitrary addresses and other allocation classes are untested.

The 128-bit vector load takes its 14-byte form when it carries bit 37 or a displacement above 255 (decoder, MM 6). Tensor loads carry up to 255 bytes in the 12-byte form and 32,767 in the 16-byte form; past 32,767 the offset is folded into a per-tile base register (Apple's compiler emits, MM 25.114.3). This compiler does the same, visible as a kernel growing from 74 to 80 instructions, with no failure out to 1 MiB of leading dimension (measured, MM 15). An identical offset-bearing body was bit-exact at stream positions 1 to 12, after 0 to 4,096 bytes of padding and at offsets 2 to 40,002 bytes straddling the boundary; within that range there is no stream-position constraint (233 + 193 dispatches, MM 15). Tensor B offsets are measured from 2 to 47,104 bytes; beyond is outside the measured domain (MM 25.114.3).

Tensor addresses are ordinary arithmetic (measured, MM 7). Apple's library computes one per-lane index register, R32, the element offset of the lane's first element in a row-major tile, and its eight tensor loads read `[R32*2 + disp]` at byte displacements {0, 2048, 32, 4096, 6144} (eight rows, sixteen columns), each moving an 8-row by 16-column sub-tile. This arithmetic is what produces the fragment layout; the index formula and the fragments are [section 10.4](#). The row stride has no field either: a non-default stride changes the mode immediate from 241 to 2289 and moves into the index (decoder, MM 7); a stride built as $K \times 2$ regardless of element size was wrong, with no error reported, on 1,024 of 1,024 elements of a $32 \times 32 \times 64$ int8 product (measured, MM 17 rule 6; [section 10.8](#)).

7.2 Load and store forms by width

[Table 7.1](#) lists, by width, the forms this chapter uses; [appendix C](#) lists every memory opcode. Lengths are those observed in Apple's 6,594-program corpus (`isa/g17-wait-mask-census.json`), with lengths the cited section adds. Operand orders are given where the forms are explained: the plain 32-bit store's are value 0, operand 1 at 1, index 5 and index lifetime 6 ([section 9.1](#)), the threadgroup forms' in [section 7.6](#) and the imageblock forms' in [section 7.7](#).

Table 7.1. Load and store forms used in this chapter, by width, with observed lengths in bytes and standing.

Glossary name	Opcode	Lengths	Width and notes	Standing
plain 32-bit load	op12682	8, 10, 14	4 bytes, element code 4	measured (MM 25.197; below Metal)
16-bit load	op12646	8, 10, 14	2 bytes (half, bf16, short, char); optional high-half destination	measured (MM 25.141.3)
plain 64-bit load	op12691	8, 14	8 bytes into a pair, element code 8	measured (glossary)
three-component vector load	op12700	8, 14	12 bytes into a 3-register tuple; the access steps 16	measured (MM 25.90)
128-bit vector load	op12709	8, 10, 14	16 bytes into a 4-register tuple, element code 16; byte displacement	measured (MM 25.141.18)
immediate-address one-word load	op12688	8, 10, 14	word at (index + displacement) × element; mask 2066	measured (MM 25.177)
tensor load, half/bf16	op12674	12, 16	two words (four half/bf16), element code 2; immediate element mask	measured (MM 7)
masked tensor load, half/bf16	op12675	12, 16	as op12674, with a 16-bit mask register	measured (MM 7)
int8 tensor load, one word	op12656	12, 16	one word (four int8), element code 1	measured (MM 7)
plain 32-bit store	op17229	8, 10, 14	4 bytes (sub-form 00)	measured (MM 25.117, MM 25.145.4)
128-bit vector store	op17256	8 (corpus)	16 bytes from a 4-register tuple; uses every wait bit 24–31	measured (MM 6)
store, sub-form 01, texel/word-slot	op17235	8, 10, 14	one word at an immediate address; reads a texture fetch; mask 2066	measured (MM 25.62)
two-component store	op17244	8, 10, 14	words k and k+1 from a pair; mask 2098	measured jointly with immediate-address four-word store (op17262) (MM 25.177)
tensor store	op17257	10, 12, 16	four words; immediate mask	measured (MM 6, MM 7)
masked tensor store	op17258	10, 12, 16	four words; mask register, bit j enables word j	measured (MM 7)
threadgroup-memory load	op12364	8, 14	2 or 4 bytes	measured (section 7.6)
threadgroup-memory store	op13288	8, 10, 14	2 or 4 bytes	measured (section 7.6)
imageblock load, 32-bit	op12151	14	one 32-bit member	measured (section 7.7)
imageblock store, 32-bit	op13075	14	one 32-bit member	measured (MM 25.96)

Also used, outside [table 7.1](#): plain 64-bit load, pair address (op12673), which Apple’s compiler emits to load pointer + immediate from a register-pair base; device load, register address (op12073), a runtime 32-bit address for thread-private arrays (Apple’s compiler emits, MM [25.39.2](#)); tensor load, fp32 (op12692), element code 4 (measured, MM [25.37](#)), and the masked twins of the int8 and fp32 tensor loads; 16-bit store (op17193, lengths 10 and 14, measured, glossary) and the 64-bit store op17238 (no glossary entry; lengths 8 and 10; Apple’s compiler emits); word store at word index (op17202), measured in the packed int8 epilogue (MM [25.105](#)); the rest of the immediate-address family (section 7.3); the spill stores ([chapter 2](#)). Two forms seen only in MLX’s quantized matmul have an open address space: MMA operand load, space unconfirmed (op12352) and dequantized-block store, likely threadgroup (op13276) (MM [25.144.1](#)). Decoder-only and unmodelled forms are in MM [25.88](#) and MM [25.192](#). The repository’s encoders round-trip 1,145 of 1,190 and 3,419 of 3,793 loads and stores; the unclaimed sub-forms are named in MM [15](#). 132 memory opcodes declare implicit uses of SR_LOCAL_X and SR_LOCAL_Y (op12118 to op12135 among them, no glossary entry for either); none occurs in the shipped corpus (decoder, isa/g17-local-indexed-memory.txt).

7.3 Immediate-address and vector forms

The immediate-address family is measured in MM [25.177](#). Its operands, in the order of store, sub-form 01, texel/word-slot (op17235), are value or destination, operand 1, mask, binding, 0, index, displacement, element; the address is the word at $(\text{index} + \text{displacement}) \times \text{element}$ of the binding, with index and displacement both immediates. The mask field is a component mask in bits 4–7 above 0x802: 2066 one word, 2098 two, 2162 three, 2290 four (bits 4–7 = 1, 3, 7, 15), filled from the tuple’s lowest register up; 2065 is the halfword form (MM [25.195](#)). Members: immediate-address one-word load (op12688), immediate-address two-word load (op12697, MM [25.184](#)), immediate-address three-word load (op12706), immediate-address four-word load (op12715), two-component store (op17244), immediate-address three-word store (op17253), immediate-address four-word store (op17262), 16-bit store at an immediate address (op17199). isa/g17-addressing-rules.json proposed the same $(\text{index} + \text{displacement}) \times \text{element}$ equation for the register-indexed form; it holds for this family and not for that one.

Worked example: a wrong-width model of this family (emulated, then executed on hardware; MM [25.177](#)). The first model of this family loaded each word with element size 1, keeping only its low byte, and still matched Apple’s kept GPU hash on 9 of 10 census sources, because the census inputs are small integers, $(7i + 3) \bmod 61$. Only the one source with float data failed; it read as zero (5.0 is 0x40a00000). Rerun with full-width random buffers (full-mantissa floats with exponents to $2^{\pm 20}$, uints over all 32 bits), all 10 sources match Apple’s GPU byte for byte, and each deformed model is refuted per form (sources agreeing, then sources where a low-byte load, a reversed load and a first-only store differ): one-word load 5 / 5, 3, 3; four-word load 6 / 6, 6, 6; four-word store 8 / 6, 6, 8; two-component store 2 / 1, 1, 2, receipted only jointly with the four-word store. The same protocol later receipted the three-word store (6 of 6, MM [25.192](#)) and the 16-bit store at an immediate address (3 of 3, MM [25.195](#)). A memory form is marked measured in this reference only if its test data could have exposed a wrong width.

In constant programs the base is a register pair holding a binding pointer that argument read, binding-table word (op14061) fetched; the address is pointer plus byte offset (measured, MM [25.179](#); [section 7.8](#)).

The vector forms:

- 128-bit vector load (op12709): byte displacement ([section 7.1](#)). Apple’s compile of MLX’s qmv carries 16, 32, 48 for $x[k+4]$, $x[k+8]$, $x[k+12]$. *Correction*: asm.encode_vec4 calls the field “units of eight bytes”; for loads it is bytes (MM [25.141.18](#)). The store’s units, witnessed only at 0, 16 and 32, are open.

- Tensor loads and stores move `popcount(mask)` consecutive elements into the tuple, lowest address in the lowest half (measured, `tools/g17emu.py EVIDENCE[12674]`). `int8` operands need `int8` tensor load, one word (`op12656`), not the generic four-word load (measured, MM 7).
- This compiler, measured: vectors placed past a 32,767-byte displacement have their base folded into the index register once, and two per-channel vectors must lie within one field of each other (MM 25.165); with loop-carried addresses (`ptr_addr`), later loads read one carried index at displacements 16, 32, 48 and instructions per trip fall 121 to 109, 184 to 171, 69 to 59 and 309 to 295 in four `qmv` loops, bit-exact against a `0x7f` sentinel (MM 25.141.18).

7.4 Masked and predicated access

- range mask, immediate width (`op612`): bit `j` of `dst` = `[lo <= src + j < lo + w]` for `j` in `0..3`, unsigned. The second immediate is a *width*. It was first read as an upper bound because nearly every witness had `lo = 0`, where the two readings coincide; two cases that separate the readings falsified the bound (measured, MM 7). `lo` and `w` are 8-bit, so lowerings compute tile-relative predicates.
- range mask, 16-bit source (`op621`): `dst` is a mask of `min(max(W - src, 0), 4)` low ones, `W` = operand 5, measured at `W = 16, 32, 48` and `64` (`src W-3..W` gives `7, 3, 1, 0`); the cap of four did not move with operand 3 at `8, 16, 32, 56` (`isa/g17-address-formula.txt`). *Correction:* the earlier fit (`0xFFFFFFFF >> src`) `& 0xF` is the `W = 32` case only. Every dispatch behind it had been authored on one witness whose operand 5 was 32, which coincides with the fitted saturation point. The source of the cap of four is open; there is no four-bit field.
- masked tensor load, half/bf16 (`op12675`) and masked tensor store (`op17258`): mask bit `j` enables word `j` of the lane's four-word access; bits above 3 are ignored; unwritten words keep their contents; only the declared `M × N` region is written and poisoned padding never leaks (measured, six patterns across 32 lanes, MM 7).
- Ordinary predicated access (measured, `isa/g17-predication.json`): a masked-off load suppresses the *value* (inactive lanes keep their sentinel); a masked-off store suppresses the *access* (seven masks); effects are per lane and nested masks intersect. Whether a masked-off load still issues its access is unknown: the intended observable was a fault, and no out-of-range load faulted (section 7.5). Apple's compiler emits predicated memory branchlessly between `exec-mask` instructions, with a skip branch for two-armed effects (chapter 5).

7.5 Out-of-range accesses

The masking experiment needed a load that faults, to separate a suppressed access from a suppressed value (measured, `isa/g17-predication.json wild_address`). Unconditional plain 32-bit load (`op12682`) reads at 4,194,304, 67,108,864, 268,435,456 and 1,073,741,824 words past a 256-word buffer (16 MiB to 4 GiB) ran through Metal, each in a fresh process so that a device loss would cost only that process. All completed with status 0 and returned 0. An in-bounds control returned its data; at the 2^{32} -byte offset the load returned 0, not the in-buffer word a 32-bit wrap would select. No fault occurred, so the masking question is still open.

Out-of-range store behaviour has not been measured. A timing driver once read and wrote up to 2.78 MB outside its buffers, with uncharacterised effect (MM 17 rule 34). The emulator refuses a store outside a buffer, a model bound (`tools/g17emu.py`).

Observed faults:

- Through Metal, one BIF0 read page fault came from a `regprobe_multi` tensor-kernel probe whose configuration was not recorded; a different process, sized from a sibling kernel’s layout, hung the GPU for about 320 s and the machine rebooted (MM 17 rule 34, as corrected; [appendix A](#)).
- Below Metal, malformed launch state produced BIF0 read page faults at unmapped addresses while `Submit` still returned 0, both callbacks ran and the recovery counter advanced: a guessed 2,048-byte threadgroup resource word (fault at address 0; the matched word then ran), an all-zero allocation 0 (fault at `0x10000010000`, its end), an oversized allocation layout (`0x100000d0000`, `0x100000d8000`) and a malformed code-placement packet (`0x80`) (measured, `docs/g17-first-dispatch-trials.md`). The requester of those reads was not identified.

Correction: MM 17 rule 34’s “only the accelerator’s own reads have faulted” is scoped to work submitted through Metal; below Metal, launch-state errors fault too.

Two silent failures resemble range errors. Undersizing a host dispatch’s completion scope (`dim^2 * esz`) returns wrong or zero data past that offset with no error (measured, MM 17 rule 5). A dead load landing late can overwrite a reused address register so that the next load reads out of range and returns 0 ([section 4.7](#), MM 25.114).

Through Metal, a stray plain 32-bit load inside the tested range returns 0 without faulting. A reader may not assume: that other load forms, resource kinds or offsets behave the same; that an out-of-range store is dropped; that reading an unmapped address is harmless below Metal; or that a 0 means “out of range”, since an unwaited load commonly reads 0 too ([section 9.1](#)). Keeping accesses in bounds is left to the program. Threadgroup accesses past 32,768 bytes and imageblock coordinates outside the tile are refused by the emulator only as a model bound; one imageblock read at `x = 32`, outside a 32-wide tile, completed and was not scored (MM 25.104.2).

7.6 Threadgroup memory

Size limits:

- 32,768 bytes per threadgroup is Apple’s compiler’s hard refusal; 48, 60 and 64 KiB are refused and 96 KiB drops the compiler connection (measured, MM 7). Metal itself does not check it for an authored container: a container declaring 1 MiB of static threadgroup memory built a pipeline that reports `staticThreadgroupMemoryLength = 1048576` (measured, MM 25.147, nothing dispatched). What the hardware does with an oversize declaration is unmeasured.
- 57,344 bytes (56 KiB) per core, inferred from a residency step: with 128 threads per threadgroup and 4,096 threadgroups, a static request of 28,672 bytes ran in 0.00304 s and 28,688 bytes in 0.00382 s, so two threadgroups fit at exactly half the budget and one above it (measured step, MM 25.49). Residency rules are [chapter 1](#).
- `maxTotalThreadsPerThreadgroup` stays 1,024 across threadgroup memory from 64 B to 32,768 B (measured, MM 25.47); `staticThreadgroupMemoryLength` is the value the kernel’s metadata declares (MM 25.147).

The forms are threadgroup-memory store (op13288) and threadgroup-memory load (op12364), plus a constant-index threadgroup load op12367 (no glossary entry) (Apple’s compiler emits, isa/g17-threadgroup-pair.txt). Their operands are value or destination, operand 1, width, base, 0, index, index lifetime, displacement, element. The base’s binding expression has scale 2 where device forms use 8 (notation list), with 0 exceptions in 25,553 corpus memory instructions (decoder, isa/g17-address-formula.txt). The address is base + index x element + displacement (the emulator’s model, measured on its receipts, tools/g17emu.py EVIDENCE[13288]).

Worked example (compiled by Apple, decoded; the same shape executed below Metal). From one kernel, so the two addresses agree by construction (isa/g17-threadgroup-pair.txt, a6-tgidx):

```
threadgroup-memory store (op13288), 10 bytes 0f040302030611c00020
  operands: R105 (the stored value), operand 1 = 0x80000010, 2066, bin(op0,const(4),2), 0,
            index register, 16, 0, 4      (op0: uniform register file)
threadgroup-memory load (op12364), 14 bytes 0f0403021a2610c0410080000000
```

The store’s operand 1 is 0x80000010: bit 31 set, a wait on slot 7, the slot this kernel’s loads fill (section 9.1); the trailing 4 is the element width. Below Metal, a program of this shape, each lane storing A[i] to scratch, a threadgroup barrier, then lane i reading slot i XOR 32, produced 64 of 64 values 11 * (i XOR 32) + 5 across two simdgroups, and a 2,048-byte allocation held 512 separately addressable words with every written value recovered (measured, docs/g17-first-dispatch-trials.md, “Parallel static-threadgroup launch mapping”). The static size is a launch resource word, not an instruction field (chapter 13): a guessed value for 2,048 bytes faulted, and the matched value Metal writes (0f00080c00000000) does not follow the simple size/8 pattern of the smaller sizes.

Further measurements:

- The threadgroup region is provisioned by an ordinary threadgroup load or store: removing the one threadgroup store made a threadgroup atomic write nothing (measured, ledger g17-the-threadgroup-atomic-subtracts-and-needs-a-region).
- The threadgroup load’s bit sensitivity: clearing byte4[3], byte4[4] or byte6[4] made the read return 0; byte5[5], byte8[6] and byte10[7] are inert; of the three-bit variants, 000 read correctly and 010, 100 and 110 read 0 (measured, isa/g17-execution-tgbits-results.json, -tgtrio-results.json). These are the positions of the 128-bit vector load’s fill-slot field (MM 6); that they are a slot field here fits, but is not established.
- A cross-simdgroup combine (word sg per simdgroup, one threadgroup barrier, an ascending fold) is exact for S = 2 and 4, sum and max (measured, MM 25.90). Ordering between simdgroups needs the barrier (section 9.2); hand-off costs are chapter 12.

7.7 Imageblock (per-core tile memory)

The imageblock is a per-threadgroup tile indexed by thread position and shared within the threadgroup (measured, isa/g17-imageblock-execution.txt): 32 lanes wrote 32 distinct values to their own pixels, and every lane that read pixel (0, 0) wrote lane 0’s value into a buffer prefilled with 0xdeadbeef.

The forms are imageblock store, 32-bit (op13075) and imageblock load, 32-bit (op12151), with operands [value, operand 1, width 18, member byte offset, 1, coordinate register, coordinate lifetime, dx, dy] (measured, MM 25.145.2). The coordinate register packs x in its low 16 bits

and y in its high 16; Apple’s compiler builds it with 16-bit plus 16-bit, 32-bit result (op10286). The declared pipeline allocated 128 bytes for 32 lanes of one 32-bit member (MM 25.104.1).

Worked example: the imageblock store’s wait bit (eleven rounds; compiled, decoded and executed on hardware; MM 25.96, MM 25.104). The target was a tensor GEMM, every byte from this compiler, that stages its output through the imageblock. Three defects gave the same symptom, a neighbour read returning the reader’s own value or 0.

1. *The released coordinate.* The store released its coordinate register (operand 6 = 16) while the following read reused it. Release is destructive (chapter 2): a one-bit same-run control, keep (0) against release (16), collapsed all 32 lanes to element 0 (measured, MM 6; isa/g17-imageblock-dseries-receipt.json). Fix: operand 6 from liveness.
2. *The dx operand.* With the coordinate kept, the x offset (operand 7 = 1) still had no visible effect in a tensor program. Apple’s compiler never uses it for a neighbour: it computes (tid.x + 1) & 31 with 16-bit add immediate (op10289) and 16-bit AND with immediate (op435) and reads with operand 7 = 0. This compiler now addresses the neighbour by an explicit column (measured, MM 25.104.2).
3. *Operand 1.* Apple’s own tensor kernel then read its neighbour on all 32 lanes and this compiler’s did not. The only field-level difference was the store’s operand 1: Apple’s 0x80000010 against this compiler’s 0x01000010. Swapping it alone moved the result both ways, and bit 31 decided: 0x80000010 and 0x81000010 passed, 0x00000010 and 0x01000010 failed. The reading recorded at the time was that bit 31 selects storage other lanes can read.

A later experiment refuted that reading. Bits 24–31 of the imageblock store’s operand 1 are its *wait mask* (bit 24 + s waits on slot s), the same field general stores and MMAs carry (section 9.1). In a census of all 57 corpus imageblock-store instances the bits equal exactly the slots the store’s sources are pending on: 47 need none and carry none, 8 carry bit 24 for the coordinate’s special-register reads, 2 carry bit 25. A prediction made before compiling, that an Apple kernel storing a freshly loaded value (its load fills slot 7) would carry bit 31, was confirmed. Table 7.2 gives the hardware runs of that kernel ($h[t] = 5t + 3$, a device barrier, $v = h[t \wedge 1]$, an own-pixel store of v, read back), three per arm.

Table 7.2. Imageblock store operand 1 against lanes that read their neighbour’s value correctly, in Apple’s kernel that stores a freshly loaded value. Three runs per arm.

store operand 1	lanes correct
0x80000010, as compiled (waits slot 7)	32 / 32
0x10, wait cleared	0 / 32, every lane 0
0x20000010, waits slot 1, which nothing fills	0 / 32, every lane 0

Without the right slot the store takes its value register before the load writes it; the failure is in the store, not in the neighbour read. Earlier deformations of operand 1 in Apple kernels had found the bits inert because those kernels stored ALU results, which need no wait. Round 11 routed the stored value through a waiting copy and kept the slot-0 wait for the coordinate (0x01000010); every arm was bit-exact (MM 25.104.4). A store’s bits 24–31 must name every slot its value and coordinate registers are pending on.

Correction: the “bit 31 selects coordinate-addressed staging” wording in MM 25.89’s round-9 entry, in MM 25.145.2 and in the tools/g17emu.py comments on the imageblock store and load is the superseded reading; bit 31 is the wait on slot 7 (MM 25.96, MM 25.104.3). There is no per-thread versus shared

distinction in the evidence. The emulator’s admission of cross-lane reads only for words stored with bit 31 is a conservative restriction of the model; no hardware result calls for it.

Declaration and API requirements (measured): an undeclared image reads 0 even when the tile is sized at encode time with `setImageblockWidth: C[0, 0..31]` were all 1.0, the read having returned 0, in three queries (MM 25.104). The pipeline must come from `newComputePipelineStateWithDescriptor::` a function-name pipeline reports `imageblockMemoryLengthForDimensions:` as 768 bytes at 32×1 ; `setImageblockWidth:height:` must match the threadgroup (`isa/g17-imageblock-execution.txt`). The declaring metadata is [chapter 11](#)’s.

The imageblock barrier is threadgroup barrier (op447) at scope 532, bytes 475100060000 (`threadgroup_barrier(mem_threadgroup_imageblock)`), which Apple’s own-element kernel carries; Apple’s compiler elided a source `simdgroup_barrier(mem_threadgroup)` between a one-simdgroup imageblock store and load (Apple’s compiler emits, `isa/g17-barrier-scopes.txt`, `isa/g17-imageblock-execution.txt`).

Not known. Use by more than one simdgroup or threadgroup (never witnessed, MM 25.145.2); the bit-4 flag (0x10) every store’s operand 1 carries (MM 25.96); the read’s operand-1 word Apple emits, 0x2000800000 (MM 25.104.2). Staging a fragment through the imageblock is bit-exact but frees no registers; its cost is in [section 10.9](#) (MM 25.106).

7.8 Constants and uniforms

A constant is an immediate inside the instruction, an entry in the *constant pool* (compile-time data in metadata slot 13 of the container; Apple’s compiler emits, `agxforge/g17/cooperativemetadata.py`), or a *published uniform*, a value Apple’s compiler computes per dispatch in a separate *constant program* that runs once before the main program and writes the uniform register file. Main reads pool entries and published uniforms through the same operand syntax, `bin(op0, const(K), W)`, so the operand alone does not say which one it is. Treating a published uniform as a pool entry gave 6 silent wrong answers (measured, MM 25.179).

The constant program was measured in MM 25.179:

- The emulator reproduces Apple’s GPU output by running the constant program first, as one thread over the same buffers, and handing its uniform writes to main: 6 of 6 sources byte-identical on full-width random buffers; three deformed models (every published uniform reads 0; the pointer names the next binding; a 32-bit uniform lands with halfwords swapped) each differ on all 6.
- Binding pointers: in all 21 constant programs studied, argument read, binding-table word (op14061) at halfwords $8 + K$ and $10 + K$ reads binding K ’s 64-bit pointer, low word then high. With $K = 4 \times \text{rank}$ ([section 7.1](#)) this is the same rule as MM 25.59’s “slots $8 + 4k$ and $10 + 4k$ for the k -th buffer”. The pointer is then the register-pair base for the immediate-address loads ([section 7.3](#)).
- Uniform destinations are main-program operations with one extra mode operand: integer to float into the uniform/staging file (op11185) is 32-bit integer to fp32 (op11179); uniform / staging write (op592) is 32-bit register move (op586); fp32 add with immediate into the uniform/staging file (op1038) is fp32 add with float immediate (op1006). A uniform destination must be written with one value across the active lanes.
- Of 15,256 corpus constant-program uniform writes, 14,814 are read by main; the 442 never-read writes are all uniform / staging writes and unexplained (decoder, MM 25.120).

In main, 16-bit AND, 32-bit result (op428) takes its mask from a pool operand (section 4.1), and the shipped decode replay refused until the pool was loaded, at 768 uses (measured, MM 25.197). 16-bit AND with immediate (op435) takes an 8-bit immediate; a wider mask such as 0xFF0 goes to the uniform-operand 16-bit AND (op437) (Apple’s compiler emits, MM 25.192).

In the uniform / staging write (decoder, MM 25.120.1), the value the decoder printed as a “publish index” is a scoreboard slot + 1. With a uniform destination it is 0 (47,269 of 47,269); with a destination in the message staging file (op4) it is 1–8 (131,313 of 131,313). The reader of a staging write is a message instruction (texture sample (op14661), texture write (op10909), texture read (op15813)), and it waits on slot index - 1: of 189,898 staging writes, 174,909 are waited on by the reader and 14,989 by an intervening instruction; none by neither. A staging write is a late value, like a load (section 9.1).

The binary format of the constant program and pool is chapter 11’s.

7.9 Textures

The texture forms are texture read (op15813), texture sample (op14661), texture write (op10909) and texture-coordinate staging write (op11564) (glossary). No operand carries the coordinate. Two reads at different coordinates are byte-identical, and the coordinate is published into the message staging file (op4) at [op4+0*4] and [op4+2*4] before the message (measured, tools/g17texrun.py).

An authored read, `A[400] = TX.read(x, y).x`, was executed (tools/g17texrun.py, isa/g17-execution-texture-results.json) on a texture holding $y*1000 + x + 7$, chosen so a wrong texel, a wrong row, a broadcast coordinate and an unbound texture give four different wrong answers: coordinate (5, 1) gave 1012, (2, 3) 3009, and lane 31 alone at (7, 3) 3014; all 32 lanes racing for one word gave an ambiguous 3007. The same runs showed that the texture operand is a dense index over the textures the function uses, not the Metal binding index (five constructed kernels); and publishing a register coordinate with a uniform / staging write needs operand 2 = 1048576; with 16777216 every lane silently reads texel (0, 0).

Which store reads a texture fetch is unresolved. store, sub-form 01, texel/word-slot (op17235) reads a texture fetch (measured, MM 25.62), and two-component store (op17244, 14 bytes) read one once (texel 201 at (1, 2), one 4×4 R32Uint texture, one thread; MM 25.146). For plain 32-bit store (op17229):

- A dispatched bisection of Apple’s reading program found that swapping only its store for this compiler’s 8-byte plain store stopped the read; that store, and a 12-byte ALU copy placed after the fetch, saw the destination register’s previous contents, at zero and at six intervening instructions. The ledger concluded “not timing” (ledger g17-only-one-consumer-can-read-a-texture-fetch).
- Apple’s own tex2d-read stores the texture read’s result with an 8-byte plain store whose operand 1 is 16777232 (0x1000010, a wait on slot 0), and that read publishes on slot 0 (Apple’s compiler emits, MM 25.146, tools/g17applestorefetch.py). This compiler’s encoder writes no wait on the 8-byte plain store, and the ALU copy in the bisection waited on slot 7, not slot 0.
- The bisection’s arms are consistent with a missing wait on the fetch’s slot; “not timing” does not rule out a wait, since unwaited late values stayed wrong after 64 intervening instructions elsewhere (section 9.1). The settling dispatch, Apple’s tex2d-read run and read back, has not been made.

A fetch result can safely be treated as a late value whose consumer waits on the fetch’s slot, and the sub-form 01 store can be relied on to read it. Do not assume that the plain 32-bit store cannot read a fetch, or that distance substitutes for the wait.

Pixel format is descriptor-side (Apple's compiler emits, `isa/g17-texture-sampler-surface.json`): `texture2d<uint>` and `texture2d<float>` (R32Uint, R32Float) compile to identical code and metadata; R16Float changes only the half-flavoured instructions. A sampler adds one register and a 4-byte sampler-state descriptor and swaps the texture read for the texture sample. Texture metadata slots are [chapter 11](#)'s. Sampled and write texture classes have no dispatched receipt (open).

Safe assumptions. The register-indexed law with unscaled byte displacements for the forms of [section 7.1](#); the immediate-address law for its eight measured members; width and index width as opcode choices; materialised strides; 64-bit address formation; and that every late value (load, imageblock read, staging write, texture fetch) needs its consumer's wait.

Not known. Out-of-range stores; negative displacements; misaligned scalar access; the vector store's displacement units; whether a masked-off load issues its access; what the threadgroup load's sensitive bits are; multi-simdgroup imageblock use; the imageblock store's bit-4 flag; the address space of the two MLX-only forms ([section 7.2](#)); the texture-fetch consumer; sampled and write textures; the origin of range mask, 16-bit source's cap of four; hardware behaviour for an oversize threadgroup declaration.

Evidence. MM [0.8](#), MM [6](#), MM [7](#), MM [15](#), MM [16](#), MM [17](#) rules 5, 6 and 34, MM [25.47](#), MM [25.49](#), MM [25.62](#), MM [25.89](#), MM [25.90](#), MM [25.96](#), MM [25.104](#) (25.104.1 to 25.104.4), MM [25.105](#), MM [25.106](#), MM [25.114](#), MM [25.114.3](#), MM [25.120](#), MM [25.120.1](#), MM [25.141.18](#), MM [25.144.1](#), MM [25.145.2](#), MM [25.146](#), MM [25.147](#), MM [25.165](#), MM [25.177](#), MM [25.179](#), MM [25.184](#), MM [25.192](#), MM [25.195](#), MM [25.197](#); `isa/g17-address-formula.txt`, `isa/g17-addressing-rules.json`, `isa/g17-predication.json`, `isa/g17-threadgroup-pair.txt`, `isa/g17-imageblock-execution.txt`, `isa/g17-barrier-scopes.txt`, `isa/g17-wait-mask-census.json`, `isa/g17-texture-sampler-surface.json`, `docs/g17-first-dispatch-trials.md`, `tools/g17emu.py`, `tools/g17texrun.py`, `tools/g17applestorefetch.py`.

8 Cross-lane operations

Cross-lane instructions move a value between the 32 lanes of one simdgroup without memory: a shuffle reads another lane’s register, a broadcast reads one lane’s register into every lane, and the native reductions combine all 32 (or 4) lanes in one instruction. Registers never cross a simdgroup; combining across simdgroups goes through threadgroup memory.

An out-of-range lane mask returns 0.0; it does not wrap to another lane. The native sums use a fixed adjacent-lane tree that rounds at every step (measured for fp32 and half, [section 8.4](#)), so their results can be predicted bit for bit and differ from a correctly rounded sum. Cross-lane instructions consume late values like any other instruction and read a loaded value too early unless they wait. All three behaviours were measured after an earlier model had predicted otherwise. The one established result with inactive lanes is that a broadcast from an inactive lane 0 returned 0 ([section 8.3](#)).

Opcodes and symbols:

- SIMD shuffle-xor, immediate mask (op14169) and SIMD shuffle-xor, runtime mask (op14170): Metal `simd_shuffle_xor`, the 32-lane forms. quad shuffle-xor, immediate mask (op13944) and quad shuffle-xor, runtime mask (op13945): the 4-lane forms.
- SIMD shuffle, constant lane, broadcast (op14157) reads a constant lane; SIMD shuffle, register lane (op14158) reads the lane named in a register.
- SIMD sum, fp32 (op16842) and SIMD max, fp32 (op16858): Metal `simd_sum` and `simd_max` on float. Apple’s `quad_sum` compiles to op13857/10 (no glossary entry); quad prefix sum, fp32 (op13856) is reachable from source but absent from the corpus (MM [25.82](#)).
- simdgroup active-thread mask (op14208) writes the mask of active lanes. `SR_PVSIMD` and `SR_TVSIMD`, the vote registers, are in [chapter 6](#).
- “L” is a lane index 0–31; “in[L]” is lane L’s source value.

8.1 Shuffle-xor mask range

For an immediate mask m below 32, SIMD shuffle-xor, immediate mask (op14169) gives every lane L the value $\text{in}[L \wedge m]$, 32 of 32 lanes for each mask. For m of 32 or more every lane receives 0.0, so the mask is neither reduced modulo 32 nor ignored. quad shuffle-xor, immediate mask (op13944) is bounded the same way to four lanes and returns 0.0 for any mask of 4 or more (measured, MM [12](#), MM [17](#) rule 25). The glossary still tags these forms “compiled”; MM [12](#) is the measurement.

With a runtime mask the two widths differ. quad shuffle-xor, runtime mask (op13945) follows each lane’s own mask exactly. SIMD shuffle-xor, runtime mask (op14170) does not, so its mask must be lane-uniform (measured, MM [12](#)); what it computes for a non-uniform mask is not known. The SIMD runtime form appears nowhere in Apple’s 5,010,028-instruction corpus, although Apple’s compiler emits it for a variable `simd_shuffle_xor` at /10 (MM [25.73](#), MM [25.82](#)).

The immediate-mask shuffle-xor has also been executed below Metal (measured, docs/g17-first-dispatch-trials.md, “SIMD cross-lane shuffle below Metal”). A 52-byte program authored by this compiler loads $A[\text{lane}]$, applies the immediate-mask shuffle-xor with mask 1 and stores $C[\text{lane}]$, with a waiting copy between the load and the shuffle. Under two input profiles with byte-identical code (normal

fp32 patterns $0x3f800000 + 256 \times \text{lane}$, and the small words $11 \times \text{lane} + 5$, which are fp32 subnormals), all 64 words came back as the bit pattern of lane $L \wedge 1$. The subnormal patterns crossed lanes unchanged, so the shuffle does not apply the fp32 ALU's flush ([section 4.3](#)); no arithmetic rule is inferred from this.

8.2 Late operands

`simd_shuffle_xor` and `simd_broadcast` reading a loaded value directly read it before it arrived (measured, MM [25.90](#)). In a two- and four-simdgroup combine that should have been exact, each arm returned one wrong value, a max below every simdgroup's own max; the first butterfly step's shuffle had read a load that had not landed. A waiting copy before the first shuffle made all four arms exact, and this compiler now inserts that copy. The rule is that of any late operand ([section 9.1](#)).

8.3 Broadcast and inactive lanes

SIMD shuffle, constant lane, broadcast (`op14157`) reads lane 0's value; it is measured only with every lane active, so lane 0 and "the first active lane" are not separated there (measured, MM [25.145.1](#)). A predicated region separates them: with lane 0 inactive (first active lane 1), every active lane received 0, and `fma(x, x, broadcast(x))` returned exactly $a*b$ on 11 of 11 mismatched lanes (measured, MM [25.144.6](#)). The constant-lane broadcast implements `simd_broadcast(x, 0)`, not `simd_broadcast_first`.

Apple's compiler lowers a divergent `simd_broadcast_first` as four instructions (decoded; the roles are inferred, not measured one by one, MM [25.144.6](#)): a special-register read, 16-bit (`op14060`) of register 38 (`SR_PVSIMD`, [chapter 6](#)), SIMD boolean reduce (`op14214`) over it (unconfirmed: likely selecting the first active lane), msb bit scan, 16-bit result (`op9989`) turning that into a lane index, and SIMD shuffle, register lane (`op14158`) from that lane. This compiler does not lower that sequence: it refuses a broadcast inside a predicated region and keeps the constant-lane broadcast for uniform code.

Two lifetime facts are specific to cross-lane forms (*this compiler*; the release rules are [chapter 2](#)'s):

- The constant-lane broadcast's source lifetime is operand 3. A template that left it at release made `y = broadcast(x); z = fma(x, x, y)` read a released `x`, and those lanes stored 0 on hardware (MM [25.144.6](#)).
- In the release checker, the value a shuffle fetches from its source lane (operand 2 of the register-lane shuffle) is another lane's register, not a read of this lane's (`agxforge/g17/releasecheck.py`).

Shuffles and broadcasts may be assumed to work with every lane of the simdgroup active. No shuffle or vote in a partially active simdgroup is established, and the emulator refuses them (MM [25.145.1](#)).

8.4 Native reductions

`simd_sum`, `quad_sum` and their product forms are a balanced adjacent-lane binary tree, every operation rounded to the element type, exchanging over lane bits 0 to 4 in order; in half precision nothing accumulates at higher precision (measured, MM [12](#)). For fp32, SIMD sum, fp32 (`op16842`) was tested on 64 rounding-sensitive 32-lane vectors with values spread over $\pm 2^{20}$: it matches the adjacent pairwise tree, which is the xor butterfly in mask order 1, 2, 4, 8, 16, on 64 of 64. It is neither the order this project's own butterfly

used (1, 8, 2, 4, 16) nor exact-then-rounded (measured, MM 25.141.16). SIMD max, fp32 (op16858) is exact on 32 lanes against an order-free reference (MM 25.90).

Worked example: 2048 and thirty-one ones in half (measured result, MM 12; the arithmetic below reproduces it). Lane 0 holds 2048.0 and every other lane 1.0. Half has spacing 2 in [2048, 4096). Table 8.1 follows lane 0’s partial sum up the tree.

Table 8.1. Lane 0’s partial sum at each level of the half simd_sum tree for 2048 plus thirty-one ones, against the exact sum.

Tree level (lane bit)	Lane-0 partial	Exact	Note
0	$2048 + 1 = 2049 \rightarrow \mathbf{2048}$	2049	a tie; RNE keeps the even significand
1	$2048 + 2 = 2050$	2051	other partials are now 2
2	$2050 + 4 = 2054$	2055	
3	$2054 + 8 = 2062$	2063	
4	$2062 + 16 = \mathbf{2078}$	2079	simd_sum returns 2078.0

quad_sum stops after lane bit 1 and returns 2050.0. A correctly rounded sum would give 2080 and 2052 (2079 and 2051 are ties that round to even). The 1 lost at the first level is not recovered, because every later partial is computed and rounded in half. A sequential sum, a wider accumulator and a correctly rounded sum each give a result different from the tree’s on this input.

Reductions across simdgroups go through threadgroup memory: each simdgroup writes its value to threadgroup word sg, one threadgroup barrier follows, and every lane folds words 0 to S-1 in ascending order, so the result does not depend on arrival order. The combine is exact at $S = 2$ (sum and max, 64 of 64 lanes) and $S = 4$ (128 of 128); a rival with no exchange matches 0 lanes for sum (measured, MM 25.90). The barrier’s semantics are chapter 9’s.

The library tile reductions (reduce_rows, reduce_columns on tensor fragments) use a per-lane sequential chain and then a butterfly over the reduction axis’s lane bits, with identities and NaN handling of their own (section 10.12).

8.5 Other cross-lane forms

- simdgroup active-thread mask (op14208) writes the mask of active lanes (measured, glossary).
- fine x-derivative, saturated (op1062) is a cross-lane dfdx with saturation; its records ran one lane, so its function is unconfirmed (MM 25.90).

Safe assumptions. The shuffle-xor mask bounds (section 8.1), shuffles and broadcasts with every lane active, and the native-sum trees of section 8.4, each within the inputs it was measured on.

Not known. Shuffles, broadcasts and votes under partial activity; the behaviour of the SIMD runtime-mask shuffle-xor with a non-uniform mask; the roles of the four instructions in Apple’s divergent broadcast; the reduction tree for half simd_sum products and int types beyond the stated measurements; cross-lane latency (chapter 12 has no measurement of it).

Evidence. MM 12, MM 17 rule 25, MM 25.73, MM 25.82, MM 25.90, MM 25.141.16, MM 25.144.6, MM 25.145.1; docs/g17-first-dispatch-trials.md, agxforge/g17/releasecheck.py.

9 Synchronization

A G17 program has four ordering mechanisms, one per scope, and little is ordered implicitly between them.

- Inside one simdgroup, an eight-slot *scoreboard* orders late values. A producer (load, atomic, imageblock read, special-register read, staging write) names a slot; a consumer names the slots it waits on in an eight-bit mask in its own operand 1. There is no wait instruction and, for late values, no interlock: a consumer that does not name the slot reads whatever the register held before.
- Between simdgroups of a threadgroup, *threadgroup barrier* (*op447*) is an execution rendezvous whose immediate names the memory kinds it covers. Registers never cross simdgroups, so every hand-off goes through memory and a barrier.
- Between threadgroups of one dispatch, the threadgroup barrier does not help. Visibility of device stores across cores needs *device fence* (*op14156*) on both sides, and the order of arrival comes from an *atomic*.
- Between dispatches, the driver's encoder places a *barrier token* in the command stream.

Forward progress is treated in [section 1.5](#). In the tested configurations threadgroups were dispatched in order and no spinning threadgroup gave way to a producer dispatched after it (MM [25.141.1](#)). No guarantee of that behaviour has been established, so a program must never depend on preemption, and any grid whose threadgroups wait on one another must be sized to measured residency.

Several failures measured on this machine came from getting one of visibility, ordering and progress right and assuming the others. A device-scope barrier gave a perfect atomic count but stale data, so the count was ordered and the data not visible. In a spin-wait the producer ran only after the waiting wave had spun to its cap: visibility was in place, progress was not. And one atomic updated memory correctly while its consumer read 0, because the return had not yet arrived in the register. Where the three differ they are stated separately below; [table 9.1](#) gives them for each scope.

Table 9.1. Ordering mechanism for each scope, with what it provides for visibility, ordering and forward progress.

Scope	Mechanism	Visibility	Ordering	Forward progress
one simdgroup	scoreboard slot + consumer wait mask	the register holds the value only for a consumer that waits on its slot	program order on one scoreboard; same-address stores: lowest active lane wins	not an issue
simdgroups of one threadgroup	threadgroup barrier (op447) at the matching scope	threadgroup memory: any scope; device memory: see section 9.2	across the barrier	barriers among a threadgroup's simdgroups completed in every measured case
threadgroups of one dispatch	device fence (op14156) on writer and reader, plus an atomic count	measured for the last-threadgroup pattern	from the atomic; a threadgroup barrier gives none	observed only among resident threadgroups; do not depend on preemption
dispatches	encoder barrier token	measured	serial encoder: after every dispatch	in order (observed)

Notation.

- *Slot*: one of eight scoreboard entries, 0–7. *Fill slot*: the slot a producer names, printed by the decoder as slot + 1 in operand 1 bits 20–23. *Wait mask*: operand 1 bits 24–31; bit 24 + s waits on slot s.
- *Late value*: a result that arrives after its producer issues and needs a waiting consumer ([section 9.1](#)).
- *Scope immediate*: operand 1 of threadgroup barrier (op447) (20, 154, 276, 532, 1144; [section 9.2](#)).
- *Residency*: how many threadgroups a core holds at once (rules: [section 1.4](#); measurements: [section 12.5](#)).
- Registers are named by role. Where the bits named here sit is [section 3.4](#); the release lifetime (modifier 16) is [chapter 2](#); costs (dispatch floors, hand-off times, fence costs) are [chapter 12](#). “Below Metal” is as defined in [chapter 7](#); those runs are recorded in docs/g17-first-dispatch-trials.md.

9.1 The scoreboard

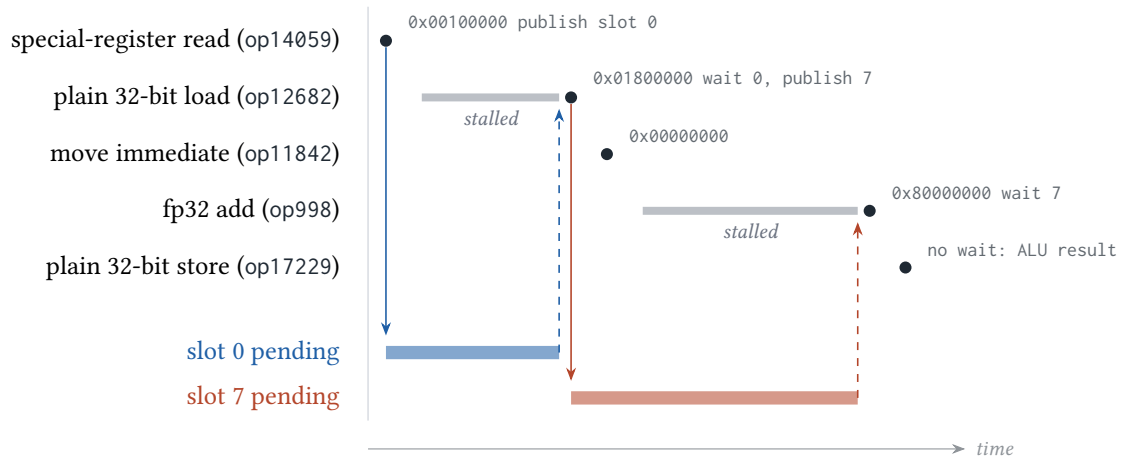
A late-value producer names its fill slot in its own control word (bits 20–23; measured, MM 0.8, MM 6); the decoder prints slot + 1, so this compiler’s first group reads 8 and means slot 7. A consumer carries an eight-bit wait mask in bits 24–31, bit 24 + s for slot s. Nothing else orders them; there is no separate wait instruction. What earlier work called a “tag ring” and separately a “load-use wait bit” are this one field.

If the hardware interlocked on the register, the mask would be only a hint; clearing one wait bit tests this. On an MMA whose operands come from loads filling slot 7, clearing its slot-7 wait bit (byte0[3]) made it read the fragments before the loads landed and produce zeros, deterministically; for tensor loads, 50 of 50 dispatches across five processes returned exactly zero (MM 6). The mask’s layout was established causally (43 dispatches) and then by preregistration: for a load filling slot v - 1 the consumer must set bit 24 + (v - 1), 20 of 20 predicted and confirmed. The imageblock store adds a wrong-slot arm. Storing a freshly loaded value (the load fills slot 7), the compiled wait on slot 7 gave 32 of 32 lanes correct; with the wait cleared, and with the wait moved to slot 1, which nothing fills, every lane stored 0 (three runs each,

MM 25.96). A set bit therefore waits on its own slot only; a late value outstanding on another slot does not hold it.

If the consumer's bit for the producer's slot is clear, the consumer does not stall. It reads the register before the late value lands and gets whatever the register held (figure 9.1(b)). Figure 9.1(a) follows the running example's epilogue through its two late values (section 0.4).

(a) The running example's epilogue



(b) A missing wait (MM 6)

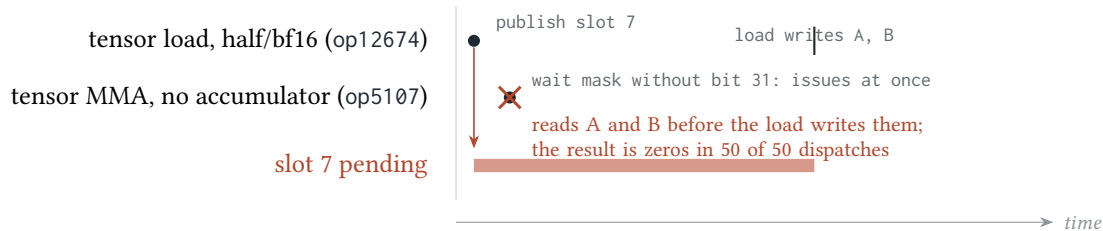


Figure 9.1. Late values and their waits, as the control words specify them. A dot is an instruction's issue; grey is an instruction held at issue by its wait mask; a coloured bar is a scoreboard slot that a producer has published and that has not yet arrived. A dashed arrow is a slot's arrival releasing its waiter. The control words in (a) are read from the running example's bytes (the low 32 bits of operand 1: bits 20–23 publish slot + 1, bits 24–31 are the wait mask). Panel (b) is the measured missing-wait experiment. Only the ordering is measured: positions on the time axis are schematic.

The same field orders every late value (loads, atomics, imageblock reads, staging writes and special-register reads). The per-form carriers (general loads, the 128-bit vector load, tensor load, the tensor MMA's scattered mask bits, the 12-byte ALU wait bits, uniform / staging write (op592)) are section 3.4's table; this section gives what they mean. Two carriers are not in that table: in 14-byte stores byte9[5] is slot 7 in this compiler's encoder (agxforge/g17/cc.py), and for tensor load, half/bf16 (op12674) MM 6 called the whole field bits 20–27, which includes the load's own wait mask; the fill slot is bits 20–23 alone. The general-load field is measured through executed consumers (MM 25.115, MM 25.121; tools/g17emu.py load_hazards), the staging-file slot only through the decoder (MM 25.120.1).

The rule behind these comes from a decoder census: for 126 forms across 6,594 Apple programs, every instance's wait bits equal exactly the slots its source registers are still pending on (`isa/g17-wait-mask-census.json`, MM 25.96). It covers stores: the 128-bit vector store (`op17256`) uses every bit 24–31 (13,140 instances in a 314-object census, each one-hot value present), which refuted an earlier “three-bit store limit” that was an encoder-table artifact (MM 6). Tensor stores are a separate class: all 876 tensor store (`op17257`) and 118 masked tensor store (`op17258`) instances carry a zero wait byte, and the same-simdgroup MMA-to-tensor-store path is exact with no wait (measured, MM 6).

Correction: the 8-byte plain 32-bit store (`op17229`) does carry wait bits (15 corpus instances match their pending slots, `isa/g17-wait-mask-census.json`; Apple's `tex2d-read` carries `0x1000010` on one, MM 25.146). The “no wait bit” note in `agxforge/g17/cc.py` describes this compiler's encoder, which has no carrier for them (appendix A).

Which consumers need a wait has been measured:

- *Not needed:* ordinary ALU to ALU. 52 of 52 ordered adjacent pairs of ordinary ALU instructions read the previous result bit-exactly with no wait (MM 25.90, MM 6).
- *Needed:* loads, imageblock reads, vector-load lanes, the packed half2 load, atomic returns, texture fetches and staging writes. “Needed” applies to every consumer kind: an indirect load whose index is a loaded value read index 0 on every lane (1 of 32 right) until its operand 1 also named slot 7 (32 of 32; MM 25.104, MM 25.90); `simd_shuffle_xor` and `simd_broadcast` reading a loaded value directly read it before it arrived (MM 25.90); 32-bit NOT (`op11190`), find most significant bit (`op9986`), bit reverse (`op14047`) and 16-bit register move (`op590`) do not interlock on a load (MM 25.133); a store takes its value register at issue (MM 25.96).
- *Special-register reads:* special-register read (`op14059`) publishes a slot and Apple's compiler always waits on it, but a slot-0 read is readable at distance 0 with no wait, and the wait has no measurable cost (measured, MM 25.117, MM 25.121; the probe and the counts are section 6.3).
- *Open:* whether an ALU consumer of an MMA result is interlocked. One ALU consumer placed directly after an MMA carried no wait and still read the result; no measured pair had an MMA producer (MM 6, MM 25.90).

An unwaited consumer could read arbitrary data, the late value if it happened to arrive in time, or the register's previous contents. To tell these apart the producer's destination was pre-set. With a poison in it, the four non-interlocking unary forms returned exactly `op(poison)` on every lane; with a fresh register they returned `op(0)`; a negative control, an add with its load-wait bit cleared, read the poison on 1,024 of 1,024 lanes, so the instrument can see a race (measured, MM 25.133). The unwaited consumer reads the previous contents, which is why most observed failures read 0 and why detecting them takes a sentinel or poison value; a zero-filled buffer cannot show them. Added distance does not help; an atomic whose return was published on a slot its consumer did not name returned 0 with 64 independent instructions in between (below).

Worked example: an atomic return published on the wrong slot (decoded, then executed on hardware; MM 25.115, `docs/g17-first-dispatch-trials.md`). This compiler's residency probe counts arrivals with uniform-address atomic, returning old (`op10094`) and reads the returned old value. At $S = 256$ simdgroups every arrival read 1 (a returned 0, plus one) while the departure maximum was 256. Of the two explanations that fit, the one first recorded, and later withdrawn, was that simultaneous same-address atomics combine and each requester sees the pre-combine value. The other is that the consumer read the register before the return arrived. The decoded bytes favoured the second. Marking the atomic's result late gave its consumer, 32-bit add immediate (`op10279`), this compiler's load-use wait, mask `0b10000000`,

slot 7, because this compiler’s loads fill slot 7. But the atomic’s operand 1 bits 20–23 were 1: it filled slot 0, copied from Apple’s witnesses, where Apple’s first consumer waits on bit 24 (120 of 120 corpus instances). The consumer waited on a slot the atomic never touched. At larger S arrivals had agreed with departures, which had been read as the wait working; in fact the arrivals were staggered enough that each return landed before its consumer read it. The discriminating run changes only the slot: publishing the atomic on slot 7 (the two programs differ in four bytes) gave an arrival maximum of 256, equal to the departure maximum, 3 of 3, while the slot-0 arm read 1. Combining would not depend on which slot the return is published on. Below Metal, the per-lane form behaved the same way: a 12-byte per-lane-address atomic, returning (op10090) publishing on slot 0 (operand 1 0x80100000) updated all 64 cells correctly and returned 0 to every lane, still 0 with 64 independent two-byte moves inserted before the consumer; changing two atomic bytes to publish on slot 7 (operand 1 0x80800000) returned all 64 old values. In both cases the fix was to name the producer’s actual slot in the consumer’s mask.

Rules for choosing and reusing slots:

- Past eight groups the slot sequence Apple’s compiler assigns is scheduler-chosen (twelve loads got 8, 1, 2, 3, 4, 4, 4, 4, 7, 7, 6, 5 in decoded values), so an emitter should use an explicit rule, such as K-step k in slot $k \bmod 8$; this compiler’s $K = 256$ lowerings use slot $k \bmod 7$, exact in 3 of 3 cases (MM 6, tools/tensorops-model/scoreboard-store-census.json).
- Reusing a slot after its named consumer has waited is safe (measured); reuse before that consumer is unmeasured and forbidden (MM 0.14, MM 6).
- Not every slot works for every producer. Loads and atomic returns work on slot 7 (measured), but a special-register read publishing on slot 7 stored nothing, waited or not, recorded and not explained (measured, MM 25.121; section 6.3). Use a slot that has been measured for that producer kind.
- Atomic return slots (contradiction resolved by scope): the glossary’s “the return fills scoreboard slot 7” for the uniform-address atomic and threadgroup atomic, returning old (op11765) describes this compiler’s emission. Apple’s corpus fills slots 0 and 1, and every first consumer waits on exactly that slot (25 of 25 for the uniform atomic, MM 25.115). Slot 7 is this compiler’s convention, checked on hardware; Apple’s corpus does not use it, and the hardware does not require it.

The release lifetime (16) is destructive and per lane (chapter 2): a store naming one register as both value and index, with the index slot releasing it, stores 0 (measured, MM 25.117), and an instruction no lane executes releases nothing (MM 25.95).

9.2 Barriers and fences

The threadgroup barrier (op447) is one six-byte instruction with scheduling class 3, MayStore, NotDuplicable and side effects. Only operand 1, the scope immediate, changes with the Metal barrier (table 9.2; Apple’s compiler emits, isa/g17-barrier-scopes.txt).

Table 9.2. Metal threadgroup barriers with the bytes and scope immediate Apple’s compiler emits for each.

Metal barrier	Bytes	Scope
threadgroup_barrier(mem_none)	075100060000	20
threadgroup_barrier(mem_device)	0f6900060000	154
threadgroup_barrier(mem_threadgroup)	275100060000	276

Table 9.2, continued

Metal barrier	Bytes	Scope
<code>threadgroup_barrier(mem_texture)</code>	876700060000	1144
<code>threadgroup_barrier(mem_threadgroup_imageblock)</code>	475100060000	532

`mem_threadgroup` | `mem_device` emits two barriers, one per flag; `simdgroup_barrier` emits nothing at all (Apple’s compiler emits, `isa/g17-atomic-semantics.json` barriers). Scopes 16, 528 and 1138 substituted for 276 each carried a threadgroup-memory round trip (measured, the threadgroup-barrier notes in `tools/g17emu.py`).

Between simdgroups a threadgroup barrier is mandatory (measured, MM 14, MM 17 rule 7). With no barrier, and with `air.simdgroup.barrier` alone (which emits no instruction), a cross-simdgroup hand-off was wrong in 12 of 12 trials and the consumer read exactly the buffer’s pre-write state, a race rather than garbage. Every workgroup-barrier variant (execution-only, threadgroup memory, device memory, both) was correct in 12 of 12. Below Metal, threadgroup store, barrier and load exchanged values between the two simdgroups of a 64-thread group, and also in a half-full 32-thread second group (measured, `docs/g17-first-dispatch-trials.md`; other partial sizes and inactive-lane accesses untested).

MM 14 and the emulator treat device-memory ordering between simdgroups differently without conflicting. MM 14 observed that the execution-only barrier (scope 20) sufficed, and says so as “an observation about this chip, not a guarantee”. The emulator’s race check (MM 25.197) orders threadgroup memory across every threadgroup barrier but device memory only across scope 154, a conservative choice that no measurement forced. Scope 276 (or any threadgroup barrier scope) is safe for threadgroup memory between simdgroups, and scope 154 for device memory between simdgroups of one threadgroup. Code must not rely on scope 20 to order memory, on any threadgroup barrier to order anything across threadgroups, or on `simdgroup_barrier` to do anything.

Worked example: threadgroup barrier against device fence (executed on hardware, MM 25.140.5). Every threadgroup writes a marker, crosses a barrier, and counts itself with uniform-address atomic, returning old (`op10094`) on its rank-0 written buffer; the last threadgroup to arrive sums all markers. With the barrier at scope 154 the count was a perfect permutation, yet the last threadgroup read stale markers: it summed 1,560 of 2,080 at 64 threadgroups and 123,408 of 131,328 at 512, differing by round. Apple’s compiler, given `atomic_thread_fence(mem_device, seq_cst, thread_scope_device)`, emits device fence (`op14156`), operands (0, 186), six bytes `0f2b00060000`, no registers, and places one after the stores and one again inside the last threadgroup’s region; the loads stay ordinary. With those two fences both sums were exact. This compiler now emits those bytes, read back through Apple’s decoder. A fused attention merge built on the pattern was bit-exact at `q0 = 0, 5, 128 and 271` over five rounds each, with NaN-poisoned rows (costs are chapter 12’s).

The imageblock barrier is the threadgroup barrier at scope 532 (section 7.7). The emulator admits it only for one simdgroup in lockstep; more than one simdgroup was never witnessed (MM 25.145.2). A barrier inside an exec-mask region is not modelled there either (`tools/g17emu.py`).

Encoder-level barriers come from a static read of the AGX user driver (decoder, MM 25.141.20); their costs are measured in chapter 12.

- A serial compute encoder emits a barrier token after every dispatch. A concurrent encoder emits one only where `memoryBarrierWithScope:` asks; in a serial encoder that call only posts a notification. For a chain where every dispatch depends on the last, both emit the same tokens.
- The token is one 32-bit command word, 0x60000160, OR 0x1000 when a pending flag is set; with its boolean arguments it ORs 0x400 or appends 0x60000502, both of unknown function. Its cost is the GPU draining at the token.
- Hazard tracking never enters the per-dispatch path: tracked and untracked buffers encode identically inside an encoder. Consecutive compute encoders in one command buffer are merged, with an unconditional barrier after a concurrent one.
- Measurements agree with this: tracked buffers with one serial encoder per token beat untracked buffers ordered by an `MTLSharedEvent` between command buffers, and untracked buffers without ordering race (MM 0.15).

Below Metal, two dependent Submits on one queue (the second reading the first’s output) computed exactly, also with a program switch between them; and one Submit carrying two or three command records, each running a per-lane +7 read-modify-write on the same words, left 14 and 21 in every lane, so each record saw the previous record’s result (measured, docs/g17-first-dispatch-trials.md, “Dependent two-stage pure GPU pipeline” and “One to three dependent GPU computations in one Submit”). The ordering of other command shapes, and whether records can overlap, is unmeasured.

9.3 Atomics

In the decoder (isa/g17-atomic-family.toml), 208 opcodes share the scheduling class of uniform-address atomic, returning old (op10094) and flags (classes 286 and 287 with flags 0x402002400 for device scope, class 329 with 0x402002800 for threadgroup scope) and one operand skeleton, [dest] imm imm [address] imm [index] imm imm imm [value] imm, with parts present or absent. Five occur in the corpus; 29 are reachable from one encoding of it by bit flips. The other 179 were never produced by single-bit walks to saturation, 132,240 two-bit mutations, 1,365,504 three-bit mutations or 800,000 random encodings; the boundary is width (a bounded negative). Its structural bits: byte4[0] returns old (to uniform-address no-return atomic, 32-bit (op10022)); byte4[1] selects a threadgroup GPR16 address over a device register pair (op11769, no glossary entry); byte5[2] selects an index register (per-lane-address atomic, returning (op10090)); byte6[2] leaves the family. Table 9.3 lists the forms this chapter uses.

Table 9.3. Atomic forms by address kind.

Glossary name	Opcode	Address	Standing
uniform-address atomic, returning old	op10094	device, uniform, byte offset in operand 6	measured (below Metal, all operations below; MM 25.140.3)
uniform-address compare-exchange, returning	op10095	device, uniform	measured (below Metal)
uniform-address no-return atomic, 32-bit	op10022	device, uniform	Apple’s compiler emits
uniform-address no-return compare-exchange	op10023	device, uniform	Apple’s compiler emits
per-lane-address atomic, returning	op10090	device, base + index register	measured (below Metal)

Table 9.3, continued

Glossary name	Opcode	Address	Standing
per-lane-address compare-exchange, returning	op10091	device, base + index register	measured (below Metal)
per-lane-address no-return atomic	op10019	device, per lane	Apple’s compiler emits
64-bit no-return atomic, max/min	op9996	device	Apple’s compiler emits
threadgroup atomic, returning old	op11765	threadgroup	measured (MM 25.115)
threadgroup atomic, no return	op11701	threadgroup	measured (glossary)

Below, “the uniform atomic” means uniform-address atomic, returning old (op10094) and “the per-lane atomic” per-lane-address atomic, returning (op10090). “Uniform-address” describes the address, not the issue: the uniform atomic executes once per active lane, so 32 lanes add 32 and receive 32 distinct old values (measured, MM 25.140.3). Its operand 6 is the address’s byte offset: values 4, 8 and 16 moved the cell by 1, 2 and 4 words. *Correction:* earlier claims called operand 6 a source lifetime (“16 writes nothing”); the probe had watched the wrong word (MM 25.140.3).

The operation is a four-bit field, named by the decoder’s operation operand. The decoder prints operand 2 as 0x40200 + code, so every executed operation has an unambiguous name: 262656 + code. For the uniform atomic the four bits are byte4[5], byte5[3], byte6[3] and byte7[7]; table 9.4 reproduces all ten encodings Apple’s compiler emits for `atomic_fetch_{add,sub,and,or,min,max}` (both signednesses), `atomic_exchange` and `atomic_fetch_xor` (decoder plus Apple’s compiler, `isa/g17-atomic-family.toml`) and gives the uniform atomic’s shortest length for each code.

Table 9.4. Atomic operation codes, with the operand value the decoder prints, the uniform form’s shortest length in bytes and whether it has executed below Metal.

Code (operand)	Operation	Shortest length, uniform form	Executed below Metal, uniform form
0 (262656)	add	10	yes, returns consumed
1 (262657)	sub	12	yes
2 (262658)	exchange	12	yes
3 (262659)	compare-exchange: selects uniform-address compare-exchange, returning (op10095)	10	yes
4 (262660)	unsigned min	12	yes
5 (262661)	signed min	10	yes
6 (262662)	unsigned max	12	yes
7 (262663)	signed max	10	yes
8 (262664)	and	10	yes
9 (262665)	or	10	yes
10 (262666)	xor	12	yes
12 (262668)	fp32 add, emitted by no tested Metal builtin	10	yes

Four combinations do not decode. The same operand values name the per-lane operations, but the bits carrying them differ by form and length: in the 12-byte per-lane atomic the fourth bit is byte11[4], and a decoder sweep of that form maps the assembler’s three-bit operation with byte11[4] set and clear

onto these codes (docs/g17-first-dispatch-trials.md, “Per-lane AND old-value return”). An authored “OR” with byte11[4] set executed exchange, updating every cell to the new value; clearing that one bit gave operand 262665 and OR (measured). *Correction:* the “eight unassigned raw operation codes” of isa/g17-atomic-antics.json were read off one witness’s field placement and are retracted (isa/g17-atomic-family.toml; [appendix A](#)); a census of unassigned codes must be made per form.

The following semantics were measured below Metal (docs/g17-first-dispatch-trials.md; isa/g17-atomic-family.toml), all in relaxed order, single dispatch, each case in two fresh processes with output guards.

- *Atomicity.* 64 lanes adding to one cell left exactly the sum (5 to 69 with +1; 5 to 2085 with lane i adding $i + 1$), and every lane received a distinct old value; with lane-varying addends the returned values form a unique chain through all 64 lanes. Two simdgroups’ elected lanes on one uniform atomic received non-overlapping intervals (5–36 and 37–68).
- *Integer operations* behave as named, including 32-bit modular sub ($5 - 32 - 32 = 4294967237$), signed against unsigned extrema on 0xFFFFFFFF and 0x80000000, and exchange.
- *Compare-exchange.* The register tuple is (desired, expected), established from the compiler’s witness; matched and failed compares both returned the old value.
- *fp32 add (code 12),* uniform and per-lane: $1.0f + 1.0f + 1.0f$ gave returned values 1.0f, 2.0f and final 3.0f; 64 per-lane updates of 1.0f from 1.0f reached 65.0f and returned 1.0f through 64.0f once each. Each update rounds to nearest even: half-ULP ties left an even mantissa unchanged and moved an odd one up once (0x3F800001 to 0x3F800002) and no further. Subnormal inputs, operands and results flush to zero. A quiet NaN seed 0x7FC12345 was returned raw to its first caller and then stored as canonical 0x7FC00000; -0 plus $+0$ returned -0 and stored $+0$; infinities were preserved. Signaling NaNs and other payloads are untested.

Atomics have been run only on the rank-0 buffer, through Metal and below it; this compiler refuses an atomic whose identity base and binding rank disagree (open, agxforge/g17/cc.py; MM [25.140.3](#)). Threadgroup atomics need a threadgroup region, which only an ordinary threadgroup load or store provisions ([section 7.6](#)).

Atomic return values are late; the fill slot is operand 1 bits 20–23 minus 1, and slot choice is in [section 9.1](#). The 10-byte uniform atomic also releases its value: operand 9 is the value’s lifetime and only 16 is representable, so a second atomic sharing a constant read 0 ($A[0] = 0$ against 24; measured by this compiler’s fuzzer, MM [25.144.6](#), agxforge/g17/cc.py); a value read again must be copied first.

Validation with a consumed return, by form:

- The uniform atomic, below Metal: all eleven of its operations in [table 9.4](#) (every code except 3), each under two-group contention with the returned old value consumed; uniform-address compare-exchange, returning (op10095) at match and mismatch. Through Metal, add (the residency probe, MM [25.115](#)) and the compiler’s wave-broadcast program (agxforge/g17/cc.py).
- The per-lane atomic, below Metal: update-only for add, and, or, sub, xor, signed and unsigned min and max, exchange and fp32 add; returns through slot 7 for add (also contended, both fixed and lane-varying addends), and, or, sub, xor and fp32 add (also contended, half-ULP and NaN cases). Compiler-generated 12-byte programs for add, and, or, sub and xor passed with a direct indexed-store consumer, plus an ordinary add consumer for add and xor.

- Per-lane-address compare-exchange, returning (op10091): hand-authored slot-7 return (one matching and 63 failing compares), then compiler-generated programs at index placements R0, R1, R5, R7 and R12 (ten Submits); other placements are decode-checked predictions, and placement 13 is refused.
- Threadgroup atomic, returning old (op11765): dispatched through Metal with its return consumed; its 32-bit subtract (op11667) consumer had read both returns with an empty wait mask until fixed, where Apple waits 10 of 10 (MM 25.115).
- Not validated: per-lane min and max returns, unsigned long-form returns, other consumer shapes (refused by this compiler), 64-bit atomics (64-bit no-return atomic, max/min (op9996), compiled only), and any atomic on a buffer other than rank 0.

An atomic does not order other memory. The count in [section 9.2](#)’s worked example was exact while the data it guarded was stale. Ordering of ordinary stores through an atomic needs device fence (op14156). The emulator does not track atomics in its race check and does not model acquire or release through atomics or the fence, so it refuses any program that synchronises threadgroups through an atomic counter (MM 25.197).

9.4 Ordering by scope

Lanes of one simdgroup

- Program order holds on one scoreboard: a store followed by a load in one simdgroup needs no fence, as two separately generated bodies joined at a shared buffer showed, including a cross-lane stress where each lane read another lane’s write (measured, MM 13). Late values still need the consumer’s wait.
- Same-address stores with plain 32-bit store (op17229): the lowest active lane wins, by lane and not by value (all lanes: lane 0; lanes 0–15 under an exec region: lane 0; odd lanes: lane 1; reversed values: lane 0; five runs each; measured, MM 25.145.4, isa/g17-execution-storewinner-results.json). Across simdgroups the winner is unmeasured.
- Registers never cross a simdgroup: a cross-simdgroup simd_shuffle returns the caller’s own value, 256 of 256 (measured, MM 14).

Simdgroups of one threadgroup

Simdgroups are ordered only across a threadgroup barrier (op447) ([section 9.2](#)). Same-value writes by several simdgroups are harmless by construction (no schedule changes the bytes), and shipped code relies on that: the fused attention appends a new K/V row from every simdgroup of every threadgroup and each reads its own copy back (modelling rule, MM 25.197).

Threadgroups of one dispatch

MM 0.14’s map says “cross-threadgroup handoff requires a dispatch boundary”. It was written about tensor fragment hand-offs before any in-dispatch mechanism had been measured, and MM 25.140.5 and MM 25.141.1 have since measured two, so the statement is superseded in part:

- *Visibility.* Device stores from one threadgroup are visible to another in the same dispatch after device fence (op14156) on the writer side and again on the reader side (measured for the last-threadgroup pattern, MM 25.140.5). A threadgroup barrier at any scope does not provide it (measured, same section).
- *Ordering.* An atomic count decides who is last; atomicity is measured (section 9.3). Without the fences the count can be right and the data stale.
- *Forward progress.* The last-threadgroup pattern needs none, because no threadgroup waits: whoever counts last does the follow-on work. Any pattern that waits needs the producer resident: in the forward-progress probe the first resident wave spun to its cap before the producer, dispatched last, ran at all (measured, MM 25.141.1; the grid sizes and counts are section 1.5). In the tested configurations a grid barrier among resident threadgroups, fenced and capped, kept totals exact in every run and never reached its cap (measured, MM 25.141.1). No progress guarantee follows beyond those configurations, and the barrier costs more than the dispatch boundary it replaces (section 12.7).

Correction: “cross-threadgroup handoff requires a dispatch boundary” (MM 0.14) is the safe default, not a hardware limit; fenced atomic patterns work within one dispatch under the conditions above.

Residency of large threadgroups (unresolved). For 1,024-thread threadgroups the forward-progress probe implies at least 10 co-resident per core and the driver formula gives 3 (MM 25.141.1, MM 25.141.20); no residency count may be assumed, and a spin-waiting grid of them must not be sized from either number (section 1.4, rule 5).

Dispatches

Dispatch order and the absence of observed preemption are as stated at the head of the chapter (section 1.5). A serial encoder places a barrier token after every dispatch; a concurrent encoder lets a dependent kernel start early, but its threadgroups queue behind all of the producer’s, so only the producer’s last-wave tail overlaps (measured, MM 25.141.1; costs chapter 12). The GPU’s L2 is not coherent with CPU stores: a recently read line of a write-combined buffer the CPU updates reads stale (measured, MM 0.15). A submitted command buffer outlives its host process (section 13.9).

Safe ordering patterns

1. Within a simdgroup: program order, plus a wait on the producer’s actual slot for every late value. Never rely on distance.
2. Between simdgroups: one threadgroup barrier at a scope covering the memory used (276 for threadgroup memory, 154 for device memory).
3. Between threadgroups in one dispatch: a device fence after the stores, an atomic count, a device fence again before reading; or a dispatch boundary. Never a threadgroup barrier alone.
4. Waiting on another threadgroup only when every participant fits in measured resident capacity at the kernel’s own register and threadgroup-memory use, with capped spins and work pulled from an atomic queue (MM 17 rule 40); never spin on a producer that may not be resident.
5. Between dispatches: a serial encoder, or a concurrent encoder with barriers at every true dependency.

Not known. MMA-to-ALU interlock; the cross-simdgroup winner of same-address stores; whether an execution-only barrier orders device memory by design; why a special-register read on slot 7 stores nothing; per-lane min, max and unsigned long-form atomic returns; atomics beyond rank 0; 64-bit atomics

on hardware; acquire and release semantics in general (only the fence-count-fence pattern is measured); the function of the barrier-token variants 0x400 and 0x60000502; the residency count for 1,024-thread threadgroups.

Evidence. MM 0.8, MM 0.14, MM 0.15, MM 6, MM 13, MM 14, MM 17 rules 4, 7, 35 and 40, MM 25.90, MM 25.95, MM 25.96, MM 25.104, MM 25.115, MM 25.117, MM 25.120.1, MM 25.121, MM 25.133, MM 25.140.3, MM 25.140.5, MM 25.141.1, MM 25.141.20, MM 25.144.6, MM 25.145.2, MM 25.145.4, MM 25.146, MM 25.197; isa/g17-barrier-scopes.txt, isa/g17-atomic-family.toml, isa/g17-atomic-semantics.json, isa/g17-wait-mask-census.json, isa/g17-execution-storewinner-results.json, docs/g17-first-dispatch-trials.md, agxforge/g17/cc.py, tools/g17emu.py, tools/tensorops-model/scoreboard-store-census.json.

10 The tensor unit

The tensor unit (Apple’s “neural accelerator”) performs one $16 \times 16 \times 16$ matrix multiply-accumulate per instruction, $D = C + A.B$ or $D = A.B$: 4,096 multiply-accumulates and 8,192 floating-point operations per issue, with one tensor-issue slot per SIMD and four per core (measured, MM 0.1, MM 8; a logical resource model, not a floorplan). It is distinct from the older 8×8 simdgroup-matrix engine (section 10.13) and from the Apple Neural Engine, which this reference does not cover.

The instruction’s operands are register tuples spread across the 32 lanes of one simdgroup, and no unit between memory and those registers handles them as a matrix. Most of the rules summarised here follow from that.

- A 16×16 tile has 256 elements; each lane holds 8 of them in fixed slots. The map is closed-form (section 10.4), and it is produced only by per-lane address arithmetic in ordinary ALU code; a “tensor load” is a gather, and there is no tile-layout unit (section 10.8).
- A lane’s 8 elements occupy an aligned tuple: 2 registers for int8, 4 for 16-bit types, 8 for fp32 operands and for every accumulator. The 10-byte instruction names each tuple in units of its width, attaches a type code to A and B (a transpose adds 32 to it) and a lifetime modifier to every register (sections 10.2 and 10.3). A transpose changes how the registers are read and moves no data.
- For half inputs, the case the rule was fitted on, the sixteen products of each output are exact and are reduced by a fixed tree with an fp32 rounding at every node, the accumulator entering first (section 10.6); bfloat and truncated-fp32 inputs are recorded only at single points. fp32 operands are truncated to 10 fraction bits on the way in; int8 sums wrap or, with one bit, saturate once per issue.
- The MMA waits for its operand loads only through an 8-bit mask naming the scoreboard slots those loads fill; a slot it does not name is not waited on, and in every measured case the MMA then read zeros, with no error (section 10.7).
- Because D is eight ordinary registers per lane in the same lane map, it can be the next MMA’s C, A or B with no store (section 10.9). What the next MMA then computes depends on how the producing kernel labeled its rows, not on the instruction: the same hand-off computes the logical $A2 \cdot D$ in this compiler’s measured kernels and a row-rotated product in Apple-compiled `simdgroup_matrix` code (section 10.5).
- None of the ten admitted forms takes an fp8, int4 or fp4 operand, applies a block scale or accumulates in 16 bits, and no cross-simdgroup register path was found; in every route tested these are built from ordinary instructions around the ten forms (sections 10.10 and 10.11). int8 and uint8 are native.

Four of these replace earlier readings; the corrections are recorded where each is measured. The “two layouts” are one map and a row rotation (section 10.4); the reduction tree is now fixed by 1,680 discriminating stimuli (section 10.6); the “result ring” is a wait mask (section 10.7); and the feed relabelings come from the way Apple’s kernels pack their tiles (section 10.9).

Notation.

- **MMA**: one tensor multiply-accumulate instruction. “With-C” forms read an accumulator C; “no-C” (from-zero) forms do not.
- **The 16-bit MMA**: tensor MMA, 16-bit $\times$ 16-bit, accumulate (op5106), the form most measurements use; the other nine forms are named in full (section 10.2).

- **Tile:** a 16×16 matrix. A is $M \times K$ (16×16), B is $K \times N$, C and D are $M \times N$. Larger GEMMs are grids of tiles.
- **Fragment:** the part of one tile held by one lane. Every lane holds 8 logical elements of a tile, numbered by a **slot** j in $0..7$; lane 1 is $0..31$ (SR130, the lane index within the simdgroup, [chapter 6](#)).
- **Tuple:** the aligned group of consecutive registers holding a fragment: 2 registers for int8, 4 for half or bfloat, 8 for fp32 operands and for every accumulator. A tuple's base register must be a multiple of its width.
- **Accumulator group:** an 8-register fp32 or int32 tuple (C and D). Legal bases are R0, R8, ..., R112 (15 groups).
- **Modifier:** the immediate that follows every register operand in the decoded form, carrying the operand's lifetime bits (release and keep). The general rule is in [chapter 2](#), the tensor-specific use in [section 10.3](#).
- **Feed:** using one MMA's D as a later MMA's A, B or C without a store.
- **Memory bridge:** storing D to row-major memory with a tensor store and reloading it with a tensor load.
- $\text{rotl1}(x)$ / $\text{rotr1}(x)$: rotate a four-bit row index left or right by one bit.
- $\text{RNE32}(x)$: round x to fp32, nearest-even.
- "pos" layout and "pos_b" layout: the two (lane, slot) maps of [section 10.4](#).
- **Silent:** a violation produces wrong or zero results with no error from compiler, driver or API.
- The statement kinds are those of [section 0.7](#), with *decoder* written *decoder only*.

Every result in the chapter is from one M5 Pro (H17s, 20 cores) and one toolchain; the Metal-path evidence files behind most of it do not record the OS build ([section 1.1](#); MM 0.1, MM 20).

10.1 Access through Metal

Metal 4's TensorOps API reaches the unit through four layers, and only the last is a matrix instruction (Apple's compiler emits, MM 1):

1. `mpp::tensor_ops::matmul2d<descriptor, execution_simdgroups<N>>::run` is header code. It calls extern "C" symbols named `__tensorops_impl_matmul2d_op_run_<left>_<right>_<dest>[_v2]`, declared in the `air.externally_defined` section, so a TensorOps kernel's AIR holds only a call to one of these symbols.
2. The symbols resolve at native compile time from `MetalPerformancePrimitives.framework/Resources/libTensorOps.rtlb`, an ar archive of 1,970 `air64_v28` bitcode members, 1,475 of them `matmul2d_op_run_*`.
3. Inside the library, `air.supports_family(i32 1010)` (MTLGPUFamilyApple10: M5 and A19 Pro) selects `air.simdgroup_matrix_16x16x16_multiply_accumulate`; family 1009 selects the older `air.simdgroup_matrix_8x8_multiply_accumulate`. Which engine a TensorOps kernel uses is decided by this predicate in Apple's library.
4. The $16 \times 16 \times 16$ intrinsic lowers to one native instruction, tensor MMA, 16-bit $\times$ 16-bit, accumulate (`op5106`), or a sibling from [section 10.2](#).

The AIR intrinsic surface is `16x16x16_multiply_accumulate.f.f.v8f32.{v8f16|v8bf16|v8f32}.{v8f16|v8bf16|v8f32}.v8f32` (nine A/B combinations, fp32 accumulator only) plus the integer widening form .

`{s|u}. {s|u}.v8i32.v8i8.v8i8.v8i32`. A fragment is a per-lane $\langle 8 \times T \rangle$ value (Apple’s compiler emits, MM 1). Hand-written AIR reaches the intrinsic directly, which is how the saturating int8 form, fp8 and the scaled spelling were reached (sections 10.2 and 10.11).

`relaxed_precision` selects the mixed float forms. Without it, the mixed float intrinsics fall to the legacy datapath with a different layout; with it they reach tensor MMA, fp32 $\times$ fp32, accumulate (op5098), tensor MMA, fp32 A $\times$ 16-bit B, accumulate (op5100) and tensor MMA, 16-bit A $\times$ fp32 B, accumulate (op5104). It is inert for forms already native (Apple’s compiler emits, measured, MM 17 rule 29). *Correction:* the reading that the fp32-A forms, with and without an accumulator (the latter tensor MMA, fp32 A $\times$ 16-bit B, no accumulator (op5101)), are never compiler-emitted (MM 19) is withdrawn; they are emitted under `relaxed_precision`.

Apple’s library body changes with the `simdgroup` count (Apple’s compiler emits, MM 25.93). At `execution_simdgroups<1>` the slot-29 system-register vector is [52] and the body reads only SR130; at `<2>` and `<4>` it is [52, 53] and the body also reads SR133 (the `simdgroup` index within the threadgroup). The $N > 1$ body is a per-simdgroup partition: no barrier, no threadgroup memory, only the 16-bit MMA for the matrix work, plus offset arithmetic (16-bit shift right, 32-bit result (op17016); 16-bit AND with immediate, 32-bit result (op426); 16-bit plus 16-bit, 32-bit result (op10286); add 16-bit to 32-bit (op10285)). How many MMAs each `simdgroup` runs is a library tiling choice: for $32 \times 32 \times 64$, `<2>` halves 32 to 16 and `<4>` stays at 16; for $64 \times 64 \times 64$, `<2>` keeps 64 and `<4>` halves to 32.

The API imposes rules of its own that the hardware does not (measured unless marked):

- `execution_simdgroups<N>` must equal the `simdgroups` launched. A narrower scope returns zero with no error (MM 14, MM 17 rule 8). **Silent.**
- `reduce_rows` and `reduce_columns` refuse to compile at `execution_simdgroups<2>` or above; an input cooperative tensor must be single-simdgroup and cannot be transposed, although the hardware transposes a fed operand (Apple’s compiler, MM 12, MM 0.9).
- Apple’s header `MPPTensorOpsMatMul2dImpl.h` (macOS 26.4 SDK, lines 3763–3765) asserts that M and N are multiples of 8 and at least one is a multiple of 16. Per-lane capacity $M \times N / 32$ is therefore always a multiple of 4 (4, 8, 12, 16, 20, 24 ...); 24×24 fails the assert (Apple’s compiler, MM 12).
- The Metal 4.1 fp8 path is blocked at four points on this OS: the frontend needs `-std=metal4.1`, the loader refuses the 4.1 language tag and the AIR 2.9 target, the OS `libTensorOps.rtlib` has no fp8 or scale symbols, and the OS tensor library lacks the builtins. Hand-written AIR still reaches fp8 (MM 11).

Evidence. MM 1, MM 12, MM 14, MM 17, MM 19, MM 25.93.

10.2 MMA instruction forms

Every admitted tensor MMA is 10 bytes, scheduling class 172 for the 16-bit MMA, and writes an 8-register fp32 or int32 destination (decoder and measured, MM 0.3, MM 2). In every Apple-emitted with-C instance D and C are the same tuple (481 of 481), so that census contains only in-place accumulation (MM 2). The sections this chapter draws on report no execution of an out-of-place form (D a different tuple from C); treat it as untested. Table 10.1 lists the ten admitted forms.

Table 10.1. The ten admitted tensor MMA forms, with operand types and tuple widths, opcodes with and without C, and arithmetic.

Form	A: type, tuple	B: type, tuple	D	With C	No C (from zero)	Arithmetic
16 × 16	half or bfloat, 4	half or bfloat, 4	fp32, 8	tensor MMA, 16-bit × 16-bit, accumulate (op5106)	tensor MMA, 16-bit × 16-bit, no accumulator (op5107)	exact products, the tree of section 10.6
16 × fp32	half or bfloat, 4	fp32, 8	fp32, 8	tensor MMA, 16-bit A × fp32 B, accumulate (op5104)	tensor MMA, 16-bit A × fp32 B, no accumulator (op5105)	fp32 operand truncated to 10 fraction bits
fp32 × 16	fp32, 8	half or bfloat, 4	fp32, 8	tensor MMA, fp32 A × 16-bit B, accumulate (op5100)	tensor MMA, fp32 A × 16-bit B, no accumulator (op5101)	fp32 operand truncated
fp32 × fp32	fp32, 8	fp32, 8	fp32, 8	tensor MMA, fp32 × fp32, accumulate (op5098)	tensor MMA, fp32 × fp32, no accumulator (op5099)	both operands truncated
int8	signed or unsigned int8, 2	signed or unsigned int8, 2	int32, 8	tensor MMA, int8 widening to int32 (op10384)	tensor MMA, int8 widening, no accumulator (op10385)	exact products, mod 2 ³² by default; saturating variant below

All ten execute as tabulated (measured, MM 2, MM 5, MM 11). The no-C form “starts from the first reduced group rather than an encoded zero tuple” (MM 0.3).

Decoded, the 16-bit MMA’s operands are (decoder only, MM 2) D:GPR32tup8, imm, imm, A:GPR32tup4, imm, imm, B:GPR32tup4, imm, imm, C:GPR32tup8, imm, imm; its no-C sibling drops the C triple; the running example’s first MMA ([section 0.4](#)) is this no-C form decoded and annotated. Operands are numbered from 0 (D). Each register is followed by two immediates: the first is its modifier ([section 10.3](#); operands 1, 4, 7 and 10), and the second after A and after B carries that operand’s type code (operands 5 and 8); the second after D (operand 2) carries the saturation flag on the int8 form. Register fields are encoded in units of the tuple width, so a tuple must be aligned. An encoder must refuse an unaligned base and must decode its output back to the same opcode, length and operands; this project’s encoder once rounded an odd index silently (MM 0.4, MM 2).

[Table 10.2](#) gives the operand type codes (decoder and measured, MM 0.3, MM 2).

Table 10.2. Operand type codes of the tensor MMA. The first row gives the field’s position for A and for B.

Type	Code	Where
float (fp32)	1	A at byte6[2], B at byte7[6]
half	2	
bfloat	3	
unsigned int8	11	
signed int8	75	

A transpose flag adds 32 to the corresponding operand’s type code (byte8[2] for A, byte8[3] for B). The signedness of an int8 operand is selected independently for A and B. A transpose changes only how a fragment is read: all four flag combinations compute exactly $D = C + A.B$, a transposed operand’s loads

keep the same index register and displacements, and transposition has no measured issue cost (measured, MM 3, MM 8).

The 10-byte MMA’s structural selector bits are these (decoder and measured, MM 0.4, MM 2): byte8[1] selects the no-C sibling; byte5[7] makes B fp32; byte8[4] makes A fp32 on the float forms; byte6[0] selects the int8 family; byte0[7] is bit 0 of the destination tuple index (40 of 40 instructions in six objects match `byte0[7] == N & 1`), not a flag; flags bits 24–31 are the scoreboard wait mask (section 10.7; carriers in section 3.4). Use the JSON field map in the repository, not this list, to construct bytes; the field map classifies all 80 bits of the 16-bit MMA (39 in mapped fields, 25 structural, 16 inert) and `mmaenc.py` round-trips 800 of 800 compiler-emitted MMAs byte for byte (MM 2, MM 6).

A single-bit census (measured, MM 2) dispatched all 80 one-bit variants of the 16-bit MMA: none was refused by the driver or faulted, every decoder-inert bit was hardware-inert, and every flags-word bit was inert on a single MMA except byte0[3] (the slot-7 wait bit). Of the decoder-refused one-bit variants, six left the accumulator unchanged (`D == C`) and eight changed the result, so decoder refusal does not predict what the hardware executes. No encoding property separates the eight from the six (decoder only, MM 25.36). Authored forms Apple’s compiler never emits (swapped A/B tuples, both transposes, A reinterpreted as bfloat) execute as layout and arithmetic predict (measured, MM 2, MM 15).

The int8 form has a saturating variant (measured, MM 11, MM 25.42). byte8 bit 4 on the int8 MMA, tensor MMA, int8 widening to int32 (`op10384`), or its no-C form, tensor MMA, int8 widening, no accumulator (`op10385`), selects saturation; on the int8 form byte8 carries three independent executed features: bit 2 transposes A, bit 3 transposes B, bit 4 saturates. Apple’s compiler bytes are 0x40 untransposed and 0x44 with A transposed; with saturation 0x50 and 0x54. In the decode, transposing A moves operand 5 only, transposing B moves operand 8 only, and saturation moves operand 2 (immediate 9 to 41) identically under all four transpose combinations; both int8 forms accept the bit (decoder only, MM 25.11).

- *AIR route.* widening_multiply_accumulate_saturate in hand-written AIR lowers to the int8 MMA with the bit set. `libTensorOps.rtl` contains no string containing saturate, so Apple’s library never emits it (Apple’s compiler, MM 11). The public matrix interfaces declare no saturating operation either: the MPP headers of the macOS 27.0 SDK (where `matmul2d_descriptor::mode` is `multiply` or `multiply_accumulate`) and the `metal_simdgroup_matrix`, `metal_tensor` and `metal_cooperative_tensor` headers of the Metal 32023 toolchain contain no string saturat (header search, 2026-10-04).
- *AIR suffix.* In the four-letter suffix A’s signedness is letter 2 and B’s letter 3; in the two-letter form A is always signed, so unsigned A cannot be spelled. The outer letters are parser-dead, not hardware fields (MM 11, MM 20).
- *Cost.* Same issue rate as the wrapping form to three places, identical register classes (measured, MM 11).
- *Encoder coverage.* This project’s MMA encoder emits the saturating form with C (`mmaenc.mma(saturate=True)`), byte-exact against Apple’s `_saturate` spelling) and refuses the no-C saturating form, whose bytes 6 and 7 differ; the eight no-C saturating MMAs are the ones a coverage count of 405 of 413 MMAs across 152 kernels misses (this compiler, MM 15, MM 25.11). Its saturating chains run on hardware and clip once per issue (MM 25.89).

The arithmetic is in section 10.6.

Apple’s tables declare many more matrix forms than the ten, and only the ten above are decoder-admitted, all of them 32-bit-accumulator forms (decoder only, MM 2, MM 0.3, MM 25.6). The two declaration counts

in the sources (132 by scheduling class, 535 by opcode range) count different things; [appendix A](#) reconciles them.

- Admission is gated on the CPU name passed to the decoder: g17s, g17g and g18 admit the same ten; g17, g16, g16s, g16g, g15 and g15s decode the same bytes as unrelated instructions (exhaustive three-flip sweep). The other declared forms are a bounded negative: none was reached within three flips of the ten seeds, nor from 1.1 million flip candidates plus a five-deep search (decoder only, MM 2, MM 25.4 “F4, narrowed”).
- Three operand classes are exclusive to the accelerator across all declared accelerator opcodes: class 335 (the 8-word tuple, D of every 32-bit-accumulator form, 18 of 18), class 159 (the 4-word tuple, D of every 16-bit-accumulator form, 36 of 36) and class 41 (the 2-word tuple). There is no fourth (decoder only, MM 25.6).
- The 36 narrow-accumulator rows, table-only narrow-accumulator MMA, first (op5108) through table-only narrow-accumulator MMA, last, no accumulator (op5143), use the same operand classes and counts as forms that decode, yet none was admitted from 1.1 million mutations plus a five-deep frontier, under any CPU name, and no witness emits one. Do not select or synthesize them (bounded negative, MM 25.13).
- The 475 _shifted declarations use none of the three accelerator classes; they are a structurally different family that shares scheduling classes 174 and 175 (decoder only, MM 25.6).
- No admitted form accumulates in 16 bits. Hand-written AIR with a half accumulator lowers to the 16-bit MMA with eight half to fp32 widening (op1004) before and eight fp32 to half narrowing (op1016) after (the code section grows from 42 to 52 instructions); a bfloat accumulator uses the 16-bit MMA with bit operations and eight fp32 to bfloat16 narrowing, unconfirmed (op1048) (62 instructions) (Apple’s compiler emits, MM 25.13).
- No declared MMA form has a scale operand in Apple’s tables (decoder only, MM 11).

Not known. A structural bit-71 variant of the int8 widening MMA that Apple’s decoder agx3dis does not admit runs bit-exact in all three kernels that carry it; its 10-byte boundary is settled, its field meaning and decoder identity are open (MM 20, T11).

Evidence. MM 0.3, MM 0.4, MM 2, MM 11, MM 15, MM 25.4 “F4, narrowed”, MM 25.6, MM 25.11, MM 25.13, MM 25.36, MM 25.42.

10.3 MMA operand modifiers

The modifier enum {0, 16, 32, 48} (weight 16 **release**, weight 32 **keep**; decoder only, MM 6) and the destructive release semantics are defined in [chapter 2](#) ([section 2.5](#); measured, MM 6, MM 0.4, MM 17 rule 37).

The modifier after A or B is 16 exactly when that tuple is not read again (Apple’s compiler emits, MM 6). The running example ([section 0.4](#)) shows the pattern: each fragment is read by two MMAs, the first keeps it (0) and the second releases it (16). Because the release is destructive, a misplaced one changes results. A released register read zero in every controlled run, but what it reads is not established in general ([section 2.5](#)): a later reader may assume neither zero nor the old value. A release set while any later consumer remains loses the fragment, with no error in any measured case. The release acts per lane, and nothing reclaimed a released register within the measured bound ([section 2.5](#), MM 25.95); reclamation beyond that bound is not excluded.

Whether a release point is safe depends on where the fragment came from; this project’s lowering got it wrong on hardware (this compiler, measured, MM 25.89). With three accumulators free, tlower ran a 2×2 stage as two accumulator groups and released A at its last read within each group. That is correct for an A loaded from memory, which each group reloads, and wrong for a register-fed A, which has no memory copy: the second group read released registers and its tile came back all zero. The fix releases a register-fed A at the last group that reads the row. `agxforge/g17/tensorlife.py` (`released_reads`) decodes every emitted body and refuses any MMA read of a released, unrewritten register; it fires on the faulty program at exactly the two faulty issues and passes all 720 Apple corpus programs that contain MMAs. A lowering that splits a stage into accumulator groups has to count the fragment’s reads in every group before placing the release, and the decoded-body check is what catches it when it does not.

The destination keep flag (0x20, mmaenc keyword `more`) is the keep bit of operand 1, the modifier after D: byte4 bit 3, instruction bit 35 (`auth.carriers(5106, 1)[5] = [(4, 3, 0)]`) (decoder only, MM 6, MM 25.22). The destination modifier carries keep and never release (`carriers(5106, 1)` has no bit-4 entry).

- *Rule (Apple’s compiler emits, MM 25.22):* set implies that the next MMA writes the same destination tuple. In a run of n MMAs accumulating into one tuple, the first n-1 carry it and the last does not. It is not a chain-boundary marker.
- *Hardware effect:* none observed. Flipping it is inert on a single MMA and on a two-step chain (measured, MM 2); of keep and release it is the conservative bit, since it only asserts reuse (MM 6).
- *Emitter rule:* set it exactly when the next MMA reuses the tuple, as Apple does. Clearing it where Apple sets it was not shown to cost anything; setting it where the next MMA does not reuse the tuple has no witness and was not tested.

Evidence for the one-way rule. No population that tested it contains a counterexample. Over 2,192 MMAs in 140 programs (1,093 carry the flag), “the next MMA reuses the tuple” has precision 100 percent (1,093 of 1,093) and recall 99.7 percent (1,093 of 1,096), while “some later MMA reuses it” has recall only 69.0 percent (MM 25.22). A larger census has zero counterexamples among 540,396 flag-set instances and 155,963 counterexamples to the converse (MM 6). The three misses in the 2,192-MMA census are one pattern: kernel `conv-c64k1`, destination D=3382, with the same 16 intervening fragment spill store (`op595`) instructions (MM 25.22). A reader may take a set flag to mean “the next MMA reuses the tuple” but should infer nothing from a clear one. (The population sizes quoted in the sources, including MM 0.4’s, are reconciled in [appendix A](#).)

Correction: MM 20 (“Hardware identity”) still lists this flag’s meaning (H5) as open; it is resolved (MM 25.22, MM 6; ledger row H5, MM 24).

The accumulator’s own modifier is operand 10 (the accumulator itself is operand 9). No field maps to it: the field map’s single-bit census classifies all 80 bits of the 16-bit MMA and none reaches operand 10, the decoder prints 0, and `auth.carriers(5106, 10)` raises (decoder only, MM 6). An author cannot set it; a release of C therefore cannot be encoded in one bit. A multi-bit encoding is not excluded, because the census is single-bit (open).

Evidence. MM 0.4, MM 2, MM 6, MM 17 rule 37, MM 25.22, MM 25.89, MM 25.95.

10.4 Fragment layouts

A tile has 256 elements and a simdgroup 32 lanes, so each lane holds 8 elements in slots $j = 0..7$. The map between matrix coordinates and (lane, slot) is closed-form, measured with one-hot probes, five preregistered predictions, 6,651 dispatches and no findings against (measured, MM 3). With lane 1 in 0..31:

```
A (transA = 0) and D:   row = 8*(l>>4) + 2*((l>>1)&3) + (j>>2)
                        col = 8*((l>>3)&1) + 4*(l&1) + (j&3)

B (either flag):       k  = 4*(l>>4) + ((l>>1)&3) + 8*(j>>2)
                        col = 8*((l>>3)&1) + 4*(l&1) + (j&3)

A (transA = 1):        k  = 4*(l>>4) + ((l>>1)&3) + 8*(j>>2)
                        row = 2*(j&3) + 8*(l&1) + ((l>>3)&1)
```

An A or D lane holds a 2×4 block: slots 0–3 are row $2m$, four consecutive columns; slots 4–7 are row $2m+1$, the same columns. A B lane holds K rows k_0 and k_0+8 of four consecutive columns. A transposed A uses the B formula for its K index. “transB” does not change the B map (MM 3).

Figure 10.1 draws the A/D map.

Lanes 0–7 hold rows 0–7 and columns 0–7, lanes 8–15 hold columns 8–15 of the same rows, and lanes 16–23 and 24–31 do the same for rows 8–15. Within a quadrant each lane holds a 2×4 block of two adjacent rows and four adjacent columns, slots 0–3 in the upper row and slots 4–7 in the lower. Because a lane’s four slots in a row are consecutive columns, one 8-byte access per row loads them. Rows r and $r + 8$ sit in the two halves of the same lane group, which matches how the tensor loads move an 8-row by 16-column sub-tile (section 10.8).

The same map, by lane quadrant (MM 0.5), is in table 10.3.

Table 10.3. A/D fragment map by lane quadrant, with the rows and columns each eight-lane group holds.

Lanes	Rows	Columns	Within the eight-lane group, local lane q
0–7	0–7	0–7	rows $2*(q>>1)$ and $+1$, columns $4*(q&1)$ to $+3$
8–15	0–7	8–15	same, column base $+8$
16–23	8–15	0–7	same, row base $+8$
24–31	8–15	8–15	same, both bases $+8$

A packer needs the inverse maps. Both are bijections onto the 256 (lane, slot) positions (re-verified, MM 3):

```
pos(r, c) = (16*(r>>3) + 8*(c>>3) + 2*((r>>1)&3) + ((c>>2)&1), 4*(r&1) + (c&3))   A and D
pos_b(k, c) = (16*((k>>2)&1) + 8*(c>>3) + 2*(k&3) + ((c>>2)&1), 4*(k>>3) + (c&3))   B
```

Each returns (lane, slot).

Worked example: four sample lanes (derived from the formulas above). Table 10.4 gives the fragments of four lanes.

		column															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
row	0	0.0	0.1	0.2	0.3	1.0	1.1	1.2	1.3	8.0	8.1	8.2	8.3	9.0	9.1	9.2	9.3
	1	0.4	0.5	0.6	0.7	1.4	1.5	1.6	1.7	8.4	8.5	8.6	8.7	9.4	9.5	9.6	9.7
	2	2.0	2.1	2.2	2.3	3.0	3.1	3.2	3.3	10.0	10.1	10.2	10.3	11.0	11.1	11.2	11.3
	3	2.4	2.5	2.6	2.7	3.4	3.5	3.6	3.7	10.4	10.5	10.6	10.7	11.4	11.5	11.6	11.7
	4	4.0	4.1	4.2	4.3	5.0	5.1	5.2	5.3	12.0	12.1	12.2	12.3	13.0	13.1	13.2	13.3
	5	4.4	4.5	4.6	4.7	5.4	5.5	5.6	5.7	12.4	12.5	12.6	12.7	13.4	13.5	13.6	13.7
	6	6.0	6.1	6.2	6.3	7.0	7.1	7.2	7.3	14.0	14.1	14.2	14.3	15.0	15.1	15.2	15.3
	7	6.4	6.5	6.6	6.7	7.4	7.5	7.6	7.7	14.4	14.5	14.6	14.7	15.4	15.5	15.6	15.7
	8	16.0	16.1	16.2	16.3	17.0	17.1	17.2	17.3	24.0	24.1	24.2	24.3	25.0	25.1	25.2	25.3
	9	16.4	16.5	16.6	16.7	17.4	17.5	17.6	17.7	24.4	24.5	24.6	24.7	25.4	25.5	25.6	25.7
	10	18.0	18.1	18.2	18.3	19.0	19.1	19.2	19.3	26.0	26.1	26.2	26.3	27.0	27.1	27.2	27.3
	11	18.4	18.5	18.6	18.7	19.4	19.5	19.6	19.7	26.4	26.5	26.6	26.7	27.4	27.5	27.6	27.7
	12	20.0	20.1	20.2	20.3	21.0	21.1	21.2	21.3	28.0	28.1	28.2	28.3	29.0	29.1	29.2	29.3
	13	20.4	20.5	20.6	20.7	21.4	21.5	21.6	21.7	28.4	28.5	28.6	28.7	29.4	29.5	29.6	29.7
	14	22.0	22.1	22.2	22.3	23.0	23.1	23.2	23.3	30.0	30.1	30.2	30.3	31.0	31.1	31.2	31.3
	15	22.4	22.5	22.6	22.7	23.4	23.5	23.6	23.7	30.4	30.5	30.6	30.7	31.4	31.5	31.6	31.7

Figure 10.1. Lane and element slot of each element of a 16×16 A or D tile. Each cell is *lane.slot*: the lane that holds the element and which of its eight element slots it occupies (a half A fragment packs the eight slots into four registers, an fp32 D into eight). The coordinates are canonical: the instruction’s own, before any packing convention (section 10.5). The four quadrants are the four groups of eight lanes (blue, red, teal, violet: lanes 0–7, 8–15, 16–23 and 24–31); alternate lanes are shaded darker and lighter to mark each lane’s 2×4 block. Lane 5, the lane followed in section 0.4, is highlighted; this compiler’s B-row packing stores those elements at application rows 2 and 10. The map was measured with one-hot probes over 6,651 dispatches (MM 3); the figure is generated from the formulas in section 10.4, which are checked against one another exactly.

Table 10.4. A/D and B fragment contents of four sample lanes, derived from the layout formulas.

Lane	A/D: slots 0–3	A/D: slots 4–7	B: slots 0–3	B: slots 4–7
0	D(0, 0..3)	D(1, 0..3)	B(0, 0..3)	B(8, 0..3)
9	D(0, 12..15)	D(1, 12..15)	B(0, 12..15)	B(8, 12..15)
17	D(8, 4..7)	D(9, 4..7)	B(4, 4..7)	B(12, 4..7)
22	D(14, 0..3)	D(15, 0..3)	B(7, 0..3)	B(15, 0..3)

Lane 22 step by step: $1 \gg 4 = 1$, $(1 \gg 1) \& 3 = 11 \& 3 = 3$, $(1 \gg 3) \& 1 = 0$, $1 \& 1 = 0$. For A/D, $\text{row} = 8 + 6 + (j \gg 2)$ and $\text{col} = 0 + 0 + (j \& 3)$, so slot 6 is D(15, 2). Inverse: $\text{pos}(15, 2) = (16 * 1 + 8 * 0 + 2 * (7 \& 3) + ((2 \gg 2) \& 1), 4 * (15 \& 1) + 2) = (22, 6)$. For B, $k = 4 + 3 + 8 * (j \gg 2)$, so slot 6 is B(15, 2), and $\text{pos}_b(15, 2) = (16 * 1 + 0 + 2 * 3 + 0, 4 * 1 + 2) = (22, 6)$. With $\text{transA} = 1$, lane 22 slot j holds A($\text{row} + 2 * (j \& 3), k + 7 + 8 * (j \gg 2)$): slot 6 is A(4, 15).

One fragment map serves all opcodes: $\text{pos_b}(k, c) = \text{pos}(\text{rot11}(k), c)$ at 256 of 256 positions (measured, re-verified, MM 3). The B map is the A/D map with rows relabeled by a one-bit rotation. Check on lane 22: $\text{rot11}(7) = 14$ and $\text{pos}(14, 2) = (22, 2) = \text{pos_b}(7, 2)$. Whether a destination “reads as pos or as pos_b” depends only on the row labeling used by the kernel that produced it (derived in section 10.5).

Withdrawn: two destination layouts. The record once stated that the destination of the 16-bit MMA, tensor MMA, 16-bit $\times$ 16-bit, accumulate (op5106), and its no-C sibling used a different layout from the fp32 $\times$ fp32 MMA, tensor MMA, fp32 $\times$ fp32, accumulate (op5098), because measured destinations of different kernels disagreed (recon section 100). A check for a fixed relabeling between the two maps found that they differ exactly by rot11 on the row index at all 256 positions, the same rotation Apple’s cooperative-tensor coordinates carry. The difference came from the producing kernel’s row labeling, and the opcode claim was withdrawn (MM 3, MM 19). The same check later scoped the feed-mode relabelings (section 10.9).

For other operand types (measured, MM 3, MM 19):

- fp32 and int8 operands use the same A, B and D maps as half.
- tensor MMA, int8 widening to int32 (op10384) is among 23 forms whose destination is pos_b inside every 16×16 tile with row-major tile blocks, 1,024 of 1,024 lane and slot (MM 3). This describes Apple-compiled multi-tile destinations, consistent with the one-map rule above.
- In the fp32 $\times$ fp32 MMA the fp32 A operand and the fp32 D/C share one layout ($A_{\text{row}} == D_{\text{row}}, A_k == D_{\text{col}}, 256$ of 256), so a D can feed the next A with no store and no permutation (MM 19).
- Apple’s cooperative-tensor coordinates for a 16×16 single-simdgroup destination equal the B map with index 0 as column and index 1 as row; relative to the D map, rows are rotated by $y = ((r \& 7) < 1) \mid (r >> 3)$ (Apple’s compiler, MM 3).

Table 10.5 gives register widths and legal bases (measured, MM 0.7, MM 0.1).

Table 10.5. Registers per lane, alignment and highest legal base register for scalars, fragments and accumulators.

Value	Registers per lane	Alignment	Highest base
scalar	1	1	R125
int8 or uint8 fragment	2	2	R124
half or bfloat fragment	4	4	R120
fp32 A or B fragment	8	8	R112
fp32 or int32 accumulator	8	8	R112 (15 groups: R0, R8, ..., R112)

R120–R127 cannot hold an 8-register group, because any instruction naming R126 or above is squashed whole (section 2.1). MM 0.7 describes the int8 pair as “four byte elements per lane packed across the pair”; how the eight byte elements of a lane are ordered within the pair is not stated in the sections this chapter draws on (open).

There is no tile-layout unit: a tensor load is a per-lane gather at an ALU-computed address, and that arithmetic produces the layout (measured, MM 7, MM 19). Apple’s library computes, once per kernel, the index register $R32 = m(1) \times 128 + \text{col}(1)$ with $m(1) = 4 * (1 >> 4) + ((1 >> 1) \& 3)$ and $\text{col}(1) = 8 * ((1 >> 3) \& 1) + 4 * (1 \& 1)$: element (m, col) of a row-major tile at leading dimension 128. Its loads read $[R32 * 2 + \text{disp}]$ with displacements {0, 2048, 32, 4096, 6144} (2048 steps eight rows, 32 steps sixteen columns). Each access moves an 8-row by 16-column sub-tile, which is why rows r and r+8 sit in the two slot halves

(Apple’s compiler emits, MM 7). *Correction:* “the hardware performs the matrix-to-fragment transposition” is refuted (MM 7, MM 19).

Evidence. MM 0.5, MM 0.7, MM 3, MM 7, MM 19.

10.5 Coordinate systems and packing

Tensor code involves three kinds of coordinate, which have to be kept apart.

1. *Canonical instruction coordinates:* the map from (lane, slot) to a matrix position that the MMA itself uses, pos for A and D and pos_b for B (section 10.4). The MMA computes $D = A B + C$ in these coordinates.
2. *A kernel’s packing convention:* which memory element its loads place in each (lane, slot), and where its stores write each slot. This is software: a tensor load is a per-lane gather at an address the kernel computes.
3. *Application coordinates:* element (i, j) of the row-major matrix in memory.

A kernel is correct when its packing convention, carried through the canonical arithmetic, produces the right product in application coordinates. Two conventions are in use.

- *Canonical packing.* Apple-compiled `simdgroup_matrix` code packs A and D canonically (application row equals canonical row) and B canonically; the chaining relabelings were measured on such kernels (MM 3, MM 13).
- *B-row packing.* This compiler packs every tile the same way, A, B and the C/D tile it loads and stores: lane l, slot j holds application row $m(1) + 8*(j>>2)$ and column $col(1) + (j\&3)$, with $m(1) = 4*(l>>4) + ((l>>1)\&3)$ and $col(1) = (l\&8) + 4*(l\&1)$ (`agxforge/g17/indexgen.py`; `agxforge/g17/tlower.py`: “A rows m and m+8 of the tile, B rows k and k+8”, stores “rows m and m+8”). This is the B map applied to every tile. Apple’s tensor-ops library computes the same per-lane index for its tensor loads, and Apple-compiled multi-tile destinations read as pos_b (section 10.4; MM 3, MM 7).

For B the two conventions agree. For A and D they differ by a one-bit rotation of the row index: because $pos_b(r, c) = pos(rot11(r), c)$, this compiler puts application row r of A in canonical row $rot11(r)$. The rotation does not change the result. Rows of A are the free M index, so canonical row $rot11(r)$ of D holds application row r of the product, and the store, which uses the same convention, writes canonical row $rot11(r)$ back to application row r. Neither the columns nor the K index are permuted, so the reduction over K runs in canonical order and the result is the one the arithmetic rule of section 10.6 predicts in application coordinates.

Lane 5 of the running example. In canonical coordinates its first MMA’s D holds elements (4, 4)–(4, 7) in slots 0–3 and (5, 4)–(5, 7) in slots 4–7 (figure 10.1). This compiler computes $m(5) = 2$ and $col(5) = 4$, so its store writes slots 0–3 to application row 2 and slots 4–7 to row 10, columns 4–7. The two descriptions name the same registers: $rot11(2) = 4$ and $rot11(10) = 5$.

D fed as B. A feed hands D’s registers unchanged to the next MMA as its B operand. The MMA reads lane l, slot j as canonical B row $k = 4*(l>>4) + ((l>>1)\&3) + 8*(j>>2)$, and that register holds canonical D row $rot11(k)$.

- Under B-row packing (this compiler), canonical D row $\text{rot11}(k)$ is application row k of the product, so the consumer reads $B_{\text{eff}}[k][c] = D[k][c]$: the logical operand, with no instruction in between.
- Under canonical packing, canonical D row $\text{rot11}(k)$ is application row $\text{rot11}(k)$, so the consumer reads $B_{\text{eff}}[k][c] = D[\text{rot11}(k)][c]$: the row-rotated operand that [section 10.9](#) measures, which the other operand's K index must then be permuted to undo.

[Table 10.6](#) follows three slots through both conventions. For lane 0, slot 4, canonical D row 1 is read as B row 8; this compiler's producer placed application row $m(0) + 8 = 8$ of D in that slot, so the consumer receives D row 8 as B row 8. With canonical packing the same register carries D row 1, and $\text{rot11}(8) = 1$. A D fed as A needs no relabeling in either convention, because the consumer's A is packed the way the producer packed D .

Table 10.6. Three slots in canonical coordinates and in the two packing conventions, for a D tile fed as the next B .

Lane, slot	Canonical D	Read as B	D row, B-row packing	D row, canonical packing
0, 4	(1, 0)	(8, 0)	8	1
5, 0	(4, 4)	(2, 4)	2	4
5, 4	(5, 4)	(10, 4)	10	5

So the relabeling a feed needs is set by the producer's packing alone. The two measurements agree with this derivation ([section 10.9](#)): in this compiler's kernels the B , A_t and B_t feeds read the logical operand bit for bit and a reference that applied the relabelings failed, in the one square 32×32 stage measured ([MM 25.103](#)); in canonically packed kernels the feeds follow the fixed relabelings, the identity for A ([MM 13](#)).

10.6 Arithmetic by input type

Each result below is labelled an **exact rule** (a closed form that reproduces every measured output) or a **measured case** (single points or edge values); unresolved behaviour is marked open.

Half inputs, fp32 accumulator: exact rule (measured, [MM 4](#), [MM 0.6](#); fitted on half $\times$ half, 1,680 of 1,680 eps-triples and every stimulus). For one output element, with k the A -column index of [section 10.4](#):

```
p_i = RNE32(a[2i]*b[2i] + a[2i+1]*b[2i+1])    i = 0..7    adjacent pairs; products exact
q_j = RNE32(p_j + p_(j+4))                    j = 0..3    pairs eight apart
acc = C
acc = RNE32(acc + q_0); acc = RNE32(acc + q_1); acc = RNE32(acc + q_2); acc = RNE32(acc + q_3)
D   = acc
```

[Figure 10.2](#) shows the measured order.

The tree is neither balanced nor sequential. The first stage adds adjacent products, which is why $(-0) * 1$, with every other product zero, gives $+0$ (the signed-zero case below). The second stage pairs partial sums eight apart. The accumulator then enters before the four partial sums, within one issue. The from-zero form, tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107), follows the same tree without C and starts the chain from q_0 ("starts from the first reduced group", [MM 0.3](#)).

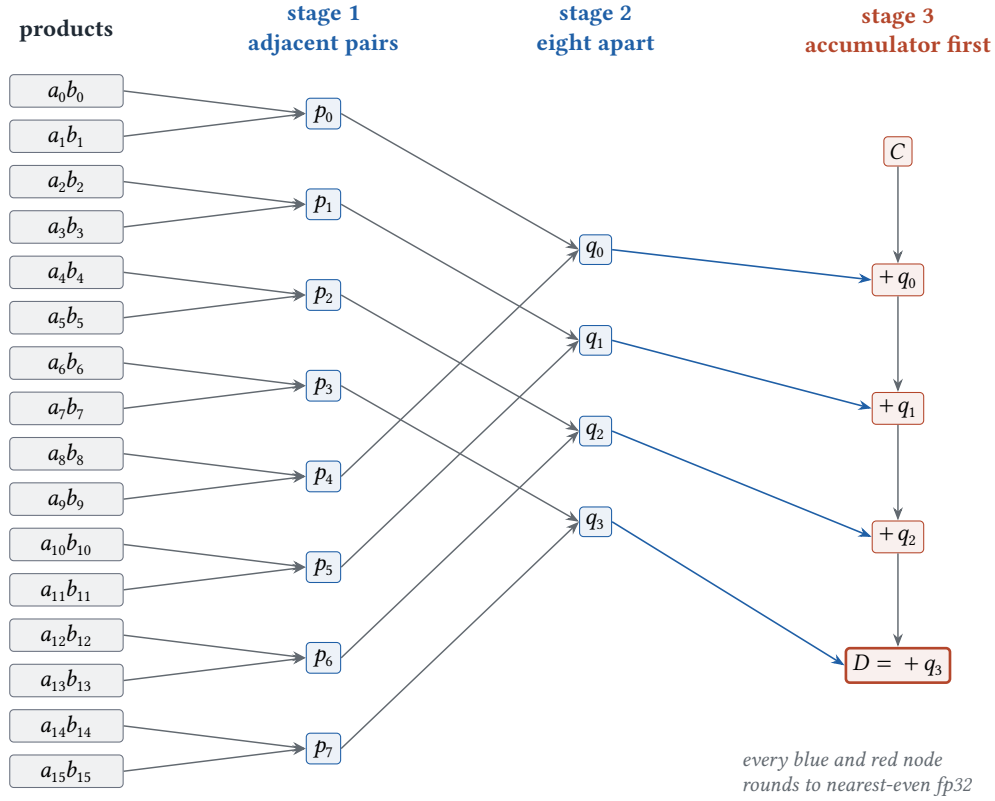


Figure 10.2. The measured reduction of one output element. One $16 \times 16 \times 16$ MMA computes each output element from sixteen exact products. Stage 1 adds adjacent products, $p_i = \text{RNE}_{32}(a_{2i}b_{2i} + a_{2i+1}b_{2i+1})$; stage 2 adds partial sums eight products apart, $q_i = \text{RNE}_{32}(p_i + p_{i+4})$; stage 3 starts from the accumulator C and adds $q_0, \dots, q_3$ in order, rounding at every step. The from-zero form, tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107), has no C and starts the chain from q_0 . The diagram shows data dependence, not physical adders (MM 4, MM 0.6).

How the tree was identified. The candidates were a sequential chain, a balanced tree over the natural K order, a balanced tree over K interleaved by eight and an exactly rounded sum, and, for the accumulator, adding C first or last. They differ only in the order of additions and where rounding happens, so the stimuli were placed on single output elements with half inputs, whose products are always exact in fp32 (an 11-bit by 11-bit significand product fits in 24 bits, and every such product lies between 2^{-48} and about 4.3×10^9 , inside fp32's normal range). A single-element stimulus then isolates the order of additions and the rounding. The main probe was the **eps-triple**: one product equal to 1 at a K position i and products equal to a small ϵ at two other positions j and k , over every placement ($16 \times 105 = 1,680$ stimuli). Whether the two ϵ s terms are added to each other before either meets the 1 changes the rounded result, so each triple answers one question about the tree's shape (MM 4; the triple design is from the record's section 4, which MM 4 cites).

- Rival fits on the 1,680 triples: balanced tree on natural order 976, interleave-by-8 balanced tree 1,392, the rule above 1,680. The triples fixed the pair and interleave stages.
- Round-to-nearest-even applies at every intermediate; round-toward-zero is rejected on every stimulus.
- Over a whole tile: 5,120 of 5,120 exact, against sequential 2,493, balanced 2,764, exact sum rounded once 2,742. Re-confirmed later with no correction (MM 4).

- Stimuli built for the purpose placed C; the worked example below shows how one separates C-first from C-last.

Worked example: why C enters first (executed on hardware for the measured value; the arithmetic below is the rule applied by hand). Let $\text{eps} = 2^{-24}$, half an fp32 ulp at 1.0. Take $C = 1.0$ and choose products so that the first two quad sums are $q_0 = q_1 = 3 \text{ eps}$ and $q_2 = q_3 = 0$ (for example products eps at K positions 0, 1 and 8 give $p_0 = 2 \text{ eps}$, $p_4 = \text{eps}$, so $q_0 = 3 \text{ eps}$ exactly; positions 2, 3 and 10 do the same for q_1 ; $\text{eps} = 2^{-12} \times 2^{-12}$, both normal half values).

```
C first (the measured rule):
acc = RNE32(1 + 3 eps)      = 1 + 4 eps      1.5 ulp is a tie; RNE picks the even neighbour, 2 ulp
acc = RNE32(1 + 4 eps + 3 eps) = 1 + 8 eps    3.5 ulp is a tie; RNE picks the even neighbour, 4 ulp
C last (any order that sums the q terms first):
q_0 + q_1 = 6 eps exactly;  RNE32(1 + 6 eps) = 1 + 6 eps    3 ulp, representable
```

Adding C first exposes each small partial sum to a tie against 1.0; summing the q terms first keeps them exact and rounds once. The hardware returns $1 + 8 \text{ eps}$ on the discriminating stimuli (measured, MM 4), which rules out every model that adds C at the end, including an exactly rounded sum.

Checks against independent code. The repository’s bit-exact model tools/tensorops-model/tensorops_model.py, which was not used to fit the rule, reproduces Apple’s $32 \times 32 \times 64$ library kernel on 6,144 of 6,144 elements (exact-sum-rounded-once: 1,948) and a previously failed tile on 1,024 of 1,024; Apple’s matmul2d is bit-exact to it on all eight shapes of the speed comparison (measured, MM 4, MM 25.98). The converse failure was also seen: this project’s tensor runtime once used a reference MMA that added C after the products, and the released multigemm class had been failing its own exact check on 417 of 1,024 elements for that reason (MM 25.89).

Accumulate-mode composition in Apple’s library: exact rule (measured, MM 4, MM 0.6, MM 15). The library’s own accumulate mode orders C the other way. With a nonzero C, matmul2d’s multiply-accumulate mode runs the MMA chain from zero and then loads C and adds it with ordinary fp32 adds (product first, C last), one extra rounding compared with putting C in the first MMA. The one-rounding difference between that mode and a single MMA with C has the same cause as the worked example. This describes how the library composes calls; inside one issue C still enters first, so a reference that models matmul2d needs both orders, and a compiler that puts C in the first MMA will differ from the library’s accumulate mode by that rounding.

Longer K: exact rule (measured, MM 4). Issues compose through the rounded fp32 accumulator: two-issue chains reproduce the rule applied twice on 17 of 17 stimuli, three-issue chains 17 of 17; an unrounded inter-issue accumulator is rejected (8 of 17, 12 of 17). K order is ascending in memory whether the library unrolls or loops: for $K=64$, chunk orders 3210, 0213 and 1032 are each rejected by at least one stimulus; a $K=128$ runtime loop gives $1 + 6 \text{ eps}$ on seven asymmetric probes where descending gives $1 + 8 \text{ eps}$.

Other input types: measured cases. The machine model states that bfloat and truncated-fp32 operands follow the same tree (MM 0.6, MM 5), but the discriminating fit above was made on half inputs only. Their products are exact only while fp32 can hold them, and unlike half products they can leave fp32’s range. The record has single points at the extremes, not a rule: a bfloat product at 2^{-133} or 2^{-140} (fp32 subnormals) is kept, a bfloat product of $3e38 \times 3e38$ gives Inf, a truncated-fp32 product $2^{100} \times 2^{-100}$ is exactly 1, and truncated-fp32 subnormal operands keep their surviving bits (recon section 33; MM 5).

fp32 operands: exact rule (measured, MM 5). An fp32 A or B is masked, not rounded: the low 13 bits of the raw pattern are cleared, keeping sign, 8 exponent bits and 10 fraction bits, for normals and subnormals

alike (TF32-shaped truncation toward zero). $1 + 2^{-10} + 2^{-11}$ becomes $1 + 2^{-10}$; $2 - 2^{-23}$ becomes $2 - 2^{-10}$; the full exponent range survives ($2^{100} * 2^{-100} = 1$ exactly). The truncated operands then follow the same tree. The fp32-operand forms cost no issue rate over the half forms (measured, MM 8; chapter 12).

- *Correction:* “fp32 operand subnormals flush to zero” (recon section 5; MM 19) is withdrawn: only the low 13 bits are cleared. The module docstring of `tools/tensorops-model/tensorops_model.py` (line 12, “subnormals flushed”) still carries the old reading; its `to_f32_operand` docstring and code (`&0xffffe000`) keep subnormals, matching MM 5.

Subnormals: measured cases (measured, MM 5). In the recorded cases nothing in the half or bfloat path or the accumulator flushes: half $2^{-24} * 1$ gives 2^{-24} ; a bfloat product at 2^{-140} is kept; an fp32 accumulator subnormal passes through. fp32 ALU operations flush subnormal inputs and results (MM 5, MM 17 rule 22; chapter 4), so a subnormal the MMA produces is flushed to zero by the next fp32 ALU instruction that reads it.

Special values: measured cases (measured, MM 5).

- The 51 recorded edge cases (recon section 33; MM 5) give, case by case: $\text{NaN} \times 1$, $\text{NaN} \times 0$, $\text{Inf} \times 0$, $\text{Inf} + (-\text{Inf})$ inside the sum, and a NaN in C each yield the fp32 quiet NaN `0x7fc00000`; $\text{Inf} \times 1$ yields Inf ; a bfloat product past fp32’s maximum ($3e38 \times 3e38$) yields Inf ; $C = \text{fp32 max} + 1$ stays fp32 max.
- $\text{half } 65504 * 65504 = 4.29e9$ exactly, and sixteen such products sum exactly.
- A NaN in an A column poisons the result even where the matching B row is zero, so K padding must be zeros (MM 17 rule 14).
- Whether $\text{Inf} - \text{Inf}$ gives the canonical NaN at every position of the tree, and which NaN payloads survive, is not recorded beyond these cases (open).

Signed zero: measured case (measured, MM 5). $(-0) * 1$ with every other term zero gives $+0$, because the pair stage adds a neighbouring $+0$ first under RNE; a model with $(-0) + (-0) = -0$ mispredicts.

int8 inputs, int32 accumulator: exact rule (measured, MM 5, MM 11). Products are exact, with signed-ness selected independently for A and B. The default form accumulates modulo 2^{32} with no saturation, including inside one MMA: $C + 2^{31}$ gives -2^{31} , and $16 * 127 * 127 + \text{INT_MAX}$ wraps to -2147225585 . 608 accumulators were exact modulo 2^{32} out to a million iterations in all four sign combinations, 429 of them after leaving the int32 range. One $K=16$ tile cannot overflow on its own ($16 \times 16,384 = 262,144$).

Saturating int8: exact rule (measured, MM 11, MM 25.42, MM 25.89).

- One MMA computes $D = \text{clip}(C + A.B, -2^{31}, 2^{31} - 1)$: one signed clip of the exact sixteen-term sum, also for unsigned operands, which saturate at the signed limit. The rule scored 2,400 of 2,400 tiles in every sign combination; wrapping scored 1,746, 1,759, 1,742 and 2,129, staged saturation 1,975, 1,886, 1,897 and 2,400. The unsigned-by-unsigned column (the last) cannot discriminate, because every product is non-negative; the signed cases carry the result. Saturation is not sticky.
- Under transpose, with D seeded to $\text{INT32_MAX} - 32$ and all-ones int8 operands (exact sum 2147483679), wrap predicts -2147483617 and saturation 2147483647. With A transposed, Apple’s bytes (byte8 `0x44`) give -2147483617 on all 1,024 outputs; with byte8 bit 4 set on all sixteen tensor MMA, int8 widening to int32 (op10384) instances (`0x54`), 2147483647 on all 1,024; an in-range sum (64) is unchanged by the bit (MM 25.42). The untransposed pair (`0x40`, `0x50`) behaves the same.

- A chain of saturating MMAs clips once per issue: on inputs near both rails, wrap, clip after every product, clip after every issue and clip once at the end differ pairwise on 21 to 56 of 1,024 elements, and the hardware matched clip-per-issue (MM 25.89).
- *Correction:* MM 20 (“Correctness hazards left open”) says saturation with transposes, chains and high registers “was not run”; its own inline note marks this stale. Transposes were run in MM 25.42 and chains in MM 25.89: saturation applies per issue and is independent of the transpose bits.

Destination conversions: exact rules (measured, MM 5, MM 6, MM 0.6, MM 17 rule 21). These are the conversion instructions a kernel applies to the fp32 accumulator, not part of the MMA. Over 1,510,400 fp32 patterns:

- fp32 to half is IEEE round-to-nearest-even with zero mismatches, subnormal results included (65520 goes to infinity, 2^{-25} to zero).
- fp32 to bfloat is RNE except that an fp32 subnormal input returns a zero carrying the input’s sign (5,879 of 5,912 subnormal patterns).
- Every NaN input maps to one canonical NaN per type: half $0x7e00$, bfloat $0x7fc0$.
- Denormal and fast-math settings change none of this.

16-bit destinations across calls: measured case (measured, MM 12, MM 17 rule 18). A 16-bit destination rounds after every accumulating library call: four contributions to a half destination give 2054 where one rounding gives 2052; the same contributions inside one call with K=64 give 2052, because internal K slices stay fp32. Use a float destination across calls.

In every tested route, fp8, int4 and scaled inputs are reached through conversions in front of these forms; see [section 10.11](#).

To match the hardware bit for bit, a reference model has to add C first inside an issue (the rule fitted on half inputs), truncate fp32 operands rather than round them, carry subnormals through the MMA (in the measured cases) and wrap int32 accumulation by default; an ordinary dot product gets the bits wrong, as do the sequential and exactly rounded rivals above.

Evidence. MM 0.6, MM 4, MM 5, MM 6, MM 11, MM 12, MM 15, MM 17, MM 19, MM 25.42, MM 25.89, MM 25.98.

10.7 Ordering and the tensor enable

Ordering

The scoreboard’s semantics (slots, wait masks, slot choice and reuse, what an unwaited consumer reads) are [section 9.1](#)’s, and the carriers of every field are [section 3.4](#)’s. What is specific to the tensor unit:

- A tensor MMA carries an 8-bit wait mask at flags bits 24–31: for a load filling slot s , the consuming MMA must set bit $24 + s$ (measured, MM 6, MM 0.8). The bits are physically scattered (slot 7 is byte0[3]; [section 3.4](#)). In the running example ([section 0.4](#)) all eight tile loads fill slot 0 and each MMA’s flags word is $0x01000000$, bit 24.

- *Earlier reading as a result ring.* The same bits were first read as an eight-slot “result ring” of in-flight results: in Apple’s code each compiled MMA waited on exactly its own load group, so the bits looked one-hot and positional (MM 6, MM 19; the correction is told in [sections 3.4](#) and [9.1](#)). The wait-mask reading ties each bit to the slot a load fills, and predicting each consumer’s bit from its load’s slot was confirmed 20 of 20 (preregistered, MM 6). In the removal control, clearing byte0[3] on an MMA whose operands come from slot 7 makes it read fragments before the loads land; it produces zeros deterministically (50 of 50 dispatches across five processes for tensor loads) (measured, MM 6, MM 17 rule 4). An unmasked slot is not waited on, and nothing reports it. **Silent.** An emitter must set the wait bit of every slot that an operand’s load fills; this can be checked from the decoded bytes alone.
- The tensor load names the slot it fills in operand 1 bits 20–23, as slot + 1, a different carrier from the ordinary 128-bit vector load (op12709) (measured, MM 6, MM 25.4 “R6 closed and B2 decoded”; [section 3.4](#)).
- On the result side, an ALU instruction placed directly after an MMA, with no wait, read the MMA’s result correctly; whether the hardware interlocks MMA-to-ALU or the consuming form waits implicitly is open (MM 6, MM 25.90). In this compiler’s kernels an ALU read right after the last MMA needed no extra wait and was bit-exact (this compiler, measured, MM 25.105.1). Assume the pattern holds for the measured kernels and no further; it is not a proven interlock.
- Feeding D as the next MMA’s C, A or B in the same simdgroup needs no wait in the measured forms, and no instruction when the consumer takes fp32; a 16-bit consumer needs eight narrowings ([section 10.9](#); measured, MM 0.9, MM 13).
- Tensor store (op17257, 876 instances) and masked tensor store (op17258, 118 instances) carry a zero wait byte in every compiled instance, yet immediate MMA-to-store paths are exact (measured and decoder, MM 6; [section 9.1](#)).
- *Hazards caused by the host harness.* Two apparent MMA hazards (a stored result feeding a register-resident consumer; independent back-to-back MMAs) were the harness’s dim parameter, which bounds how many output bytes are complete and host-visible on return; both kernels pass when it covers the output ($\text{dim}^2 * \text{esz}$) (MM 19, MM 17 rule 5).
- Slot assignment past eight load groups and slot reuse follow [section 9.1](#); this compiler’s K=256 half and int8 lowerings (16 K slices, slots reused two to three times) match Apple and the reference in all three trials (measured, MM 6, MM 20).

Per-kernel metadata slot 44

The enable’s effect was measured by patching 14 pipelines (MM 6, MM 0.1). The metadata format is in [chapter 11](#).

- Slot 44 is present exactly when the code contains a tensor MMA: in all 50 such kernels and in no legacy-engine kernel. That correlation alone could be bookkeeping that follows from the code; the patches below show it is required.
- If its low byte is zero, every tensor MMA produces zeros, the pipeline still builds and the dispatch completes without error. In the tested patches any nonzero low byte, with the other three bytes arbitrary, is bit-identical to the original, so the low byte acts as an on/off switch.
- A disabled unit produces zeros quickly and neither faults nor stalls: a 1,400-MMA chain runs in 14.75 μs instead of 32.0 (8.4 ns per issue instead of 22), a saturated dispatch in 0.587 ms instead of 22.7 (a factor of 39). The MMAs still issue, at roughly ALU cost, while writing zeros. The timing fits a power or clock gate better than a validity check; the mechanism itself was not measured.

- An image author must give every kernel that contains a tensor MMA the enable with a nonzero low byte. A zero low byte is not refused at pipeline build and shows up only as zero results. **Silent.**
- With an explicit imageblock in the same kernel, per-kernel slot 19 sits at 55 and the tensor enable moves to slot 54; the slot-29 vector is [48, 49, 52] (Apple’s compiler emits, MM 25.89). “Slot 44” is the position in the tensor-only layout, not a fixed index in every kernel.

Metal caches pipelines by AIR function hash, so a pipeline built earlier in the same process is reused and every byte patched into the archive afterwards silently does nothing. Build the archive in one process, and patch and dispatch in another, with a positive control that shows a predictable bit moving the output (measured, MM 17 rule 36, MM 25.42).

Evidence. MM 0.1, MM 0.8, MM 6, MM 13, MM 17, MM 20, MM 25.4 “R6 closed and B2 decoded”, MM 25.42, MM 25.89, MM 25.90, MM 25.105.1.

10.8 Tensor loads and stores

The tensor unit has no memory port of its own. Fragments are filled and drained by per-lane loads and stores at addresses ordinary ALU code computes; partial tiles are handled by masks, not by the MMA (measured, MM 7). The general memory mechanisms (addressing, displacement folding, threadgroup memory) live in chapter 7; bandwidth and latency figures in chapter 12.

Table 10.7 lists the tensor load and store forms (measured, MM 0.8, MM 7).

Table 10.7. Tensor load and store forms, with per-lane payload and mask kind.

Form	Per-lane payload	Mask
tensor load, half/bf16 (op12674)	two 32-bit words, four 16-bit elements; 12- or 16-byte encoding	immediate (15 or 0)
masked tensor load, half/bf16 (op12675)	two words	16-bit register
int8 tensor load, one word (op12656)	one word, four int8 elements, byte-addressed	immediate
masked int8 tensor load (op12657)	one word	16-bit register
tensor load, fp32 (op12692)	fp32 elements (element width 4)	not stated in the sources
tensor store (op17257)	four 32-bit words	immediate
masked tensor store (op17258)	four words	16-bit register: bit j enables word j (j = 0..3); bits above 3 ignored; unwritten words keep their prior contents

An int8 operand needs the byte-addressed one-word form, not the generic four-word load (measured, MM 7).

The operand list of the tensor load and the tensor store is (decoder only, MM 7) dst, mode imm, flag imm, base (a relocation expression naming the buffer), imm, index:GPR32, last-use imm, byte-offset imm, element code, mask.

- The element code is the access width in bytes: 1 for the int8 one-word load, 2 for half (all 7,853 half/bf16 tensor-load instances carry 2), 4 for float (the fp32 tensor load and plain float accesses), and 8 and 16 for wider accesses (16 witnessed on 128-bit vector load (op12709) and 128-bit vector store (op17256)) (measured and decoder, MM 7, MM 25.37). *Correction:* MM 25.5 “R7 narrowed” is superseded by MM 25.37 on code 4 (appendix A).
- In Apple’s code the mask is only 15 or 0 and last-use only 0 or 16 (Apple’s compiler emits, MM 7). The last-use immediate follows the index register and takes the release modifier’s values (section 10.3); a reader should treat 16 as releasing the index register.
- Field positions from a single-bit census: the mode field’s variable content is byte11[3]; the index register is a 7-bit selector at byte3[1..7]; the byte offset is byte8[5..7] plus byte9[0..4] (decoder only, MM 7).
- The slot the load fills is operand 1 bits 20–23, as slot + 1 (section 10.7). A load can also carry a wait mask, used so the first tile loads wait on the index register’s own load (measured, MM 6).

The row stride is not an instruction field. A non-default stride changes the mode immediate from 241 to 2289 (bit 11 of the mode word), switches to a per-lane computed index register and adds ALU code; the stride lives in the register’s precomputed value (decoder and Apple’s compiler, MM 7). Default strides must come from the operand’s element size: a lowering that used $K \times 2$ for every type was twice too small for float and twice too large for int8, and the $32 \times 32 \times 64$ int8 case was wrong on 1,024 of 1,024 elements (measured, MM 17 rule 6). **Silent.**

Partial tiles are masked with range mask, immediate width (op612), whose 8-bit lo and width immediates make masks tile-relative (measured, MM 7, MM 19; semantics in section 7.4). Multi-simdgroup sub-tile ownership is expressed as per-simdgroup store masks (Apple’s compiler emits, MM 7).

A tensor load is a per-lane gather whose addressing is section 10.4’s; displacements are bytes (section 7.1; MM 25.141.18). `air.simdgroup_async_copy_2d` lowers to a synchronous load/store loop and its wait to nothing; latency hiding is the scoreboard (Apple’s compiler emits, MM 25.89).

Through Metal, the one fault seen (section 7.5) came from a `regprobe_multi` tensor-kernel probe whose configuration was not recorded (measured, MM 17 rule 34, as corrected there); size every buffer from the kernel’s own layout. A dead load’s destination is still in flight, and reusing it lets the late load overwrite it (measured, MM 25.114).

Evidence. MM 0.8, MM 6, MM 7, MM 17, MM 19, MM 25.37, MM 25.89, MM 25.114, MM 25.141.18.

10.9 Chaining and the memory bridge

A feed hands one MMA’s destination D (8 fp32 registers per lane) to a later MMA in the same simdgroup without a store. There are four modes: D as A, as B, as A with transA (“At”) and as B with transB (“Bt”); D can also be the next C (the ordinary K chain).

An fp32 D fed into an fp32 operand needs no instruction (measured, MM 13); a 16-bit consumer needs exactly eight conversions, fp32 to half narrowing (op1016), per D tile. The direct chain is bit-identical to the memory bridge on 200 of 200 random matrices (204,800 elements).

The measured relabelings admit two readings: a property of the MMA’s operand ports, which every compiler must apply, or a consequence of how the producing kernel packed its tiles. Section 10.5 derives

the second, which predicts different feeds for the two packings in use. One measurement per packing separates the two:

1. In kernels that pack A and D canonically (Apple-compiled `simdgroup_matrix` code), register-direct chaining is exact in all four modes with fixed relabelings: 1,596 of 1,596 random chained pairs over four modes, five operand forms and grids from 1×1 to 4×4 , extended to 304 of 304 over eight grids and five transpose combinations (measured, MM 13, MM 0.10):

fed as A	$A_{\text{eff}}[r][k] = D[r][k]$
fed as B	$B_{\text{eff}}[k][c] = D[\text{rot11}(k)][c]$
fed as A, transA	$A_{\text{eff}}[r][k] = D[\text{rot11}(k)][\text{rotr1}(r)]$
fed as B, transB	$B_{\text{eff}}[k][c] = D[\text{rot11}(c)][k]$

A logical $A2 \cdot D1$ through the B feed then needs A2 loaded with its k index permuted by `rot11` (wrong in 20 of 20 trials without it), and matches the unpermuted product to rounding, not bit for bit, because the permutation changes the association order. In the transposed feeds the accumulator C must carry the same relabeling (MM 17 rules 9–11).

2. In kernels that use B-row packing for every tile (this compiler's `tlower` emission), every feed reads the logical operand bit for bit (measured, MM 25.103): B computes $A2 \cdot D$, At computes $D^T \cdot B2$, Bt computes $A2 \cdot D^T$. The test was preregistered with the relabeled product as the prediction and the logical product as the control expected to fail. The relabeled reference failed in every arm, with maximum errors of 290 (B), 305 (At) and 477 (Bt), and the logical product was bit-exact. The two references differ on 1,024, 1,024 and 896 of 1,024 elements (Bt's remaining 128 are columns where $\text{rot11}(c) = c$), so the test could separate them. Decoded, At differs from A only in operand 5 (the type field with the transpose bit) and, off the diagonal, the fed register. The domain is one square stage ($M = N = 32, K = 64$), one `simdgroup`, no accumulate, with the fed operand in fp32 or narrowed to half; mode A was measured separately in this compiler's chains (MM 25.89).

Both measurements place the relabelings in the producer's packing. A compiler applies them only when the producer packed D canonically; under B-row packing of every tile, D included, it applies none (MM 17 rule 9 as scoped by MM 25.103), and B, At and Bt feeds outside the measured stage remain unmeasured. *Correction:* the relabeling table read as an instruction property (MM 0.10, MM 13 before their scope notes) is scoped by MM 25.103.

Other feed cases have been measured:

- *Accumulator (C) feed* (this compiler, measured, MM 25.125.2): the register C feed is bit-identical to the memory bridge (M32 and M16, 3 of 3 each; swapped-tile and claims-no-C controls fail on 1,024 of 1,024). At M32 it removes eight fp32 C loads and the producer's eight stores but splits the consumer into two accumulator groups; the instruction count is unchanged (192).
- *int8 chains:* in hand-authored AIR (Apple's labeling), int8 accumulators, including register-resident requantized chains, chain through all four modes with the half chain's rotations, 272 of 272 runs (measured, MM 3, MM 11).
- *Transposed consumers across a quantize boundary* inside a hidden-chunk loop: nn reads $P_t \cdot W$, tn $P \cdot W$, nt $P_t \cdot W_t$, tt $P \cdot W_t$, 12 runs exact (measured, MM 25.45).
- *S and P kept in registers* for prefill attention (a register-fed A from a named accumulator): exact (measured, MM 25.163; the time saved is in section 12.10).

- *Staging D through the imageblock* is bit-exact but frees no registers and is slower: register feed 13.4 / 20.3 μ s, memory bridge 14.6 / 23.3 μ s, imageblock 14.4 / 27.3 μ s at M16 / M64; staging adds 254 instructions at M64 (measured, MM 25.106).

Two earlier readings were host errors (MM 19). That `matmul2d` allows one `run()` per kernel and that a chain must cross a kernel boundary was a misreading of slice offsets, which are (column, row); read correctly, both calls are bit-exact in one kernel. And certain operand ranges once excluded from register-direct chaining were errors of the host model, which rounded each product where the hardware forms products exactly; no operand-range condition remains.

This compiler emits a feed only for a receipted key (producer and consumer form with every modifier, role A/B/C, transpose, tile grid, locality), all one simdgroup and one threadgroup (this compiler, MM 25.125.3, MM 0.10); any absent key stays on the memory bridge. A decoded chain containing any stack store is not register-resident (MM 17 rule 12).

Apple's library has a chaining hazard (measured, MM 13, MM 17 rule 13). Its cooperative input tensor exposes only the first $\min(\#D1 \text{ tiles}, \#D2 \text{ tiles})$ blocks of the producer in row-major block order; a narrower consumer silently reads zeros or wrong elements for the rest. The rule predicts all 27 configurations measured per side (M and both widths in 16, 32 and 64, one simdgroup), of which the nine narrower consumers per side fail; `is_compatible_as_*_input`, a run-time member function of `matmul2d`, returns true in every one, so it cannot serve as a guard. Pad the consumer to the producer's tile count or use the memory bridge. **Silent**.

The memory bridge, a tensor store to row-major memory followed by a tensor load, worked in every measured case (measured, MM 13): two separately generated bodies joined at a buffer offset are bit-exact, including a cross-lane stress and a K remainder. In one kernel and one simdgroup it needs no fence, because the second body's loads follow the first body's stores in program order on the same scoreboard. It is the correctness baseline and the default outside the receipted feed domain. Streaming the free tile axis runs grids up to 12×8 producing 12×12 (1,344 MMAs, 4,807 instructions) with zero stack stores; spill edges are not a closed-form function of shape (two preregistered rules failed at 6 of 12 and 82 of 144).

Evidence. MM 0.10, MM 3, MM 11, MM 13, MM 17, MM 25.45, MM 25.103, MM 25.106, MM 25.125.2, MM 25.125.3, MM 25.163.

10.10 Data movement across simdgroups

Registers did not cross a simdgroup in any measured case. A cross-simdgroup `simd_shuffle` returns the caller's own simdgroup's value in 256 of 256 cases. Only the accumulator role can stay register-resident across simdgroups, and only inside one library cooperative scope where both simdgroups take part in both calls; no register state crosses a simdgroup boundary there either (measured, MM 14).

A hand-off between simdgroups takes three steps (measured, MM 14, MM 17 rule 7):

1. a fragment-order store (16 bytes per lane for a converted A or B tile, 32 for an fp32 accumulator);
2. a workgroup barrier: with no barrier, or with `air.simdgroup.barrier` alone, the hand-off is wrong in 12 of 12 trials; every workgroup barrier variant (execution-only, threadgroup memory, device memory, both) is correct in 12 of 12. Without one the consumer reads exactly zero, the buffer's pre-write state. That an execution-only barrier sufficed is an observation about this chip, not a memory-model guarantee; section 9.2 says what each barrier orders. **Silent** when omitted;

3. a fragment-order load, which returns the producer’s register bits in the same lane and slot: no relayout exists.

The round costs roughly a quarter of a microsecond including both barriers (measured, MM 14). Store + barrier + load per round: 38.0 ns at one simdgroup and one threadgroup per core, 81.9 ns at 32 simdgroups, 100–131 ns at 64 per core through threadgroup memory; through device memory 58–83 ns and 151–161 ns (1 percent above threadgroup memory at low occupancy, 17 percent at high) (measured, MM 7). Threadgroup and device memory are within 8 percent in the scheduling study (MM 14).

Three scheduling rules follow from the measurements (216 of 216 exact, MM 14, MM 17 rule 32):

- never recompute a shared stage per simdgroup (1.16 to 6.66 times the shared time);
- share at low occupancy (1.7 to 10 times faster than one simdgroup holding all tiles), and when one simdgroup would hold 16 fp8 output tiles and spill 134–138 stores;
- otherwise keep the tiles in one simdgroup (sharing costs 3 to 22 percent).

Shared fused stages are exact through 16 simdgroups (measured, MM 0.14, MM 20); at 32 the recompute form stays exact but both shared forms exceed the 32 KiB threadgroup-memory limit. Sharing between threadgroups needs a dispatch boundary; a threadgroup that waits on another inside one dispatch depends on both being resident, which is the restriction of [section 1.5](#).

At `execution_simdgroups<2>` and `<4>` the library partitions output tiles per simdgroup with no barrier and no threadgroup memory (Apple’s compiler emits, MM 25.93; [section 10.1](#)). This compiler’s `gemm_generic` admits 2 and 4 simdgroups and up to 8 threadgroups, bit-exact on hardware (this compiler, MM 0.13, MM 25.89).

Evidence. MM 0.14, MM 7, MM 14, MM 17, MM 20, MM 25.89, MM 25.93.

10.11 Quantized formats

Of the ten admitted forms, none takes an fp8 or sub-8-bit operand and none applies a block scale. In every route tested (Apple’s compiler, Apple’s library, hand-written AIR), low-precision formats reach the unit by conversion into an int8, half or bfloat fragment with ordinary instructions, and scales are applied in software. This describes the recovered forms and the routes tried, not a complete inventory of the hardware.

int8 and uint8 are native (measured, MM 0.11, MM 8, MM 11): tensor MMA, int8 widening to int32 (op10384) and its no-C form, at exactly twice the fp16 issue rate ([section 12.1](#)). Arithmetic in [section 10.6](#).

Requantization from int32 to int8 has no instruction; it is a scalar sequence (load, integer to float, multiply, round, float to integer, clamp, store) (measured, MM 11). Rounding is round-half-to-even (± 0.5 , 1.5, 2.5, 3.5 give 0, 2, 2, 4); the clamp saturates (32768 gives 127, not -128; -25600 gives 0, not 156). This compiler’s requant epilogue computes $b = \text{acc} + \text{bias}[\text{col}]$ (int32, wrapping), $f = \text{float32}(b)$ (RNE), $y = \text{float32}(f \cdot \text{scale})$ (RNE), $r = \text{rint}(y)$ (half-even), $q = \text{clamp}(r + z_p, \text{lo}, \text{hi})$, lowered as 32-bit add (op10282), 32-bit integer to fp32 (op11179), fp32 multiply (op3290), round to nearest even (op3770), a clamp to $[\text{lo} - z_p, \text{hi} - z_p]$ with fp32 compare-select, fmax/fmin (op9700) before narrowing, fp32 to 32-bit integer (op9320), and a second 32-bit add for the zero point (this compiler, measured, MM 25.130.1).

- The float-to-int conversion saturates at both widths, signed and unsigned, and sends NaN to 0; a prior maxnum/minnum clamp is harmful (it maps a NaN block to -127). Guard with `isnan(x)` or `(as_type<uint>(x) & 0x7fffffff) > 0x7f800000`; under default fast math `x != x` folds to false (measured, MM 6, MM 17 rule 17). **Silent.**

For fp8 (e4m3fn and e5m2), eight fp8 unpack to bf16 (op17642) instructions, each taking two fp8 values in a 16-bit register to two bf16 values in a 32-bit register (format immediate 97 for e4m3fn, 98 for e5m2), feed one ordinary bf16 use of the 16-bit MMA (measured, MM 11). Decode is exact for all 254 finite e4m3fn codes and 248 finite e5m2 codes; arithmetic is bit-identical to the bf16 rule on 96 of 96 tiles across four format pairs; transposes exact on 36 of 36. Cost: bf16's when the unpack is hoisted (4.96 ns per MMA per core, 14 accumulators); 5.3 ns at four unpacked fragments per MMA and 8.4 ns at eight. Keep unpacked fragments per MMA at four or fewer (MM 17 rule 28).

For packing, fp8 pack, vector, from fp32/half/bf16 (op13618) (AIR `air.convert`) rounds to nearest even with no saturation (measured, MM 11, MM 25.105, MM 25.105.1), verified over all 2^{32} fp32 patterns and all 65,536 bf16 and fp16 patterns. Largest finite values are 448 (e4m3fn) and 57,344 (e5m2). Signed zeros are kept. Table 10.8 gives the results at and beyond the range limits and for NaN.

Table 10.8. fp8 pack results for large magnitudes and NaN inputs, by target format.

Input	e4m3fn result	e5m2 result
magnitude rounds to 448 or less (464 itself rounds to 448)	RNE code	–
magnitude above 464, positive	0x7f (NaN)	–
magnitude above 464, negative	0xff (NaN, sign kept)	–
magnitude 61,440 or above, positive	–	0x7c (+infinity)
magnitude 61,440 or above, negative	–	0xfc (–infinity)
NaN input, either sign	0x7f	0x7e

An fp8 overflow keeps its sign; only NaN inputs become the positive canonical NaN. Two e4m3fn overflow arms whose reference sent every overflow to 0x7f failed in exactly the negative overflows (174 of 372, 297 of 575); with the sign kept, both arms match word for word, and e5m2's 963 overflows were signed infinities (measured, MM 25.105). The quantize-out epilogue of this compiler and the pack measured in MM 11 are the same instruction, the fp8 pack, so the rule covers both. *Correction:* MM 0.11's quantizer table ("Above 464 $\rightarrow$ canonical NaN") is superseded for negative overflows by MM 25.105 (corrected at MM 25.105.1); MM 11's "Every NaN input maps to the positive canonical NaN" is about NaN inputs only and stands.

Apple's fp8 store encoding (Apple's compiler emits, MM 25.105.1): per eight floats, four packs of the 12-byte form of fp8 pack, vector, from fp32/half/bf16 (op13618/12), each writing one 16-bit half from two fp32 registers in place (`R4L <- R4, R5`, then `R4H <- R6, R7`, where R4–R7 are four accumulator registers), then one word store at word index (op17202) per packed register (width 4, index $\text{lane} \times 5$).

int4 and uint4 likewise reach no admitted form directly (measured and Apple's compiler, MM 5, MM 25.6, MM 25.28). Nibbles are unpacked by ordinary ALU instructions, for example 16-bit AND with immediate (op435), 16-bit AND (op437) and 16-bit integer to half (op11183) with the integer-to-float conversions, into a supported fragment, then the 16-bit MMA, tensor MMA, 16-bit $\times$ 16-bit, accumulate (op5106), or the int8 MMA. int2 needs no new mechanism; its refusal is at the `tensor_inline` constructor, an API limit (MM 25.6). *Correction:* the two AND forms were once read as shifts.

No tested route reaches fp4 (e2m1) or fp6 (e2m3, e3m2) in hardware (bounded negative and Apple’s compiler, MM 0.11, MM 20, MM 25.94): no vendor route tested produced a hardware unpack, pack or MMA for them. After a software decode the ordinary fp16 or bf16 use of the 16-bit MMA applies; the decode costs about 24 instructions per value in Apple’s newer backend and about 4 in MLX’s bit trick. Packed fp4 and fp6 tokens crash buildConvert; air.pack and air.unpack crash buildPack; air.quantize_pack crashes NumericPackUnpackPass. This compiler has not built the software operand unpack (MM 0.14).

No tested route applies a block scale in hardware (measured, MM 11, MM 17 rule 26). The scaled matmul2d form is MX-style: a ue8m0 scale per 32 elements along K, one plane per row of A and per column of the transposed B, no bias, no zero point. The eleven-operand scaled intrinsic compiles but has no effect: across 36 configurations with scale exponents from 2^{-30} to 2^{+30} , every output equals the unscaled result and the emitted 16-bit MMA bytes are identical; per-lane scale operands are dropped silently. Never emit the scaled spelling expecting scaling. **Silent**. The scaled int8 form with fp32 accumulate aborts at Failed to encode. (Apple’s compiler, MM 20).

Scaling in software was measured for placement and cost (MM 11, MM 25.105.2):

- *Cheapest placement*: per 32-element K block, two MMAs into a zeroed temporary (the first is tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107)), then one fp32 multiply-add of the block sum by the product of the two scales: 8 instructions per MMA, 1.03 to 1.08 times the unscaled MMA time on 2×2 to 4×2 grids; 477 byte-table plus 128 predecoded comparisons bit-identical.
- *Latency hiding*: the block’s MMAs do not depend on the accumulator, so the multiply-add’s round trip hides behind the next block’s MMAs. Against a chained form it costs 8 ns per step (15 percent), where a multiply-add inside the dependent path costs about 27 ns per step at low occupancy; the forms converge at 12 or more simdgroups per core.
- *Operand-side scaling*: 5.7 to 10.1 ns against 5.0 ns per MMA, 74 percent of elements exact for one block; the integer exponent add is wrong for zeros and subnormals.
- *This compiler’s MX step* (mx32) uses the post-MMA placement with $T = (T * SA) * SB$ then $D += T$ (two fp32 multiplies and an fp32 add). The multiplies are exact while the result stays in fp32’s normal range, because the factors are powers of two; a result below 2^{-126} is flushed by the ALU (section 4.3), and the add into D rounds as any fp32 add does. The step is bit-exact for e4m3 $\times$ e5m2, e5m2 $\times$ e4m3 and bf16 at K = 64 to 128, refused with transposes, accumulate, more than one simdgroup, the K loop and register-fed B, and untuned (29.4 μ s against 17.6 μ s unscaled at K = 64) (this compiler, measured, MM 25.105, MM 25.105.2).
- *Where scaling time goes in fused kernels*: block scaling is 25–27 percent of fused fp8 time and 44–49 percent of int8 (measured, MM 16).

The ceil quantizer rule (measured, MM 11, MM 17 rule 27) flushes fp32 subnormals, takes the block’s integer maximum of absolute bit patterns, reduces across lanes with two simd_shuffle_xor steps, and then computes

```
code = clamp(((amax_bits + BIAS) >> 23) - EMAX, 0, 254)
BIAS = 0x1fffff, EMAX = 8   (e4m3fn)
BIAS = 0x1fffff, EMAX = 15  (e5m2)
BIAS = 0x1ffff,  EMAX = 6   (int8)
factor = 2^(127 - code), built as (254 - code) << 23
```

It costs 127 instructions and 15.0 ns per 16×32 block, never needs a clamp, and is 12 to 31 percent more accurate on e4m3fn. The OCP floor rule costs 157 instructions and 19.4 ns and needs a software clamp

(without it 202 of 1,024 gaussian blocks produce NaN in e4m3fn). Both are exact on 112 adversarial cases. Because the pack does not saturate, only the scale rule keeps values in range.

- Scale codes run 1 to 254. Code 0 (2^{-127} , an fp32 subnormal) is flushed by the ALU, so it is unusable (this compiler refuses it by name); code 255 yields NaN for the block. In-kernel E8M0 decode ($e \ll 23) + (((e + 1) \gg 8) \ll 22)$ gives $2^{(e-127)}$ for codes 1–254 and $0x7fc00000$ for 255 (measured, MM 11, MM 25.128).
- *Accuracy* (relative Frobenius error against fp64, $64 \times 512 \times 64$ gaussian): int8 block-32 with fp32 scale 0.0076, power-of-two ceil scale 0.0112; e4m3fn block-32 ceil 0.0376; e5m2 0.0757. fp32 accumulation contributes $1.4e-8$ to $5.1e-8$, never the limit. int8 is 1.2 to 1.4 times faster block for block (measured, MM 11).
- *Format choice by working set*: bf16 and fp16 are fastest in cache (5.4–6.7 ns per MMA per core), then block-scaled int8, then fp8; int8 overtakes bf16 between 21 and 42 MiB of working set, fp8 between 84 and 169 MiB. fp8 is the choice for capacity and bandwidth; in cache it is the slowest of the three (measured, MM 16).
- `air rint` builds and rounds half to even; `llvm.roundeven` and `llvm.nearbyint` crash Apple’s compiler (MM 20).

Evidence. MM 0.11, MM 5, MM 6, MM 8, MM 11, MM 16, MM 17, MM 20, MM 25.6, MM 25.28, MM 25.94, MM 25.105, MM 25.105.1, MM 25.105.2, MM 25.128, MM 25.130.1.

10.12 Reductions

A reduction here means summing (or taking max/min of) a tile’s elements along rows or columns, either inside Apple’s library (`reduce_rows`, `reduce_columns`) or in code a compiler emits from cross-lane primitives (chapter 8).

The library’s FP32 reductions follow one rule (measured, MM 12; 215,040 of 215,040 outputs in one census and 184,320 of 184,320 over seven shapes in another, no disagreement among replica lanes):

1. each lane sums its own elements of the tile in slot order, sequentially with RNE32;
2. a butterfly of lane exchanges then runs over the lane bits along the reduction axis, in ascending lane-bit order: a row reduction over lane bits 0 and 3, a column reduction over lane bits 1, 2 and 4.

These are the lane bits the axis runs along, which is why they read transposed against the layout formulas of section 10.4. *Correction*: “the two reduction axes follow structurally different orders” (recon section 120; MM 19) is withdrawn: one rule on two axes.

Summation direction is fixed per compiled object (Apple’s compiler, MM 12, MM 25.35, MM 25.39). The per-lane chain may run ascending or descending. Of 20 objects, 4 ascend and 16 descend. Under strict math an explicit reduction runs as written; under default fast math an ascending source is normalized to descending unless every leaf has another use (partial extra uses still normalize). The direction is readable from the compiled object without a dispatch (`tools/g17reduxdir.py`). N congruent to 8 modulo 16 makes the sum ascending on its own (7 of 7, against 20 of 20 descending where N is a multiple of 16). The read-all flip needs the destination tile spilled and per-lane capacity of at least 12 (no exceptions on 45 fitted pairs; held out 106 of 109, the three misses at shapes whose baseline already spills); the reduction tree itself is never rebuilt (46 of 46). When M is 16 one object can contain both directions (a long chain family and

twelve short chains disagree in 10 of 33 objects). The library’s order cannot be relied on; a compiler that needs bit-stable sums emits its own chain under strict math (MM 17 rule 24).

Among the hardware primitives (measured, MM 12, MM 25.141.16, MM 17 rule 25; chapter 8 is their home), `simd_sum` and SIMD sum, fp32 (op16842) are a balanced adjacent-lane tree, an xor butterfly in mask order 1, 2, 4, 8, 16, rounded to the element type at every step (section 8.4); SIMD shuffle-xor, immediate mask (op14169) returns 0.0 for a mask of 32 or more, and SIMD shuffle-xor, runtime mask (op14170) needs a lane-uniform mask (section 8.1). A compiler emitting its own reduction builds it from these.

The library’s semantics depend on the destination type (table 10.9; measured, MM 12, MM 17 rules 19–20).

Table 10.9. Library reduction semantics by destination type, for sum and for max and min.

Destination	Sum	Max / min
FP32	per-lane sequential RNE32 chain, then the axis butterfly	exact; the identity participates; NaN ignored
half	every addition rounds to half	exact at half precision, finite extrema as identities
bfloat	adds in fp32, one final bfloat rounding	exact at bfloat precision, finite extrema as identities
int32	exact over the measured domain	exact, integer extrema as identities

max and min fold in the identity, not the extremum, and ignore NaN like fmax: an all `-inf` row returns `-FLT_MAX` (float), `-65504` (half), `-3.3895e38` (bfloat). A masked-softmax lowering expecting `-inf` from a fully masked row will not get it.

Only the FP32 reduction is lowered by this compiler (this compiler, measured); it emits an explicit fp32 local chain plus a shuffle-xor butterfly rather than calling `reduce_rows`, and refuses other precisions, shapes and multi-simdgroup reductions by name (MM 12, MM 15). The register-resident column reduction folds accumulators over row tiles, then runs an xor butterfly over lane masks 2, 4 and 16 with the immediate-mask shuffle-xor: six arms bit-exact; an exactly rounded control fails by up to $9.5e-6$ (MM 25.103). A KV-split attention changes values, not only association, and needs a KV-split reference (MM 17 rule 23).

Evidence. MM 12, MM 15, MM 17, MM 19, MM 25.35, MM 25.39, MM 25.39.1, MM 25.103, MM 25.141.16.

10.13 Legacy simdgroup-matrix engine

The older `simdgroup_matrix` path is a separate engine: 8×8 tiles, 14-byte instructions, different scheduling classes, no shared bytes with the tensor MMA, and a different reduction order (measured and decoder, MM 2, MM 9).

Legacy opcode selection is by accumulator type (table 10.10; Apple’s compiler emits, MM 9, MM 0.3).

Table 10.10. Legacy simdgroup-matrix MMA forms by input and accumulator type, with scheduling class, accumulator registers and arithmetic.

Form	Inputs, accumulator	Class	Accumulator registers	Arithmetic
legacy 8×8 simdgroup MMA, f16 (op838)	fp16, fp16	43	one 32-bit register	lower-precision accumulator
legacy 8×8 simdgroup MMA, f16 in fp32 out (op2862)	fp16, fp32	118	two-register tuple	sequential fp32 chain
legacy 8×8 simdgroup MMA, fp32 (op2842)	fp32, fp32	118	two-register tuple	sequential fused fp32 FMA chain
legacy 8×8 simdgroup MMA, bf16 (op2902)	bf16, fp32	118	two-register tuple	fp32-accumulate class

The rest of this section calls them the f16, f16-in-fp32-out, fp32 and bf16 legacy forms. legacy 8×8 simdgroup multiply, f16 (op839) (simdgroup_multiply, class 44) issues at the cost of the f16 legacy MMA: slope 3.435e-05 against 3.436e-05 seconds per op, ratio 0.9998, null arm flat (measured, MM 25.55 as corrected by MM 25.56, which fixed the unit from milliseconds to seconds).

As measured (MM 4), the f16-in-fp32-out form is $\text{acc} = C$; for k in $0..7$: $\text{acc} = \text{RNE32}(\text{acc} + a_k * b_k)$ (168 of 168 eps-triples; pairs-first 144, balanced 112). The fp32 form is a sequential fused fp32 FMA chain at full precision: $1 + 2^{-23}$ survives and $(1+u)(1+u) - (1+2u)$ returns 2^{-46} , which is why Apple routes exact fp32 there rather than to the tensor unit’s truncating fp32 forms. The two engines’ results differ bit for bit, so an output tile stays on one engine (MM 9, MM 17 rule 31).

Beside the tensor unit (measured, MM 9), at 16 and 32 simdgroups per core the legacy stream is hidden under the tensor stream completely (overlap 1.00 for every legacy form); 0.91–1.00 at 8. A dependent hand-off between the engines is free at 8 or more simdgroups per core; at 2 per core tensor-to-legacy runs 9.7–13.5 percent above control and legacy-to-tensor 1.1–6.2 ns above. The partial overlap at one or two simdgroups is in-order issue, not a shared resource. *Correction*: “legacy MMA and accelerator share one issue slot” (recon section 31; MM 19) is withdrawn.

Beside the fp32 ALU (measured, MM 9), the f16 form overlaps fp32 fma by 0.66–0.95 (mostly different resources); the class-118 forms overlap it by 0.37 or less (0.07–0.37 for the f16-in-fp32-out and bf16 forms), so the pair costs 89–97 percent of the plain sum. *Correction*: “the legacy MMA overlaps fp32 fma by 81–86 percent” is true of the f16 form only (MM 19).

The legacy engine’s rates and joint budget were measured (MM 9):

- Legacy: 183–205 MAC per ns per core against 122–133 for the fma stream; issue 2.79–2.80 ns at 32 simdgroups per core and 2.48–2.58 at 8 and 16, which is 226–253 FLOP per cycle, 4.0–4.5 times less per FLOP than the tensor MMA. What the legacy engine shares with fp32 fma is not identified; “about 8 fma of budget per legacy issue” is a measured exchange rate, not a mechanism (open).
- Beside a saturated tensor stream: free up to 8 fma per MMA, or 8 legacy issues per 8 tensor MMAs; both together about 64 fma-equivalents per 8 tensor MMAs. Free legacy issues per 8 bf16 tensor MMAs: 8 with no fma, 4 at 4 fma per MMA, 1 at 8. Past the budget each legacy issue costs about 5 percent. Arithmetic gain is at most 12.5, 6.3 and 1.6 percent; 7 tensor plus 8 legacy issues run 1.142 times the throughput of 8 tensor issues.

- Do not schedule legacy MMAs in layer kernels; at about 46 instructions per tensor MMA they are already past the joint budget (MM 17 rule 31).

The two engines can share one tile through memory (measured, MM 9, MM 25.33). A 16×24 by 24×16 product split by K (tensor 16, legacy 8) through a row-major memory bridge was exact on 32 of 32. `air.simdgroup_matrix_8x8_load` and `_store` take origin and strides as `<2 x i64>` in (column, row) order, so for a row width W, `strides = {1, W}`. The cross-engine register map is measured but the in-register bridge is unbuilt: the legacy 8×8 spreads 64 elements over 32 lanes at 2 each, the tensor layout packs them into 8 lanes at 8 each, both elements of a legacy lane travel together, and tensor lane = L >> 2 with bits 0 and 1 swapped (tensor lanes 0..7 receive quads 0, 2, 1, 3, 4, 6, 5, 7).

Evidence. MM 0.3, MM 2, MM 4, MM 9, MM 17, MM 19, MM 25.33, MM 25.55, MM 25.56.

10.14 Compiler rules for tensor code

MM 17 states the consequences of this chapter as numbered compiler rules. Table 10.11 gives, for each rule, the section that explains it together with its mechanism.

Table 10.11. MM 17 compiler rules for tensor code, with the section that explains each.

MM 17 rules	Where explained
1 (never name R126 or above, the MMA’s tuples included; silent, MM 10)	section 10.4 and section 2.1
2 (at most 15 live accumulator groups straight-line, 14 in a loop; $8N + K \leq 115$, $8N + 4M \leq 108$, MM 11)	section 2.8
3 (Apple’s spill counts are not hardware limits, MM 13)	section 12.4
4, 5, 36 (wait masks; host completion scope; build and patch in separate processes)	section 10.7
6, 34 (strides from element size; buffer sizing)	section 10.8
7, 32 (workgroup barrier for hand-off; never recompute a shared stage)	section 10.10
8, 29 (<code>execution_simdgroups<N></code> ; <code>relaxed_precision</code>)	section 10.1
9–13 (feed relabelings, scoped by MM 25.103; forwarding domain; the compatibility oracle)	section 10.9
14–16, 18, 21–22 (zero K padding; fp32 truncation; int32 wrap and saturation; destinations and conversions)	section 10.6
17, 26–28 (float-to-int saturation and NaN, MM 25.50; no hardware scaling; the ceil quantizer; fp8 unpack)	section 10.11
19–20, 23–25 (library reductions; KV-split reference; summation order; runtime shuffle masks)	section 10.12
30, 33 (never unroll the K loop; fusion boundaries, MM 16)	section 12.2
31 (no legacy MMAs in layer kernels; one engine per output tile)	section 10.13
37 (keep a loop phi’s entry value live)	section 10.3 and section 2.6

Not known. Each item gives the safe course where there is one; chapter 14 indexes them.

- MMA-to-ALU ordering is observed to work with no wait in the measured kernels; the mechanism is open, and no scoreboard slot for an MMA result has been identified (MM 6, MM 25.90). Assume nothing beyond the measured patterns.
- The meaning of the destination keep flag on hardware: inert as observed; set it as Apple does (MM 25.22).
- A multi-bit encoding of the accumulator's modifier: not excluded (MM 6).
- The 36 narrow-accumulator and 475 _shifted declarations: not admitted; do not emit (MM 25.13, MM 25.6).
- Fragment shapes other than $16 \times 16 \times 16$: not measured (MM 0.14).
- The bit-71 int8 variant: runs, meaning unknown (MM 20).
- The software fp4/fp6 operand unpack: not built; its fp16-subnormal behaviour unmeasured (MM 0.14).
- What a released register reads: zero in every controlled run, not established in general; read one only where zero and the old value give the same answer (section 2.5). Reclamation beyond the measured bound: not excluded (MM 25.95).

Evidence. MM 0.14, MM 17, MM 20, MM 25.95, MM 25.103.

11 Kernel metadata and the image

A compute kernel reaches the GPU as a binary archive whose machine code is accompanied by metadata sections the driver reads when it builds a pipeline: which special registers the kernel reads, how its buffers are bound, whether it uses the tensor unit, what its constant program writes, how much threadgroup memory it declares. Apple documents none of these sections.

The GPU executes the project's machine code, and a change made to Metal's copy of it after pipeline creation reaches execution. Pipeline creation validates only the container's format: undecodable code, a missing end of program and impossible metadata values all build a pipeline ([section 11.4](#)). Mutating one field at a time, against a control that could fail, separated the fields the driver reads and acts on (the tensor-unit enable, the system-register vector, the binding records) from those it reads only for their presence and those it ignores. Damaged structure is not refused: it crashes the driver inside pipeline creation, and an earlier pointer-slot defect hung the GPU. For these reasons the compiler computes metadata from the program it emitted instead of copying it from a neighbouring kernel, and it refuses layouts it has not witnessed.

Names used for the image and its parts:

- *Image*: the delivered `scan.arc.metallib`. *Object*: the applegpu Mach-O inside it. *Section*: a Mach-O section such as `__GPU_METADATA,__compute`.
- *Per-kernel slot N*: field N of the per-kernel FlatBuffers table in `__GPU_METADATA` ([section 11.1](#)). Not to be confused with scoreboard slots ([section 9.1](#)).
- *Constant program*: Apple's per-dispatch program `agc.main.constant_program` ([section 11.2](#)). *Uniform uK*: the uniform register file ([section 2.2](#)).
- *Kind-3 record*, *kind-6 record*, *argument map*: records of the constant-program description ([section 11.2](#)).
- *Class*: a measured metadata layout (which vtable sits where, which slots exist) that this project can author ([section 11.3](#)).
- *Contract*: the compiler's statement of a program (`program.contract()`: code identity, entry, bindings, instruction inventory) and of its emission choices (`program.abi()`: register count, forms, system-register set, metadata class, launch) (MM 23.5).

11.1 __GPU_METADATA slots

`__GPU_METADATA` is a FlatBuffers document: a root table pointing at a per-kernel table with a vtable, binding-record vectors, and further vectors and records (MM 25.143, `agxforge/g17/mdgen.py`). Over 6,000 cached sections, per-kernel slots 2, 4, 6, 8, 10, 12, 13, 26, 27 and 29 are pointers to length-prefixed vectors; 6, 8, 10 and 12 are empty in 2,828 of 2,828; 2, 4 and 26 are built from the binding list (`agxforge/g17/authorobj.py`). Writing an integer into a pointer slot creates a dangling pointer in a section the loader walks; that failure class hung this GPU on 2026-09-08, and the author now refuses it. The per-kernel table declares 60 bytes, and every corpus object leaves at least 68 bytes before the next structure: packed to its 52 bytes of fields the loader segfaults, and a slack sweep put the boundary between 8 and 12 bytes (`agxforge/g17/mdgen.py`). Binding records come in six measured shapes (long, short, elided for buffer 0, written, mid, elided-written); across 9,207 records every slot set lies within {0, 1, 2, 3}, field 3 is the written

flag, and no record carries a type: a buffer's element type never reaches `__GPU_METADATA` (agxforge/g17/mdgen.py, agxforge/g17/authorobj.py).

The loader does not reach the binding records through the FlatBuffers root. In the measured scalar class the root's slot 3, the only pointer that could reach the two binding vectors, is zero, yet the records are read: swapping two buffer indices moved the store off its buffer and left the sentinel untouched. Whatever walks the section finds those records another way (measured, agxforge/g17/mdgen.py). One field found inert on early kernels mattered on a later one: the first two classes were built from blobs whose zeroing sweep had erased a whole single-record vector; that was harmless while a kernel stored to a device buffer and fatal (a loader segfault) once a third binding record made slot 26 point at the wrong vector (agxforge/g17/mdgen.py).

Table 11.1 lists the per-kernel slots whose meaning is established.

Table 11.1. Per-kernel `__GPU_METADATA` slots with established meaning, with content and standing.

Slot	Content	Standing
0	declared register count = highest 32-bit register named in the main program + 1 (2,022 of 2,022 corpus objects; 247 of 247 delivered bundles)	Apple's compiler emits; inert for execution and residency (section 1.4 ; MM 10, MM 25.115); a corrupted slot-0 <i>offset</i> crashes the driver (MM 25.147)
1	kind-6 field 2 + field 3 (section 11.2)	Apple's compiler emits, law S1 (MM 25.120)
2, 3, 4	built from the bindings; in the scalar class slot 3 = 8 × bindings, slot 4 = Q - 4, slot 2 = Q - 4 + 4 × bindings, where Q is a per-kernel multiple of 4 correlating with no readable field	measured: only the right slot-4 value executes (agxforge/g17/mdgen.py)
13	the constant pool, a vector whose count is a byte length (region 4 + count + 16 bytes): loop trip counts, offsets, division-magic constants (0xAAAAAAB for /3, 0xCCCCCCD for /5); changing one literal in the source moves slot-13 bytes while <code>__TEXT</code> is byte-identical	Apple's compiler emits (MM 25.118, MM 25.120); the uniform-halfword pool read by ordinary forms (section 7.8)
15, 16, 17	one byte each, value 1 whenever present; slot 16 = writes a device buffer, 17 = writes a texture, 15 = 16 or 17: over 21,131 sections exactly four combinations (none 286, texture only 70, buffer 20,757, both 18). Threadgroup and imageblock stores set none	Apple's compiler emits (agxforge/g17/cc.py)
18	threadgroup memory used (static or dynamic), value 1 when present (2,020 of 2,020)	Apple's compiler emits (agxforge/g17/cc.py, agxforge/g17/mdgen.py)
19	the explicit imageblock declaration (with <code>__GPU_LD_MD</code> slot 23 and an imageblock form of <code>__GPU_ARCH_LD_MD</code> carrying the element size); in the tensor class slot 19 sits at 55 and slot 44 moves to 54	Apple's compiler emits (MM 25.89, MM 25.104); an undeclared image reads 0 (measured, section 7.7)
27	binding-index words, a word count: 0, 1, 2 name buffers A, B, C	Apple's compiler emits (MM 25.118, MM 25.120)
28	statically declared threadgroup bytes; Metal's <code>staticThreadgroupMemoryLength</code> reports exactly this (0, 16, 64, 128, 16,896 over eight kernels)	measured (MM 25.147)

Table 11.1, continued

Slot	Content	Standing
29	the system-register vector: $4 + 4n$ bytes, one entry per special register read, ordered by entry value and keyed by register, not position; the numbering is section 6.2 's (for example SR130, the lane index, is entry 52; SR156, threadgroup position x , is 0)	measured for the active classes (MM 0.8 , MM 15)
31	present iff the constant program writes at least one uniform (726 of 726 MMA programs; 22,299 of 22,305 objects); value $16 + 16 \times \text{ceil}(\text{buffers} / 2)$	Apple's compiler emits, laws L4 and S31 (MM 25.120)
32	instruction-count class: absent at 30 main-program instructions or fewer, 3 with a back edge, otherwise 1 up to 300 and 2 from 301 (43 witnesses)	Apple's compiler emits (agxforge/g17/mdgen.py)
33	present for programs with a runtime loop	Apple's compiler emits (MM 25.100 , agxforge/g17/mdgen.py)
38, 42	the texture-state byte count, in a texture kernel	Apple's compiler emits (agxforge/g17/mdgen.py)
44	the tensor-unit enable, one byte, value 1; present exactly when the code contains an accelerator MMA (50 of 50 such kernels, no legacy-engine kernel)	measured, causal over 14 pipelines (MM 6 ; section 10.7)

Clearing slot 44's low byte tests whether it is a validity flag, which the driver would check and refuse or fault on, or an enable for the unit. The pipeline still builds and the dispatch completes without error, every tensor MMA writes zeros, and the MMAs run fast: a 1,400-MMA chain takes 14.75 μs instead of 32.0 (8.4 ns per issue instead of 22), a saturated dispatch 0.587 ms instead of 22.7. The MMAs still issue, at roughly ALU cost, but the unit does no work. Any nonzero low byte, with the other three bytes arbitrary, is bit-identical to 1 (measured, MM [6](#)). The timing fits a power or clock gate on the unit better than a validity check, though the mechanism was not measured ([section 10.7](#)). Since neither pipeline creation nor the dispatch rejects a cleared value, a missing enable shows up only as zeros, and a compiler has to derive the slot from the presence of an accelerator MMA in the emitted code. Its byte position in the per-kernel table depends on the class (an imageblock declaration moves it to 54, MM [25.89](#); the constant-program re-layout places it at 55, [section 11.2](#)); "slot 44" in this document means the field, wherever the class puts it.

For slot 32, MM [10](#) calls the low byte inert, while MM [15](#) and agxforge/g17/mdgen.py treat slot 32 as an instruction-count property a body must satisfy; the first concerns the value and the second its presence. The *value*'s low byte was mutated without changing execution in the measured kernels (MM [10](#)). *Presence* changes the table's layout: the tail's byte slots (33, 32, 16, 15) are right-aligned and the slots after them move, so a layout with the wrong presence places other fields, among them slot 0 and slot 44, at the wrong offsets (agxforge/g17/mdgen.py, MM [25.120](#)). This compiler therefore emits both presence and value by Apple's law ([section 3.5](#) records the earlier padding to 31 instructions; MM [15](#)). Not established: whether a wrong value with the right presence matters for any kernel shape beyond those tested.

Worked example: the running example's contract and the metadata it implies (compiled; delivered image executed below Metal). Each row of [table 11.2](#) gives a field of the contract this compiler records for the running example (example-listing.txt, the line after the listing) and the metadata that field implies:

Table 11.2. Contract fields this compiler records for the running example, with the metadata each implies.

Contract field	Value	Metadata consequence
register_count	76	slot 0 = 76: the highest register named is R75 (top of the B tile R72_R75)
main_instruction_count, back edge	101, none	slot 32 = 1
system_registers	130, 156	slot 29 = [0, 52]: special-register read, 16-bit (op14060) of SR_SIMD_ELEM (the lane index) and special-register read (op14059) of SR_TG_X (threadgroup x)
execution.tensor bindings	true (four tensor MMAs) A (index 1, offset 0), B (2, offset 2), C (3, offset 4, written)	slot 44 present, value 1 binding records, C in the written shape; descriptor bytes 0, 4, 8 (rank x 4) in the code (MM 23.2)
pk_values	slots 15 and 16 = 1	writes a device buffer, no texture
constant_pool, prologue	empty; 0e000000 + thirty 0600	no slot 31, no constant-program record; the 64 bytes before entry 64 are the constant-program position, holding only end of program (op684) and two-byte filler (op13483)
launch	exact grid required, not bounds-checked	runtime policy of the released class, not metadata

register_count, main_instruction_count, forms, the system-register vector and the launch facts come from the emitted program and are never copied from a nearby kernel; tensors metadata.layout refuses an unwitnessed tail or system-register set (this compiler, MM 23.2). The contract's register_count and main_instruction_count count _agc.main only (MM 25.146).

Below Metal, the register count also travels in the launch state: the USC Registers packet (tag 0x8d) carries it in bits 8–12 in groups of eight, plus a spill size (measured by capture; chapter 13).

11.2 Constant program and record

Every Apple MMA program carries a second program, `agc.main.constant_program` (with a `.cfg` suffix when it branches), which runs once per dispatch, as one thread, before the main program, and computes thread-invariant values into uniform registers that main reads (Apple's compiler emits; modelled and checked against Apple's GPU, MM 25.120, MM 25.179). Its inputs are binding entries read by argument read, binding-table word (op14061): for the k -th buffer, halfwords $8 + 4k$ and $10 + 4k$ hold its 64-bit pointer, low word then high word, in all 21 constant programs examined (MM 25.59, MM 25.179; the two sections' notations agree, appendix A). *Correction:* MM 25.113 read this metadata as a register-spill description; it describes the constant program (MM 25.120).

A constant-program instruction writes a uniform exactly when Apple's decoder prints its first operand as `bin(op0, K, S)` (`op0` being the uniform register file, decoder tag (`op0`)); it writes uniform $K // 2$ (the uniform file itself is section 2.2). Writers are uniform / staging write (op592) in both forms and ALU forms with a uniform destination, among them 32-bit add immediate into the uniform/staging file (op10298), integer to float into the uniform/staging file (op11185), fp32 add with immediate into the uniform/staging file (op1038) and fp32 FMA into the uniform/staging file (op2318); appendix C lists the rest (MM 25.120).

Three pair exactly with main-program forms plus one extra mode operand: the integer-to-float writer with 32-bit integer to fp32 (op11179), the uniform / staging write with 32-bit register move (op586), and the fp32 add writer with fp32 add with float immediate (op1006) (MM 25.179). K is $2 \times ((\text{byte0} \gg 4) | ((\text{byte2} \gg 6) \& 3) \ll 4)$ in the 4-byte uniform / staging write, reaching 126 (section 3.1), and a nine-bit field in the 8-byte and longer forms (MM 25.120). The worked bytes 8b040924 write u8 (K 16) from R2 and wait on slot 0; b308070256a0a402 writes u9 (K 18) from R0 and waits on slot 1 (decoder, MM 25.120.1).

Table 11.3 gives the laws of the constant program’s metadata (Apple’s compiler emits; each checked on every MMA program it applies to with zero misses, and predicted before compiling held-out variants, MM 25.120).

Table 11.3. Laws relating the constant program to its metadata records and per-kernel slots.

Law	Statement
L4, S31	per-kernel slot 31 present iff the constant program writes a uniform; value $16 + 16 \times \text{ceil}(\text{buffers} / 2)$, $\text{buffers} = \text{len}(\text{argument map}) / 4$ (32 or 48 in the corpus; 64 at 5 buffers, held out)
L1	kind-3 record field 0 = $8 + 4 \times \text{len}(\text{argument map})$
L2	kind-3 field 2 = the constant program’s register count (1 if it writes no uniform)
L5	the argument map (count at record + 20) is [4b, 4b+1, 4b+2, 4b+3] per buffer b whose binding entry the constant program reads
L3	kind-6 field 3 = $1 + \max(K // 2)$ over every uniform write
S1	per-kernel slot 1 = kind-6 field 2 (the main program’s uniform count) + field 3

L3 was reached by refutation: “6 + the number of writes” failed on a variant that skipped u6-u7; “1 + the highest uniform written by uniform / staging write (op592)” failed where a 32-bit add immediate into the uniform/staging file wrote the top uniform; four compile rounds with committed predictions converged on “every writer counts” (MM 25.120). Over the wider corpus L3 fits 2,650 of 2,720 programs with a constant-program write; all 70 misses are non-MMA and under-predict, and in 50 of them the main program reads uniforms up to Apple’s value, a rule not pinned. (The u0-u5 reservation in MMA programs is section 2.2.)

With a constant program the serializer re-lays the per-kernel tail: slot 44 moves to byte 55, slot 31 (a 32-bit value) goes at 56, byte slots 33, 32, 16 and 15 are right-aligned to end at 63, and slot 0 goes at 64; the argument map follows the kind-3 body; every later position moves (relocate_layout), and Q is recomputed. All 726 Apple MMA programs re-serialize byte-identically after the change (this compiler, MM 25.120).

Of 726 Apple MMA programs, 265 have metadata sections this compiler reproduces byte for byte from recovered facts alone, 286 more reproduce only when Apple’s slot-13 and slot-27 values are supplied, 72 differ and 103 are refused as class limits (bindings beyond the class, unmeasured system-register sets, missing records) (MM 25.120). The section’s size is fully determined by table shapes, the lengths of the per-kernel vectors (slots 13, 14, 26, 27, 29, 41) and the constant-program record’s length (56 classes, none ambiguous; MM 25.113). The two blocking inputs, the slot-13 prefix values and the argument map, are facts about the kernel *source*: one program reads 301/400/500 where its sibling reads 300/400/500; only 631 of 1,824 nonzero prefix words appear anywhere in the code; two programs with byte-identical argument reads list different buffers. They cannot be recovered from Apple’s code, and for a program this compiler builds they are known by construction (MM 25.120).

The project’s emulator, running the constant program first as one thread over the same buffers and handing its uniform writes to main, equals Apple’s GPU byte for byte on 6 of 6 sources with full-width random data, and every deformed model (all published uniforms 0; the argument read naming the next binding; halfwords swapped) differs on all 6 (measured against Apple’s GPU, MM 25.179).

Not known. Whether a uniform write wider or narrower than 32 bits occupies exactly one uniform (the variants built to test it emitted no such write); the main-program rule behind the 70 L3 misses; 442 constant-program uniform writes no main program reads (MM 25.120, MM 25.120.1).

11.3 ABI classes

A class is a measured layout plus the free values inside it. This project authors only classes it has witnessed and refuses the rest (this compiler, agxforge/g17/mdgen.py, tensormetadata.layout).

The released tensor class is ABI version 5 (this compiler’s platform contract, MM 23.2):

ABI version	5
entry	64
execution	{simd_width: 32, tensor: true}
public buffers	A=1, B=2, C=3
binding ranks in code	A=0, B=1, C=2 -> descriptor bytes 0, 4, 8
pointer-block offsets	0, 2, 4
system-register sets	(130,) or the measured composed class (130, 156)
constant pool	explicitly empty
pk_extra / pk_values	(15, 16) / both 1
prologue	0e000000 followed by thirty 0600 fillers
grid / threadgroup	(32,1,1) / (32,1,1)

Three of its properties are machine rules (MM 23.2). SR130 must be read in a tensor kernel. Descriptor byte values are $4 \times$ binding rank, where the rank is the position in the compiler’s ordered binding contract and differs from the public Metal buffer index. And without the tensor enable (slot 44) every MMA writes zeros (MM 6). The single-simdgroup, exact-grid launch is runtime policy: research lowerings ran 2 and 4 simdgroups and up to 8 threadgroups bit-exactly (MM 0.13, MM 25.89).

Other classes have also been measured:

- The scalar object class (500-byte section, nine per-kernel slots, Q carried by class) and the tensor class (412 bytes, slot 44 at 55) are the two base layouts; further classes add binding shapes (the written-buffer class; reproduction counts are in section 11.2), the composed system-register sets (130, 156), (130, 133, 156) and the imageblock set (130, 164, 165), the constant-program re-layout, and loop tails without slot 32 (agxforge/g17/mdgen.py, MM 25.104, MM 25.108, MM 25.120).
- The system-register class is keyed on the full set. (SR130, SR156) authors and runs as a measured class; it is not a universal rule for every form whose decoder byte 1 is 130 (13 of 385 such forms cannot inherit entry 52 from the coarse key) (MM 0.13, MM 0.14).
- An empty system-register set must be refused: 7 real programs declaring one crashed the driver inside pipeline creation (measured, MM 25.147).

The delivered manifest (manifest.json, format g17-common-pipeline-v3) records the ABI version, system registers, per-binding index, offset, written flag and element type, the entry, and the (opcode, length)

forms. 222 of 247 manifests carry a template’s `register_count` and `code_size`; the true values are in the container, and a tool reading the manifest’s count is misled (MM 25.147).

The other metadata sections were measured by one-at-a-time mutation (`agxforge/g17/authorobj.py`):

- `__GPU_LD_MD`: format known and entry PC named causally, but its slot set is a compiler input. A foreign but structurally valid `__GPU_LD_MD` (a buffer kernel’s in a texture kernel) builds a pipeline without complaint, which is why this section is never defaulted. It carries a per-process build counter that reads like a field (MM 25.91).
- `__GPU_ARCH_LD_MD`: one boolean, set in 6,234 of 21,001 objects; no contract fact and no opcode determines it (the best of seven tested predictors left 3,144 exceptions), so it is a compiler input.
- `__GPU_STATS_MD`: 96 bytes with 20,999 distinct patterns in 20,999 objects, not read by the loader: zeros, 0xFF, another kernel’s bytes or removal all left an authored texture read exact.
- Damaging `__GPU_METADATA`, `__GPU_LD_MD` or `__GPU_ARCH_LD_MD` kills Apple’s driver inside pipeline creation.

11.4 Object, library and archive bytes

A bundle’s `scan.arc.metallib` is a fat container holding an `air64 MetalLib` and an `applegpu Mach-O GPU executable`. It is loaded with `newBinaryArchiveWithDescriptor` and `newComputePipelineStateWithDescriptor` under `MTLPipelineOptionFailOnBinaryArchiveMiss`, which makes Metal use the precompiled code rather than compile the AIR (MM 25.143). The project’s tools author each layer: `g17mtlb.py` (the `MetalLib`), `g17container.py` (the Mach-O head and AIR tables), `g17arc.py` (the archive’s `__reflection`, `__compute`, `__descriptor`, `__metallib`), `g17authorobj.py` (the object), `g17mdgen.py` (`__GPU_METADATA`), `g17ldmd.py` (`__GPU_LD_MD`) (MM 25.143). Metal caches pipelines by AIR function hash, so a patched archive must be built in one process and dispatched in another, with a positive control; without one, a “no change” result cannot distinguish an inert bit from a cached pipeline (measured, MM 17 rule 36).

A census (`tools/g17imagecensus.py`, MM 25.148) assigns every byte of 421 distinct delivered archives (12,770,955 bytes, 1,253 index entries over ten delivery roots) to exactly one kind (table 11.4), and the tool refuses to finish if a byte is left over.

Table 11.4. Bytes of 421 delivered archives by kind, with each kind’s share and how it is checked (MM 25.148).

Kind	Share	What is checked
authored: <code>__text</code> program bytes and prologue, <code>__GPU_METADATA</code>	85.6%	program bytes equal the bundle’s <code>program.bin</code>
generated: fat header, both <code>MetalLibs</code> , container head, object header and symbols, <code>__GPU_LD_MD</code> , <code>__GPU_ARCH_LD_MD</code> , <code>__text</code> filler	10.6%	the whole envelope is re-emitted and compared
class: constants execution shows are read	0.2%	each compared at its offset (72 bytes)
inert: constants nothing reads	0.3%	each is zero (76 bytes)
tail: the descriptor’s (392, 16)	0.1%	its value; inert, and not one of the 148
zeroed: <code>__reflection</code> , <code>__descriptor</code> , <code>__GPU_STATS_MD</code> , <code>__GPU_REMARKS_MD</code>	3.2%	all zero
unexplained	0	

The envelope is a function of the object and the entry name: the module hash is sha256(object + name), both MetalLibs are built from (name, hash), and the archive is re-emitted byte for byte in 421 of 421. `__text` is prologue + program.bin + filler in 421 of 421. Rebuilding from recorded recipes reproduces 1,249 of 1,253 entries byte for byte; the other four are stale roots whose recipe now compiles a different program, not authoring failures (MM 25.148).

Apple’s own reader (`xcrun metal-objdump --macho --private-headers`) names the load commands, which identifies most of the 148 class constants: what the project had called `LC_TABLE` (0x31) is `LC_NOTE`, whose `data_owner` is the AIR name; 0x32 is `LC_BUILD_VERSION`. With the `TOOL_*` values from `<mach-o/loader.h>` most constants are toolchain identifiers (table 11.5; MM 25.148).

Table 11.5. Class constants by byte offset in slice 1 of the archive, with whether execution reads them, their value and meaning (MM 25.148).

Offset	Bytes	Kind	Value	Meaning
824	4	read	1	<code>LC_BUILD_VERSION</code> platform = <code>PLATFORM_MACOS</code>
828, 832	8	inert	0	minos, sdk (Apple writes 0x1A0600 = 26.6 as minos)
836	4	read	2	ntools
840, 844	8	inert	0	tools[0] (Apple writes <code>TOOL_GPUARCHIVER</code> 0x407, version 0x7D17FF00 = 32023.255)
848, 852	8	read	0x403, 0x20001	tools[1]: <code>TOOL_AIRNT_PLUGIN</code> , 2.0.1
1088	32	inert	0	the <code>AIR_DESCRIPTOR</code> key (Apple’s varies)
1260	32	read	e3b0c442...	SHA-256 of the empty string, checked by value
1296, 1300	8	read	0x402, 0x403	<code>TOOL_AIRNT</code> , <code>TOOL_AIRNT_PLUGIN</code>
1304	4	read	0xC0000000	open: required, meaning unknown
1324	4	read	1	open: required, meaning unknown
1428	4	read	0x408	<code>TOOL_METAL_FRAMEWORK</code>
1440	4	read	0x40000004	open: required, meaning unknown

60 of the 72 read bytes are named; 12 are open. They belong to a record with a position rule, not to fixed slots: over 22,721 Apple archives the eight-word record starting with `TOOL_AIRNT` sits at 1296 in 22,625, one word later in 58, and is absent in 38, and the word at 1440 is missing in exactly the 96 archives where the record moved (MM 25.148). *Correction:* an older “76 inert” list and an older one-kernel “image is ours” census are superseded by this census (appendix A).

The GPU executes the machine code the project authored, from the copy Metal places in its heap. Two things were tested: whether Metal recompiles the kernel from the archive’s AIR (or serves a pipeline cached by AIR hash), and whether a change to that heap copy after pipeline creation reaches the program that runs.

- `program.bin` appears byte for byte in the archive’s `applegpu` slice. 1,620 operand-byte flips, each checked to leave the opcode stream unchanged on decode, were written into that region (no rebuild, AIR and its hash unchanged) and dispatched in a fresh process; all 2,048 output floats and the kernel’s time changed (MM 25.143). The executed code is the archive’s machine code, not a recompilation of its AIR.
- For 10 delivered kernels of six kinds, the program’s exact bytes appear at offset +0x2740 of the dispatch’s first mapped 64 KiB region. The causal test patched that heap copy after `newComputePipelineState`

and before the dispatch, on $C[i] = 3 \cdot i + 1$, changing the add constant (program byte 19) from 1 to 2 only if the bytes occurred exactly once. Unlike the operand flips above, this flip is in a byte adjacent to an opcode. The unpatched control read $3i + 1$; the heap patch read $3i + 2$, the same result as the flip authored in the container; and a patch asking for absent bytes wrote nothing and read $3i + 1$, so the patcher alters nothing else (measured, MM 25.147). This establishes that the executed program incorporates the patched bytes; it does not determine whether subsequent copying or validation occurs.

Metal reports `maxTotalThreadsPerThreadgroup` 1,024 and `threadExecutionWidth` 32 for every kernel and `staticThreadgroupMemoryLength` equal to slot 28; the declared register count it reports is the container's own (measured, MM 25.147). Where the driver takes the thread limit from is [section 1.2](#) (MM 25.141.20).

What pipeline creation validates was mapped by handing it the known-answer container with one change at a time (`spike/agxsub/g17validprobe.py`; nothing dispatched; MM 25.147). Two of the inputs are negative controls that pipeline creation should reject. The outcomes:

- *Rejected* (“invalid format”): a corrupted archive header, an archive truncated to half. These controls show that pipeline creation does check its input, so the acceptances below are not explained by an absent check.
- *Accepted*: code replaced by undecodable `0xff`, the end of program removed, declared register count 0, 1, 200 or 256, static threadgroup memory declared at 1 MiB (reported back as 1,048,576, far past the 32 KiB a threadgroup can have).
- *Crashed the driver* (a CPU fault inside `newComputePipelineState`): a corrupted per-kernel slot-0 offset, and the 7 programs with an empty system-register set.

Since pipeline creation checks neither the instruction bytes nor the metadata values, and a bad metadata layout faults the driver, errors inside the format have to be caught when the image is authored. This project's guards ([section 3.8](#)) and refusals run before the image is written for that reason.

Code generation and the whole container are this project's; no Apple compiler process runs to produce them, and Apple's decoder is read in-process only as a check. Turning Metal source into AIR still runs Apple's front end, and the knowledge of which forms exist was learned from Apple's compiler output (MM 25.147).

Evidence. MM 0.8, MM 0.13, MM 0.14, MM 6, MM 10, MM 15, MM 17 (rule 36), 23.2, 23.5, 25.59, 25.89, 25.91, 25.100, 25.104, 25.108, 25.113, 25.118, 25.120, 25.120.1, 25.141.20, 25.143, 25.146, 25.147, 25.148, 25.179; `agxforge/g17/mdgen.py`, `agxforge/g17/authorobj.py`, `agxforge/g17/cc.py`.

12 Performance

The measured resource model of one G17 core has six resources, and a kernel's time is set by whichever it exhausts first:

1. Instruction issue: one simdgroup issues a simple instruction about every 3.1 ns; there is no fp32/int32 dual issue ([section 12.3](#)).
2. The tensor unit: about 4.96 ns per $16 \times 16 \times 16$ MMA per core at saturation, 1,024 flop per cycle per core ([section 12.1](#)).
3. Dependent latency: a dependent MMA or a load-to-use chain in one simdgroup costs far more than the issue interval (74 ns per MMA in one chain, 30.5 ns per L1 load, 405–415 ns per DRAM load); only independent work, from the same simdgroup or from other resident simdgroups, hides it ([sections 12.1](#) and [12.6](#)).
4. Residency: how many simdgroups a core holds, which is what supplies that independent work ([section 12.5](#)).
5. Memory bandwidth by footprint: on-core, SLC (520–528 GB/s) and DRAM (269.6–298.5 GB/s depending on the instrument) ([section 12.6](#)).
6. Instruction fetch and dispatch: a hot loop body over 16 KiB loses the instruction cache at low occupancy; each dependent dispatch costs microseconds ([sections 12.6](#) and [12.7](#)).

In the kernels this project studied, the tensor unit's peak was rarely the limit. The fused attention and FFN kernels execute about 46 instructions per MMA, so the tensor pipe sits idle most of the time and they are issue- and ALU-bound at 35–57 percent of peak ([section 12.2](#)). The other studied workloads are bound elsewhere: the one-token decode projections are DRAM-bound at q8 and issue-bound by counter at q4 (yet the q4 MLP did not speed up when a quarter of its instructions were removed); batched decode is bound first by occupancy and, after split-K, by the fp32 pipe; prefill attention by latency on a scratch round trip; and prefill GEMMs reach 80–89 percent of peak once measured in the graph ([section 12.10](#)).

Rate and timing terms:

- **Per core / per simdgroup:** a rate quoted per core aggregates every simdgroup on it; a per-simdgroup rate is one instruction stream. The device has 20 cores and four SIMD slots per core ([chapter 1](#)).
- **Occupancy or simdgroups per core:** simdgroups launched divided by 20 (the launched figure) or resident at once (the residency figure); the text says which.
- **Saturated:** enough simdgroups per core that adding more does not raise throughput.
- **Warm clock:** timed after a saturating dispatch long enough to raise the GPU to its top performance state ([section 12.8](#)). Every absolute time in this chapter is at a warm clock unless stated.
- **Footprint:** unique bytes a kernel touches.
- MMA opcode names are those of [chapter 10](#); the tensor MMA used for rate measurements is tensor MMA, 16-bit $\times$ 16-bit, accumulate (op5106).

Beyond the 0.4–1.3 percent in-process and 9–13 percent cross-process spread of [section 1.6](#), one row differed by 18 percent between processes, and the rebuilt harness moved 25–44 percent between its own processes; ratios timed together in one process reproduce to about 1.5 percent (measured, MM [16](#), MM [25.124.2](#)), which is why the comparisons here are ratios. Under load the second run of a fixed A-then-B pair ran about 0.1 ms per token faster, so every A/B here is order-balanced (measured, MM [17](#) rule 43).

12.1 Tensor issue rate and latency

At saturation (measured, MM 8), 16,384 threadgroups x 4 simdgroups x 1,400 issues of the 16-bit MMA, tensor MMA, 16-bit × 16-bit, accumulate (op5106), is 91.75 M issues in 22.44 ms: 4.09×10^9 issues per second, 33.5 TFLOP/s dense fp16/bf16 into fp32 at 8,192 flop per issue, 1.67 TFLOP/s per core, 128 MAC per cycle per SIMD (four times a 32-lane FMA) at an implied 1.63 GHz. int8 runs at exactly twice the rate: 13 ns per issue in a chain, 11.50 ms saturated, 8.1×10^9 issues per second, 67 TOPS, 256 MAC per cycle per SIMD, using tensor MMA, int8 widening to int32 (op10384).

An independent later harness gives 4.96–4.97 ns per MMA per core at 16 simdgroups per core, 8,192 flop in 8.0 cycles at 1.62 GHz, 1,024 flop per cycle per core; int8 2.47 ns. About 5.5 simdgroups per core saturate the unit (measured, MM 8, MM 0.1). Percent-of-peak figures in the sources use 33.1 TFLOP/s, carried from that harness and not re-derived; 33.5 is the derived figure. They differ by 1.2 percent, inside harness spread (MM 8). This chapter quotes percentages against 33.1, as the sources do.

Each core has four tensor slots (measured, MM 8): one, two or four simdgroups each running the chain take the same time (32.8 μ s), 8 take 57.5 and 32 take 224.3, flat to four and linear beyond (the full row is in section 1.3). Each SIMD has one slot and holds it for the whole interval.

The cost of one MMA depends on how much independent work surrounds it (table 12.1).

Table 12.1. Cost per MMA by occupancy and by dependence between MMAs.

Configuration	Cost per MMA	Source
dependent chain at saturation	22.2 ns; eight independent accumulators over the same 1,400 issues take the same 31.96 μ s, so result latency is no longer than the issue interval	measured, MM 8
one simdgroup, 8 independent accumulators	20.5 ns	measured, MM 8
one simdgroup, one dependent chain	74 ns (3.6 times)	measured, MM 8
one simdgroup per core: 8 dependent MMAs vs four independent chains of 2	both 218.0 ns per iteration, so chaining adds nothing and the cost is that of a single chain	measured, MM 8
fit over 321 kernel timings, bf16 class	62.6 ns inside a dependent chain at 1 simdgroup per core; 5.8 ns at 64	measured fit, MM 16
same fit, fp8 and int8	68.2 ns at 1; 7.0 ns at 64	measured fit, MM 16
tensor MMA, 16-bit × 16-bit, no accumulator (op5107) back-to-back into the same destination	34 ns (a write-after-write stall), gone across 8 destinations or at saturation	measured, MM 8

The same fit charges 1.6 ns per other instruction at one simdgroup and 0.21–0.34 ns at 64. Its six worst outliers are all independent-MMA ablations, so the large per-MMA figure measures dependence rather than the MMA itself (MM 16). The fp32-operand forms (section 10.2) and a transposed A cost nothing extra (measured, MM 8).

At low occupancy a compiler should interleave independent accumulators (one simdgroup goes from 74 to 20.5 ns per MMA); at 12 or more simdgroups per core the forms converge and the interleave buys nothing (MM 8, MM 11). A 255-trip one-slice K loop in one simdgroup averages 1.45 μ s per trip (loads then four MMAs, nothing overlapping), the low-occupancy extreme (measured, MM 25.101).

The static model’s tensor constants are fitted, not measured (section 12.9; MM 25.144.12).

Evidence. MM 0.1, MM 8, MM 11, MM 16, MM 25.101, MM 25.144.12.

12.2 Bottlenecks in the studied kernels

Fused attention and FFN are issue- and ALU-bound (measured, MM 8, MM 0.12, MM 16). Fused attention reaches 35–42 percent of 33.1 TFLOP/s (11.5–13.8 TFLOP/s) and the FFN 47–57 percent. A real attention chunk loop executes about 46 instructions per MMA: 1,480 instructions for 32 MMAs is 356 ns of the 433 ns a key block costs per core, while the tensor pipe is busy for 158 ns. Attention throughput stays at 11.5–13.8 TFLOP/s from an 8 MiB working set to 128 MiB, so it is issue-bound, not bound by the KV stream. One simdgroup’s dependent chain costs 5.35 μ s per 32-key block; about 247 simdgroups saturate the GPU.

In the attention kernels (measured, MM 16, recon kernels), simdgroups per threadgroup from 1 to 16 are within noise (keep at least 20 threadgroups so every core has one); transposed-B flags cost 1.3 percent; unrolling K by 2 or 4 gains nothing and adds 98–110 spill stores; two token tiles per task doubles the time (354 spill stores). The retained rebuild costs 1.25–1.29 times the recon per key block (6.9 against 5.35 μ s for one simdgroup; 558 against 433 ns per block and core) despite 1,033 instructions and no spills, against the recon’s 1,925 with 42 (measured, MM 25.126).

Loop form and fusion were measured in MM 16. The runtime K loop is 1.3–1.8 times faster than unrolling at 8 blocks and 5.9–7.2 times at 32 (unrolled kernels spill 24–923 stores). A spill-free fused chain is 5–10 percent faster than any cut and 8–20 percent faster than three kernels; at 15 or more spill stores the cuts win. Attention stays fused in all 27 measured shapes even while spilling 42 stores (cutting P costs 1.03–1.30 times, cutting both 1.14–2.03 times). For a gated FFN at $d = 2048$, $Hh = 8192$, one wave is 20 threadgroups at about 1.8 ms in fp8, and the fused form needs $nsg*mt*640$ bytes of threadgroup memory (fp8, int8) or $nsg*mt*1,088$ (bf16, fp16) against 32,768: $nsg*mt \leq 51$ and ≤ 30 .

Table 12.2 compares this compiler’s GEMM with Apple’s matmul2d (measured, interleaved medians, $M \times 128 \times 256$ half, MM 25.107).

Table 12.2. GEMM time for $M \times 128 \times 256$ half: Apple’s matmul2d against this compiler’s K-loop and unrolled forms.

Shape and grid	Apple	this compiler, K loop	unrolled
M512, 8 threadgroups	106.6 μ s	113.7 μ s (1.07 $\times$ behind)	262.7 μ s
M1024, 8 threadgroups	183.6 μ s	244.7 μ s (1.33 $\times$)	376.0 μ s
M4096, 256 threadgroups x 1 simdgroup (equal geometry)	58.7 μ s	62.0 μ s (1.06 $\times$)	78.6 μ s

Apple’s own 64×4 geometry takes 69.4 μ s for M4096; at 12.8 simdgroups per core the K loop reaches 4,330 GFLOP/s against Apple’s 4,570. *Correction:* MM 25.98’s earlier ratios are superseded by MM 25.107 (appendix A).

- Occupancy mattered more than instruction scheduling. An occupancy sweep (one 16-row tile row per simdgroup, a 12.5 KB body) goes from 132 GFLOP/s at 8 simdgroups (0.4 per core) to 3,329 at 256 (12.8 per core) and 4,497 at 512 (25.6 per core, 124 registers); work grew 32 times and time 1.27 times

(measured, MM 25.98.1). Software pipelining (rotating MMAs ahead of loads) gave 1.08 times at 8 simdgroups (113.8 to 105.6 μ s, where at least 1.2 had been preregistered) and 0.99 times at 12.8 per core (measured, MM 25.97, MM 25.107).

- In the K-loop body, one slice per trip waiting on its own loads paid a full round trip per step (about 417 ns against Apple’s about 245 at 8 threadgroups). Issuing two slices’ loads before the first MMA (kloop_unroll=2) gains 35–38 percent and closes to 3–4 percent of Apple at 4 and 8 threadgroups (about 337 to 210 ns per step against Apple’s 202); an interleaved control gains 1–4 percent (measured, MM 25.124.6).
- Unrolled bodies are fetch-bound, at about 518 ns per MMA (512 MMAs in 265 μ s) against a 20–74 ns issue interval; fully unrolled K=256 bodies are 61 KB and 116 KB against Apple’s 5,142-byte looping kernel (measured, MM 25.97, MM 25.98).
- Grid-split threadgroups run concurrently: with 512 MMAs per threadgroup, time is flat from G = 1 to 8 ($t(8)/t(2) = 0.995$); the null control (2 MMAs) is 11.5 against 265.3 μ s (measured, MM 25.89).

For the bf16 layer kernels (measured, MM 25.43), 16 simdgroups per threadgroup is fastest for the two- and three-kernel forms (2.856 and 3.103 ms); 32 is 1.46–1.86 times slower than 16 in every form, against a run spread of 0.007–0.069 ms.

A weight-stationary FFN is 3.7–4.4 times faster than a token-parallel one at 16 and 32 tokens (measured, MM 16, MM 25.126); on the retained harness fp8 weight-stationary loses from 48 tokens (1.13 times the token-parallel time) and bf16 from 64 (1.15). *Correction:* the recon model’s “about 64 tokens” crossover is not the measured one for these kernels.

A GELU register epilogue ($x * \text{sigmoid}(1.702x)$, six instructions per slot, 4 registers) takes 15.6 μ s at M16 against 52.2 μ s for a memory stage (MM 25.109). A full-tile one-dispatch attention takes 30.5 μ s at 2 rows to 104.3 μ s at 16 rows, because its softmax is straight-line (MM 25.110).

Evidence. MM 0.12, MM 8, MM 16, MM 25.43, MM 25.89, MM 25.97, MM 25.98, MM 25.98.1, MM 25.107, MM 25.109, MM 25.110, MM 25.124.6, MM 25.126.

12.3 Scalar issue

Table 12.3 gives the issue model (measured, MM 25.142.6, MM 0.15; a counted 128-trip loop over eight independent chains, op counts checked from the decoded bytes, each dispatch in its own process).

Table 12.3. Scalar issue cost per op in one simdgroup, each class alone and mixed, against the predictions of one shared port and of dual issue.

One simdgroup, ns per op	alone A	alone B	mixed	one shared port predicts	dual issue predicts
ffma + iadd	4.18	3.09	3.61	3.64	2.09
fadd + iadd	3.11	3.09	3.11	3.10	1.55
fmul + shl_imm	3.16	6.10	3.17	4.63	3.05
fadd + fmul (control, one class)	3.11	3.16	3.17	3.14	1.58

- A simple instruction costs about 3.1 ns per simdgroup whatever its class; this is the per-simdgroup issue floor.

- There is no fp32/int32 dual issue. The fma + iadd pair separates the two models, and it lands on the shared-port value. The full grid agrees: fadd 1,050, iadd 1,238, mixed 1,271 μ s at B = 128.
- fp32 fma at 4.18 shows its latency is not fully hidden by eight chains.
- shl_imm costs 6.1 alone and 3.17 mixed with fmul: a stall that other instructions fill (the narrow register file's constraint, [chapter 2](#)), not an issue slot.
- fp32 add throughput is about 4.1×10^{12} per second machine-wide, about 2.1×10^{11} per core, consistent with 128 fp32 lanes per core at about 1.6 GHz (the clock was not measured by this probe).

For a compiler, interleaving float and integer work gains nothing; independent chains do, and eight reach the floor (MM 17 rule 38). Apple's wait after every special-register read costs nothing measurable ([section 6.3](#), MM 25.121). The static model's fitted issue constants (3.105 ns interval, 12.5 issue slots per interval per core, 3.43 ns per fragment load, about 150 GB/s of fragments per core) are in [section 12.9](#) (MM 25.144.12).

ALU numerics edges that affect cost are in [chapter 4](#). The one relevant to scheduling is that fdiv costs 9 more instructions than a single-instruction operation such as hardware exp2 (op1272) (measured, MM 6).

Evidence. MM 0.15, MM 6, MM 17, MM 25.121, MM 25.142.6, MM 25.144.12.

12.4 Cost of registers and spills

The register-file and capacity rules (the 126-register flat pool, the trade lines, the accumulator limits, the two spill opcodes, spill cliffs as scheduling outcomes) are in [section 2.8](#).

A spill costs latency rather than saturated throughput (measured, MM 11). Against a 14-accumulator kernel, 15 accumulators (32 spill stores) cost 1.31 times, 16 cost 1.47, 18 cost 2.17 and 24 cost 2.30 at 1–4 simdgroups per core, but only 1.00–1.10 times at 64. Removing spills did not always help; with few stores the streaming order ran at 0.47–0.79 of the flat order's speed in several shapes. Spill counts here include both spill forms ([section 2.8](#); MM 11, MM 13, MM 19).

For this compiler's one-tile GEMM body (measured, MM 25.111), a declared count of 44 against 124 (one metadata byte) is equal in time within 1.2 percent at 25.6 simdgroups per core (41.6 against 42.1 μ s) and 0.7 percent at 51.2 per core (69.9 against 69.4 μ s). The result is specific to this body, and equal time past 32 simdgroups per core does not by itself show residency past 32. Register peaks of this compiler: 16×16×64 R42, 32×32×64 R82, 64×64×64 R122; M512 with GELU reaches R125; declared 44, 60, 92 and 124 at N16, 32, 64 and 128 (this compiler, MM 25.98.1, MM 25.109.1).

Registers did cost time in the decode kernels (measured, MM 25.144.4). In the decode qmv kernel at 4 rows per simdgroup, the same loop at 81 registers instead of 114 is 4.1 μ s faster (43.2 against 47.3 μ s). The B 8 batched qkv kernel uses 109 (q4) and 108 (q8) registers and holds occupancy at 25–27 percent where its threadgroup memory (64–128 bytes) and threads per group would allow 96 simdgroups per core, so the register file sets the cap. Halving its accumulators moved the count only from 109 to 107 and did not lift occupancy: its registers are dominated by weight words, dequantized values, field temporaries and addresses. Instruments' "Register Pressure Influence" counter read 0 for every kernel recorded, including these, so that counter does not see this cap.

Registers and residency are measured in [section 12.5](#); the declared count is inert for execution and residency (rule 1 of [section 1.4](#); MM 10, MM 25.115).

Evidence. MM 10, MM 11, MM 13, MM 19, MM 25.98.1, MM 25.109.1, MM 25.111, MM 25.115, MM 25.144.4.

12.5 Residency and occupancy measurements

The residency rules (what the hardware guarantees, forward progress) are in [chapter 1](#). Each residency count here is given with its kernel and conditions, because the counts differ by kernel and must not be merged.

Metal’s public API has no occupancy counter; it offers only GPUtimestamp (measured, MM 25.48; [section 12.7](#)). `maxTotalThreadsPerThreadgroup` reads 1,024 for every pipeline, from 4 to 512 live scalars per thread and 64 to 32,768 bytes of threadgroup memory, so it does not respond to pressure; `staticThreadgroupMemoryLength` tracks the request (measured, MM 25.47, MM 11). Residency was therefore measured with probes built for the purpose, and later read through Instruments counters ([section 12.7](#)).

An early reading (recon section 167), since corrected, found time flat from 20 to 116 registers per lane and concluded occupancy was “slot-limited, not register-limited”. Above the ALU utilisation ceiling (prior art places it at about 24 simdgroups per core) occupancy can fall a long way with no change in time, so the flat curve said nothing about occupancy. A direct atomic counter (arrival add, peak max, fixed spin, exit decrement) then read occupancy falling 44.6 to 32.1 simdgroups per core over 22 to 126 registers. The timings stand; the occupancy claim was withdrawn (MM 19, MM 20, MM 25.48, MM 25.49).

[Table 12.4](#) gives the counts, each with its kernel.

Table 12.4. Resident simdgroups by kernel and probe, with the conditions of each count.

Kernel and probe	Conditions	Resident simdgroups	Source
Apple-compiled direct atomic counter (recon 170)	22 to 126 registers	44.6 falling to 32.1 per core (x1.39); about 40 at 44 registers (44.5 at 30, 39.6 at 46); 32.0 at 98	measured, MM 19, MM 25.49, MM 25.111.2
Recon 134 kernel, 1,024-thread threadgroups	40 threadgroups at 126 registers	peak exactly 640 resident, 32 per core; resident falls from about 890–930 at 22 registers to 530–660 at 126 (25–30 percent, no step), worth 1.00 to 0.95 in throughput capacity	measured, MM 11
This compiler’s residency counter, straight-line, 12,800 instructions, one simdgroup per threadgroup	warm clock, launched past capacity (S = 4,096), 3 rounds	95.0 per core at 19 registers used; 71–74 at 33; 67–69 at 49; 66–69 at 65; 65–66 at 81; 64–66 at 97; 68–69 at 113; 64–68 at 125	measured, MM 25.121
Same probe, declared against used	S = 1,024 and 2,048	declaring 125 while using 19 peaks with using 19 (1,814–1,896 at S = 2,048); warm, using 125 holds 1,024 of 1,024 in 5 of 5	measured, MM 25.115
Threadgroup-memory handle, 128-thread threadgroups, 4,096 threadgroups	9 to 94 registers	at least 8 simdgroups per core at 94 registers (two threadgroups still co-resident)	measured, MM 25.49
Forward-progress probe, single-simdgroup threadgroups	spinning residents	up to 1,200 threadgroups progress concurrently; first resident wave at 2,000 launched about 1,690 (about 84 per core at that register count); 1,024-thread threadgroups: 200 ran with no consumer starving; at 800, 480 of 799 consumers spun to the poll cap, because the producer, dispatched last, was beyond residency. That run does not measure how many were resident at once, since waves can reach the cap one after another. These are the tested configurations, not a guarantee; the restriction they motivate (do not depend on preemption; size a waiting grid to residency) is section 1.5 ’s	measured, MM 25.141.1
Instruments, B 8 batched qkv decode kernel	109 registers	occupancy 25–27 percent of a 96-simdgroup-per-core budget	measured counters, MM 25.144.4

- The Apple-compiled counter (32.1 at 126) and the recon-134 kernel (32 per core at 126) agree on the floor near 32 for Apple-compiled kernels at high register counts; MM 11’s gradual 25–30 percent fall and the counter’s 1.39-fold fall describe the same shape on different kernels.
- This compiler’s counter reads about twice as many (64–69 per core from 49 to 125 registers) on a different kernel (straight-line code, its own allocator). Why the two kernels differ is open (MM 25.121). A scheduler must use the floor measured for its own kernel family; `agxforge/g17/tensorsched.py` carries both (64 per core from 49 to 125 registers for this compiler’s kernels, 32.1 at 126 for Apple’s).

- Residency follows the registers a program uses, not the count it declares (rule 1 of [section 1.4](#); MM 25.115).
- *Correction:* MM 25.115’s provisional per-core figures are superseded by MM 25.121 ([appendix A](#)).
- A fixed-pool model predicts 7.8 simdgroups at 126 registers where 32.1 are measured, so residency is not one pool divided by demand; the physical capacity and allocation granularity are open ([section 1.4](#); MM 25.49, MM 20).

Threadgroup memory also acts as a residency handle (measured, MM 25.49). The 57,344-byte per-core budget (rule 4 of [section 1.4](#)) shows as a step: 28,672 bytes per threadgroup runs 0.00304 s, 28,688 bytes 0.00382 s (+25 percent for 16 bytes). The step persists at 94 registers (ratio 1.047), and its size tracks arithmetic intensity, not registers: a 24-fma control at 10 registers gives 1.041. The 32,768-byte per-threadgroup limit is [chapter 7](#)’s.

Per-core simdgroup staircase

[Table 12.5](#) gives the median ns per iteration (8 legacy issues) of a retained Apple-compiled legacy simdgroup_multiply_accumulate loop with 8 independent accumulators and 2,048 trips, at a warm clock (a dispatch of at least 4.7 ms before every timed one, a fixed 4-simdgroup-per-core reference dispatch after each), grid = 20 × simdgroups per core, threadgroup 32 (measured, MM 25.122).

Table 12.5. Median time per iteration of the legacy MMA loop against simdgroups per core.

Simdgroups per core	ns per iteration
1–8	24.5–24.6
9–12	30.7–31.2
13–17	54.6
18–21	60.2–60.9
22–30	83.6–84.3
31–34	85–90.6
35–42	113.4–114.5
43–45	119.6–120.7
46–48	143.0–143.6

- Rungs at 24.6 + 29.6k ns (k = 0..4: 24.6, 54.0–54.6, 83.7, 113.3, 142.9), each with a +6 ns half-step about 4 simdgroups before the next full rung (30.6, 60.2, 89.9, 119.5); a full rung spans about 11 simdgroups per core.
- Models: total work as a line, refuted (steps of 6–24 ns against a 0.5 percent floor); a threadgroup granule, refuted (threadgroup sizes 32–128 equal within 1.5 percent at equal work); a residency or wave granule, consistent.
- The first two boundaries fall exactly per core, at 161 and 241 threadgroups (20 × 8 + 1, 20 × 12 + 1). Later boundaries are windows (358–368, 442–460, 613–641, 675–686, 840–855, 891–923 threadgroups) inside which each dispatch of the same grid lands on one of two adjacent rungs; the reference reads 24.54 ns either way, so it is not the clock; outside the windows one grid repeats within 0.33 percent.
- The rung counts simdgroups rather than chains in flight. With 2, 4, 8 or 16 accumulators per simdgroup the boundaries sit at the same simdgroups per core within one; ns per MMA at 1–8 simdgroups per core is 11.87, 5.99, 3.06 and 1.61, a constant about 24 ns per iteration, so up to 16 independent chains

hide behind one dependent-chain latency. Time = (one simdgroup’s iteration time) × f(simdgroups per core), with f about 1 up to 8, 1.25 at 9–12, 2.2 at 13–17, 2.5 at 18–21.

- The cause inside the core is open; it is not a residency cap, since the residency counter holds 64–95 per core (MM 25.121).

Resolution of U3. A “bimodal legacy rate” that flipped from dispatch to dispatch was first read as a two-state per-issue rate with a constant 9/8 ratio (recon 158), then as a 79.4 ns quantum (recon 164, in fact the slope of a line). Both were measurement artefacts: the occupancy axis had been mislabelled by 32 times and oversubscribed (MM 25.85, MM 25.86), and runs shorter than about 1.5 ms ran at the idle clock (a v2 sweep with a 0.5 ms warm-up saw medians of 29.9 and 49.6 ns in the two halves of one run). On the corrected axis at a warm clock the bimodality is confined to the boundary windows above (MM 25.122, MM 19).

In a related legacy measurement, one accumulator chained into itself measures latency rather than throughput: 1 chain 129.5 GMAC/s, 2 give 259.1, 4 give 518.1, then 8 and 16 fall to 395.6 and 199.0 as accumulators outgrow the registers (measured, MM 25.56).

Evidence. MM 11, MM 19, MM 20, MM 25.47, MM 25.48, MM 25.49, MM 25.56, MM 25.111.2, MM 25.115, MM 25.121, MM 25.122, MM 25.141.1, MM 25.144.4.

12.6 Memory bandwidth, latency and instruction fetch

Sustained read bandwidth by footprint (measured, MM 7, streaming microbenchmark) is 1.3–4.8 TB/s up to 5 MiB (on-core, reported as a lower bound), 520–528 GB/s at 10–20 MiB (the SLC plateau) and 281–294 GB/s from 40 MiB (DRAM), with the step between 20 and 40 MiB; independent of threadgroup shape at 1, 8 and 32 simdgroups.

DRAM bandwidth depends on the instrument; table 12.6 gives the figure the sources report for each.

Table 12.6. DRAM read bandwidth by measuring instrument.

Instrument	Read	Source
streaming microbenchmark by footprint, 40 MiB and above	281–294 GB/s (row M7: 283–294)	MM 7, MM 20
1,024-thread threadgroups, 256 MB cold streams	298.5 GB/s (peak; supersedes the 272 GB/s used before)	MM 0.15, MM 25.141.13
grid-stride float4, 1,048,576 threads, 256 MiB and 1 GiB, two rounds	269.6–275.7 GB/s; write 241.0–252.9; copy 243.5–255.6 (read plus write counted); 64 MiB reads 218–264	MM 25.121
static performance model floor	280 GB/s (a model constant)	MM 25.144.12

The sources use 298.5 GB/s as the roofline peak, as this chapter does; a simple grid-stride kernel sees about 270. The third instrument agrees with M7 to within about 6 percent (MM 25.121).

Table 12.7 gives load-to-use latency by footprint (measured, MM 25.121; 32 lanes each chasing a random cycle, warm clock, least-squares slope over five chain lengths).

Table 12.7. Load-to-use latency of a dependent load by footprint.

Footprint	ns per dependent load
16 KiB (L1)	30.5 (about 49 cycles)
4 MiB	195 (192–238)
1 GiB, 16k–64k hops (DRAM)	405–415

The static model uses 30 / 96 / 195 / 313 / 422 ns at 16 KiB / 256 KiB / 4 MiB / 64 MiB / 256 MiB (MM 25.144.12); the 256 KiB and 64–256 MiB rows do not fit one line in the probe and are model inputs. Issuing a load costs 9.3 ns in an unwaited chain (measured, MM 25.97). *Correction:* MM 25.97’s “about 121 ns” L1 latency was the L1 row at an idle clock (MM 25.121).

Measured cache effects in two real kernels:

- Decode attention keeps K and V resident in the last-level cache between dispatches up to about 4.2 MB (1,024 keys), at 18–20 ns per key; past it about 29 ns per key, about 140 GB/s, under half the DRAM peak (MM 25.181).
- Spare threadgroups warming the cache save up to 10 μ s per small-to-big kernel pair (7 percent) when the warm set fits the small kernel’s duration (4 MB with 256 extra threadgroups: 143.2 to 133.3 μ s); decoy loads save nothing; 8 MB lengthens the small kernel (MM 25.141.5).

Instruction fetch in hot loops

The coarse sweep and the step-location sweep in table 12.8 measure different things.

Table 12.8. Instruction-fetch cost against loop size, from two sweeps and two follow-up measurements.

Measurement	Result	Source
Per-instruction time against loop code size, coarse sweep	grows sixfold from 1 KB to 26 KB at one simdgroup per core (0.411 to 2.49 ns), 1.85-fold at 64, then flattens	measured, MM 7
Location of the step, seventeen-point sweep at fixed instruction mix	the variable is the loop body: 15,971 bytes fast, 16,595 partial, 17,247 slow, bracketing 16 KiB (1,363 bytes of the kernel are prologue)	measured, MM 19
Cost of crossing it	a loop body over 16 KiB costs 2.81 times on one simdgroup and 1.15 times at 64 simdgroups per core; cold straight-line code costs 1.91 ns per byte for one simdgroup	measured, MM 25.144.12
At high occupancy	at 26–64 simdgroups of the same code per core fetch is shared and crossing costs nothing extra: register-O attention bodies of 21.6–22.5 KB folded to 13.1 KB ran at exactly the instruction-count ratio (1,169 against 739 μ s = 1.58; 1,829 against 1,141 instructions = 1.60)	measured, MM 25.144.12

The 16 KiB loop-body bracket is where the per-instruction cost steps; the 26 KB figure is where the coarse sweep stopped rising. The hot-loop guideline “about 26 KiB, roughly 2,800 instructions” (MM 0.1, MM 7) is superseded as a design rule: keep a hot loop body under 16 KiB (the fast point is 15,971 bytes). At 26 or more simdgroups per core the cliff costs nothing measurable; at intermediate occupancy fetch is

unmeasured (open). Neither fetch form improved the static model (section 12.9). An earlier whole-kernel bracket is superseded (appendix A).

Evidence. MM 0.15, MM 7, MM 19, MM 20, MM 25.97, MM 25.121, MM 25.141.5, MM 25.141.13, MM 25.144.12, MM 25.181.

12.7 Dispatch costs and counters

Table 12.9 lists the costs (measured, MM 0.15).

Table 12.9. Measured dispatch, encoder and command-buffer costs.

Item	Cost	Conditions
dependent tiny dispatch	3.7 μ s each in a serial encoder; 4.2 μ s with a concurrent encoder plus scoped or resource barriers (MLX’s pattern)	chain of one-thread dispatches each reading the previous word (MM 25.141.10, 25.141.13)
extra compute encoder in one command buffer	about 6.5 μ s each	172 encoders per token: 9.275 against 8.163 ms as one encoder (MM 25.142.1)
concurrent encoder with scoped buffer barriers	0.03–0.07 ms per token slower than serial	pipelined q8 token, 196 dispatches (MM 25.142.1)
tracked buffers in one serial encoder against untracked buffers ordered by MTLSharedEvent	tracked faster by 0.2–0.3 ms per token	ABBA, both precisions (MM 25.142.1)
indirect command buffer	direct against indirect 4.82 against 5.83 μ s per dispatch (64 $\times$ 1 threadgroups), 4.66 against 5.81 (64 $\times$ 32); no gain for large dispatches (143.98 against 145.59 μ s; 131.80 against 128.41 at 32 MB)	MM 25.141.7
pipeline change	not measurable: one uber pipeline for FFN and w2 behind skip branches 101.9 μ s per dispatch shared, 101.8 separate, 99.9 same-pipeline floor	MM 25.141.2
grid barrier instead of dispatch boundaries	133.1 against 120.8 μ s per 32 MB phase (fenced, capped barrier among resident threadgroups; no barrier reached its cap in these runs, section 1.5)	MM 25.141.1

- Pipeline changes should not be counted as a cost (MM 17 rule 39). A constant-drain fusion model (6.54 μ s per pipeline change) was refuted: removing a change saved 0.6–2.5 μ s, not 6.5; an in-chain difference alone is not a per-switch cost, and a fusion saves about the work it removes (measured, MM 25.142.4).
- An in-chain layer A/B predicted the graph at 0.75–1.07 times for changes of 8–22 μ s per layer, and small savings reached the graph only on the path every token waits for, so changes are judged in the whole graph (MM 17 rule 41).
- A prefill GEMM grid with row groups in the high bits of the threadgroup id ranked differently alone and in the graph: 9 percent faster alone with weights resident and 15 percent slower in the graph,

because row groups reading one column block were `grid_n` threadgroups apart and each re-streamed it from DRAM (measured, MM 25.157).

Counters by instrument

What the sources say about counters differs by instrument, and each statement is correct for the instrument it describes (table 12.10).

Table 12.10. Performance counters available through each instrument.

Instrument	What it gives	Source
Metal’s public counter API	one counter set, timestamp (<code>GPUPerformanceTimestamp</code>), sampled at stage (encoder) boundaries only: supportsCounterSampling stage 1, dispatch 0; sampling at a dispatch boundary asserts	measured, MM 25.48, MM 25.141.6, MM 0.15
Metal System Trace	command buffers and encoders only; the only GPU counter is “RT Unit Active”; it records the GPU performance state (Minimum, Medium, Maximum) per interval	measured, MM 0.15, MM 25.142.1
Instruments <code>xcctrace</code> record with a GUI-saved “Performance Limiters” template (<code>counterscounterprofile = 13, g17-limiters.tracemplate</code>)	89 counters, including Kernel Occupancy, Compute SIMD Groups Inflight, Instruction Throughput Limiter and Utilization, ALU / F32 / F16 / Integer limiters, L1 Cache Limiter, Buffer L1 Read/Write Bandwidth, GPU Read/Write Bandwidth, Last Level Cache Bandwidth and Limiter, Memory Request Stalls Influence, Register Pressure Influence, Occupancy Manager Target, Neural Accelerator Limiter	measured, MM 25.144.4

The private `GPURawCounter.framework` route fails: the sampler-source query (selector 261) returns `kIOReturnNotPrivileged`, and root does not help; it needs the entitlement `com.apple.private.agx.performance-spi`, and Apple’s `MTLReplayer`, which carries it, is killed on launch by AMFI (measured, MM 25.144.4). *Correction:* the “M4 status” paragraph inside MM 25.144.4 predates the template recording later in that section (appendix A). “Timestamps only” (MM 0.15, MM 25.48, MM 25.141.6) remains true of Metal’s public API.

The Instruments counters are reduced as follows (MM 25.144.4). Per-core counters are medians over busy samples (Kernel Occupancy at least half its own 90th percentile); bandwidths are 90th percentiles from a separate sampler. Absolute “GPU read bandwidth” is not a DRAM figure when a looped weight stays LLC-resident (MLX’s about 500 GB/s exceeds the 298.5 GB/s DRAM peak). Register Pressure Influence read 0 for every kernel recorded (section 12.4).

No hardware clock is readable inside a kernel: none of the 67 special-register indices of special-register read (`op14059`) is a clock, `llvm.readcyclecounter` crashes the compiler service, and a GPU-side ticker slows five-fold under load. The working clock is the CPU’s `mach_absolute_time` (41.667 ns per tick) written into 512 lines of an untracked write-combined buffer, read by a one-thread stamp dispatch (about 1 µs each); the stamped span matches the command buffer to 0.14–0.5 percent (measured, MM 25.141.13). One encoder per dispatch is not a faithful timeline (a layer read 276 µs that way against 330 in one encoder) (measured, MM 0.15; MM 17 rule 42).

Below Metal, the submission interface is [chapter 13](#)’s: a command queue is created through selectors 7, 16 and 28, and each Submit enters the kernel through IOConnectTrap4 (one record) or IOConnectCallMethod selector 0x1d (several records) ([section 13.6](#)). The resident runtime’s warm Submit-to-callback split is in [section 13.7](#).

Evidence. MM 0.15, MM 17, MM 25.48, MM 25.141.1, MM 25.141.2, MM 25.141.6, MM 25.141.7, MM 25.141.13, MM 25.142.1, MM 25.142.4, MM 25.144.4, MM 25.157.

12.8 GPU clock and power

The top clock and the governor are described in [section 1.6](#). Clock effects seen in these measurements:

- Cold runs are slow (the rule, the 1.5 ms idle-clock threshold and the warm-up are [section 1.6](#)). For example, a cold one-threadgroup pointer chase was linear in chain length (206, 405, 802, 1,640 μ s at 4 MiB) and then fell for longer chains (measured, MM 25.121, MM 25.122).
- Short command buffers drop the performance state: mlx-lm’s seven per token spent 27–78 percent of decode in “Medium” ([section 1.6](#); MM 0.15).
- A clock reference does not detect contention for cores. Under another workload (Device Utilization 31 percent) the same FFN projection timed 185–194 μ s against 145–155 idle while the reference dispatch read 50.3–50.6 μ s in both (measured, MM 25.124.6).

[Table 12.11](#) gives measured power on the GPU rail. Its rows come from two instruments and two engines, so they do not form a like-for-like ratio.

Table 12.11. GPU-rail power, work rate and efficiency for each arm.

Arm	GPU rail	Work	Efficiency	Source
idle	32.3 mW	–	–	MM 25.56
fp32 fma	4,948 mW	12.4 GMAC/s	2.5 GMAC/J	MM 25.56
legacy engine, legacy 8×8 simdgroup MMA, f16 (op838), four independent accumulators	9,973 mW	184.4 GMAC/s	18.5 GMAC/J	MM 25.56
legacy engine, legacy 8×8 simdgroup MMA, f16 (op838), one dependent chain	19,001 mW	129.6 GMAC/s	6.8 GMAC/J	MM 25.56
idle (second instrument)	31.6 mW	–	–	MM 25.60
fp32 FMA, serially dependent chain	27,917.5 mW	0.071 TFLOP/s (latency-bound)	–	MM 25.60
TensorOps matmul2d 32×32×64 half to float	62,768.4 mW (reproduced at 62,518.0, 0.40 percent)	4.00 TFLOP/s, 12 percent of 33.5	2.06 uJ per 32×32×64 MMA	MM 25.60

- MM 25.56: 5 s windows, three trials, grid 2²⁰, GPU at 1,620 MHz and 100 percent active residency in every working arm. The dependent chain draws 1.9 times the power of the four-chain kernel for less work (trials within 40 mW). A kernel stalled on memory behaves differently: the repository README

gives 5,607 mW computing against 1,681 mW stalled on memory. Its “MMA” rows are the legacy engine, not the tensor unit.

- MM 25.60: filled 6 s windows; liveness ratios (runtime against an eight-fold loop increase) tensor 8.05, ALU 7.82. These are operating points, not peaks; “2.25 times” is not the accelerator’s power ratio over a saturated ALU.
- *A dead-code measurement.* The first tensor attempt stored its result under `if (lane == 1024)`, the compiler deleted the loop, and it read 9.5 W; dispatch time did not move with the loop count (256 and 32,768 iterations both 0.010 ms), which showed that the loop was gone. Every power figure here carries a liveness ratio for that reason (MM 25.60).
- Killing the host left the GPU at 1,620 MHz, 5,588 mW and 0 percent idle with every process reporting 0 GPU ms/s (MM 17 rule 35).

Correction: MM 20 (“Hardware identity”) says “Power was not measured”; MM 25.56 and MM 25.60 measured it.

Evidence. MM 0.15, MM 8, MM 17, MM 20, MM 25.56, MM 25.60, MM 25.121, MM 25.122, MM 25.124.6.

12.9 Cost models

The project has two cost models. `tensorsched` prices uniform work in measured kernel families and bounds non-uniform work from below; `g17perfmodel` prices any compiled program from its bytes, with a median error of 15 percent, good for attention and poor for GEMM tile shapes and DRAM-bound kernels.

Scheduler model (`agxforge/g17/tensorsched.py`)

The model (MM 16, MM 25.124, and the module docstring) is:

```
T_latency    = max over simdgroups of (steps of that simdgroup) x L_step
T_throughput = (total steps) x C_step / cores
T_memory     = unique bytes / BW(footprint)
time         = max(T_latency, T_throughput, T_memory)
```

This is the standard makespan lower bound, tight only when every simdgroup does the same number of steps.

- *Accuracy on uniform work* (measured, MM 16, MM 25.124): attention median 2.7 percent (20 of 23 rows within 10 percent; worst –16.5 percent at $128 \times 2,048$ keys fp8); weight-stationary stage 1, 5.1 percent stated, 3.3 percent on the retained rows (worst –6.0); token-parallel FFN, 3.3 percent stated over 16 rows, of which 7 come from an unretained log; the 9 retained rows give 4.7 percent (worst +11.9, bf16 at 10 threadgroups).
- *Domain limit from a non-uniform workload* (measured, MM 16). A causal attention gives query tiles triangular trip counts; the model over-predicts the diagonal skip’s benefit by 20.9 percent at 256 tasks and 37.1 percent at 1,024, and refitting both constants on the same kernels’ uniform runs leaves the error. For unequal work its predicted speedup is a ceiling and its time a lower bound. The uniform two-regime shape is also too sharp (time rises 1.46 times for 8 times the tasks, then 3.14 times for the next 4 times; the best fit leaves ± 12 percent).

- *The scheduling term* (model and measured, MM [25.124.1](#)). `T_schedule = list_makespan_ns`: tasks dealt in threadgroup index order round-robin to 20 cores, then one per freed slot; a core with n resident tasks advances each one step per $\max(L_step, n \times C_step)$. On the causal points (e4m3, head width 128, L_step 2,868 ns, C_step 327 ns) speedup optimism falls from 20.9 to 7.5 percent at 256 tasks and from 37.1 to 26.5 percent at 1,024; it stays a lower bound, and speedup refuses to form a ratio from it.
- *Constants and domain checks* (model, MM [25.124](#)): attention step pairs 5.35 μ s / 433 ns (fp8, int8) and 5.0 μ s / 381 ns (bf16); a bandwidth ladder of 3 TB/s to 5 MiB, 0.52 TB/s to 20 MiB and 0.28 TB/s from 40 MiB; the chain fit 62.6 / 5.8 ns (bf16) and 68.2 / 7.0 ns (fp8, int8) with `chain_ns` refusing any occupancy but 1 and 64 per core; residency floors 64 per core from 49 to 125 registers (this compiler's kernels) and 32.1 at 126 (Apple's). Every prediction carries checks (kernel family, dtype, head width or layer shape, registers 19–126 with the residency floor holding the needed simdgroups, footprint on a measured plateau, token range, uniform work); a failed check yields no number, and `choose` returns the known-safe schedule with `fallback=True`.
- *Calibration across kernel families* (measured, MM [25.124.2](#), MM [25.126](#)): the recon constants chose the wrong FFN schedule at fp8 48 and bf16 64 tokens on the retained harness; the layerbench constants pick the measured-fastest at all 11 points (blocked weight-stationary to token-parallel between 32 and 48 e4m3 tokens and between 48 and 64 bf16; one-pass weight-stationary wins at 48 e4m3, 1.119 against 1.254 ms). Head width scales by the MMA count (dh/128 for multiples of 32 from 96 to 256, 12 ratios at -6.4 to $+2.3$ percent; 144 costs 1.25, measured 1.26–1.32; width 64 takes 0.355); the affine per-MMA extrapolation under-predicted width 256 by 16–32 percent and is retired.
- Simdgroups per threadgroup 1, 2, 4 and 8 predict the same attention time; the model keeps the measured default 8 (1.455 ms, 4.2 percent behind the best, 2 at 1.396) and excludes 16 (16 threadgroups leave 4 cores idle) (MM [25.124](#)).
- It chooses schedules but does not lower them, and the production runtime cannot express most schedules it chooses between (module docstring).

Static model from compiled bytes (`tools/g17perfmodel.py`)

The figures in this subsection are model and measured values from MM [25.144.12](#).

- *Mechanics*. One simdgroup is simulated in order over decoded bytes: each instruction issues one 3.105 ns interval after the last and not before its operands; loads publish scoreboard slots and consumers stall until they land (tensor fragment loads, tensor load, half/bf16 (op12674) and 128-bit vector load (op12709), take the working-set latency; plain 32-bit load (op12682) the 16 KiB latency); a 16-bit MMA, tensor MMA, 16-bit $\times$ 16-bit, accumulate (op5106), result is ready after a dependent-MMA latency; loops are walked 4 trips and extrapolated (latches `cnt < imm` and `p > 0` only). Threadgroups are dealt to 20 cores in index order; residency comes from the program's highest register (the MM [25.121](#) table) and 3,072 threads per core; per-core rate is an exact mean-value analysis of three stations (issue slots, load path, tensor unit) fed into the MM [25.124.1](#) list schedule; a 280 GB/s DRAM floor; a fixed per-dispatch cost per timing protocol.
- *Nine fitted constants* (bounded least squares on log error; the tensor unit may not beat 4.95 ns per MMA per core): 12.5 issue slots per interval per core; 3.43 ns per fragment load; a scalar load at 0.43 fragment loads; 6.83 ns per MMA (72 percent of peak); load latency $\times 0.50$ of the pointer chase; dependent MMA latency 31.6 ns; MMA issue 1.0 interval; per-dispatch cost 1.4 / 1.0 / 1.0 μ s. The latency and issue constants sit at their lower bounds: they are not identifiable from these timings.

- *Accuracy* over 108 recorded timings (45 GEMM, 54 attention, 9 graph; 16 set aside as noisy): leave-one-table-out median 14.8 percent (47 of 92 within ± 15 percent); fixed split 15.5 percent; in-sample 9.6 percent; graph attention 8.3 percent; hot attention at $M \geq 512$ 9.9 percent; warm GEMM 19.6 percent; noisy rows 35 percent. The ± 15 percent target is met in the median and for attention, not for GEMMs.
- *Where it fails*. GEMM tile shapes do not extrapolate (a 2×2 tile up to 40 percent slow, tall tiles 16–37 percent fast; operand sharing among a threadgroup’s simdgroups is invisible to it); 1,024-token prefill projections predicted 144 ms against 113 measured (+27 percent); SwiGLU 9.0 against 7.2 ms (DRAM-bound, and the model has one 280 GB/s plateau); per-simdgroup latency is not identifiable. It refuses to run without the opcode class table, because an UNKNOWN fallback moved 81 of 108 predictions.
- *Fetch*. Two fetch terms built from the recorded cliff (1.91 ns per byte cold, 2.81 times past 16 KiB on one simdgroup, 1.15 times at 64) both worsened held-out error (23.2 and 17.5 percent against 14.8), so fetch is not in the model of record.
- *Predictions it made* (prediction, then built): SwiGLU fused into w3’s GEMM 7.6 ms (built); hardware exp2 at 512-row chunks 1.5 ms (built; 679 to 490 instructions a trip); K and V shared across 4 simdgroups through threadgroup memory –2.8 ms (slower).

Evidence. MM 16, MM 25.124, MM 25.124.1, MM 25.124.2, MM 25.126, MM 25.144.12; agxforge/g17/tensorsched.py.

12.10 Limiting resources by workload

The workload is InternLM2.5-1.8B at 4 bits unless stated. Throughput against mlx-lm for each workload is in the machine model (MM 25.190; the headline comparison is in the [showcase](#), section 2); the evidence for which resource bounds each workload is kept here.

Single-stream decode

Wide projections are memory-bound and narrow ones issue-bound.

- In the plan (model and measured, MM 25.124.5), one bf16 decode step (d 2048, 16 heads $\times$ 128, KV 256, one query row) has a memory floor of 0.487 ms (136.3 MB), 0.247 ms with fp8 weights.
- A one-token projection with too few threadgroups is bound by its K chain (measured, MM 25.124.6): M 16, N 128, K 2048 times the same with cold and cache-resident weights (within 1 percent), at 6.5–9.9 GB/s ours and 7.5–16.7 GB/s Apple’s, against 270. With enough threadgroups the column grid reaches the floor: N $8192 \times$ K 2048 (33.5 MB) on 64 threadgroups of 128 columns takes 143–145 μ s = 231–234 GB/s, 1.2 times its 0.120 ms floor, equal to Apple’s matmul2d (142.8–145.2 μ s). Split-K on N 2048, K 2048: G 1 84.2, 2 52.1, 4 41.1, 8 42.0, 16 43.3 μ s; the plateau (about 200 GB/s, cached equal to cold) is not DRAM.
- Table 12.12 shows where 4.73 ms per token goes at a 1K context (measured, MM 25.181).

Table 12.12. Single-stream decode time per token at a 1K context by part, with bytes streamed and effective bandwidth.

Part	ms	Bytes per token	Effective GB/s
FFN (w1, w3, SwiGLU)	1.76	453 MB	257
w2 + residual	0.90	227 MB	252
qkv	0.48	113 MB	236
wo + residual	0.30	57 MB	188
head	0.36	107 MB	296
attention	0.74	107 MB of KV	144
norms and the rest	0.18		

Only the head streams at the DRAM peak.

- Counters on the decode qmv kernels (measured, MM 25.144.4): the q4 w2 kernel is issue-bound (instruction throughput limiter 77.2 percent, zero memory-stall influence; pinned-weight floor 32.9 μ s above the 29.8 μ s DRAM floor); q8 w2 is memory-bound (instruction limiter 46.7 percent, memory-stall influence 26.6 percent, LLC limiter 100; 89 percent of the 298.5 GB/s peak); MLX’s qmv is latency- and memory-bound at 16.7 percent occupancy (64-thread threadgroups), 37–59 percent memory-stall influence.
- In q4 w2, compute and stream barely overlap (measured, MM 25.141.15, at the 4-rows-per-simdgroup shape): real 46.2 μ s; x index cache-resident 40.9; weight index resident 29.5; both 28.5 μ s (the instruction floor); streaming 8.9 MB at 298.5 GB/s needs about 30 μ s. The real time is close to the sum.
- The single-stream q4 MLP is load-bound (measured, bounded negative, MM 25.200): removing 26 percent of its instructions (FFN 393 to 291 instructions, 10 loads either way) moves nothing (186.4–192.1 against 177.9–193.4 tok/s at 1,792 tokens).
- Context costs about 20 ns per key-layer (measured, MM 25.199); long-context attention variants (keyblock, tgsplit) do not help in the graph at 1,792 keys.
- Apple-compiled twins of this project’s decode kernels, bit-identical in output, give 10–15 percent higher median throughput, so this project’s instruction-level code generation costs that much (measured, MM 25.211; showcase section 2).

Batched decode

The batched projections are occupancy-bound, then fp32-pipe-bound after split-K (all figures measured).

- The tensor-unit projection (g17qsm: $Y^T = W^T X^T$, W dequantized once into registers as A, 16 sequences as B) costs 446 μ s per layer for 16 tokens (27.9 μ s a token) against batched qmv’s 126 at B 8: 4.5 times per token; about 76 GB/s, latency-bound per simdgroup (about 700 dependent scalar instructions per trip before 8 MMAs) (MM 25.166).
- Split-K to about 512 threadgroups is the knee at every role (qkv split 4: 39.8 μ s at 512 threadgroups; wo split 8: 23.2; w1/w3 split 2: 76.4; w2 split 8: 75.8) (MM 25.185).
- At B 16 the projections are fp32-pipe-bound, not memory-bound: F32 Limiter 100, F32 Utilization 99.9, instruction throughput limiter 78 (p90 98), kernel occupancy 38 (p90 50), memory request stalls 0, GPU read bandwidth p90 122 GB/s, Neural Accelerator utilization p90 25, F16 utilization 0. Projections are

7.7 ms of a 10.98 ms step (70 percent) for 0.95 GB of q4, about 2.6 times the DRAM floor. Ablation on w1: base 76.3, no dequant 59.6, no weight load 60.9, neither 35.1 μ s (MM 25.194).

- Moving the dequant to the fp16 pipe with fp16 fma, register form (op798), two per weight, relieves the fp32 pipe: w1/w3 74.6 to 65.7 μ s (−12 percent), w2 73.9 to 66.6, qkv 39.6 to 35.4, wo 22.1 to 20.0 (MM 25.196).
- At B 8 the w2 projection hits the LLC limiter (100 percent) for this compiler and for MLX’s affine_qmv_wide alike, so both are held by the same memory limit. The B 8 qkv is register-capped at 25–27 percent occupancy (section 12.4) (MM 25.144.4).
- MLX’s own batched decode is compute-bound past B 2 (quantized) and B 8 (bf16); every precision converges on 19–21 ms per step at B 16 (q4 841 tok/s, bf16 750): dequantization costs more than the bytes saved (MM 25.142.5).
- Which mlx-lm batched path is the baseline depends on the model (MM 25.193).

Prefill

Every figure in this subsection is measured.

- *A corrected GEMM measurement.* An earlier comparison put this compiler’s prefill GEMMs at 60–69 percent of peak against MLX’s 82–88. The per-dispatch profile used there did not close (181 ms of GPU time in a 150 ms prefill), and the isolated GEMM bench runs about four times the in-graph per-call time. With a closure check (dispatch-alone sum 135.8 ms against chained step spans 137.1 and in-process medians 139.6, within 1 percent) the in-graph GEMMs are at 80–89 percent of 33.1 TFLOP/s: qkv $256 \times 4096 \times 2048$ 148 μ s (88 percent), wo 81 μ s (80), w1 290 μ s (89), w2 292 μ s (89). Pinning the weights cache-resident changed per-role time by 0 to ± 5 percent, so weight traffic is not the GEMM cost (MM 25.160).
- Prefill attention is latency-bound on the per-block scratch round trip: 32-key blocks cut instructions per 16 keys from 490 to about 390 (−20 percent) for −3 percent time (MM 25.162). Keeping S and P in registers made attention 14.7 to 12.1 ms (exact) or 9.6 ms (hardware exp2) (MM 25.163).
- The SwiGLU kernel was bandwidth-bound: 10 bytes an element over 201 M elements is 6.7 ms at 298.5 GB/s, exactly its 6.74 ms; an fp16 intermediate (6 bytes an element) made it 5.54 ms and arithmetic-bound (MM 25.183).

Table 12.13 summarizes what binds each workload.

Table 12.13. Binding resource for each studied workload.

Workload	Binding resource in the studied kernels	Evidence
fused attention, FFN layer kernels	instruction issue and ALU (about 46 instructions per MMA)	MM 8, MM 16
single-stream decode, q8 projections and the head	DRAM bandwidth	MM 25.144.4, MM 25.181
single-stream decode, q4 w2	instruction issue (counter) yet insensitive to removing dequant work (load-bound)	MM 25.144.4, MM 25.200
one-token projection with few threadgroups	one K chain’s latency	MM 25.124.6
decode attention past 4.2 MB of KV	LLC capacity, then about 140 GB/s	MM 25.181
batched decode B 16 projections	occupancy first (about 256 one-simdgroup threadgroups a projection); after split-K to about 512, the fp32 pipe, relieved by moving the dequant to the fp16 pipe	MM 25.185, MM 25.194, MM 25.196
batched decode B 8 w2	LLC bandwidth (both stacks)	MM 25.144.4
prefill GEMMs	tensor and issue, at 80–89 percent of peak	MM 25.160
prefill attention	latency of a scratch round trip, until S and P stay in registers	MM 25.162, MM 25.163

Not known. The cause of the per-core staircase (MM 25.122); why this compiler’s residency counter reads about twice Apple’s kernel’s (MM 25.121); fetch cost at intermediate occupancy (MM 25.144.12); physical register capacity per core (MM 20); why Apple-compiled twins are 10–15 percent faster (MM 25.211). All figures are from one machine, and the Metal-path evidence files behind them do not record the OS build (section 1.1).

Evidence. MM 8, MM 16, MM 25.124.5, MM 25.124.6, MM 25.141.15, MM 25.142.5, MM 25.144.4, MM 25.160, MM 25.162, MM 25.163, MM 25.166, MM 25.181, MM 25.183, MM 25.185, MM 25.190, MM 25.193, MM 25.194, MM 25.196, MM 25.199, MM 25.200, MM 25.211.

13 Submission below Metal

Without Metal, a process reaches the GPU through GPU and shared-memory allocations, a set of launch words, a submission record, a call into the kernel and a completion notification. The resident runtime that assembles these into a model execution, and validates its results, is in [section 17.2](#).

A Metal application asks Metal to build a pipeline and encode a command buffer; Metal’s driver library (AGXMetal) then writes command pages and calls Apple’s private IOGPU framework, which enters the kernel driver. A below-Metal process does the same work itself: it allocates GPU memory and shared memory through the AGX driver’s user client, writes the code, the launch state and the command pages, builds the submission record, and calls the same IOGPU submission function. The kernel driver, the GPU firmware and the hardware are unchanged.

The project’s process owns the program bytes, the launch words, the command pages and the record. IOGPU took ownership of the submitted callback blocks on every observed successful Submit; ownership on failure, timeout or teardown is not established ([section 13.7](#)). The kernel driver and firmware own everything after the trap.

Launches are known to work in the configurations whose layout was measured. Within the measured graphs the words a launch needs are known by role, except for some known only by value, such as the shader-state bits and the kernel-page word at offset 0x3c0; those are copied as captured. The runtime refuses every configuration outside the envelope. Some of those might work, but none has been measured.

Interface terms:

- *User client*: the IOKit connection to the AGX kernel driver. Its numbered entry points are *selectors*.
- *Allocation n*: the n-th GPU memory allocation in a measured resource graph, created through selector 9. Each has a GPU virtual address (VA) and a CPU mapping at the same address.
- *Command pages*: two 16 KiB shared-memory regions per submission, created through selector 14: the *kernel page* (kernel-command shared memory) and the *segment page* (the segment list). Each has a shared-memory ID.
- *Launch state*: the per-launch words in measured allocations that select a program, its grid and its buffers.
- *Submit record*: the record handed to `IOGPUCommandQueueSubmitCommandBuffers`.
- *Measured graph*: a complete set of allocations, pages and launch words whose layout was captured from Metal once, then rebuilt from fields. Four sizes are in use: 3 MiB, 4 MiB, 128 MiB and 1 GiB.

Standing in this chapter: *measured* means executed below Metal with a checked output and a control; *observed* means traced in a live process (Frida) or read from Apple’s binary without executing a GPU command; *static* means resolved by reading the installed IOGPU code (`tools/g17submitinspect.c`).

13.1 Apple-owned layers

Table 13.1. What the below-Metal process writes and what remains Apple’s, by layer.

Layer	Below Metal	Still Apple’s
program	every instruction byte, from the project’s compiler	encoding rules recovered with Apple’s decoder
resources and launch	allocations, command pages, launch words, bindings, the Submit record, written by the project’s host code	the formats themselves, recovered by measurement
submission	the call into <code>IOGPUCommandQueueSubmit</code> <code>CommandBuffers</code>	<code>IOGPU.framework</code> , <code>IOKit</code> , the AGX kernel driver
execution		the GPU firmware and the hardware

The process never creates an `MTLDevice` or a Metal command buffer, and it checks that `AGXMetal` is not loaded (MM 25.210.9).

13.2 Device, context and queue setup

- The AGX user client opens with no entitlement. A process with no Metal driver loaded creates the device and a context (measured; `tools/g17dispatch.c`, README rows 1–2).
- Creating a command queue calls selectors 7, 16 and 28, in `IOGPUCommandQueueCreate` (observed in a Metal process that created only a device and a queue). Selector 28 registers the queue’s completion notification queue (static). Calling the group a second time by hand ends in a selector-28 error (`docs/g17-first-dispatch-trials.md`, “Submission transport and record, resolved statically” and the preflight that follows it).
- The sampler-source query (selector 261) returns a privilege error; nothing below depends on it (MM 25.143).

Correction: an earlier reading named selector 7 the submit and fence, and selector 14 a telemetry ring (MM 25.143). The static resolution places 7, 16 and 28 in queue creation and identifies selector 14 as the shared-memory allocator (section 13.3), superseding the earlier attribution.

13.3 Allocations and address windows

- GPU allocations come from selector 9. Each request names an allocation class (a request *type*), a size, and an address; the driver returns a buffer mapped at the same CPU and GPU virtual address (measured; MM 0.15, README row 2).
- The address window depends on the request type. The exact type value `0x58000000` places an allocation in the window at `0x6f_00000000`; other tested values do not reach the `0x1ea_` window that Metal’s command buffer names. Whether that window is the shader window is not shown (measured; MM 25.143.O).
- Shared memory comes from selector 14 (`IOGPUDeviceCreateDeviceShmem`). Its 16-byte output is `{cpu_address: u64, size: u32, shmem_id: u32}`. A submission uses two 16 KiB regions, the kernel page and the segment page, with IDs such as 2 and 1 (static and observed; trials, “Submission transport and record”).

- Allocations of 32 KiB, 8 MiB, 96 MiB and 1 GiB were created and CPU-mapped with sentinels and no Submit. This shows allocation and mapping only; GPU visibility was not established, so the runtime reuses measured graphs instead of a size formula (MM 25.210.3).

13.4 Code placement and the program counter

- Every program is uploaded into allocation 0, at GPU address 0x10000000000, and the allocation is treated as immutable after upload. Programs start at offset 0x6c0 and are 64-byte aligned (measured; MM 25.210.6).
- A launch's program counter is allocation 0's base plus the program's offset. It is written in pieces into allocation 23: the low 16 bits OR 7 at offset 0x40, bits 16–31 at offset 0x46, bits 32–47 at offset 0x48 (MM 25.210.6).
- The shader-state word at offset 0x42 of allocation 23 is written separately. A single 32-bit program-counter write once corrupted it (measured; MM 25.210.6).
- *Not known*: the low program-counter tag bits outside the measured entry class.

13.5 Launch state and command pages

Table 13.2 lists the words a launch needs, all written by the project's runner (MM 25.210.6, unless noted).

Table 13.2. Launch words by location, with width, meaning and standing.

Where	Width	Meaning	Standing
allocation 23, offsets 0x40, 0x46, 0x48	16 bits each	program counter: low half OR 7, bits 16–31, bits 32–47	measured
allocation 23, offset 0x42	16 bits	shader state	measured value; bits unnamed
allocation 23, offset 0x38	two 32-bit words	scratch/config word, then zero	measured value; bits unnamed
allocation 22 offset 0xa8; allocation 25 offset 0x10	three 32-bit words each	grid x, y, z, written twice	measured
allocation 25, offset 0x1c	32 bits	threads per threadgroup (32 in the measured classes)	measured
allocation 28, offset 0x1ba0	four 64-bit pointers	program buffer pointers; at most three are admitted	measured
allocation 23, offset 0x06	16 bits	scaled metadata address coordinate: (VA28 >> 13) &0xffff	measured (see below)
allocation 25, offset 0x09	16 bits	scaled address coordinate: (VA23 >> 14) &0xffff	measured equality
kernel page, offset 0x3c0	word	needed for the tensor contribution: clearing it removes the MMA result while Submit and callbacks succeed	measured; meaning unknown
kernel page, offsets 0x3ec and 0x3f8	words	kept as measured	unknown

Launch state comes in two classes. Scalar and packing programs use the state word 0x0e40 with the scratch word 0x0c000007. Tensor projections and the cooperative LayerNorm use 0x0e58 with 0x0c00100f. Their individual bits are not named (MM 25.210.6).

MM 25.210.6 lists the halfwords at allocation 23 offset 0x06 and allocation 25 offset 0x09 as “graph-identity correlates; no formula”. The trial log fits both exactly at six graph sizes from 17 to 34 MiB, and in the 4 MiB graph, as the shifted addresses above (docs/g17-first-dispatch-trials.md, the address-coordinate table). For allocation 23’s halfword the evidence is causal in one graph: changing only that halfword between the high and low coordinate decided pass or fail. Treat the formula as established for the measured graphs; do not assume which firmware step reads it, or that every allocation with that encoding has the same role.

In the 1 GiB class the complete 32 KiB shader record is copied into allocation 17 at offset 0x10000 and selected by allocation 25’s halfword (0x001a) at offset 0x09; the originally wrapped selector value (0x0048) page-faulted and must not be reused. The 48 KiB binding metadata is copied from allocation 28 into allocation 17 and selected by allocation 23’s halfword (0x002c) at offset 0x06. The executor rewrites the selected low copy of the shader record for each stage; writing the original high record after the copy does not change the program that runs (measured; MM 25.210.3).

13.6 Submit record and IOGPU call

The Submit record’s base layout is 48 bytes; Metal’s observed records are 64 bytes, one per command buffer (table 13.3; static and observed; trials, “Submission transport and record”).

Table 13.3. Fields of the Submit record by byte offset.

Offset	Field
0x00, u32	shared-memory ID of the kernel page
0x04, u32	shared-memory ID of the segment page
0x08, u32	optional sideband shared-memory ID
0x10, u64	scheduled callback block
0x18, u64	completed callback block
0x20, u32	optional debug shared-memory ID
0x28, u64	declared by the framework’s metadata; meaning not established
0x30 on	possible driver extension; kept uninterpreted

The runner must also meet these conditions (measured; MM 25.210.6):

- IOGPUDeviceGetNextGlobalTraceID must return consecutive IDs for the trace-ID words in the command pages.
- The coordination word at offset 0x24 of shared region 0 must hold the fire value 0x800000f0. The staged value 0xf0 was rejected.
- Exactly one scheduled and one completed callback are required, each a fresh Block_copy. Reusing blocks crashed the CPU notification dispatcher; the crash is in host code and says nothing about the GPU.

The call reaches the kernel in one of two ways (static; trials, “Submission transport and record”):

- A one-record Submit calls `IOConnectTrap4`: a kernel trap with trap index 0, the queue ID, the record size, the record pointer and an output pointer. The Metal reference runs observed exactly this, with count 1 and stride 64.
- A multi-record Submit calls `IOConnectCallMethod` with selector 0x1d, passing the contiguous record array.

Correction: MM 25.143 and README row 6a describe a Submit with no per-dispatch kernel call (shared-memory writes only, a candidate store at `Submit+0x11c`). The static resolution shows each Submit enters the kernel through the trap or the method call above; the store at `Submit+0x11c` is an output-argument write (appendix A).

Open: what consumes the fire value. In a Metal process, a control that never set the ready bit still executed (MM 25.143). In the pure process, the staged value 0xf0 was rejected and only 0x800000f0 works (MM 25.210.6). The two observations come from different paths and do not settle what consumes the word. *Safe assumption:* a pure process writes 0x800000f0 before Submit. *Not safe:* treating that write as the mechanism that starts the GPU.

13.7 Completion and callback ownership

- Completion arrives as a notification. `IOGPUNotificationQueueDispatchAvailableCompletion` Notifications dequeues 40-byte notifications (`IODataQueueDequeue`) and invokes the referenced callback block. In the Metal reference runs, two notifications per submission carried, as their first 64-bit values, exactly the record’s scheduled and completed callback pointers, in that order (observed; trials, “Submission transport and record”).
- In the MiniLM FFN block, a warm request spends 2.385 ms from Submit to the completed-callback entry and 0.0495 ms from callback entry to the waiter’s return (measured; MM 25.210.5). The first interval mixes GPU execution, driver work and completion delivery and is not GPU time.
- Callback ownership was observed on macOS 26.6.2 (MM 25.210.9a). On the successful count-one path, each of six observed callback blocks was invoked, returned, released by the notification dispatcher, and freed through `_Block_release`. No caller release occurred, and no block accumulated. *Policy this supports:* allocate fresh copied blocks per Submit, hand them to IOGPU, and neither release nor reuse them. *Not known:* ownership on failure, timeout, or queue teardown with pending callbacks, and behaviour on other OS versions.

13.8 Multi-record Submits and dependent dispatches

- Several records in one Submit execute. Count-two and count-three Submits ran distinct programs and bindings into separate outputs, and later trials reached seven separately bound records, the seventh predicted without a Metal capture (measured; trials, “Two command records in one pure Submit” through “Seventh record predicted”).

- Dependent stages, as the resident runtime runs them, are consecutive count-one Submits: 7 for the MiniLM FFN, 916 for one Qwen decode token (measured; MM 25.210.3, MM 25.210.5). A two-stage dependent pipeline, with and without a program switch, passed in the trials (measured; trials, “Dependent two-stage pure GPU pipeline”).
- *Not built*: many dependent dispatches in one Submit with explicit barriers between them. Metal’s serial-barrier word and fence semantics are decoded (README row 10), but the runtime does not use them.

13.9 Faults, hangs and recovery

- Malformed kernels have hung the GPU on this machine. Killing the host process does not cancel a submitted command buffer; recovery from that hang was a reboot (rule 35 of MM 17).
- The GPU’s recovery counter and event reports are observable before and after a run. The resident runtime refuses a run if either changes (MM 25.210.9).
- The runtime bounds the wait for each Submit’s callbacks to five seconds (MM 25.210.6).
- *Not available*: detecting or recovering from a hang beyond that bound, and resetting the GPU (README row 11).

13.10 Scope of the measured graphs

Table 13.4. Measured resource graphs by size, with the work each carries and its source.

Graph	Used for	Source
3 MiB	the MiniLM FFN and attention blocks	MM 25.210.4, MM 25.210.6
4 MiB	the connected layer (allocation 20 enlarged to 4 MiB, allocation 21 moved)	MM 25.210.6
128 MiB	the MiniLM encoder	MM 25.210.3
1 GiB	the Qwen2.5-0.5B decoder	MM 25.210.3

Outside these graphs the program-counter placement, the grid and threadgroup words, up to three buffer pointers, the two state classes, the record layout, and the address-coordinate halfwords within the tested range still apply. Not yet carried over:

- arbitrary graph allocation and publication (the runtime recovers only the request and page sets of the measured graphs, and refuses every other size);
- a fourth buffer pointer (earlier controls wrote 0 of 32 outputs);
- new shapes, data types, simdgroup counts, OS versions or GPU generations;
- GPU-only timestamps.

Evidence. MM 0.15, MM 25.143 (with 25.143.N and 25.143.O), MM 25.147, MM 25.150, MM 25.210 (25.210.3, 25.210.5, 25.210.6, 25.210.9, 25.210.9a); docs/g17-first-dispatch-trials.md; README rows 1–16.

14 Open questions

Each item is stated in full in its chapter, beside the rule it qualifies, under “Not known” and with the safe assumptions that apply; the lists below only index them. Items about submission below Metal are in [section 13.10](#) and the “Not known” notes of [chapter 13](#).

14.1 The device, registers, encoding and metadata (chapters 1, 2, 3, 11)

- Whether a released register can read its old value: a sentinel-checked probe of the old-value branch (MM 25.145).
- What IR0-IR15 hold and whether any executed program names one (Apple’s class table; MM 25.59).
- Whether anything reserves R126/R127 versus the part populating 126 of 128 (MM 24.2 row H1).
- Physical register capacity and allocation granularity per core (MM 24.2 row H4, MM 25.20, MM 25.49).
- Why this compiler’s residency counter reads about twice Apple’s at high register counts (MM 25.121).
- Simultaneous residency count for 1,024-thread threadgroups: fp.m’s implied ≥ 10 per core vs the driver’s 3 (MM 25.141.1, MM 25.141.20).
- Whether the encoding holes of half to fp32 widening (op1004) and fp16 fma, register form (op798) are unreachable in the encoding or only in the field maps (MM 25.133, MM 25.196).
- Release order inside instructions other than the measured ALU forms and the word store (MM 25.117, MM 25.145).
- Which narrow-pool policy gives better occupancy (MM 25.146).
- Class-7 byte6 = 0xa1 lengths (MM 25); move immediate, three-operand form (op11842) width-10 driver; store, sub-form 01, texel/word-slot (op17235)/two-component store (op17244) width-14 sufficient condition (MM 25.62).
- Meaning of int8-MMA structural bit 71 (MM 25.71, row T11).
- Why the 36 narrow-accumulator table rows are refused (MM 25.6).
- tensor load, half/bf16 (op12674) residue bit byte1[6] beyond the rendering split (MM 25.32).
- Bits 33 and 37 of operand 1 in ALU and load forms (seen in the running example; not explained in the sources read).
- An independent decoder derived from per-length field maps with encode-then-compare refusal (MM 25.209).
- Whether a wrong slot-32 value with correct presence affects any kernel shape beyond those tested (MM 10).
- The 12 open class-constant bytes (0xC0000000 at 1304, word 1 at 1324, 0x40000004 at 1440) (MM 25.148).
- Q in the scalar metadata class (agxforge/g17/mdgen.py).
- The main-program rule behind the 70 L3 misses; whether a non-32-bit uniform write occupies one uniform; 442 unread constant-program uniform writes (MM 25.120, MM 25.120.1).
- Whether the register count in the below-Metal USC Registers packet affects residency (MM 25.143; [chapter 13](#)).
- Slot 44’s byte position rule across classes (MM 25.89, MM 25.120, agxforge/g17/mdgen.py).

14.2 Scalar arithmetic, control flow, special registers and cross-lane operations (chapters 4, 5, 6, 8)

- Released-register read: zero or old value; a sentinel-checked probe of the old-value branch is owed (MM 25.145).
- 32-bit compare-select, four registers (op11375) bit test on operands with high bits set, and field code 0's signedness above 2^{31} (ledger/claims.toml).
- 32-bit compare with immediate into a flag (op10369)/10 mode-SET layout (operand 1 bit 2^{31}): meaning unknown, unreached from source (MM 25.73, MM 25.76).
- Integer source-modifier value 8 as a signedness bit beyond 32-bit saturating add (op10239) and 32-bit arithmetic shift right immediate (op16805) (MM 25.59, MM 25.145.1).
- NaN into fp32 saturate (op904) and fp32 truncate toward zero (op3818) unmeasured (MM 25.145.1).
- Negate/abs modifiers on fp32 compare-select, fmax/fmin (op9700) unmeasured (MM 25.145).
- hardware log2 (op2570) accuracy not measured; trig-family op (op3946) function unknown (glossary).
- fp32 to bfloat16 narrowing, unconfirmed (op1048): whether it is the fp32-to-bfloat narrowing is unresolved (glossary).
- Load-wait field of fp16 fma, register form (op798) not located (MM 25.196).
- Wait and source lifetime of fp32 to 32-bit integer (op9320) and 32-bit integer to fp32 (op11179) unmeasured (MM 25.196).
- MMA-to-ALU interlock (MM 6, MM 25.90).
- Special-register read publishing on slot 7 stores nothing: unexplained (MM 25.121).
- SR_PVSIMD / SR_TVSIMD under partial activity; roles of Apple's four-instruction divergent broadcast (MM 25.144.6, MM 25.145.1).
- y and z builtins on multi-axis dispatches never dispatched (MM 25.145.1).
- Exec auxiliary operand 0 meaning; count 3 and novel exec encodings never executed (isa/g17-exec-mask.txt).
- while-exec, set mask in place (op579) and exec pop, restore mask (op577) n = 2 admitted on whole-program evidence only (MM 25.145.2).
- Indirect calls (MM 25.90).
- Shuffles, broadcasts and votes in a partially active simdgroup (MM 25.145.1); SIMD shuffle-xor, runtime mask (op14170) with a non-uniform mask (MM 12).
- Flag numeric value per producing form (op10379, which has no glossary entry, against 32-bit compare, OR-chained into a flag (op11313)).
- Same-address store order across simdgroups (MM 25.145.4; chapter 9).

14.3 Memory and synchronization (chapters 7, 9)

- Out-of-range store behaviour on hardware; other load forms and resource kinds out of range (MM 17 rule 34, section 7.5).
- Negative displacement; misaligned scalar access legality (isa/g17-addressing-rules.json).
- Why Apple's compiler uses the index add, not the displacement, for unsigned tid + k (isa/g17-addressing-rules.json).

- Displacement units of the 128-bit vector store (op17256) (MM 25.141.18).
- Whether a masked-off load issues its access (no observable; isa/g17-predication.json).
- Meaning of the threadgroup load's sensitive bits byte4[3], byte4[4], byte6[4] (isa/g17-execution-tgbits-results.json).
- Hardware behaviour for a threadgroup-memory declaration over 32 KiB, which Metal accepts (MM 25.147).
- Imageblock with more than one simdgroup or threadgroup; the store's operand-1 bit-4 flag; the read's 0x2000800000 word (MM 25.145.2, MM 25.96, MM 25.104.2).
- MMA operand load, space unconfirmed (op12352) and dequantized-block store, likely threadgroup (op13276) address space (MM 25.144.1).
- Settling dispatch for plain 32-bit store (op17229) reading a texture fetch (Apple's tex2d-read) (MM 25.146).
- Sampled and write texture classes executed (agxforge/g17/texnarrow.py).
- Origin of range mask, 16-bit source (op621)'s cap of four (isa/g17-address-formula.txt).
- MMA-to-ALU result interlock (MM 6, MM 25.90).
- Why a special-register read publishing on slot 7 stores nothing (MM 25.121).
- Cross-simdgroup winner of different-value same-address stores (MM 25.145.4).
- Whether an execution-only barrier orders device memory by design (MM 14).
- Per-lane atomic min/max returns and unsigned long-form returns; atomics on non-rank-0 buffers; 64-bit atomics on hardware (docs/g17-first-dispatch-trials.md, agxforge/g17/cc.py).
- Acquire/release semantics beyond the fence-count-fence pattern; emulator race check for atomics and device fence (op14156) (MM 25.197).
- Function of barrier-token variants 0x400 and 0x60000502 (MM 25.141.20).
- Residency count for 1,024-thread threadgroups: fp.m's 200 vs the driver's 60 (MM 25.141.1, MM 25.141.20) - chapter 1.
- Overlap and ordering of multiple command records in one below-Metal Submit beyond the self-accumulation shape (docs/g17-first-dispatch-trials.md).
- Barrier behaviour in other partial threadgroup sizes and with inactive lanes (docs/g17-first-dispatch-trials.md).

14.4 The tensor unit and performance (chapters 10, 12)

- MMA-to-ALU result ordering: interlock or implicit wait not established (MM 6, MM 25.90).
- Destination keep flag: no hardware effect found; what the 758,739-instance population counts is unstated (MM 0.4, MM 25.22).
- Accumulator modifier (operand 10): a multi-bit encoding not excluded (MM 6).
- 36 narrow-accumulator declarations, table-only narrow-accumulator MMA, first (op5108) through table-only narrow-accumulator MMA, last, no accumulator (op5143): refused although structurally like admitted forms; reason unexplained (MM 25.4 "F4, narrowed", MM 25.13).
- MM 11's "56 MMA forms" not reconciled with the 60 non-_shifted declarations (MM 11, MM 25.6).
- Bit-71 int8 MMA variant: executes bit-exact; field meaning and decoder identity unknown (MM 20, T11).

- Eight decoder-refused bits of tensor MMA, 16-bit × 16-bit, accumulate (op5106) that change the result vs six that do not: no encoding property separates them (MM 25.36).
- Out-of-place accumulation (D a different tuple from C): no execution reported in the sections read (MM 2).
- int8 fragment byte order within the register pair: not stated (MM 0.7).
- Fragment shapes other than 16×16×16: not measured (MM 0.14).
- Whether a release clears or frees per-lane storage (MM 25.89, MM 25.95).
- What the legacy engine shares with fp32 fma (“8 fma per legacy issue” is an exchange rate) (MM 9).
- Software fp4/fp6 operand unpack not built; fp16-subnormal behaviour of the decode unmeasured (MM 0.14).
- Cause of the per-core simdgroup staircase inside the core (MM 25.122).
- Why this compiler’s residency counter reads about twice Apple’s kernel’s (64–69 vs 32–45 per core) (MM 25.121).
- Physical register capacity per core and allocation granularity (MM 20, MM 25.49).
- Instruction-fetch cost at intermediate occupancy (between 1 and 26 simdgroups per core) (MM 25.144.12).
- Why Apple-compiled twins of this project’s decode kernels are 10–15 percent faster (MM 25.211).
- Static model: per-simdgroup latency not identifiable; GEMM tile shapes do not extrapolate (MM 25.144.12).
- The read_sr-on-slot-7 observation (stored nothing, waited or not) is recorded, not explained (MM 25.121).
- Another G17 product or OS version: nothing measured (MM 0.1, MM 20).

Part II

Compilation and delivery

15 The compiler

A program reaches native G17 bytes through one of two input routes and a shared backend (figure 15.1). Section 15.2 follows the running example through each stage; the general rules for selection, allocation and tensor lowering come after it.

15.1 Input routes

Metal source goes first through Apple’s front end, which produces AIR, and the project translates AIR into its own intermediate representation (IR). Python builders construct that IR directly. Both routes then use the same native backend.

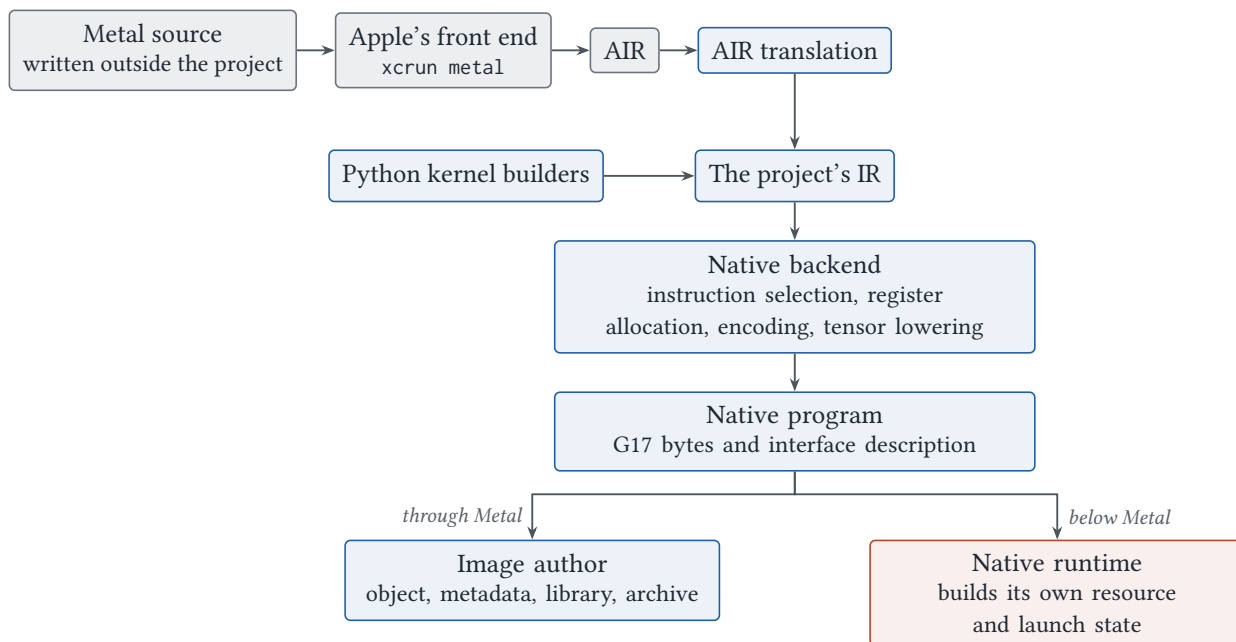


Figure 15.1. Input routes to a native program. Metal source passes through Apple’s front end to AIR, which the project translates into its own IR; the model kernels are built in that IR directly. Both routes share the native backend, whose output feeds both execution paths. Grey: Apple’s components; blue and red: the project’s.

The Metal-source route tests the compiler on programs written outside the project’s model builders. In the reconciled snapshot, 128 sources of the frozen 197-source census compile (MM 25.196). On this route only the translation from source to AIR is Apple’s.

The direct-IR route builds the model kernels: the projections, attention, normalization and generation steps. It does not invoke Apple’s Metal front end.

The backend outputs native instruction bytes together with a description of the program’s buffers, bindings and execution requirements. The two execution paths then use it differently:

- For the Metal path, the image author packages that output into the object, metadata, library and archive that Metal loads.

- The below-Metal runtime takes the native bytes and builds its own measured resource and launch state ([section 17.2](#)).

Both routes currently use Apple’s decoder to check the emitted instructions. That dependency is part of building and validating a program; the decoder is not involved in delivering the finished program to the GPU.

15.2 The running example in the compiler

The program is the one of [section 0.4](#), compiled and decoded on the CPU. Its bytes are the ones that ran below Metal, but nothing in this section executes them.

IR

The IR is SSA over typed values. A function declares its buffers, each with a binding index and an element type, and tensor operations are first-class.

```
import struct
from agxforge.g17 import cc, ir, scanlink

a = ir.Buffer("A", 1, elem=ir.F16)
b = ir.Buffer("B", 2, elem=ir.F16)
c = ir.Buffer("C", 3, elem=ir.F32)
fn = ir.Function("tensor_example", [a, b, c])
bd = ir.Builder(fn, fn.block("entry"))
bd.tensor_matmul(a, b, c, M=17, N=19, K=16)
i = bd.builtin("threadgroup_position_in_grid")
one = struct.unpack("<I", struct.pack("<f", 1.0))[0]
bd.store_at(c, i, bd.fadd(bd.load(c, i, type=ir.F32), bd.const(one)))
bd.ret()
ir.verify(fn)

program = cc.compile_function(fn)      # 1,232 bytes, 101 instructions; SHA-256 8bb67a3c...c667
image = scanlink.author(program)      # object, metadata, library and archive for the Metal path
```

Selection

`cc.select` maps each IR operation to a recovered G17 form ([table 15.1](#)).

Table 15.1. Selection in the running example: each IR operation and the G17 form it maps to.

IR operation	G17 form
tensor_matmul	a tensor body generated by <code>agxforge/g17/tlower.py</code> (section 15.4)
<code>builtin("threadgroup_position_in_grid")</code>	special-register read (op14059) of the threadgroup position in x
load	plain 32-bit load (op12682)
const	move immediate, three-operand form (op11842)
fadd	fp32 add (op998)
store_at	plain 32-bit store (op17229)
ret	end of program (op684)

Each of these forms has mapped fields and measured behaviour; selection refuses any other form by name ([section 15.3](#)).

Tensor body

`tlower` turns the $17 \times 19 \times 16$ shape into two tile rows and two tile columns, with the second of each partial:

- It computes each lane's addresses from the lane index, read with special-register read, 16-bit (op14060) into R0L, as row $m = 4 * (l \gg 4) + ((l \gg 1) \& 3)$ and column $col = (l \& 8) + 4 * (l \& 1)$, then $m * ld + col$ for each leading dimension: the B-row packing that [section 10.5](#) explains. The results sit in R1–R8.
- It builds the partial-tile masks in 16-bit halves (R11H, R12H, R13H), with range mask, immediate width (op612) and 16-bit AND (op437), because the masked forms take a 16-bit mask operand.
- It loads the A fragments into R20–R23 (tile row 0) and R56–R59 (tile row 1, masked), and the B fragments into R68–R71 and R72–R75 (the masked second column). Every load publishes scoreboard slot 0.
- It issues one tensor MMA, 16-bit $\times$ 16-bit, no accumulator (op5107) per output tile, since $K = 16$ is a single slice. Each MMA waits on slot 0.
- It stores each tile from its accumulator group (R24, R32, R40 and R48, each eight-aligned) with tensor store (op17257), or masked tensor store (op17258) where the tile is partial.

Allocation

The allocation follows the constraints of [chapter 2](#):

- Each accumulator group starts on a multiple of eight, and each 16-bit fragment on a multiple of four ([section 2.2](#)).
- Each fragment is read by two MMAs, kept (0) by the first and released (16) by the second ([section 2.5](#)). The epilogue's fp32 add releases both its sources, and the store releases its value and index: all are last uses.
- The tensor body names R0–R8, R11H–R13H and R20–R75. The scalar allocator must keep the union of every register a tensor body names out of its own choices. The epilogue's registers, R9 and R16–R18, all fall outside that set (`_prepare_tensor_composition` in `agxforge/g17/cc.py`).

- The register count, 76, is the highest register used plus one. It is reported, not chosen ([section 2.8](#)).

Waits

Each late value is tracked through a scoreboard slot:

- The threadgroup-position read publishes slot 0.
- The plain load waits on slot 0 for its index and publishes slot 7.
- The fp32 add waits on slot 7.
- No instruction waits on a slot nothing fills, and nothing reads a late value without its wait ([chapter 9](#)).

The compiler’s checks verify both rules on the emitted bytes, together with the no-read-after-release rule ([section 16.1](#)).

Executor interface

`program.abi()` records what an executor needs:

- ABI version 5;
- three bindings, A and B half and read-only, C float and written;
- an empty constant pool;
- 101 main instructions;
- the 24 distinct (opcode, length) forms used;
- the system registers read;
- that exact launch geometry is required.

The Metal path turns this into an image ([section 16.1](#)). The native path places the same bytes itself and writes the launch words that correspond to these facts ([chapter 13](#)).

15.3 General selection and allocation rules

As in the running example ([section 15.2](#)), `cc.select` uses a form only if its fields are mapped and its behaviour was measured (the form registry, `isa/g17-forms.toml` and `isa/g17-scalar-isa.toml`). Anything else it refuses by name, for example “no measured metadata layout for 5 bindings”, or an unmeasured cooperative binding class (`isa/g17-source-execution.json`). A refusal names the measurement that is missing.

Where the language leaves behaviour undefined, the compiler follows Apple’s hardware. Its first lowering of `(uint)x` for a negative float returned the magnitude, where Apple’s code returns 0. It now lowers the cast to fp32 to 32-bit integer (`op9320`), reproducing Apple’s bytes and output ([MM 25.186](#), [MM 25.196](#); [section 4.5](#)).

Allocation obeys [chapter 2](#). `cc.allocate` assigns physical registers from R0 to R125 with tuple alignment, and sets every lifetime modifier from liveness ([sections 2.2](#) and [2.5](#)). It never releases a register that is live around a back edge ([section 2.6](#)), it avoids encoding holes through the explicit tables `_encodable_dests` and `_encodable_fma16_sources` ([section 2.7](#)), and it reserves every register a tensor body names, through

`_prepare_tensor_composition` in `agxforge/g17/cc.py`. `agxforge/g17/releasecheck.py` re-checks the emitted bytes for any read after a release.

15.4 Tensor lowering

`agxforge/g17/tlower.py` generates a matrix-product body from its shape and types (MM 23.4, MM 25.165):

1. Read the lane index, and the simdgroup index when there are several. Compute row, column and element addresses in scalar code.
2. Build row and column masks for partial tiles. K beyond the matrix is zero-filled by the masked loads, never read out of range.
3. Allocate index scalars, fragments and eight-register accumulator groups.
4. Emit tensor loads with scoreboard slots and wait masks. The first K slice uses the from-zero MMA and later slices the accumulating one.
5. For a product that accumulates into C, compute from zero and add C afterwards, as Apple's library does.
6. Store the result, with masks on partial tiles.

Larger shapes run as a runtime K loop that carries the accumulators across trips. It is bit-exact from 1 to 255 trips, and at M512 it is 2.25× faster than the unrolled body, because the unrolled form spills (MM 25.101). Rule 30 of MM 17 is never to unroll the K loop.

A row grid, several simdgroups, a column grid and split-K spread a product across the GPU. Each combination is admitted only where it was receipted on hardware (MM 25.134, MM 25.165).

Epilogues run inside the same dispatch. The output can be narrowed to half, requantized to int8 with per-channel scales, or folded with a residual, without another dispatch (MM 25.165, MM 25.183).

16 Delivery and validation

A compiled program is packaged into an image that Metal loads, and its result is accepted only after a ladder of checks. Model graphs are then assembled from these programs.

16.1 Image and validation ladder

scanlink.author writes every byte of the image: the Mach-O object, the metadata, the library and the binary archive (MM 25.147, MM 25.148). The metadata holds the register count, the system registers, the bindings and the tensor-unit enable. The census of every byte of 421 delivered archives, the 148 constants Apple’s loader requires and what Metal checks are in [section 11.4](#).

Validation is a ladder with the cheapest check first ([table 16.1](#); MM 23.6).

Table 16.1. The validation ladder in the order its checks run, with the tool that performs each.

Step	Check	Where
1	every instruction decodes back to what was asked for	the encoders
2	the whole body decodes, with one final end of program (op684), no R126+ operand and no read after release	agxforge/g17/verify.py, releasecheck.py
3	the contract and metadata agree with the code	scanlink.author
4	the CPU emulator runs the program from its bytes against a reference, with the race check	tools/g17emu.py
5	one hardware dispatch per kernel, its output sentinel-filled, compared with the reference	tools/g17deliver.py
6	Apple’s compiler builds the same source, and the outputs are compared with a control input that must differ	tools/g17sourceexec.py

Steps 4 to 6 in more detail:

- Step 5 is exact, with one named exception. No CPU reference reproduces hardware exp2 (op1272), so kernels that use it are checked against an enclosure with a wrong-base control. These are the decode and prefill attention in their hw_exp2 forms, and the fast SwiGLU (MM 25.144.5, MM 25.183). One input set is checked per kernel, not every input.
- The emulator decodes the bytes, not the IR, so it reproduces the hardware from the program itself (MM 25.195, MM 25.197).
 - On shipped model graphs it is byte-identical to the GPU after every dispatch: one decode position of the batched graph (316 dispatches), one of single-stream decode (172), and the 1,024-token prefill through layers 0 and 1.
 - On Apple’s own census programs it matches Apple’s GPU output in 53 cases, with 0 differing and 45 refused by name.
 - Its “wp” tier also admits semantics that only whole programs verified on hardware.

- Step 6 uses Apple’s compiler only for comparison. Comparing outputs on the same source catches places where this compiler’s reading of the language differs from Apple’s. The cast in [section 15.3](#) was found that way.

16.2 Model graphs

A model is compiled kernel by kernel and then assembled into a static graph (MM [25.138](#), MM [25.141.15](#)):

- `tools/g17deliver.py` build compiles every kernel kind a model needs, authors each bundle, and dispatches it once. It writes the delivery index only if every check passes.
- `tools/g17q4graph.py` build turns the model’s weights and a config into `graph.json`, which holds:
 - named arenas, one buffer each with guards, and their initial contents;
 - the ordered dispatch list, with each binding’s arena and offset;
 - any per-token host writes.
- `tools/g17prefillgraph.py` adds the prefill section, which processes the prompt in M-row chunks before decoding starts (MM [25.142.10](#)).
- A config selects each kernel’s variant, so one model has many graphs (`tools/models/*.json`). They differ in their kernels, not in their structure.

Part III

Execution

17 Execution paths

The same native program runs on two paths ([figure 0.1](#)): through Metal, which builds the pipeline and submits the work, and below Metal, where the project’s runtime does both and submits through IOGPU ([table 17.1](#) lists what remains Apple’s).

17.1 Execution through Metal

The executor, `tools/g17decodegen.m`, loads each bundle’s library and binary archive. It builds the pipeline with `MTLPipelineOptionFailOnBinaryArchiveMiss`, so Metal must use the archived code. The GPU then executes this project’s code: patching one constant in GPU memory after the pipeline is built changes the output as predicted (measured, [MM 25.147](#)), though whether the bytes are copied or checked again before execution is not determined. Metal still derives the pipeline configuration from the metadata the image declares and submits the work; the native path ([section 17.2](#)) does both itself.

`decodegen` encodes each token into one command buffer. It creates one Metal buffer per arena, writes the arenas’ initial contents, and encodes the whole token, every dispatch of every layer, into that command buffer. Stage outputs stay on the GPU, in arenas that the next stage binds. In pipelined mode the GPU also runs the generation step, which advances the position and writes the next token’s embedding. The host then commits the token command buffers back to back with no wait between them, and reads the generated tokens from a log at the end. With no idle gap between tokens the GPU stays at its maximum clock state ([MM 0.15](#)).

In the current InternLM2.5-1.8B decode graph each of the 24 layers is seven dispatches:

```
RMSNorm -> qkv projection -> attention (RoPE + KV append + 32 key slices + merge) -> wo projection + residual
-> RMSNorm -> w1/w3 projections + SwiGLU -> w2 projection + residual
```

Four more dispatches run per token: the final norm, the vocabulary head, the argmax and the generation step. That makes 172 dispatches a token ([MM 25.154](#), [MM 25.190](#)). `mlx-lm` runs 16 dispatches a layer for the same model ([MM 25.142.1](#)). *Correction*: an earlier graph had eight dispatches a layer ([MM 25.142.1](#)); [MM 25.211](#) repeated that figure and is corrected in place.

Decode speed comes from these design choices:

- Split-K quantized matrix-vector products read the 4-bit weights with K split across lanes and thread-groups, and fuse the residual add or SwiGLU into the same dispatch ([MM 25.152](#), [MM 25.153](#)).
- The attention is sliced, with a parallel merge. It splits the keys into 32 slices, each a `simdgroup` with its own running softmax, and merges the partial results with a parallel butterfly. Its cost grows with the keys it reads, not with a serial merge: 9.7 μ s a layer at 128 keys, against MLX’s 12.0 ([MM 25.140](#), [MM 25.141.14](#)).
- The structure is static. Every buffer, binding and dispatch is fixed before the first token, so a token costs one command-buffer submission.

Metal twins of these kernels, with identical threads, partitions and operation order and compiled by Apple, are bit-identical and 10–15 percent faster (section 12.10), so the speed comes from these designs and this compiler’s code generation loses part of it. Which code difference costs the time is open (MM 25.211).

Batched decode uses the tensor unit. With up to 16 sequences, the batched projection dequantizes each 4-bit weight once in registers, then multiplies it against every sequence with tensor MMAs. It computes the transpose, $Y^T = W^T X^T$, because only B can advance through the key-block index register (MM 25.166).

- The dequantization runs in fp16, on the register-form fp16 fma (op798) (MM 25.196).
- Splitting K to about 512 threadgroups keeps the GPU occupied (MM 25.185).
- A 32-sequence form runs two 16-row halves over one dequantized weight (MM 25.178).

Prefill runs the prompt in chunks of up to 512 rows. It uses tensor-unit GEMMs on persistent fp16 weights, an attention that keeps its score and probability tiles in registers, and an fp16 FFN intermediate. Short prompts use a 16-row 4-bit route instead (MM 25.163, MM 25.183, MM 25.202).

17.2 Native execution below Metal

The native runtime runs a model with no Metal in the process: it never creates an MTLDevice or a Metal command buffer, and it checks that AGXMetal, Apple’s Metal driver library, is absent. Work is submitted through Apple’s private IOGPU framework to the installed kernel driver and the GPU firmware. Two pinned models share one pipeline: MiniLM-L6 for sentence embeddings, and Qwen2.5-0.5B-Instruct for text generation (MM 25.210).

Table 17.1 sets out what is the project’s and what is still Apple’s (MM 25.210.9).

Table 17.1. What the project supplies at each layer of the native path, and what is still Apple’s or outside the project.

Layer	This project	Still Apple’s, or outside
model	graph import, checkpoint conversion, lowering, memory plan	the checkpoints and tokenizers; NumPy and PyTorch for host input and validation
GPU program	every instruction byte, from the ordinary compiler	encoding rules recovered with Apple’s decoder
execution	the host worker: uploads, resource and command state, Submit, wait, checks	IOGPU. framework, IOKit, the AGX kernel driver, the firmware, the hardware

The pipeline has five stages (MM 25.210.3):

1. Import: `agxforge/g17/inferencegraph.py` reads the checkpoint’s tensors into a typed graph, re-fusing holes, overlaps and mismatched shapes.
2. Lower: `tools/g17inferencelower.py` compiles every node through the ordinary compiler. MiniLM becomes 170 stages from 16 distinct bodies. Qwen becomes 748 prefill and 916 decode stages from 28.
3. Plan: a memory planner assigns arenas, each slot with 256-byte guards.
4. Bundle: the schedule is placed into a measured resource graph: 128 MiB for MiniLM, 1 GiB for Qwen.

5. Execute: a resident worker runs it, with at most three bindings per stage and one Submit per stage.

For each stage the runner:

1. places the stage's program in allocation 0;
2. writes the stage's launch words: the program counter, the shader state, the grid, the threads per threadgroup and up to three buffer pointers;
3. builds one count-one Submit record with fresh callback blocks;
4. calls IOGPU and waits, for at most five seconds, for both callbacks.

Chapter 13 documents each of those fields and records, what is known about them, and where the measured envelope ends. The runtime refuses anything outside that envelope.

Completion and correctness are checked separately.

- Each stage reply carries preservation bits: the stage completed; the weights, guards, code and uploaded source are unchanged; and AGXMetal is absent.
- Every live region's guards are checked after every stage, and any change in the GPU's recovery or event counters fails the run.
- The captured outputs are then checked offline, at two levels. Each level's bound was registered before the first dispatch and never widened (MM 25.210.2).
 1. Each stage is checked against its actual input: it is recomputed by independent CPU arithmetic from the values its GPU predecessor actually produced. The bound is $2e-5 * (1 + \text{abs}(\text{reference}))$, and packs, gathers, bias and residual adds must be bit-exact. All 680 MiniLM stages and 9,984 Qwen stage checks pass. This level checks each stage's arithmetic but not drift across the model.
 2. Each application output is checked against the original checkpoint evaluated in FP64.
 - MiniLM's final embedding must be within a maximum error of 0.005 and a cosine of at least 0.999. Measured: $1.29e-4$, cosine at least 0.9999997786. Against an FP64 model of the same half-transport policy, the bound $2e-4 * (1 + \text{abs}(\text{reference}))$ also passes (maximum $8.21e-5$).
 - Each Qwen last-row logit vector, with the native tokens teacher-forced, must be within $0.05 + 0.003 * \text{abs}(\text{reference})$. Measured: 37 of 37 inside, and 37 of 37 agree on the top token.
 - One auxiliary control fails and is reported as failing: Qwen's prefix logits against an FP64 model of the same half-projection policy, $0.025 + 0.002 * \text{abs}(\text{reference})$. The maximum is 0.097, with 10,658 elements outside, traced to a half-rounding boundary in layer 0.
- The numerical domain is half-transported matrix operands with fp32 accumulation, not original-fp32 arithmetic.

The native path is slower (measured, MM 25.210.5):

- The matched MiniLM blocks take 2.427 against 1.797 ms for the FFN, and 3.815 against 1.962 ms for attention.
- In the paired comparison with Transformers on MPS, native Qwen decode ran 3.26–3.33 tok/s against MPS's 22.6–49.1; the lowest MPS figure is its first request, which includes first-use effects, and its warm requests ran 48–49. Later native changes raised decode to 5.15–5.32 tok/s, paired against the old guard check, and 4.62 tok/s in an unpaired replay. Those were not paired with MPS, so the two sets of numbers are not one comparison (MM 25.210.5).

- A repository-authored full-model Metal control runs the same 28 programs. Its own decode frame is 230 ms, of which the reported GPU time is 33 ms, so the control’s frame is dominated by time outside its GPU interval; this does not apportion the native runtime’s time.
- Some native costs are measured directly:
 - In the FFN block, a warm request spends 2.385 ms from Submit to the completed callback. That interval mixes GPU execution, driver work and completion delivery, and is not GPU time. Only 0.0495 ms then passes from callback to the waiter’s return, so semaphore wakeup is not the cost.
 - Replacing the host’s scalar guard check with a vectorised one made Qwen decode 1.30–1.31× faster.
 - The rest of the per-token breakdown, across 916 Submits, guard scans and padded transfers, is not measured.

17.3 Comparison of the two paths

Table 17.2. The path through Metal and the native path below it, compared.

	through Metal	below Metal
models	InternLM2.5-1.8B, Qwen3-0.6B, Qwen3-8B	MiniLM-L6, Qwen2.5-0.5B
per token	one command buffer	one Submit per stage (916 for a Qwen decode token)
pipeline configuration	Metal derives it from the image’s metadata	the runner writes it, from measured classes
speed	ahead of mlx-lm for some workloads (showcase, section 2; MM 25.190)	slower than matched Metal blocks and MPS; no advantage either way against the full-model Metal control
established	correctness on hardware; performance against mlx-lm	complete execution with no Metal; correctness against the original checkpoint
not established	why Apple’s code for the same kernels is faster	a speedup from bypassing Metal; behaviour across OS updates; reproduction outside this campaign; a result only this path can produce (MM 25.210.9)

Performance work is done on the Metal path, which the native path does not replace.

A Corrections to earlier interpretations

Many readings in the project record were later corrected, and the chapters give the current account. The tables below list earlier readings that still appear elsewhere in the repository, each with where it was made and where it was corrected, so that a stale statement can be recognized. Where an old reading is likely to be met, the chapter also carries a short “Correction” note.

A.1 The device, registers, encoding and metadata (chapters 1, 2, 3, 11)

Table A.1. Corrected readings about the device, registers, encoding and metadata.

Old reading	Where	Current reading	Where
R126 and R127 read zero and drop writes	recon section 134 (MM 19)	Instructions naming R126–R143 are squashed whole; a zero-filled buffer hid it	MM 10, MM 19, MM 25.18
A released register reads zero	MM 0.15, MM 17 rule 37	0 in every controlled run; not established as a rule; read only where 0 and the old value agree	MM 25.95, MM 25.145, MM 25.147
Releases take effect at the end of the instruction (as a general rule)	MM 25.145	True of measured ALU forms; the word store reads its value after its index’s release	MM 25.117
fp16 fma, register form (op798) holes at registers 2 and 3 mod 8	digest (not MM)	R18, R19, R26, R27, R50, R51, R58, R59 low halves	MM 25.196
half to fp32 widening (op1004) holes are d = 14, 15	MM 25.136.6	R14, R15, R30, R31, R46, R47, R62+	MM 25.133, agxforge/g17/cc.py
The register file is 32 (ALU destination five bits; memory fields four bits)	early agxforge/g17/cc.py field map	R0–R125 measured; fields re-measured at seven and eight bits	agxforge/g17/cc.py (hireg sweep), MM 10
IRGPR32 is “a file no instruction writes”	MM 25.72 / row T4	its users write ordinary Rn; the class is R0–R143 plus IR0–IR15	MM 25.59, Apple’s class table
16 bytes need both byte10 bits 7 and 0 (class 7)	docs/agx-isa-status.md	True for its population; _WIDE7_EXT rule 12 + 4 × byte10[0] for byte6 in {0x60, 0x68, 0x70, 0x78}; neither general	MM 25, corpus spans
“Our decode agrees with Apple’s framing 400/400” as independent evidence	MM 25.63	That decode calls Apple’s decoder	MM 25.209
D1 975, D2 690; only tensor MMA, 16-bit × 16-bit, accumulate (op5106)/tensor MMA, 16-bit × 16-bit, no accumulator (op5107) admitted	docs/g17-isa-cartography.md	D1 7,549 (7,546 with the 26A434 decoder), D2 1,118; ten MMA opcodes admitted	docs/g17-isa-reference.md, MM 2
Store “consumer token” with a wait counter	recon sections 15, 22–24 (MM 19)	bits 5:4 of the source register number	MM 19

Table A.1, continued

Old reading	Where	Current reading	Where
MMA high bits are a result ring	recon section 14 (MM 19)	eight-bit wait mask over load slots	MM 6, MM 19
Tensor load slot is operand 1 bits 20–27	MM 6 (and chapter 9.1’s table)	bits 20–23 slot + 1; bits 24–31 the load’s wait mask	agxforge/g17/memenc.py, decoder flips
Slot 31 and the variable record describe register spill	MM 25.113	They describe the constant program	MM 25.120
The constant program’s uniforms: “6 + number of writes”, “1 + highest uniform / staging write (op592) uniform”	MM 25.120 rounds 2, 3b	L3: 1 + max(K // 2) over every writer	MM 25.120
76 inert class constants (list itemising 92 bytes)	ledger g17-class-constants-are-mostly-inert	76 inert; the descriptor tail is not one of the 148	MM 25.148
LC_TABLE (0x31)	project tooling	LC_NOTE	MM 25.148
manifest.register_count is the program’s	delivered manifests	222 of 247 carry a template’s value; read the container	MM 25.147
The encoder may accept bytes whose field arithmetic reaches zero residual	campaign encoder	must decode back; it once rounded an odd register silently	MM 2, MM 23.3

A.2 Scalar arithmetic, control flow, special registers and cross-lane operations (chapters 4, 5, 6, 8)

Table A.2. Corrected readings about scalar arithmetic, control flow, special registers and cross-lane operations.

Old reading	Section	Current reading	Section
32-bit compare-select, four registers (op11375) field code 1 is a == b (cc CSEL_CC['eq'])	MM 25.90 “Was” column	(a & b) != 0, a bit test; printed code 15	MM 25.90; ledger/claims.toml
fine x-derivative, saturated (op1062) is fsat	MM 25.90 “Was” column	op1062 is cross-lane dfdx.fine.sat; fp32 saturate is op904	MM 25.90
32-bit integer to fp32 (op11179)/10 operand 2 inherited as 4 (unsigned) for every int-to-float	MM 25.90 “Was” column	operand 2 is signedness, 4 unsigned, 5 signed	MM 25.90
A read_sr’s first consumer must wait (read_sr is a late value)	MM 6, MM 0.8, MM 25.90	slot-0 reads are readable at distance 0; Apple’s wait is a free policy	MM 25.117, MM 25.121
A store cannot take a special register as its source (the value “has not arrived”)	ledger 2026–09-08, MM 25.117 first claim	the failure was the aliased store release of plain 32-bit store (op17229) operand 6	MM 25.117

Table A.2, continued

Old reading	Section	Current reading	Section
0x82 = quadgroup_index_in_threadgroup, 0x98 = simdgroups_per_threadgroup	isa/g17-scalar- isa.toml	0x82 = SR_SIMD_ELEM (lane), 0x98 = SR_TG_X_SIZE; the builtins are computed from them	MM 0.8, agxforge/ g17/mdgen.py
SR_SIMD_GRP has no generalized slot-29 entry	MM 0.8	entry 53	MM 25.93, agxforge/g17/ mdgen.py
A released register reads zero	MM 0.15, MM 17 rule 37	zero in every controlled measurement; checker assumes zero or old value	MM 25.95, MM 25.145
The branch is a 4-byte word with a 12-bit displacement	isa/g17-scalar- isa.toml	10-byte branch; the 4-byte reading is its first four bytes	agxforge/g17/ cf.py, isa/g17- exec-mask.txt
Codes 11 and 15 mean not-equal	isa/ g17-condition- codes.toml (prediction)	15 is the bit-test pattern; 11 unseen	same file (measured row)
(uint)x lowers as a 34-instruction decomposition (gave 5 for -5.5)	MM 25.186	one fp32 to 32-bit integer (op9320), mode 1 (gives 0)	MM 25.186, MM 25.196
special-register read, 16-bit (op14060) might zero-extend into the high half	MM 25.145	it writes only its half	MM 25.145.4

A.3 Memory and synchronization (chapters 7, 9)

Table A.3. Corrected readings about memory and synchronization.

Old reading	Where	Current reading	Where
The device load carries no displacement field; address = base + (index + displacement) × sizeof	isa/ g17-addressing- rules.json	Displacement field exists, in bytes, unscaled; the element-count add applies to unsigned tid + k; the (index + displacement) × element law is the immediate-address family's	isa/g17-address- formula.txt, MM 25.141.18, MM 25.177
Imageblock store operand 1 bit 31 selects shared / coordinate-addressed storage	MM 25.89 (round 9), MM 25.104.3 rounds 8–9, MM 25.145.2, tools/g17emu.py comments	Bits 24–31 are the wait mask; bit 31 waits on slot 7	MM 25.96, MM 25.104
Cross-threadgroup hand-off requires a dispatch boundary	MM 0.14	Fence-count-fence works inside one dispatch; waiting patterns need residency	MM 25.140.5, MM 25.141.1

Table A.3, continued

Old reading	Where	Current reading	Where
A threadgroup barrier with mem_device orders device memory across threadgroups	cc's barrier("device") use, MM 25.140.5	It does not; device fence (op14156) on both sides	MM 25.140.5
435/10 is "array", 447/6 is "threadgroup atomic"	MM 25.75	16-bit AND with immediate (op435); threadgroup barrier (op447)	glossary, MM 25.192
uniform-address atomic, returning old (op10094)'s return fills slot 7	glossary	Fill slot is operand 1 bits 20–23; Apple uses 0/1, this compiler 7	MM 25.115
Simultaneous uniform atomics combine and return a pre-combine value	MM 25.115 (struck)	Consumer waited on the wrong slot	MM 25.115
uniform-address atomic, returning old (op10094) operand 6 is a source lifetime ("16 writes nothing")	ledger g17-the-uniform-atomic-needed-one-lifetime-bit	Operand 6 is the byte offset	MM 25.140.3
per-lane-address atomic, returning (op10090) has eight unassigned raw operation codes	isa/g17-atomic-semantics.json	Retracted; codes depend on the form's field placement	isa/g17-atomic-family.toml
range mask, 16-bit source (op621) = (0xFFFFFFFF » src) & 0xF, saturating at 32	isa/g17-address-formula.txt (first fit)	mask of min(max(W - src, 0), 4) ones, W = operand 5	same file, resolved section
Vector displacement is in units of eight bytes	asm.encode_vec4	Bytes, for loads	MM 25.141.18
The 8-byte plain 32-bit store (op17229) has no wait bit	agxforge/g17/cc.py	Apple's 8-byte op17229 carries operand-1 wait bits; this compiler's encoder does not write them	isa/g17-wait-mask-census.json, MM 25.146
Only the accelerator's own reads have faulted	MM 17 rule 34	Through Metal; below Metal, launch-state errors fault	docs/g17-first-dispatch-trials.md
A "three-bit store limit" on wait masks	an earlier recon section, recalled in MM 6	Encoder-table artifact; 128-bit vector store (op17256) uses all eight bits	MM 6
Load-to-use about 121 ns	MM 25.97	L1 at idle clock; 30.5 ns warm (numbers in ch 12)	MM 25.121

A.4 The tensor unit and performance (chapters 10, 12)

Table A.4. Corrected readings about the tensor unit and performance.

Old reading	Section	Current reading	Section
An fp32 value above 464 gives the (positive) canonical NaN in e4m3fn	MM 0.11	An fp8 overflow keeps its sign (0xff / 0xfc for negatives); only NaN inputs become positive canonical NaN	MM 25.105, MM 25.105.1
Hot loops: about 26 KiB, roughly 2,800 instructions	MM 0.1, MM 7	The step is at a 16 KiB loop body (15,971 fast); no cost at 26–64 simdgroups per core	MM 19, MM 25.144.12
Instruction-footprint cliff at (16,066, 17,158] whole-kernel bytes	recon 156 via MM 19	The variable is the loop body, bracketing 16 KiB	MM 19
Hardware counters: timestamps only (read as “no counters exist”)	MM 0.15, MM 25.48, MM 25.141.6	True of Metal’s public API; Instruments’ limiter template records 89 counters	MM 25.144.4
Limiter counters not reachable without Apple’s GUI or entitlement (“M4 status”)	MM 25.144.4 (earlier bullet)	Reachable via xctrace with the GUI-saved template	MM 25.144.4 (later recording)
Power was not measured	MM 20	GPU-rail power measured on both engines	MM 25.56, MM 25.60
H5 (destination keep flag) open	MM 20	Set implies the next MMA writes the same tuple; hardware-inert	MM 6, MM 25.22
Saturation with transposes, chains, high registers not run	MM 20	Saturation holds under transpose; clips once per issue in a chain	MM 25.42, MM 25.89
fp32 operand subnormals flush to zero	recon 5 via MM 19; tensorops_model.py line 12 docstring	Low 13 bits cleared only; subnormals kept	MM 5, MM 19
tensor MMA, fp32 A × 16-bit B, accumulate (op5100) / tensor MMA, fp32 A × 16-bit B, no accumulator (op5101) never compiler-emitted	recon 27, 104 via MM 19	Emitted under relaxed_precision	MM 19, MM 17 rule 29
Destination layout is a property of the opcode	recon 100 via MM 19	One map; pos_b(k,c) = pos(rot11(k), c); the difference is the producing kernel’s row labeling	MM 3, MM 19
The hardware performs the matrix-to-fragment transposition	MM 19	No tile-layout unit; ALU address arithmetic makes the layout	MM 7, MM 19
MMA high flag bits are an eight-slot result ring	recon 14 via MM 19	An 8-bit wait mask over eight load slots	MM 6, MM 19
Two MMA hazards (stored result to register consumer; back-to-back MMAs)	recon 112 via MM 19	Host dispatch-scoping (dim) artifact	MM 19, MM 17 rule 5
matmul2d allows one run() per kernel; chain must cross a kernel boundary	recon 108, 121 via MM 19	Slice offsets misread; single-kernel chain works	MM 19

Table A.4, continued

Old reading	Section	Current reading	Section
Feed relabelings are instruction properties	MM 0.10, MM 13, MM 17 rules 9–11 (unscoped text)	Only for kernels that pack A and D canonically; under this compiler’s B-row packing the feeds read the logical operand (section 10.5)	MM 25.103
Two reduction axes follow different orders	recon 120 via MM 19	One rule: per-lane chain then butterfly over the axis lane bits	MM 12, MM 19
Reduction sums run descending	recon 19 via MM 19	Direction belongs to the compiled object (fast-math)	MM 12, MM 19
Legacy MMA and tensor unit share one issue slot	recon 31 via MM 19	Complete overlap at saturation; partial overlap is in-order issue	MM 9, MM 19
Legacy MMA overlaps fp32 fma 81–86 percent	MM 19 (first version of recon 152)	True of legacy 8×8 simdgroup MMA, f16 (op838) only; class-118 forms 0.37 or less	MM 9, MM 19
Occupancy is slot-limited, not register-limited (20–116 registers)	recon 167 via MM 19	Direct counter: 44.6 → 32.1 simdgroups per core over 22–126 registers (Apple-compiled kernel)	MM 19, MM 25.49
Legacy bimodality: two-state 9/8 rate; 79.4 ns quantum	recon 158, 164 via MM 19	Per-core staircase; bimodality only in boundary windows	MM 25.122
Tensor-load element code 4 “means 32-bit” withdrawn	MM 25.5 “R7 narrowed”	Element field = width in bytes; 4 = fp32	MM 25.37
range mask, immediate width (op612): second immediate is an absolute bound	MM 19	It is a width	MM 7, MM 19
16-bit AND with immediate (op435) and 16-bit AND (op437) are shifts	(glossary history)	16-bit ANDs	MM 25.28 / glossary
Software-pipelining gain $\geq 1.2\times$ preregistered	MM 25.97	1.08× at 8 simdgroups, 0.99× at 12.8 per core	MM 25.97, MM 25.107
Apple M512 58.6 μ s; ratios 1.17×, 2.1–4.5×	MM 25.98	106.6 μ s; 1.06–1.07× at equal geometry / M512	MM 25.107
WS/TP crossover about 64 tokens	MM 16 (model)	fp8 48, bf16 64 on the retained harness	MM 25.126
L1 load latency about 121 ns	MM 25.97	30.5 ns at a warm clock	MM 25.121
Residency used-125: ~43.6 per core, then ~58; S = 1,024 shortfall	MM 25.115 (v3-v5)	Warm: 1,024/1,024; this compiler’s counter 64–68 per core at 125	MM 25.115, MM 25.121
ac_bench per-op figures in milliseconds	MM 25.55	Seconds	MM 25.56
Prefill GEMMs at 60–69 percent of peak	MM 25.155	80–89 percent in graph with a closure-checked profile	MM 25.160
Constant-drain fusion model (6.54 μ s per pipeline change)	MM 25.142.4 (preregistration)	Refuted; removing a change saved 0.6–2.5 μ s	MM 25.142.4
mlx-lm baseline via bench loop (1.18–1.26×)	MM 25.153-25.181	generate_step baseline: 1.14–1.22×	MM 25.190

A.5 Superseded counts and wordings

The chapters no longer carry the reconciliation detail collected here: counts that changed between measurements, inventories that differ between sources, and wordings that were later replaced. Each row gives where the earlier statement was made and where it was settled.

The device, registers, encoding and metadata

Table A.5. Superseded counts and wordings about the device, registers, encoding and metadata.

Old reading or accounting	Where	Current reading	Where
“Everything was measured on macOS 26.6.2 (25G83)” / “every figure ... one OS build”	section 0.1 ; earlier section 1.1	25G83 is the stated platform and is recorded by the below-Metal records; the evidence files of the Metal-path measurements do not record an OS build	section 1.1 , MM 25.91
“A spinning threadgroup is never preempted”; “a grid barrier among resident threadgroups is sound”	earlier section 1.5 ; MM 25.141.1 wording	observations of one probe family in the tested configurations; restrictions: do not depend on preemption; size waiting grids to measured residency	section 1.5 , MM 25.141.1
A released register “reads zero” (MM 0.15), “every later read returns zero” (MM 6, MM 17 rule 37), every release observed read exactly 0 (MM 25.95), against “zero or its old value, observed both ways” (MM 25.145), “observed only sometimes” (MM 25.145.4), “returns 0 - sometimes, not always” (agxforge/g17/releasecheck.py)	MM 0.15, MM 6, MM 17, MM 25.95, MM 25.145, MM 25.145.4	every controlled probe read zero; readback not established in general; read a released register only where zero and the old value give the same answer	section 2.5
IR* is “a distinct 16-register file” (agxforge/g17/registerdomain.py) against the 160-member decoder class IRGPR32	agxforge/g17/ registerdomain.py, MM 25.59	the two agree. IR32 is the 16-register IR file; IRGPR32 is R0–R143 followed by IR0–IR15, and corpus users write only its general members	section 2.2
fp16 fma, register form (op798) holes are “registers 2 and 3 mod 8” (R34, R35, R42, R43 are not holes); half to fp32 widening (op1004) holes only d = 14 and 15 (“r119/r120” in decoder ids)	MM 25.136.6	the listed sets: the fma’s eight low-half destinations and two sources; the widening’s R14, R15, R30, R31, R46, R47, R62 and above	section 2.7 , MM 25.133, MM 25.196
Instructions are “2 to 16 bytes”	section 0.2 ; the 216-opcode census (isa/g17-opcodes.toml)	2 to 22 bytes, even: Apple’s corpora show every even length 2–16 plus 18 and 22 (texture write (op10909) at 18, texture sample (op14661) at 22), no 20; the decoder-admitted universe shows 4–16, 18, 22	section 3.1

Table A.5, continued

Old reading or accounting	Where	Current reading	Where
“Apple’s table declares 17,779” (MM 24.8 row T1, docs/ g17-isa-cartography.md, tools/ g17admit.py) against “17,796” (docs/agx3-oracle.md, agxforged/ g17/model.py)	those sources	both hold for the 25G83 build, which has 17,796 table entries, 17,779 declaring at least one operand, 17 declaring none (isa/ g17-agx3meta-instrs.txt)	section 3.2
D2 690, then 1,107 when per-form evidence was generated, then 1,118 through hardware checks of drifting bits	MM 25.87, MM 25.58	D2 = 1,118 of D1 7,549 (7,546 with the 26A434 decoder)	section 3.2
D1 975, D2 690, and only tensor MMA, 16-bit × 16-bit, accumulate (op5106) and tensor MMA, 16-bit × 16-bit, no accumulator (op5107) admitted	docs/g17-isa- cartography.md	D1 7,549 (7,546 with the 26A434 decoder), D2 1,118; ten tensor MMA opcodes admitted	sections 3.2 and 3.8
The project framer’s first check walked 112,952 instructions where Apple’s decoder finds 85,806	docs/ agx3-oracle.md section 10	the framer agrees with Apple on 1,725 of 22,187 constant programs and misframes 200 of 200 current programs; Apple’s decoder is the ground truth	section 3.1 , MM 25.88, MM 25.209
Encoder held-out re-authoring 28.2 percent (before seven fixes)	MM 25.88	80.8 percent	section 3.3 , MM 25.88
Tensor load, half/bf16 (op12674) fill-slot field is operand 1 bits 20–27	MM 6	bits 20–23 fill slot (slot + 1), bits 24–31 the load’s own wait mask	section 3.4
Constant-program binding reads at “halfwords 8 + K and 10 + K” (K the halfword offset, MM 25.179) against “8 + 4k and 10 + 4k for the k-th buffer” (MM 25.59)	MM 25.179, MM 25.59	the same rule with K = 4k	section 11.2
Constant-program metadata read as a register-spill description	MM 25.113	it describes the constant program	section 11.2 , MM 25.120
The “76 inert” list itemised 92 bytes	MM 25.148 (ledger)	76 inert bytes; the descriptor tail (392, 16) is separate and not one of the 148 class constants	section 11.4 , MM 25.148
“The image is ours” census covering one novel kernel	MM 25.148 (ledger)	the 421-archive delivered-image census, every byte assigned	section 11.4 , MM 25.148

Scalar arithmetic, control flow, special registers, memory, cross-lane and synchronization

Table A.6. Superseded counts and wordings about scalar arithmetic, control flow, special registers, memory, cross-lane operations and synchronization.

Old reading or accounting	Where	Current reading	Where
“fsat” names fine x-derivative, saturated (op1062); “saturate returns 0”	early map; the anomaly came from a record that ran one lane	op1062 is a cross-lane dfdx with saturation (function unconfirmed); saturate is fp32 saturate (op904), sat(a + K)	MM 25.90; sections 4.3 and 8.5
Printed integer condition code 15 is “something other than lt/gt/signed”	isa/g17-condition-codes.toml notes	it is the bit test (a & b) != 0, field code 1 of 32-bit compare-select, four registers (op11375)	MM 25.90; section 4.2
(uint)x lowered as a 34-instruction decomposition that decoded the float’s bits; (uint)(-5.5) gave 5	this compiler, before MM 25.186	fp32 to 32-bit integer (op9320), truncate and saturate, gives 0; Metal leaves the case undefined and each emulation matched its own program’s GPU output (a compiler difference, not a hardware error)	MM 25.186; section 4.5
Integer-to-float conversion code 4 for every source (inherited from an unsigned witness; a negative int32 became a value near 2^{32})	agxforg/g17/cc.py before MM 25.90	operand 2 is signedness: 4 unsigned, 5 signed (2/3 on the 16-bit form)	MM 25.90, MM 25.195; section 4.5
The join was called exec.restore	early isa/g17-scalar-isa.toml	exec pop, restore mask (op577) with count 1	agxforg/g17/cf.py; section 5.1
A 4-byte backward branch de 6f f3 1e with a 12-bit displacement whose sign is word bit 28	isa/g17-scalar-isa.toml	the first four bytes of the 10-byte conditional back-edge branch (op458): kind 2 with b2.4 and b2.0 set; d1-d10 are the 10-byte map’s weights 2 to 2^{10} in bytes 0–2; the “sign” b3.4 is the 2^{14} weight, reading as a sign only because a small negative displacement sign-extends through it	agxforg/g17/cf.py; section 5.1
Byte 0x82 of the 4-byte special-register read is quadgroup_index_in_threadgroup; 0x98 is simdgroups_per_threadgroup (from kernels said to differ only in that byte)	early isa/g17-scalar-isa.toml	the toml recorded registers the builtins’ kernels read; 0x82 is the lane index (SR_SIMD_ELEM), 0x98 threads-per-threadgroup x	MM 0.8, isa/g17-special-register-numbers.txt, agxforg/g17/mdgen.py; section 6.1
SR_SIMD_GRP “not generalized”, no slot-29 entry	MM 0.8 table	entry 53 is exactly the SR133 read added by the multi-simdgroup compile (execution_simdgroups sweep); single-builtin compiles carry [53]; entry 152 → 4 added later	MM 25.93, MM 25.149.1; section 6.2

Table A.6, continued

Old reading or accounting	Where	Current reading	Where
A store reading a special-register value directly stores what has not arrived (ledger, 2026-09-08); every read is a late value needing a wait	ledger; MM 6, MM 0.8, MM 25.90	the failing stores named one register as value and index with the index slot releasing it; a slot-0 read needs no wait (latency probe 2026-09-24, 90 of 90 rows reproduced)	MM 25.117, MM 25.121; section 6.3
The BIF0 read page fault came from the tensor-probe configuration the rule names	MM 17 rule 34 before its 2026-09-23 correction	that configuration was a different process (pid 76676), which hung the GPU for about 320 s and is known to hang, not to fault; the fault came from a regprobe_multi kernel whose configuration was never recorded (corrected from another lane’s reading of the crash report and system log)	MM 17 rule 34; section 7.5
The 8-byte plain 32-bit store (op17229) “has no byte9 and no wait bit”	agxforge/g17/cc.py	the encoding carries wait bits: 15 corpus instances match their pending slots, and Apple’s tex2d-read carries 0x1000010 on one; the statement describes this compiler’s encoder only	isa/g17-wait-mask-census.json, MM 25.146; section 9.1
“Eight raw operation codes of per-lane-address atomic, returning (op10090) unassigned by any Metal intrinsic”	isa/g17-atomic- semantics.json	read off one witness’s field placement; retracted; the fourth operation bit of the 12-byte form is byte11[4], and unassigned codes must be counted per form	isa/g17-atomic-family.toml; section 9.3

The tensor unit, performance and submission

Table A.7. Superseded counts and wordings about the tensor unit, performance and submission.

Old reading or accounting	Where	Current reading	Where
Two declaration counts for the tensor family: 132 opcodes in scheduling classes 170-175, against 535 declarations in opcode ranges 5090-5618 and 10384-10389 (18 forms with an 8-word destination, 5090-5107, 8 admitted; 36 with a 4-word destination, 5108-5143; 6 integer forms, 10384-10389, 2 admitted; 475 _shifted forms, 5144-5618)	MM 2, MM 0.3; MM 25.6, MM 25.4 “F4, narrowed”	The two counts use different bases and do not conflict. The 132 counts by scheduling class; the 535 counts by opcode range and includes every _shifted declaration. Ten forms are decoder-admitted in either accounting	section 10.2
“None of the 56 MMA forms the decoder tables declare” has a scale operand	MM 11	The statement stands. How the 56 relates to the 60 non-_shifted declarations (18 + 36 + 6) is not stated in the sources	section 10.2

Table A.7, continued

Old reading or accounting	Where	Current reading	Where
Destination keep-flag rule quoted over three populations: 2,192 MMAs in 140 programs (1,093 flagged); 540,396 flag-set instances with 155,963 converse counterexamples; 758,739 “instances”	MM 25.22; MM 6; MM 0.4	All three agree on the one-way rule (set implies the next MMA reuses the tuple) with zero counterexamples. The 758,739 population is not defined by its source and is not 540,396 + 155,963 (696,359)	section 10.3
“Whether release clears or frees storage is open”; “a released register reads 0 afterwards”	MM 25.89; earlier text of section 10.3	A release acts per lane under the execution mask; a released register read zero in every controlled run, but what it reads is not established in general; nothing reclaimed it within the measured bound	section 2.5 (MM 25.95, MM 25.145)
tensor MMA, fp32 A × 16-bit B, accumulate (op5100) and tensor MMA, fp32 A × 16-bit B, no accumulator (op5101) “are never compiler-emitted” (from recon sections 27 and 104)	MM 19	Emitted under relaxed_precision	section 10.1 (MM 17 rule 29)
Element code 4 does not mean a 32-bit access (the withdrawal of “code 4 means 32-bit”)	MM 25.5 “R7 narrowed”	The element code is the access width in bytes; code 4 is emitted by fp32 accesses (two witnesses)	section 10.8 (MM 25.37)
16-bit AND with immediate (op435) and 16-bit AND (op437) read as shifts	earlier opcode notes	They are ANDs (op435 executed over 4,096 lanes, MM 25.192)	section 10.11 (MM 24.5)
Apple matmul2d at M512 took 58.6 μs (round one); this compiler 1.17× behind, unrolled 2.1–4.5×	MM 25.98	Round one did not reproduce; interleaved medians give 106.6 μs for Apple at M512, the K loop 1.07× behind	section 12.2 (MM 25.107)
Residency of a used-125 program: about 43.6 per core (v3), then about 58 per core (v4), and a shortfall at S = 1,024 (v5)	MM 25.115	v3 ran beside an orphaned kernel; the v5 shortfall was a cold-clock effect. Warm, 64–69 per core from 49 to 125 registers, 95.0 at 19	section 12.5 (MM 25.121)
Instruction-fetch step as a whole-kernel size bracket (16,066, 17,158] bytes	recon section 156; MM 19	The variable is the loop body: 15,971 bytes fast, 17,247 slow, bracketing 16 KiB	section 12.6 (MM 19)
GPU performance counters “not reachable without Apple’s GUI or entitlement” (the “M4 status” paragraph)	MM 25.144.4	89 counters are recorded by xctrace with the GUI-saved “Performance Limiters” template, later in the same section; Metal’s public API still offers timestamps only	section 12.7
“Power was not measured”	MM 20 “Hardware identity”	Measured on the GPU rail; MM 20’s own later paragraph says so	section 12.8 (MM 25.56, MM 25.60)

Table A.7, continued

Old reading or accounting	Where	Current reading	Where
Below Metal, “three command buffers make one selector-7 call”, a persistent command stream with no MMIO doorbell	MM 0.15; earlier text of section 12.7	Selectors 7, 16 and 28 are called in queue creation; each Submit enters the kernel through IOConnectTrap4 (one record) or IOConnectCallMethod selector 0x1d (several records)	sections 13.2 and 13.6
Submit kicks the GPU only through shared-memory writes, with no per-dispatch kernel call; a candidate store at Submit+0x11c	MM 25.143; README row 6a	Each Submit enters the kernel through the trap or the method call; the store at Submit+0x11c is an output-argument write	section 13.6
The FinalizeShmemHeader ready bit is the doorbell	README row 3	Disproved by a live control; see the Submit path above	section 13.6

B Scalar instruction semantics

The tables cover the scalar opcodes whose function the sources state, grouped as in [chapters 4 to 6](#) and [8](#). Names are copied from `isa/g17-opcode-glossary.json`. “Lengths” are the encoded byte lengths the sources name for the form (“-” where no length is recorded with the semantic); a semantic holds for the (opcode, length) forms that ran and is not claimed for other lengths. “Standing” is this reference’s reading of the cited sections, using the statement kinds of [section 0.7](#): *measured* (executed against an independent reference with a control), *measured (receipt fit)* (32 lanes matched one candidate semantics among rivals; the replay checks consistency and is not independent evidence, MM [25.145](#)), *whole-program* (only inside programs verified bit-exact on the GPU), *Apple’s compiler emits* (compile-side only). A dagger (†) marks an opcode that the glossary still tags *compiled* although the cited section, which is newer, measured it; see the notes after the tables.

B.1 Integer add, subtract and saturate

Table B.1. Integer add, subtract and saturate instructions.

Name	Opcod	Lengths	Semantic	Standing	Source
32-bit add	op10282	–	$d = a32 + b32$, low 32 bits	measured	g17emu EVIDENCE; MM 0.16
32-bit add immediate	op10279	12	$d = a32 + \text{imm}8$; source scaled by modifier bits 8–10	measured	g17emu EVIDENCE; MM 25.115
add, 32-bit plus zero-extended 16-bit	op10283	–	$d = a32 + \text{zext}16(b)$, mod 2^{32} (zero, not sign, extension)	measured	MM 25.145.1
add 16-bit to 32-bit	op10285	–	$d32 = \text{zext}16(a) + b32$	measured	MM 25.195
16-bit plus 16-bit, 32-bit result	op10286	–	$d32 = \text{zext}16(a) + \text{zext}16(b)$; $0xFFFF + 1 = 65536$	measured	g17emu EVIDENCE; MM 25.145.2
16-bit add immediate	op10289	–	$d16 = a16 + \text{imm}$, wraps at 16 bits	measured	MM 25.195
64-bit add	op10267	–	$d64 = a64 + b64$ (base + offset addresses)	measured	glossary; MM 0.16
32-bit subtract	op11667	12	$d = a32 - b32$, wraps	measured	g17emu EVIDENCE
32-bit subtract immediate	op11666	12	$d = a32 - \text{imm}$, wraps ($4096 - 15 = 4081$)	measured†	MM 25.145.1 , MM 25.95
64-bit subtract immediate	op11655	–	$d64 = a64 - \text{imm}$	Apple’s compiler emits	glossary
32-bit saturating add	op10239	–	$d = a + b$, unsigned saturation (signed if source modifier carries 8)	measured (receipt fit)	MM 25.145 ; MM 25.59
32-bit saturating subtract	op11624	–	$d = a - b$, clamped at 0 (unsigned saturation)	measured (receipt fit)	MM 25.145

B.2 Integer multiply and multiply-add

Table B.2. Integer multiply and multiply-add instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
32-bit multiply	op10825	14	$d = a_{32} * b_{32}$, low 32 bits	measured	g17emu EVIDENCE; MM 25.73
32-bit multiply-add	op10826	–	$d = a_{32} * b_{32} + c_{32}$, low 32 bits	measured (receipt fit)	MM 25.145
multiply by immediate, 32-bit addend	op10823	–	$d = a_{32} * \text{imm} + b_{32}$	measured	glossary
multiply by immediate, 16-bit addend	op10824	–	$d = a_{32} * \text{imm} + \text{zext16}(b)$	measured	glossary
multiply-add, 16-bit multiplier	op10829	–	$d = a_{32} * \text{zext16}(b) + c_{32}$	measured	glossary
multiply-add, 16-bit multiplier and addend	op10830	–	$d = a_{32} * \text{zext16}(b) + \text{zext16}(c)$	measured	glossary
16-bit multiply by immediate plus immediate	op10831	–	$d_{32} = \text{zext16}(a) * \text{imm} + \text{imm}$	measured	glossary
multiply-add, 16-bit multiplicand	op10835	–	$d = \text{zext16}(a) * b_{32} + c_{32}$	measured	glossary
multiply-add, 16-bit multiplicand and addend	op10836	–	$d = \text{zext16}(a) * b_{32} + \text{zext16}(c)$	measured	glossary
signed-by-unsigned 16-bit multiply-add	op10838	–	$d = \text{sext16}(a) * \text{zext16}(b) + c_{32}$ (60 of 60 over seven records)	measured	MM 25.59
16-bit multiply by immediate plus immediate, 16-bit result	op10858	–	$d_{16} = a_{16} * \text{imm} + \text{imm}$	measured	glossary
16-bit multiply by immediate plus register	op10860	–	$d_{16} = a_{16} * \text{imm} + b_{16}$	measured	glossary
multiply by immediate minus immediate	op11059	–	$d = a * \text{imm} - \text{imm}$	measured	glossary
multiply by immediate minus register	op11060	–	$d = a * \text{imm} - b_{32}$	measured	glossary
multiply-subtract	op11063	–	$d = a * b - c$	measured	glossary
$32 \times$ immediate to 64-bit product	op10789	–	$d_{64} = a_{32} * \text{imm}$	measured	glossary
64-bit address multiply-add	op10790	12	$d_{64} = a_{64} + b_{32} * \text{imm}$ (address formation)	measured	glossary; MM 25.80
32×32 to 64-bit product	op10793	–	$d_{64} = a_{32} * b_{32}$	measured	glossary

Table B.2, continued

Name	Opcode	Lengths	Semantic	Standing	Source
32 × 32 + 32 to 64-bit multiply-add	op10795	–	d64 = a * b + c, unsigned	measured	glossary

B.3 Logic, shifts and bit operations

Table B.3. Logic, shift and bit-operation instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
32-bit AND with immediate	op423	–	d = a32 & imm; only the mask bits reach the result	measured [†]	g17emu EVIDENCE (confound3 receipts); MM 25.145
32-bit AND	op424	4, 10	d = a32 & b32 (the 4-byte form releases its sources)	measured	MM 24.8 row T4; MM 25.77
16-bit AND with immediate, 32-bit result	op426	–	d32 = a16 & imm8, one half read, source kept	measured [†]	MM 25.139.1
16-bit AND, 32-bit result	op428	–	d32 = a16 & mask, mask a uniform from the constant pool	whole-program	MM 25.141.17; MM 25.145
16-bit AND with immediate	op435	–	d16 = a16 & imm8 (15, 63, 255 over 4,096 lanes)	measured	MM 25.192
16-bit AND	op437	–	d16 = a16 & b16; also the form for a wide immediate through a uniform	measured	glossary; MM 25.192
32-bit OR	op13575	4, 10	d = a32 b32 (the 4-byte form releases its sources)	measured	MM 24.8 row T4; g17emu EVIDENCE
32-bit OR with immediate, seven-bit destination	op13574	10	d = a32 imm, seven-bit destination (reaches R64+), carries a load wait	measured	glossary; g17emu EVIDENCE
OR with 16-bit operand	op13576	–	d = a32 zext16(b)	measured	glossary
16-bit OR	op13588	–	d16 = a16 b16	measured	MM 25.145.1
32-bit XOR	op17771	4, 10	d = a32 ^ b32 (the 4-byte form releases its sources)	measured	MM 25.145.1; MM 24.8 row T4
XOR with 16-bit second operand	op17772	–	d = a32 ^ zext16(b)	measured	glossary
XOR with 16-bit first operand	op17774	–	d = zext16(a) ^ b32	measured	glossary
16-bit XOR, 32-bit result	op17775	–	d = a16 ^ b16, 32-bit result	measured	glossary
16-bit XOR	op17784	–	d16 = a16 ^ b16	measured [†]	MM 25.145.1
32-bit NOT	op11190	–	d = ~a32	measured (receipt fit)	MM 25.145
NOT to 16-bit result	op11193	–	d16 = ~a & 0xFFFF	measured	glossary
32-bit shift left immediate	op14391	–	d = a32 « imm	measured [†]	g17emu EVIDENCE (receipt u14391)

Table B.3, continued

Name	Opcode	Lengths	Semantic	Standing	Source
32-bit shift left by register	op14392	–	$d = a \ll k$, $k = \text{low 7 bits of the amount}$, 0 when $k \geq 32$	measured	glossary; g17emu EVIDENCE
16-bit shift left immediate, 32-bit result	op14394	–	$d32 = \text{zext16}(a) \ll \text{imm}$	measured	glossary
32-bit shift right immediate	op17013	–	$d = a32 \gg \text{imm}$, logical; also <code>extract_bits</code>	measured	MM 25.84; g17emu EVIDENCE
32-bit shift right by register	op17014	–	$d = a32 \gg b$, logical	measured	glossary; g17emu EVIDENCE
16-bit shift right, 32-bit result	op17016	–	$d32 = a16 \gg n$	measured	MM 25.145.1; g17emu EVIDENCE
16-bit shift right	op17043	–	$d16 = a16 \gg n$	measured	glossary
32-bit arithmetic shift right immediate	op16805	12	$d = a \gg n$, sign-filling; executed only with source modifier 24	measured	MM 25.145.1; MM 25.80
population count	op465	–	number of set bits of a32	measured	MM 25.84
bit reverse	op14047	–	reverses the 32 bits of a	measured (receipt fit)	MM 25.84; MM 25.145
find most significant bit	op9986	–	index of the highest set bit; $\text{msb}(0) = 0xFFFFFFFF$	measured (receipt fit)	MM 25.145
msb bit scan, 16-bit result	op9989	–	the msb scan with a 16-bit destination	Apple's compiler emits	MM 25.144.6
bit interleave, Morton encode	op14147	–	Morton: bit $2i$ of $d = \text{bit } i \text{ of } a.\text{lo16}$, bit $2i+1 = \text{bit } i \text{ of } b.\text{lo16}$	measured	glossary
range mask, immediate width	op612	–	bit j of a 4-bit result set iff $\text{lo} \leq \text{src} + j < \text{lo} + w$ (lo , w immediates)	measured	glossary; MM 25.59
range mask, register width	op615	–	as range mask, immediate width (op612) with w from a 32-bit register; window wraps mod 2^{32} (41 of 41 values)	measured	MM 25.59
range mask, 16-bit source	op621	–	as range mask, immediate width (op612) from a 16-bit source	measured	glossary
integer clamp	op11364	–	Metal integer clamp, two widths	measured	MM 25.84

B.4 Moves

Table B.4. Scalar move instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
32-bit register move	op586	4	copy one 32-bit register; source lifetime is operand 3	measured	g17emu EVIDENCE; agxforge/g17/cc.py
16-bit register move	op590	–	copy one 16-bit half (dst = low 16 of src)	measured	MM 25.145.1
move immediate, two-operand form	op554	4	writes an immediate; only 0 has run	measured for 0 ⁺	MM 25.145.1
16-bit move immediate into a half	op555	4	dst16 = imm (byte 2); only 0 is claimed	measured for 0	MM 25.184
move immediate, three-operand form	op11842	10	writes a 32-bit immediate	measured	g17emu EVIDENCE; MM 25.95
uniform / staging write	op592	4	uniform write (constant program, slot 0) or staging write (slot 1–8)	measured	MM 25.179

B.5 Compares, flags and selects

Table B.5. Compare, flag and select instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
32-bit register compare-select, immediate arms	op11372	14	d = (a cc b) ? T : F; field codes 0 eq, 1 ult, 5 slt; only T = 1, F = 0 executed	measured	ledger/notes.toml [g17-all-ten-relations-from-three-codes]
32-bit compare-select, four registers	op11375	14	d = (a cc b) ? x : y; field code 0 = a > b, field code 1 = (a & b) != 0	measured	MM 25.90; ledger/claims.toml [g17-csel-code-one-is-a-bit-test]
32-bit compare-select, three registers	op11365	8, 10, 14	d = (a cc b) ? c : ...; compare against an immediate 0..256	measured	glossary; isa/g17-comparison-semantics.json
32-bit compare-select against immediate	op11362	10	d = (a32 cc imm) ? imm : imm	Apple's compiler emits	MM 25.80
32-bit register compare to 16-bit result	op11462	10, 14	d16 = (a32 cc b32) ? 1 : 0; printed cc 9 is unsigned less-than	measured	MM 25.195
16-bit compare-select, four registers	op11507	–	d16 = (a == b) ? c : d, equality (not a bit test)	measured	MM 25.90
16-bit compare-select against immediate, 32-bit result	op11392	–	d32 = (x16 cc K) ? T : F	measured	glossary
32-bit compare against immediate, select to 16-bit	op11456	–	d16 = (x32 cc K) ? t16 : F	measured	glossary

Table B.5, continued

Name	Opcode	Lengths	Semantic	Standing	Source
32-bit compare against immediate, select to 16-bit, minifloat false arm	op11458	–	d16 = (x32 cc K) ? t16 : F, minifloat F	measured	glossary
32-bit compare against immediate, select to 16-bit, register false arm	op11460	–	d16 = (x32 cc K) ? T : f16	measured	glossary
32-bit compare against immediate to half boolean	op11461	–	d16 = (x32 cc K) ? T : F, minifloat arms (half 0.0 / 1.0)	measured	glossary
32-bit register compare, select to 16-bit	op11466	–	d16 = (x32 cc y32) ? t16 : F	measured	glossary
32-bit register compare to half boolean	op11471	–	d16 = (x32 cc y32) ? T : F, minifloat arms	measured	glossary
16-bit compare against immediate to half boolean	op11491	–	d16 = (x16 cc K) ? T : F, minifloat arms	measured	glossary
32-bit compare with immediate into a flag	op10369	6, 10	flag = (a32 cc imm); printed cc = 8 field; uint counter /10, int /6	measured	MM 25.73; g17emu EVIDENCE
32-bit compare with immediate into a flag, short form	op10378	6	the 6-byte sibling of op10369; signed test against 0	measured	glossary
16-bit register compare into a flag	op10374	–	flag = (a16 cc b16); compare only, no select	measured	glossary
32-bit compare, AND-chained into a flag	op11310	–	flag = (x cc K) ? c16 : 0 (forms &&)	measured	glossary
32-bit compare, OR-chained into a flag	op11313	–	flag = (x cc y) ? T : flag_in (forms)	measured	glossary; isa/g17-condition-codes.toml
32-bit compare, AND-chained into a flag with false arm	op11314	–	flag = (x cc y) ? flag_in : F	measured	glossary
16-bit compare, chained into a flag	op11322	–	flag = (x16 cc K) ? flag_in : F	measured	glossary
half compare, chained into a flag	op9683	–	flag = (half(x) REL 0.0) ? flag_in : 0; -0 = 0, NaN false	measured	glossary
flag to 16-bit register	op14099	–	flag to a 16-bit register; after one compare form (op10379, no glossary entry) reads 1 true, 2 false	measured	ledger/notes.toml
fp32 compare-select, fmax/fmin	op9700	–	float select; codes 0 ==, 1 <, 2 >, 4 !=, 5 ≥, 6 ≤; 3 and 7 are max/min-style	measured	MM 25.145; agxforge/g17/cc.py
float compare-select, fselect form	op9748	–	float select from a 16-bit and a 32-bit source, cc 0 (==)	measured	glossary

Table B.5, continued

Name	Opcode	Lengths	Semantic	Standing	Source
float compare-select, csel form	op9749	–	as float compare-select, fselect form (op9748); the csel-named sibling	measured	glossary
half compare-select against minifloat	op9842	–	d16 = (x16 cc K) ? a16 : b16, minifloat K; source modifier 20 = abs(x)	measured	MM 25.59

B.6 fp32 arithmetic

Table B.6. fp32 arithmetic instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
fp32 add	op998	4, 6, 8, 12	d = a + b, RNE, subnormals flushed; subtract = negate modifier	measured	g17emu EVIDENCE; MM 25.180
fp32 add, common alternate encoding	op1061	–	d = a + b (alternate encoding, 511 corpus instances)	measured	glossary
fp32 add with float immediate	op1006	12	d = a + K8, 8-bit float immediate	measured	MM 25.145.1
fp32 multiply	op3290	4, 6, 14	d = a * b, subnormals flushed	measured	g17emu EVIDENCE; MM 25.73
fp32 fused multiply-add	op2190	4, 6, 8, 12, 16	d = a * b + c, one rounding	measured	MM 25.90
fp32 fused multiply-add, alternate encoding	op2238	–	d = a * b + c (alternate encoding, 288 corpus instances)	measured	glossary
fp32 FMA, literal addend	op2192	16	d = a * b + K, one rounding (3,186 discriminating lanes)	measured	MM 25.184
fp32 FMA, literal multiplicand	op2198	16	d = a * K + c; subnormal inputs flushed; NaN → 0x7fc00000	measured [†]	MM 25.145.1; g17emu EVIDENCE
fp32 FMA with two float immediates	op2200	–	d = a * K1 + K2; subnormals flushed; NaN → 0x7fc00000	measured	MM 25.145.1; g17emu EVIDENCE
fp32 saturate	op904	12	sat(a + K), K = ±0; clamps to [0, 1], –0 → +0	measured	MM 25.90; MM 25.145.1
fp32 truncate toward zero	op3818	–	trunc; 2.5 → 2, –0.3 → –0, –1.5 → –1	measured	MM 25.145.1
round to nearest even	op3770	–	rint, ties to even	measured	MM 25.130; g17emu EVIDENCE
floor	op3786	–	floor (integral inputs checked); denormals to zero per the glossary	measured	MM 25.84; glossary

B.7 fp16 and mixed-width float arithmetic

Table B.7. fp16 and mixed-width float arithmetic instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
fp16 fma, register form	op798	12	binary16 $a * b + c$, one rounding; halves as sources	measured	MM 25.196
half add with float immediate	op776	–	half($a + K8$); half subnormals kept; NaN $\rightarrow 0x7e00$	measured	MM 25.195
fp32 add, half result	op1014	–	half($a32 + b32$), one rounding; overflow $\rightarrow$ inf; NaN $\rightarrow 0x7e00$	measured	MM 25.145.1 ; g17emu EVIDENCE
fp32 plus half, half result	op1015	–	half($a32 + b16$); half subnormals kept	measured	MM 25.145.1
fp32 multiply, half result	op3306	–	half($a32 * b32$)	measured	MM 25.145.1

B.8 Conversions

Table B.8. Conversion instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
fp32 to half narrowing	op1016	–	fp32 $\rightarrow$ half, RNE, subnormal results kept, NaN $\rightarrow 0x7e00$	measured	MM 6 ; g17emu EVIDENCE
half to fp32 widening	op1004	12	half $\rightarrow$ fp32, exact; destination holes	measured	MM 25.90 ; agxforge/g17/cc.py
32-bit integer to fp32	op11179	10	int32/uint32 $\rightarrow$ fp32, RNE; operand 2: 4 unsigned, 5 signed	measured	MM 25.90
16-bit integer to fp32	op11180	–	int16/uint16 $\rightarrow$ fp32, exact; code 2 unsigned, 3 signed	measured	MM 25.195
u32 to half	op11182	–	uint32 $\rightarrow$ half, RNE, past range $\rightarrow$ +inf	measured	MM 25.184
16-bit integer to half	op11183	–	int16/uint16 $\rightarrow$ half; operand 2: 2 unsigned, 3 signed	measured	glossary
fp32 to 32-bit integer	op9320	10	fp32 $\rightarrow$ int; code 4 uint, 5 int; mode 1 truncates, saturates, NaN $\rightarrow 0$	measured	MM 25.186
half to 32-bit integer	op9321	–	half $\rightarrow$ int32	measured	glossary
fp32 to u32 into the uniform/staging file	op9328	10	fp32 $\rightarrow$ uint32 into the uniform/staging file	measured	glossary; MM 25.73
half to 32-bit integer into the uniform/staging file	op9329	–	half $\rightarrow$ int32 into the uniform/staging file; operand 3: 4 u, 5 s; truncates	measured	glossary
fp8 pack, vector, from fp32/half/bf16	op13618	–	fp8 pack, RNE, no saturation, NaN $\rightarrow$ canonical NaN (all 2^{32} inputs)	measured	MM 0.11 ; g17emu EVIDENCE
fp8 unpack to bf16	op17642	–	fp8 $\rightarrow$ two bf16, exact	measured	g17emu EVIDENCE

B.9 Special-function instructions

Table B.9. Special-function instructions.

Name	Opcode	Lengths	Semantic	Standing	Source
rsqrt seed	op3850	10	tabulated exact function; faithful (≤ 0.82 ulp); $0 \rightarrow +\text{inf}$	measured	MM 25.141.16; MM 25.136
sqrt-safe rsqrt	op3978	–	rsqrt returning 1.0 at 0	measured	MM 6; MM 25.180
hardware reciprocal	op3658	10	$1/x$ within 1 ulp, not correctly rounded; subnormal result $\rightarrow$ signed 0	measured	MM 25.136; MM 25.109
hardware exp2	op1272	10	2^x within 1 ulp, biased high, deterministic, no compact closed form	measured	MM 25.144.2; MM 25.136
hardware log2	op2570	10	$\log_2(x)$ in one instruction	Apple’s compiler emits	MM 6; MM 25.73

B.10 Cross-lane and special-register reads

Table B.10. Cross-lane instructions and special-register reads.

Name	Opcode	Lengths	Semantic	Standing	Source
SIMD shuffle-xor, immediate mask	op14169	10	lane L receives in[L ^ m]; $m \geq 32$ gives 0.0 on every lane	measured [†]	MM 12
SIMD shuffle-xor, runtime mask	op14170	10	runtime mask; must be lane-uniform	measured [†]	MM 12
quad shuffle-xor, immediate mask	op13944	–	4-lane xor; mask ≥ 4 gives 0.0	measured [†]	MM 12
quad shuffle-xor, runtime mask	op13945	–	quad runtime mask; follows each lane’s own mask	measured [†]	MM 12
SIMD shuffle, constant lane, broadcast	op14157	–	lane 0’s value; only with every lane active	measured	MM 25.145.1; MM 25.144.6
SIMD sum, fp32	op16842	10	simd_sum fp32; xor tree in mask order 1, 2, 4, 8, 16	measured	MM 25.141.16
SIMD max, fp32	op16858	10	simd_max fp32, exact on 32 lanes	measured	MM 25.90
simdgroup active-thread mask	op14208	–	mask of active lanes	measured	glossary
special-register read	op14059	4	special register $\rightarrow$ 32-bit register	measured	g17emu EVIDENCE; MM 0.8
special-register read, 16-bit	op14060	8	special register $\rightarrow$ one half; the other half is kept	measured [†]	MM 25.145.4
argument read, binding-table word	op14061	–	binding K pointer, halfwords $8 + K$ and $10 + K$	measured	MM 25.179

B.11 Control flow

Table B.11. Control-flow instructions. [Chapter 5](#) has the detail.

Name	Opcode	Lengths	Semantic	Standing	Source
exec push, if, from flag	op582	4	push and narrow the exec state by a flag	measured	MM 25.145.2 ; g17emu EVIDENCE
exec pop, restore mask	op577	4	counter = max(0, counter - n)	measured (n = 1); whole-program (n = 2)	MM 25.145.2
while-exec, set mask in place	op579	4	inverting while; sets the state in place; does not gate conditional back-edge branch (op458)	whole-program	MM 25.145.2
conditional back-edge branch	op458	10	backward branch, taken iff any lane is active	measured	g17emu EVIDENCE; docs/g17-first-dispatch-trials.md
skip branch, no active lanes	op462	10	forward branch, taken iff no lane is active	measured	MM 25.141.2
call	op450	–	subroutine call (Apple’s own program ran 32/32)	whole-program	MM 25.90
return	op580	–	subroutine return	whole-program	MM 25.90
end of program	op684	4	end of program	measured	g17emu EVIDENCE

B.12 Notes on the tables

- The glossary tags these forms “compiled” though a cited section records an execution: 32-bit subtract immediate (op11666) (receipt D11666.l12, MM [25.145.1](#), and the release-mask arms of MM [25.95](#)); 32-bit AND with immediate (op423) (confound3 receipts); 16-bit AND with immediate, 32-bit result (op426) (MM [25.139.1](#)); 16-bit XOR (op17784) (receipt t17784, MM [25.145.1](#)); 32-bit shift left immediate (op14391) (receipt u14391); move immediate, two-operand form (op554) (immediate 0 only, MM [25.145.1](#)); fp32 FMA, literal multiplicand (op2198) (receipt t2198, MM [25.145.1](#)); the four shuffle-xor forms op14169, op14170, op13944 and op13945 (SIMD shuffle-xor, immediate mask; SIMD shuffle-xor, runtime mask; quad shuffle-xor, immediate mask; quad shuffle-xor, runtime mask) (MM [12](#)); and special-register read, 16-bit (op14060) (MM [25.145.4](#)). The glossary text for op2198 (“beyond that route its semantics are unmeasured”) predates MM [25.145.1](#).
- Some glossary text describes this compiler rather than Apple. The glossary says uniform-address atomic, returning old (op10094) returns on scoreboard slot 7; Apple’s own op10094 fills slot 0 and waits on it, and slot 7 is this compiler’s patched form (MM [25.115](#)). [Chapter 9](#) covers atomics.
- Several forms run with no glossary entry; the chapters cite them by label. They are the multiply by immediate g17emu calls mul-imm (op10822) and the XOR with immediate it calls xor-imm (op17770), both executed (g17emu EVIDENCE); the exec forms op573, op574, op575, op576, op578 and op583 ([chapter 5](#)); and the selected forms op862 (half multiply), op9697, op9737 and op9740 (fused float, half and half/bfloat compare-selects), op11373, op11374 and op11385 (immediate-arm selects) and op13857 (quad_sum), whose standing is Apple’s compiler emits only.

- The tables leave out forms whose function is not stated (trig-family op, function unmeasured (op3946); fine x-derivative, saturated (op1062), unconfirmed; fp32 op with float immediate, unconfirmed (op3298)), uniform-destination constant-program forms other than those listed (MM 25.179), memory forms ([chapter 7](#)), atomics and barriers ([chapter 9](#)) and tensor forms ([chapter 10](#)).

C Opcode index

Table C.1 lists every opcode this document mentions, as the [opcode glossary](#) records it. The names are this project’s, chosen by measured function; Apple’s spellings are not used. *Standing* is the glossary’s confidence: *executed*, the behaviour confirmed by running it on this GPU; *compiled*, seen in compiled code, including programs that ran correctly, without its own semantics separately confirmed; *static*, known only from Apple’s tables or the corpus; *unconfirmed*, a partial identification whose name says what is known and marks what is not (MM 0.16). A few opcodes the chapters mention have no glossary entry; the table marks them, and the chapter that mentions one says what is known. “Discussed in” gives the one or two sections that discuss the opcode most; the last column gives up to four machine-model sections.

Table C.1. Opcode index: glossary name, standing, and where each opcode is discussed.

Opcode	Name	Standing	Discussed in	Machine-model sections
op0	uniform register file, decoder tag	static	11.2 , 2.2	25.57 , 25.67 , 25.120.1
op4	message staging file, decoder tag	static	2.2 , 7	25.27 , 25.57
op423	32-bit AND with immediate	compiled	B.3 , B.12	24.5
op424	32-bit AND	executed	2.4 , 4.1	24.5
op426	16-bit AND with immediate, 32-bit result	compiled	4.1 , 10.1	25.114 , 25.141
op428	16-bit AND, 32-bit result	executed	2.2 , 4.1	25.141
op435	16-bit AND with immediate	executed	7.7 , 7.8	25.28 , 25.192
op437	16-bit AND	executed	0.4 , 7.8	24.5
op447	threadgroup barrier	executed	9 , 7.7	25.75 , 25.104.1 , 25.104.2 , 25.106.1
op450	call	executed	5 , 5.1	25.90
op458	conditional back-edge branch	executed	5.3 , 3.5	25.101.1 , 25.142.1 , 23.10
op462	skip branch, no active lanes	executed	3.5 , 5	25.141.2 , 25.141.12
op465	population count	executed	B.3	25.84
op554	move immediate, two-operand form	compiled	3.1 , 4.1	25.101.1
op555	16-bit move immediate into a half	executed	4.1	25.177 , 25.184
op573	(no glossary entry)		5.1	
op574	(no glossary entry)		5.1	
op575	(no glossary entry)		5 , 5.1	
op576	(no glossary entry)		5 , 5.1	
op577	exec pop, restore mask	executed	5.1 , 5	25.59 , 25.89 , 25.101.1 , 25.104.2
op578	(no glossary entry)		5 , 5.1	
op579	while-exec, set mask in place	executed	5 , 5.1	25.101.1 , 25.124.6
op580	return	executed	5 , 5.3	25.90
op582	exec push, if, from flag	executed	5.1 , 5.3	25.59 , 25.89 , 25.101.1 , 25.104.2
op583	(no glossary entry)		5 , 5.1	
op586	32-bit register move	executed	2.4 , 4.1	25.59 , 25.73 , 25.101.1

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op590	16-bit register move	executed	4.1, 9.1	25.82, 25.133, 25.141.3, 25.141.17
op592	uniform / staging write	executed	3.4, 11.2	25.27, 25.179
op595	fragment spill store	compiled	2.8, 10.3	25.8, 0.7
op612	range mask, immediate width	executed	0.4, 7.4	25.58, 23.4
op615	range mask, register width	executed	B.3	25.59, 25.72
op621	range mask, 16-bit source	executed	7.4	25.59, 25.72
op684	end of program	executed	3.5, 0.4	25.68, 25.73, 25.82
op776	half add with float immediate	executed	4.4, 4.7	25.184, 25.195
op798	fp16 fma, register form	executed	2.7, 3.4	25.194, 25.196
op838	legacy 8×8 simdgroup MMA, f16	compiled	12.8, 10.13	25.12
op839	legacy 8×8 simdgroup multiply, f16	compiled	10.13	24.4
op862	(no glossary entry)		4.4	
op904	fp32 saturate	executed	4.3	25.84, 25.90
op998	fp32 add	executed	0.4, 2.4	24.5
op1004	half to fp32 widening	executed	2.7, 4.5	25.90
op1006	fp32 add with float immediate	executed	4.3, 7.8	25.88, 25.145.1
op1014	fp32 add, half result	executed	4.4	25.145.1
op1015	fp32 plus half, half result	executed	3.8, 4.4	the glossary entry
op1016	fp32 to half narrowing	executed	4.5, 10.2	6, 13
op1038	fp32 add with immediate into the uniform/staging file	compiled	7.8, 11.2	25.120, 25.179
op1048	fp32 to bfloat16 narrowing, unconfirmed	unconfirmed	4.5, 10.2	25.13
op1061	fp32 add, common alternate encoding	executed	B.6	25.142.1
op1062	fine x-derivative, saturated	unconfirmed	4.3, 8.5	25.90
op1272	hardware exp2	executed	4.6, 12.3	25.136
op2190	fp32 fused multiply-add	executed	3.1, 4.3	25.53, 25.84
op2192	fp32 FMA, literal addend	executed	4.3	25.84, 25.184
op2198	fp32 FMA, literal multiplicand	compiled	4.3	25.82
op2200	fp32 FMA with two float immediates	executed	4.3	25.145.1
op2238	fp32 fused multiply-add, alternate encoding	executed	B.6	25.142.1
op2318	fp32 FMA into the uniform/staging file	compiled	11.2	25.120
op2570	hardware log2	compiled	4.6	6, 24.5
op2842	legacy 8×8 simdgroup MMA, fp32	compiled	2.5, 10.13	24.4
op2862	legacy 8×8 simdgroup MMA, f16 in fp32 out	compiled	10.13	0.3, 9
op2902	legacy 8×8 simdgroup MMA, bf16	compiled	10.13	24.4
op3290	fp32 multiply	executed	2.5, 3.3	25.53, 25.84
op3291	fp32 times half multiply	static	2.7	25.88
op3298	fp32 op with float immediate, unconfirmed	unconfirmed	B.12	25.84
op3306	fp32 multiply, half result	executed	4.4	25.145.1

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op3658	hardware reciprocal	executed	4.6, 4.1	25.136
op3770	round to nearest even	executed	4.3, 10.11	25.130
op3786	floor	executed	B.6	25.84
op3818	fp32 truncate toward zero	executed	4.3	25.145.1
op3850	rsqrt seed	executed	4.6	25.136, 25.141
op3946	trig-family op, function unmeasured	static	4.6	24.4
op3978	sqrt-safe rsqrt	executed	4.6	6, 24.4
op5098	tensor MMA, fp32 $\times$ fp32, accumulate	executed	10.1, 10.2	0.3, 2
op5099	tensor MMA, fp32 $\times$ fp32, no accumulator	executed	10.2	0.3, 2
op5100	tensor MMA, fp32 A $\times$ 16-bit B, accumulate	executed	10.1, 10.2	0.3, 2
op5101	tensor MMA, fp32 A $\times$ 16-bit B, no accumulator	executed	10.1, 10.2	0.3, 2
op5104	tensor MMA, 16-bit A $\times$ fp32 B, accumulate	executed	10.1, 10.2	0.3, 2
op5105	tensor MMA, 16-bit A $\times$ fp32 B, no accumulator	executed	10.2	0.3, 2
op5106	tensor MMA, 16-bit $\times$ 16-bit, accumulate	executed	2.4, 3.3	0.3, 2
op5107	tensor MMA, 16-bit $\times$ 16-bit, no accumulator	executed	0.4, 2.5	0.3, 2
op5108	table-only narrow-accumulator MMA, first	static	10.2	25.6, 25.13
op5143	table-only narrow-accumulator MMA, last, no accumulator	static	10.2	25.6, 25.13
op9320	fp32 to 32-bit integer	executed	4.5, 10.11	25.130, 25.186
op9321	half to 32-bit integer	executed	B.8	25.59
op9328	fp32 to u32 into the uniform/staging file	executed	5.3	25.59
op9329	half to 32-bit integer into the uniform/staging file	executed	B.8	25.59
op9683	half compare, chained into a flag	executed	B.5	the glossary entry
op9697	(no glossary entry)		B.12	
op9700	fp32 compare-select, fmax/fmin	executed	2.5, 4.2	25.117, 25.130
op9737	(no glossary entry)		B.12	
op9740	(no glossary entry)		B.12	
op9748	float compare-select, fselect form	executed	B.5	25.59
op9749	float compare-select, csel form	executed	3.8	25.59
op9842	half compare-select against minifloat	executed	B.5	25.59
op9986	find most significant bit	executed	4.1, 9.1	the glossary entry
op9989	msb bit scan, 16-bit result	compiled	8.3	25.144.6
op9994	(no glossary entry)		3.7	
op9995	(no glossary entry)		3.7	
op9996	64-bit no-return atomic, max/min	compiled	3.7, 9.3	25.59
op10019	per-lane-address no-return atomic	compiled	9.3	25.59
op10022	uniform-address no-return atomic, 32-bit	compiled	9.3	25.59

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op10023	uniform-address no-return compare-exchange	compiled	3.7, 9.3	25.59
op10090	per-lane-address atomic, returning	executed	9.3, 9.1	the glossary entry
op10091	per-lane-address compare-exchange, returning	executed	9.3	the glossary entry
op10094	uniform-address atomic, returning old	executed	9.3, 2.4	25.59, 25.112, 25.115, 25.115.1
op10095	uniform-address compare-exchange, returning	executed	9.3, 3.7	25.59
op10239	32-bit saturating add	executed	2.2, 2.4	25.59
op10267	64-bit add	executed	B.1	25.59
op10279	32-bit add immediate	executed	3.4, 4.1	25.114
op10282	32-bit add	executed	3.4, 4	25.114
op10283	add, 32-bit plus zero-extended 16-bit	executed	4.1, 7.1	25.145.1
op10285	add 16-bit to 32-bit	executed	4.1, 10.1	25.93, 25.195
op10286	16-bit plus 16-bit, 32-bit result	executed	4.1, 7.7	the glossary entry
op10289	16-bit add immediate	executed	4.1, 7.7	25.104, 25.192, 25.195
op10298	32-bit add immediate into the uniform/staging file	compiled	11.2	25.120
op10369	32-bit compare with immediate into a flag	executed	5.3, 4.2	25.59, 25.73
op10374	16-bit register compare into a flag	executed	B.5	25.59
op10378	32-bit compare with immediate into a flag, short form	executed	B.5	25.59
op10379	(no glossary entry)		4.2	
op10384	tensor MMA, int8 widening to int32	executed	10.2, 3.8	0.3, 2
op10385	tensor MMA, int8 widening, no accumulator	executed	10.2	0.3, 2
op10789	32 × immediate to 64-bit product	executed	B.2	25.59
op10790	64-bit address multiply-add	executed	B.2	25.59
op10793	32 × 32 to 64-bit product	executed	B.2	25.80
op10795	32 × 32 + 32 to 64-bit multiply-add	executed	B.2	25.59
op10822	(no glossary entry)		B.12	
op10823	multiply by immediate, 32-bit addend	executed	B.2	25.59
op10824	multiply by immediate, 16-bit addend	executed	2.2	25.59
op10825	32-bit multiply	executed	4.1, 4.6	25.141
op10826	32-bit multiply-add	executed	4.1	25.145
op10829	multiply-add, 16-bit multiplier	executed	B.2	25.59
op10830	multiply-add, 16-bit multiplier and addend	executed	B.2	25.59
op10831	16-bit multiply by immediate plus immediate	executed	B.2	25.59
op10835	multiply-add, 16-bit multiplicand	executed	B.2	25.59
op10836	multiply-add, 16-bit multiplicand and addend	executed	B.2	25.59
op10838	signed-by-unsigned 16-bit multiply-add	executed	4.1	25.59

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op10858	16-bit multiply by immediate plus immediate, 16-bit result	executed	B.2	25.59
op10860	16-bit multiply by immediate plus register	executed	B.2	25.59
op10909	texture write	compiled	0.2, 3.1	25.120.1
op11059	multiply by immediate minus immediate	executed	4.1	25.59
op11060	multiply by immediate minus register	executed	2.2, 4.1	25.59
op11063	multiply-subtract	executed	4.1	25.59
op11179	32-bit integer to fp32	executed	4.5, 7.8	25.90, 25.130
op11180	16-bit integer to fp32	executed	4.5	25.144.1, 25.195
op11182	u32 to half	executed	4.5, 4.7	25.179, 25.184
op11183	16-bit integer to half	executed	10.11	25.59, 25.195
op11185	integer to float into the uniform/staging file	static	7.8, 11.2	25.120, 25.179
op11190	32-bit NOT	executed	9.1	25.84
op11193	NOT to 16-bit result	executed	B.3	25.59
op11310	32-bit compare, AND-chained into a flag	executed	4.2	25.59
op11313	32-bit compare, OR-chained into a flag	executed	4.2	the glossary entry
op11314	32-bit compare, AND-chained into a flag with false arm	executed	B.5	the glossary entry
op11322	16-bit compare, chained into a flag	executed	B.5	the glossary entry
op11362	32-bit compare-select against immediate	compiled	B.5	25.84
op11364	integer clamp	executed	B.3	25.84
op11365	32-bit compare-select, three registers	executed	4.1	25.80
op11372	32-bit register compare-select, immediate arms	executed	4.2, 4.7	25.59
op11373	(no glossary entry)		4.1	
op11374	(no glossary entry)		B.12	
op11375	32-bit compare-select, four registers	executed	4.1, 4.2	25.90
op11385	(no glossary entry)		B.12	
op11392	16-bit compare-select against immediate, 32-bit result	executed	B.5	25.59
op11456	32-bit compare against immediate, select to 16-bit	executed	B.5	25.59
op11458	32-bit compare against immediate, select to 16-bit, minifloat false arm	executed	B.5	25.59
op11460	32-bit compare against immediate, select to 16-bit, register false arm	executed	B.5	25.59
op11461	32-bit compare against immediate to half boolean	executed	B.5	25.59
op11462	32-bit register compare to 16-bit result	executed	3.3	25.51, 25.80, 25.195
op11466	32-bit register compare, select to 16-bit	executed	B.5	25.59
op11471	32-bit register compare to half boolean	executed	B.5	25.59

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op11482	16-bit compare-select against immediate	compiled	3.8	25.58
op11491	16-bit compare against immediate to half boolean	executed	B.5	25.59
op11507	16-bit compare-select, four registers	executed	4.2	25.90
op11564	texture-coordinate staging write	static	7.9	25.120.1
op11624	32-bit saturating subtract	executed	4.1	25.145
op11655	64-bit subtract immediate	compiled	B.1	25.58
op11666	32-bit subtract immediate	compiled	2.5, 4.1	25.95
op11667	32-bit subtract	executed	4.1, 9.3	the glossary entry
op11701	threadgroup atomic, no return	executed	9.3	25.59
op11765	threadgroup atomic, returning old	executed	9.3, 9.1	25.59, 25.87, 25.115
op11769	(no glossary entry)		9.3	
op11842	move immediate, three-operand form	executed	3.1, 0.4	25.62, 25.95, 25.114.2, 25.114.3
op11994	ray-tracing block load, 32-bit	static	2.7	25.88
op12073	device load, register address	compiled	7.2	25.39.2
op12118	(no glossary entry)		7.2	
op12135	(no glossary entry)		7.2	
op12151	imageblock load, 32-bit	executed	7.2, 7.7	25.104.1, 25.104.2
op12352	MMA operand load, space unconfirmed	unconfirmed	7.2	25.144.1
op12364	threadgroup-memory load	executed	7.2, 7.6	25.210
op12367	(no glossary entry)		7.6	
op12646	16-bit load	executed	7.1, 7.2	25.141.3, 25.141.17
op12656	int8 tensor load, one word	executed	7.2, 7.3	25.23, 25.34, 0.8
op12657	masked int8 tensor load	executed	10.8	25.34, 25.37, 25.51, 0.8
op12673	plain 64-bit load, pair address	compiled	3.4, 7.2	25.141.3
op12674	tensor load, half/bf16	executed	3.4, 7.2	0.8
op12675	masked tensor load, half/bf16	executed	0.4, 7.2	25.23, 25.24, 0.8
op12682	plain 32-bit load	executed	0.4, 3.3	25.23, 25.24, 25.34
op12688	immediate-address one-word load	executed	7.1, 7.2	25.67, 25.120.1, 25.177
op12691	plain 64-bit load	executed	7.1, 7.2	25.37, 25.51, 25.73, 25.139.2
op12692	tensor load, fp32	executed	7.2, 10.8	25.37, 25.51
op12697	immediate-address two-word load	executed	7.3	25.177, 25.184
op12700	three-component vector load	executed	7.2	25.90
op12706	immediate-address three-word load	executed	7.3	25.179
op12709	128-bit vector load	executed	3.1, 3.4	0.8
op12715	immediate-address four-word load	executed	7.3	25.177
op13075	imageblock store, 32-bit	executed	7.2, 7.7	25.96, 25.104.1, 25.104.2
op13276	dequantized-block store, likely threadgroup	unconfirmed	7.2	25.144.1

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op13288	threadgroup-memory store	executed	7.2, 7.6	25.210
op13483	two-byte filler	static	3.5, 11.1	25.68, 25.71, 25.73, 25.82
op13574	32-bit OR with immediate, seven-bit destination	executed	4.1	25.144.8
op13575	32-bit OR	executed	2.4, 3.1	25.84
op13576	OR with 16-bit operand	executed	B.3	25.59
op13588	16-bit OR	executed	B.3	25.145.1
op13618	fp8 pack, vector, from fp32/half/bf16	executed	4.5, 10.11	25.21
op13754	camera/video kernel op, function unknown	unconfirmed	3.2, 3.3	25.58
op13856	quad prefix sum, fp32	static	8	25.82
op13857	(no glossary entry)		8	
op13944	quad shuffle-xor, immediate mask	compiled	8, 8.1	12, 24.5
op13945	quad shuffle-xor, runtime mask	compiled	8, 8.1	12, 24.4
op14047	bit reverse	executed	9.1	25.84
op14059	special-register read	executed	0.4, 3.3	25.82, 25.120.1, 25.141.13
op14060	special-register read, 16-bit	compiled	0.4, 2.2	25.144.6
op14061	argument read, binding-table word	executed	7.3, 7.8	25.35, 25.59, 25.67, 25.120
op14099	flag to 16-bit register	executed	4.2	25.59
op14147	bit interleave, Morton encode	executed	2.2	25.59, 25.72
op14156	device fence	executed	9, 9.2	25.140.5, 25.141.1
op14157	SIMD shuffle, constant lane, broadcast	executed	2.4, 8	25.102
op14158	SIMD shuffle, register lane	compiled	8, 8.3	25.144.6
op14169	SIMD shuffle-xor, immediate mask	compiled	8, 8.1	12
op14170	SIMD shuffle-xor, runtime mask	compiled	8, 8.1	12, 24.4
op14208	simdgroup active-thread mask	executed	8, 8.5	25.59
op14214	SIMD boolean reduce	unconfirmed	8.3	25.144.6
op14391	32-bit shift left immediate	compiled	B.3, B.12	25.130, 25.141
op14392	32-bit shift left by register	executed	4.1	25.145
op14394	16-bit shift left immediate, 32-bit result	executed	B.3	25.59
op14661	texture sample	compiled	0.2, 3.1	25.120.1
op15813	texture read	compiled	7.8, 7.9	25.120.1
op16805	32-bit arithmetic shift right immediate	executed	4.1	25.145.1
op16842	SIMD sum, fp32	executed	8, 8.4	25.90, 25.141
op16858	SIMD max, fp32	executed	8, 8.4	25.90
op17013	32-bit shift right immediate	executed	B.3	25.84
op17014	32-bit shift right by register	executed	B.3	25.128
op17016	16-bit shift right, 32-bit result	executed	6.1, 10.1	25.93
op17043	16-bit shift right	executed	B.3	24.5
op17193	16-bit store	executed	7.1, 7.2	25.71

Table C.1, continued

Opcode	Name	Standing	Discussed in	Machine-model sections
op17199	16-bit store at an immediate address	executed	7.3	25.184, 25.195
op17202	word store at word index	executed	7.2, 10.11	25.105, 25.105.1, 25.105.2, 25.127
op17229	plain 32-bit store	executed	0.4, 2.4	25.8, 25.30, 25.37
op17235	store, sub-form 01, texel/word-slot	executed	3.1, 7.2	25.62
op17238	(no glossary entry)		7.1, 7.2	
op17244	two-component store	executed	3.1, 7.2	25.62, 25.177
op17253	immediate-address three-word store	executed	7.3	25.184, 25.192
op17256	128-bit vector store	executed	2.4, 3.4	25.8, 0.8
op17257	tensor store	executed	0.4, 3.4	0.8
op17258	masked tensor store	executed	0.4, 3.4	25.23, 0.8
op17262	immediate-address four-word store	executed	7.2, 7.3	25.177
op17642	fp8 unpack to bf16	executed	10.11	25.12
op17770	(no glossary entry)		B.12	
op17771	32-bit XOR	executed	2.4, 4.1	25.84
op17772	XOR with 16-bit second operand	executed	B.3	25.59
op17774	XOR with 16-bit first operand	executed	B.3	25.59
op17775	16-bit XOR, 32-bit result	executed	B.3	25.59
op17784	16-bit XOR	compiled	B.3, B.12	25.59
op17789	scalar spill store, 32-bit XOR into the staging file	executed	2.8	25.59

D Source code locations

Table D.1. Source modules and data files for each part of the compiler and the two execution paths.

Concern	Module
IR	agxforgel/g17/ir.py
selection, allocation, emission	agxforgel/g17/cc.py
tensor lowering	agxforgel/g17/tlower.py, tensorgemm.py, tensor.py
field maps and encoders	isa/g17-tensor-mma-fieldmap.json, agxforgel/g17/mmaenc.py, memenc.py, auth.py, encode.py
register domain	agxforgel/g17/registerdomain.py
released-register check	agxforgel/g17/releasecheck.py
metadata and object	agxforgel/g17/mdgen.py, authorobj.py, scanlink.py, verify.py
decoder (Apple’s)	agxforgel/g17/model.py, tools/agx3dis
Metal front end	tools/g17front.py
CPU emulator	tools/g17emu.py, tools/g17shippedcheck.py
model kernels and graphs	tools/g17deliver.py, g17q4graph.py, g17prefillgraph.py, g17qmv.py, g17attn.py, g17qsm.py
Metal executor	tools/g17decodegen.m
native runtime	agxforgel/g17/inferencegraph.py, tools/g17inferencelower.py, tools/g17inferencebundle.py, spike/agxsub/g17workload_inference.h

Colophon

Built from AGXForge revision 7e39ba977e95 (2026-10-04). The $\text{\LaTeX}$ source is in docs/tex, one file per chapter; `make -C docs/tex` writes the revision stamp and rebuilds this PDF with Lua $\text{\LaTeX}$. `tools/g17docs.py --check` verifies that every machine-model citation names a real section, every cross-reference has a target and every opcode is named beside its number. Text set in Libertinus and Inconsolata; figures drawn in TikZ, the fragment map generated from the measured formula.